THÈSE

En vue de l'obtention du

DOCTORAT DE L'UNIVERSITÉ DE TOULOUSE

Délivré par : *l'Université de Toulouse (UT)*

Présentée et soutenue le *(JJ/MM/AAAA)* **par :**

Clément BLANCO VOLLE

Adaptive Control through Socially Emergent Learning of an Ensemble of Local Expert Agents

---

JURY

Premier MEMBRE	Professeur d'Université	Président du Jury
Second MEMBRE	Astronome Adjoint	Membre du Jury
Troisième MEMBRE	Chargé de Recherche	Membre du Jury

École doctorale et spécialité :
MITT : Domaine STIC : Intelligence Artificielle

Unité de Recherche :
Institut de Recherche en Informatique de Toulouse - IRIT)

Directeur(s) de Thèse :
Nicolas VERSTAEVEL, *Stéphanie COMBETTES* et *Michel POVLOVITSCH SEIXAS*

Rapporteurs :
Jean-Paul JAMONT et *Flavien BALBO*

Abstract

As the automotive industry gradually moves toward fully autonomous driving, the development of safe and intelligent transportation systems remains a crucial challenge, specifically with regard to interactions between the driver and the vehicle. The effectiveness of advanced driver assistance systems (ADAS), such as speed recommendation systems, depends on their ability to collaborate with the driver while taking into account the non-stationarity and uncertainty inherent in driver behavior.

Many machine learning approaches exploit the decomposition of the input space into specialized local models in order to better capture the variability of dynamics and facilitate continuous adaptation. This organization not only improves robustness in non-stationarity contexts, but also provides localized uncertainty estimates, facilitating the integration of learned models into safe control strategies.

In recent years, local learning based on contextual agents has emerged as a promising extension of these approaches. Inspired by the principles of self-organizing multi-agent systems, these methods allow each specialized model, represented by an agent, to cooperate and share information to enhance online adaptation, uncertainty management, and model interpretability. In this context, we study the following question: to what extent cooperative local learning architectures can serve as a foundation for safe control strategies, exploiting model introspection to guide exploration and support reliable decision-making ?

This thesis models the problem of speed recommendation as a problem of adaptive control of complex systems with unknown and non-stationary dynamics. We introduce the CELL (Context Ensemble Local Learning) formalism, inspired by self-organizing multi-agent systems. This formalism allows us to design online learning architectures capable of modeling nonlinear dynamics through the cooperation of specialized local models. This theoretical framework allows us to design systems that continuously adapt to environmental changes while preserving a spatialized representation of knowledge, which is essential for estimating the epistemic uncertainty required for reliable learning-based control.

Based on this formalism, we propose two systems derived from CELL: oCELL, in which local models are spatialized using orthotopes, and kCELL, which uses radial basis functions to obtain smoother approximations. A first series of experiments demonstrates that the geometric organization of agents offers introspective capabilities that can be linked to the notions of explainability and interpretability in machine learning. In particular, kCELL proves capable of overcoming the stability-plasticity dilemma in a non-stationary environment, while remaining competitive with classical approaches. kCELL is then integrated into a predictive control system (Model Predictive Control - MPC). We exploit the model's introspective properties to direct exploration towards areas of high uncertainty and ensure safety by avoiding poorly modeled regions.

Finally, future research directions are highlighted concerning predictive uncertainty quantification and the use of heterogeneous local models, paving the way for more robust and transparent adaptive learning systems.

Résumé

Alors que l'industrie automobile évolue progressivement vers une conduite entièrement autonome, le développement de systèmes de transport sûrs et intelligents demeure un enjeu crucial, notamment en ce qui concerne les interactions entre le conducteur et le véhicule. L'efficacité des systèmes avancés d'aide à la conduite (ADAS), tels que les systèmes de recommandation de vitesse, repose sur leur capacité à collaborer avec le conducteur tout en tenant compte de la non-stationnarité et de l'incertitude inhérentes à son comportement.

Plus largement, de nombreuses approches en apprentissage automatique exploitent la décomposition de l'espace d'entrée en sous-modèles locaux spécialisés afin de mieux capturer la variabilité de la dynamique et de faciliter l'adaptation continue. Cette organisation permet non seulement d'améliorer la robustesse face à la non-stationarité mais aussi de fournir des estimations d'incertitude localisées, facilitant l'intégration des modèles appris dans des stratégies de contrôle sûres.

Ces dernières années l'apprentissage local fondé sur des agents contextuels a émergé comme une extension prometteuse de ces approches. Inspirée des principes des systèmes multi-agents auto-organisés, ces méthodes permettent à chaque modèle spécialisé, modélisé par un agent, de coopérer et de partager l'information pour renforcer l'adaptation en ligne, la gestion de l'incertitude et l'interprétabilité du modèle.

Dans ce contexte, nous nous posons la question question suivante: dans quelle mesure des architectures d'apprentissage local coopératif peuvent-elles servir de fondation à des stratégies de contrôle sûres, exploitant l'introspection du modèle pour orienter l'exploration et soutenir une prise de décision fiable en condition réelles ?

Cette thèse modélise le problème de la recommandation de vitesse en un problème de contrôle adaptatif de systèmes complexes à dynamique inconnue et non-stationaire. Nous introduisons le formalisme CELL (*Context Ensemble Local Learning*), inspiré des systèmes multi-agents auto-organisés. Ce formalisme permet de concevoir des architectures d'apprentissage en ligne capables de modéliser des dynamiques non linéaires grâce à la coopération de modèles locaux spécialisés. Ce cadre théorique permet de concevoir des systèmes s'adaptant continuellement aux changements environnementaux tout en préservant une représentation spatialisée des connaissances, essentielle à l'estimation de l'incertitude épistémique requise pour un contrôle fiable basé sur l'apprentissage.

À partir de ce formalisme, nous proposons deux systèmes dérivés de CELL: oCELL, dans lequel les modèles locaux sont spatialisés à l'aide d'orthotopes, et kCELL qui utilise des fonctions de base radiales pour obtenir des approximations plus lisses. Une première série d'expériences démontre que l'organisation géométrique des agents offre des capacités d'introspection qui peuvent être reliées aux notions classiques d'explicabilité et d'interprétabilité en apprentissage automatique. En particulier, kCELL se révèle capable de surmonter le dilemme stabilité-plasticité dans un environnement non stationnaire, tout en étant compétitif avec les approches classiques. kCELL est ensuite intégré dans un système de contrôle prédictif (*Model Predictive Control - MPC*). Nous exploitons les propriétés d'introspection du modèle pour orienter l'exploration vers les zones de forte incertitude et sécuriser l'exploitation en évitant les régions mal modélisées. Nous introduisons une nouvelle méthode d'explicabilité pour le contrôle sous contraintes, permettant de quantifier l'influence des contraintes sur le résultat de l'optimisation.

Enfin, les orientations futures de la recherche sont mises en évidence concernant la quantification de l'incertitude prédictive et l'utilisation de modèles locaux hétérogènes, ouvrant la voie à des systèmes d'apprentissage adaptatifs plus robustes et transparents.

Contents

General Introduction

A robot can be defined by its ability to sense, think and act [208]. As of the 21st century, this definition is currently undergoing a profound transformation. In modern systems, a fourth component emerges as essential: *communication*. This evolution represents a transition from robots as isolated industrial units, to robots as collaborative partners [19].

As we move from the technology-driven era of industry 4.0 toward the anthropocentric vision of Industry 5.0, the focus is shifting back to the human operator [19] in the objective of building a synergy between machines and humans.

Robots need to move beyond being mere executors of pre-programmed tasks to penetrate consumer markets and more complex social environments. This requires developing social skills and advanced learning capabilities to achieve true synergy with human partners. This synergy strongly relies on the ability of a system to not only perceive its environment but to also adapt its behavior to the uncertain dynamics introduced by human presence.

This shift is especially present in the field of intelligent mobility. Nowadays, autonomous vehicles technologies are developing rapidly. Some companies in the US like Waymo or Baidu in China, already moved toward a driverless future by proposing autonomous taxi services. However, the reality of the consumer market must be nuanced. Current projections suggest that full market saturation by autonomous vehicles may not occur until the 2050s [146]. For the time being, the human driver will remain a central component of the transport ecosystem.

Consequently, developing Advanced Driver Assistance Systems (ADAS) is of public utility. Technologies such as Adaptive Cruise Control (ACC) and Electronic Braking Systems (EBS) are essential to assists the driver and keep road users safe.

In particular, given the obvious correlation between speed mismanagement and road fatalities, the development of Intelligent Speed Assistance (ISA) systems has become a major concern for global road safety.

In some sense, an ISA system can be conceptualized as a robot. Its "body" is the vehicle, its "senses" are the suite of on-board sensors, its "thoughts" are the algorithms running on electronic control units and its "actions" are the recommendation of speed setpoints communicated to the driver.

This setting represents a unique Human-Robot Collaboration (HRC) challenge. Including the human in the loop introduces a significant layer of uncertainty and non-stationarity. To be effective, the system cannot remain static and must be able to anticipate driver intent, consider behavioral variability and adapt to continuously changing driving contexts.

However, achieving this level of adaptation in safety-critical systems poses a dilemma: how can a controller learn and adapt in real-time without sacrificing the predictability and transparency required for safe control? This tension between adaptability and reliability

leads to the core research question of this thesis:

To what extent can cooperative local learning architectures, such as CELL, serve as a foundation for adaptive safe control strategies by exploiting model introspection to guide exploration and support reliable decision-making ?

Contributions

The foundational challenges induced by this collaboration are explored through the prism of adaptive control with a continuously learning dynamic model. To address our research question, we explore self-organizing multi-agent systems for supervised learning. We argue that the decomposition of complex tasks into local, specialized experts governed by cooperation rules, provides the necessary introspection capabilities to bridge the gap between online learning and adaptive safe control.

We demonstrate that the bottom-up social feedback and the local nature of these agents do more than just enable adaptation. They provide inherent introspection capabilities that can be linked to traditional interpretability and explainability literature. In addition to that, we show that these properties can be leveraged to enhance the optimization process in Model Predictive Control (MPC) to influence exploration or exploitation.

Our contributions are structured along two primary axes. On one hand, we contribute to the field of online learning by introducing the CELL (Context Ensemble Local Learning) formalism. It provides the theoretical foundations for local adaptations that allow for online learning in non-stationary environments from a stream of data. We propose three specific instantiations of CELL, namely oCELL, kCELL and gpCELL. We demonstrate through experiments their ability to solve regression tasks while maintaining a meaningful spatialization of knowledge. The local nature and bottom-up social feedbacks of the learning procedure leads to the emergence of introspection properties that can be linked to the notions of interpretability and explainability [30].

On the other hand, we provide contributions to the field of control with learned models. We investigate how the introspection capabilities of the CELL models can be leveraged to improve the reliability of autonomous systems. Specifically, we integrate the kCELL model into a MPC pipeline, where the model's introspection properties are used to guide data collection through uncertainty-aware exploration and to safeguard the system during exploitation by avoiding poorly modeled regions. In addition to that, we exploit the structure of local agents to develop a novel explainability approach that quantifies the influence of constraints on the final control decisions.

Finally, we provide engineering insights on the application of this thesis contributions in the industrial use case of speed recommendation, discussing the required future research directions toward real-world deployment.

Outline of the Thesis

This thesis is structured as following (cf. Figure 1 for the illustrated version):

- Chapter 1 introduces the industrial and regulatory context of this thesis which originates from industrial requirements from AUMOVIO. It establishes the scientific abstraction process from a speed recommendation task to the more generalist challenge of adaptive control for non-stationary systems with unknown dynamics.

- Chapter 2 provides a comprehensive review of literature regarding online learning methods and control strategies. It positions the research works of this thesis at the intersection of these two fields by identifying the specific research gaps related to real-time adaptation.

- Chapter 3 introduces the CELL formalism and details the iterative development of oCELL and kCELL algorithms. It also presents some experiments that demonstrate how local cooperation rules enable for learning and adaptation.

- Chapter 4 presents the integration of kCELL into a MPC scheme. The mechanisms of uncertainty-aware exploration and exploitation are detailed. In addition to that, a novel explanation method for controllers is proposed to analyze the influence of constraints in the optimization process.

- Chapter 5 exposes prospective works about predictive uncertainty quantification that showcase the possibilities offered by the modularity of the CELL formalism for machine learning. It introduces gpCELL, a new instantiation of CELL aimed at scaling Gaussian Processes to large datasets.

- Chapter 6 returns to the initial industrial motivation of this thesis discussing the applications of the developed methods for intelligent speed recommendation under an engineering point of view.

- Chapter 7 summarizes the contributions of this work and discusses the broader implications for adaptive control with learned models. It also highlights the future directions for research.

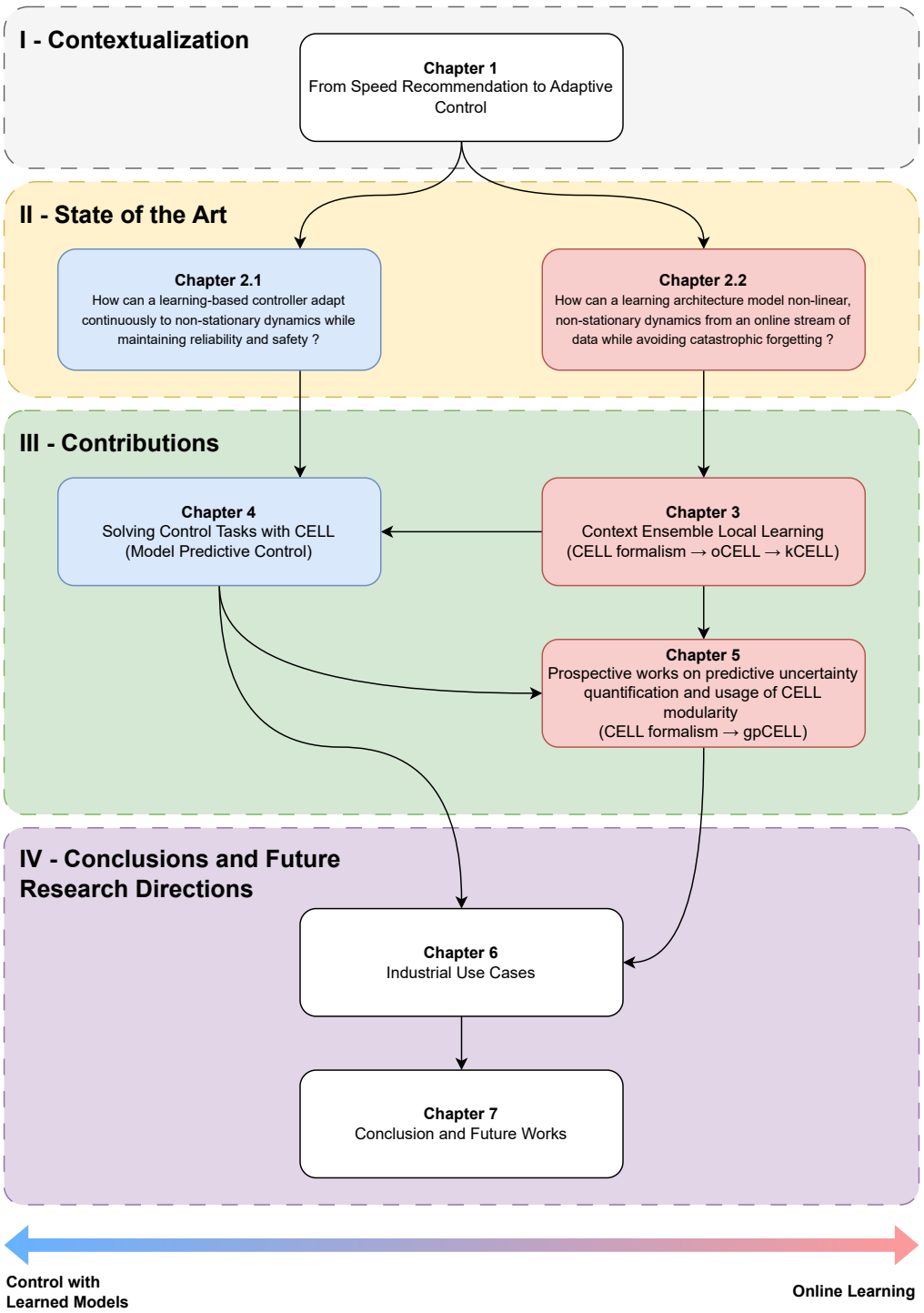


Figure 1: Outline of the Thesis

Chapter 1

From Speed Recommendation to Adaptive Control

Prior to achieving complete vehicle autonomy, the future of mobility in the short and medium term lies in the development of effective collaboration between humans and machines. There is a need to move from theoretical collaboration to functional safety-critical control systems. Thus, it is necessary to address the inherent unpredictability and non-stationarity introduced by humans in the context of this collaboration.

This work was initiated within the industrial context of developing Intelligent Speed Assistance (ISA) for the automotive sector. The core scientific challenge of this task lies in the development of a recommendation system able to adapt continuously to the behavior of a human driver that evolves over time. This property is key in order to incorporate more intelligence into the speed recommendation process to move beyond static recommendation systems. This chapters demonstrates that by treating the vehicle and the driver as a whole dynamical non-stationary system, the speed recommendation problem can be abstracted into a problem of Adaptive Control with a Learned Model.

The objective of this chapter is to detail the reasoning that leads from the specific industrial case of speed recommendation to a more generalist scientific abstraction. Section 1.1, exposes the industrial context of this thesis. Section 1.2 presents a contextualization on speed management systems and their current limitations. Then, Section 1.3 shows how a speed recommendation problem can be abstracted into an adaptive control problem with a continuously learning model. Finally, Section 1.4, exposes the research questions that frame the contributions of this thesis into the fields of *online learning* and *control with learned models*.

1.1 Industrial Context

This doctoral research is conducted under a CIFRE funding in partnership with AUMOVIO, a global supplier specialized in automotive engineering and mobility solutions. AUMOVIO develops hardware, software and systems to make mobility safe, connected, autonomous and more intelligent. The strategic vision of the company is rooted in the Software-Defined Vehicle (SDV) philosophy. By decoupling hardware and software development cycles, the

company aims to leverage its ecosystem of sensors, including radar and vision systems to build more intelligent and adaptive Advanced Driver Assistance Systems (ADAS).

Speed management remains the most significant lever for improving road safety. Excessive speed is a primary factor in serious accidents, affecting both driver's reaction time and the physical limits of the vehicle's braking. Research has consistently demonstrated a direct, nonlinear relationship between speed and fatality risk. For instance, empirical crash data suggests that the probability of a fatality in a collision grows exponentially with speed, while the risk is about 10% at 40km/h, it surges to over 80% at 80km/h [116].

Despite these risks, drivers often fail to maintain safe speeds due to environmental complexity, missing signage or adverse weather conditions. While passive safety systems such as airbags and seatbelts provide essential reactive protection, they are often insufficient to preserve physical integrity during high-speed impacts, making active speed management a priority for the automotive industry.

To address these challenges, the European Transport Safety Council (ETSC) recommends the deployment of Intelligent Speed Assistance (ISA) (cf. Figure 1.1). This system is designed to inform the drivers about legal speed limits and encourage compliance. The ETSC argues that it has the potential to reduce road collisions by 30% and fatalities by 20% [74].

Beyond safety, speed management increasingly intersects with broader logistical and environmental goals, such as reducing fuel consumption, improving passenger comfort and optimizing estimated times of arrival. This has led to significant regulatory changes:

- As of July 6, 2022: the European Union mandated that all new vehicle types must be equipped with ISA.
- As of July 7, 2024: this requirement was extended to all new vehicles sold within the EU market [73].

A critical aspect of the current European regulation [72] is the preservation of driver authority over the system. The ISA system must be overridable (e.g through a maintained pressure on the accelerator) and allow for manual deactivation during a journey. This ensures that the system acts as a collaborative assistant rather than an autonomous override.

For AUMOVIO, this creates a significant engineering and scientific challenge to make the most out of ISA systems. The device must provide recommendations that are effective enough to ensure safety and efficiency, yet intuitive enough to be accepted by a human driver who retains ultimate control. As it is explored in subsequent sections, the success of such device depends on its ability to adapt to the inherent uncertainties of human behavior under various driving contexts.

1.2 Background and Motivation

Excesses and mismanagement of speed on the roads have been proven to be positively correlated to the occurrence of accidents with injuries and fatalities [108, 70]. Studies have shown that vehicle speed control devices like Intelligent Speed Assist (ISA) could significantly contribute to the reduction of road accidents [220]. Moreover, better speed management can lead to significant reduction in energy consumption [244] making this issue a strategic concern for individuals, but also for fleet management companies [8]. Consequently, developing

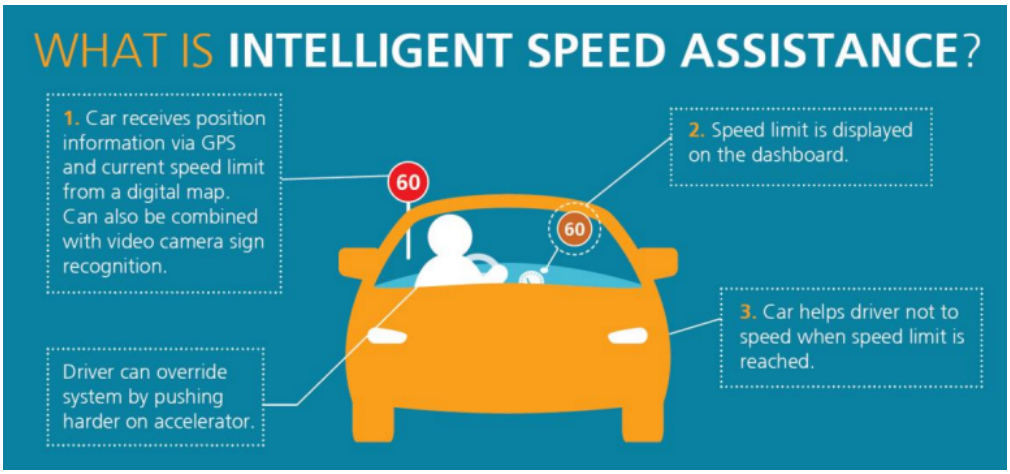


Figure 1.1: Definition of Intelligent Speed Assistance (ISA) according to the European Transport Safety Council (ETSC). [75]

speed management systems that help drivers maintain safe and efficient speeds is essential for public safety and for the sustainable development of the automotive industry.

From an industrial perspective, the main challenge for developing intelligent speed recommendation systems lies in designing a system that effectively influences vehicle speed while maintaining a high level of user acceptance, safety and energy efficiency. As the driver can chose to *accept*, *delay* or *refuse* a recommended speed setpoint, the effectiveness of such recommendation systems is largely determined by the underlying system architecture and the way it interacts with a human driver, which has an uncertain behavior.

Research on speed management can be broken down into two distinct branches as displayed on Figure 1.2. On one hand, *open-loop* systems comprise mainly advisory systems that do not consider any form of feedback from the vehicle or driver responses to recommendation setpoints. On the other hand, *closed-loop* systems solely consider feedbacks from the vehicle state, removing the driver from the execution loop. This way, a large portion of the uncertainty brought by the driver can be ignored, making it easier to provide safety guarantees and stability.

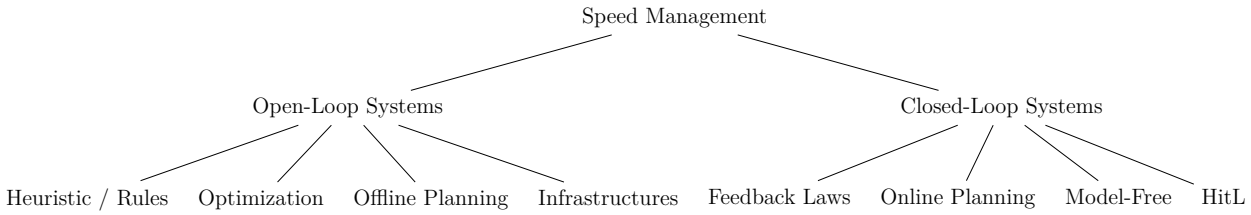


Figure 1.2: Taxonomy of Vehicle Speed Management Approaches

Open-loop systems correspond to systems that do not consider feedback from the driver or vehicle current state. In the speed management literature, these works primarily focus on translating environment constraints such as traffic light timings [143, 7], weather conditions [84] or road geometry [111] into recommended speed profiles. Among the proposed approaches, some employ advanced optimization techniques like Pontryagin's Maximum Prin-

ciple [175] to maximize energy efficiency. Others rely on passive infrastructure-based solutions ranging from physical speed enforcement via road bumps [194] to visual stimulations [254] aiming at increasing speed awareness through environmental design.

However, whether digital or physical, these approaches share the same structural limitation because they assume that the recommendation setpoints are going to be followed perfectly and instantaneously. This assumption ignores the non-stationarity and variability of the human behavior.

Definition 1. In the context of this thesis, **non-stationarity** refers to a dynamical system whose underlying transition dynamics are not constant over time [217].

Consequently, without online feedback mechanisms to account for driver acceptance, the theoretical benefits of these optimizations are rarely fully realized in real-world conditions.

Works on closed-loop speed management systems are rooted in the development of autonomous vehicles. In a sense, they resolve the main limitation of open-loop systems by completely ignoring the driver and focusing solely on vehicle control. In this context, the system plays an active role in speed regulation by acting directly on low level vehicle's actuators (throttle/brake), thereby bypassing the uncertainties of driver acceptance. The related literature can be divided into two technical branches: *model-based online planning* and *data-driven* approaches.

On the one hand, *model-based online planning* approaches rely extensively on physical models of the vehicle coupled with Model Predictive Control (MPC) [11, 134, 105], Pontryagin's Maximum Principle (PMP) [155, 176] or Dynamic Programming (DP) [215]. These approaches take into account changes in the environment by solving constrained optimization problems in real-time that mostly balance energy efficiency, safety and travel time. Since solving many optimization problems can be costly, research efforts have focused on finding analytical solutions [176, 159, 105] or using discretization techniques [128, 126] to solve the optimization problems. These methods offer rigorous safety and stability guarantees as they rely on predefined physical models coupled with control theory. However, the main limitation exposed for open-loops systems remains: the human driver is excluded from the control loop, resulting in mostly irrelevant systems when considering a human driver controlling the vehicle. Moreover, the capabilities of the resulting controllers depend on the accuracy of the physical models used. To ensure the controller operates in real-time, it is often necessary to make compromises between the complexity and accuracy of the physical model, making it difficult to handle large number of variables.

On the other hand, *learning-based* approaches have emerged to address the limitations of physical modeling, particularly for capturing complex, nonlinear behaviors that are difficult to formalize analytically. By leveraging Reinforcement Learning, data-driven approaches also simplify multi-objective optimization through the design of reward functions that balance energy, safety and comfort [243, 68, 152]. Various architectures are leveraged to handle speed control tasks such as Deep Deterministic Policy Gradient (DDPG) for car-following and heavy vehicles [216, 166], Proximal Policy Optimization (PPO) for traffic efficiency [243, 152, 253] or Soft-Actor-Critic (SAC) and Deep Q-Network (DQN) for safe speed control [123, 137]. These methods also allow to generate human-like profiles by learning directly from vast datasets of driving trajectories [251, 256]. However, these approaches rely on extensive and sample inefficient training, resulting in frozen policies that cannot adapt to the non-stationary

nature of the road environment. Furthermore, compared to model-based methods, data-driven controllers lack formal safety and stability guarantees. They mostly rely on neural networks which make enforcing hard constraints difficult in safety-critical control systems. This is due to the brittleness of their predictions outside the training distribution and to the lack of interpretability of their inner workings [249]. Neural networks can be very efficient but are black-boxes that are hard to trust and validate, specifically in industrial safety-critical use cases in which the use of neural networks is mostly reduced to small certifiable networks, largely limiting the scalability potential [249].

Despite their differences, both model-based and data-driven closed-loop architectures share a common limitation: they prioritize the autonomy of the vehicle, denying the authority of the driver. Human-in-the-Loop (HitL) systems seek to address this limitation. These approaches attempt to reintegrate the driver into the control loop by treating the human behavior as a dynamic variable of the problem rather than ignoring it. In this field, literature focuses on personalizing speed recommendations by monitoring physiological signals such as heart rate to model stress levels [156], or by estimating driver acceptance scores to modulate the speed recommendation strategy [230]. Other works leverage simple online reinforcement learning strategies to guide the static cloud-based planning toward the driver's historical preferences [174] or use context-aware models to predict expert driver reactions on blind intersections [193]. These approaches represent significant steps toward driver consideration into closed-loop systems. However, they still remain limited by their offline nature. Most of these HitL systems are built upon static databases or pretrained driver profiles that fail to account for the variability of human driving behaviors and for the non-stationarity of those behaviors that evolve over time. In reality, a driver's response is not a fixed parameter but a non-stationary process influenced by physiological states, environmental context and long-term behavioral adaptation.

The research literature on speed management systems reveals a significant gap in the state-of-the-art. Open-loop advisory systems respect driver's autonomy but lack the consideration of feedbacks on driver's behavior to ensure high effectiveness through acceptance. Conversely, closed-loop autonomous systems offer high performance and theoretical guarantees but exclude the human driver from the execution loop. Finally, HitL methods that try to reinsert the driver in the loop through personalization and data-driven approaches, are limited by their inability to adapt in real-time to non-stationary behavior and to continuously learn.

We identified a research gap in the lack of approaches that can continuously learn and adapt to the non-stationarity of the human driver behavior. The next section shows that from the identified research gap, the industrial speed recommendation problem can be abstracted to a more fundamental scientific challenge: the control of complex non-stationary systems. By considering the vehicle-driver dynamical system as a whole with unknown non-stationary dynamics, the problem is reformulated as a task of Adaptive Control with a Learned Model, which is the core focus of this thesis.

1.3 Problem Statement

To address the gaps identified in the literature (mainly the failure to effectively taking into account the human driver), this thesis proposes a shift in perspective. Rather than viewing speed recommendation as a static recommendation task, we treat it as a Human-Robot Collaboration (HRC) problem.

Definition 2. Human-Robot Collaboration (HRC) is defined as the process where a human and a robot work together as a team, through a shared workspace and a shared task, to achieve a common goal by leveraging the complementary strengths of both actors [203].

In the context of speed recommendation, the components of this HRC definition can be interpreted as follows:

- **Shared Workspace**: This corresponds to the vehicle and its immediate driving environment. The recommendation system (robot) and the driver (human) act within the same physical and informational space: the system communicates speed recommendations to the driver, while the driver's inputs (e.g., pedal control) simultaneously influence vehicle motion.
- **Shared Task**: This corresponds to longitudinal control and speed regulation. The recommendation system computes optimal speed setpoints, and the driver executes them, forming a continuous loop of recommendation and response.
- **Common Goal**: This represents the desired driving behavior that balances safety, fuel efficiency, passenger comfort, and travel time.
- **Complementary Strengths**: This refers to the synergy between the computational capabilities of the digital recommendation system, enabled by on-board sensor data, and the human driver's perceptual and reasoning abilities, such as recognizing complex driving contexts or social cues that the recommendation system might overlook.

Effective HRC depends on the robot's ability to anticipate human actions [54]. The robot must infer the driver's intended motion and behavior to synchronize its recommendations effectively. Forecasting a human's response, enables the recommendation system to provide setpoints that are complementary to the human's intent rather than being reactionary or disruptive.

Consequently, we propose to tackle the speed recommendation problem as a control problem under the assumption that the vehicle-driver dynamical system is considered as a whole. Given the non-stationarity and complexity of human behavior, this dynamical system cannot be described by a static physical model. Instead, as it is non-stationary, it must be continuously learned online from experience. When formulated this way, the speed recommendation problem becomes a problem of **Adaptive Control** that can be decomposed into two subproblems: **online learning** and **control with learned model**.

First, the **online learning** subproblem consists in the continuous refinement of a predictive model $\hat{f} : \mathbb{R}^d \to \mathbb{R}^m$ of the vehicle-driver dynamics from a stream of state-action pairs $[s_t, u_t] \in \mathbb{R}^d$. Function $\hat{f}$ maps a state s_t and an action u_t to the next state such as

$$\hat{f}\left(s_t, u_t\right) = s_{t+1} \tag{1.1}$$

Then, the **control with learned model** subproblem consists in the derivation of a control law that minimizes a cost function J over a horizon H based on the predictions of the learned model $\hat{f}$.

To solve these subproblems in a non-stationary environment, we argue for the necessity of using a local learning approach in which the feature space is partitioned into regions managed by local experts. This choice is driven by the requirements for interpretability and real-time adaptability. In the context of HRC, interpretability may not be a strict mathematical requirement for control but it is a functional necessity to:

- Ensure the **certifiability** of the system [144].
- Improve **human trust** toward the system [10].

Regarding safety, industrial standards require that the internal predictive models driving the decision-making process be **traceable and verifiable**. This task remains notoriously difficult with black-box global approximators like neural networks. For HRC, a human is more likely to accept and cooperate with a system whose behavior is predictable and based on transparent logic.

Moreover, global approximation methods often struggle with the trade-off between **model adaptability** and **knowledge retention** [163]. With local learning methods, the model can locally adapt to new behaviors and incrementally without corrupting unrelated knowledge. Thus, the model remains accurate across different driving contexts without requiring computationally heavy, and often impossible, offline training.

Local learning provides transparency by decomposing complex global dynamics into a set of specialized local models. Unlike global architectures where a single weight update can unpredictably affect the entire model, a local learning approach can ensure that any update stays local. This spatialization ensures that any prediction is traceable to a specific region of the feature space.

The long-term goal of this research is to enable efficient HRC, specifically in the automotive sector. This goal depends on the reliability of the underlying control architecture. Therefore, this thesis does not address the high-level social or psychological aspects of HRC but rather the foundational subproblems required to make the best out of HRC: **local online learning** and **control with learned models**. The following section details the research questions that shape the goal of this thesis.

1.4 Research Questions

The industrial and scientific challenges detailed in previous sections highlight a tension between the need for continuous adaptation and the requirement for reliability. To guide the works of this thesis and provide robust contributions, we investigate the two following research questions:

- **Online Learning of Non-Stationary Dynamics**: How can a learning architecture continuously model nonlinear, non-stationary dynamics from an online stream of data without losing previously acquired knowledge?

– **Reliability in Learning-Based Control**: How can a learning-based controller adapt continuously to non-stationary dynamics while maintaining reliability and safety ?

The answer to these questions lies at the intersection of two research fields: **machine learning** and **control theory**.

The following chapter provides a comprehensive review of literature organized around these two research axes.

Chapter 2

State of the Art

The previous chapter showed how the industrial use case of speed recommendation can be generalized into a broader scientific problem. When considering the human in the loop, the speed recommendation problem naturally fits within the context of **Human-Robot Collaboration** (HRC). By explicitly modeling the driver-vehicle pair as a single dynamical system, this HRC formulation can further be abstracted as an **Adaptive Control** problem. In contrast to traditional robotics, which often focuses on autonomous execution in static or isolated environments, this thesis addresses HRC challenges where the developed system must anticipate and adapt to the **uncertainty** and **non-stationarity** introduced by human behavior.

The objective of this chapter is to review the existing research landscape to address the two research questions formulated in Section 1.4:

- **Online Learning of Non-Stationary Dynamics**: How can a learning architecture continuously model nonlinear, non-stationary dynamics from an online stream without losing previously acquired knowledge?
- **Reliability in Learning-Based Control**: How can a learning-based controller adapt continuously to non-stationary dynamics while maintaining reliability and safety ?

This literature review is structured around two axes that mirror the subproblems defined in Section 1.3.

Section 2.1 traces the evolution of control theory from reactive regulation to predictive optimization with Model Predictive Control (MPC). It analyzes the shift from the use of first-principle models toward learning-based methods such as Reinforcement Learning (RL).

Section 2.2 addresses the specific modeling requirements for a continuously learning model that could be integrated in a MPC setting and satisfy the requirements of HRC. This section examines the evolution of learning paradigms from static batch processing to online methods capable of handling data streams. The emphasis is put on the inherent trade-off between model adaptability and knowledge retention. This section argues in favor of the architectural advantages of **local online learning** and more specifically for the exploration of multi-agent systems for online supervised learning. Finally, Section 2.3 concludes on this chapter by positioning our works within these fields and highlights the specific gaps that we intend to fill.

2.1 Control with Learned Models

The design of autonomous systems capable of seamless and efficient interactions with humans requires a control architecture that is both mathematically rigorous and adaptive. This section provides a review of literature about control theory aiming at answering the following question:

How can a learning-based controller adapt continuously to non-stationary dynamics while maintaining reliability and safety?

Section 2.1.1 introduces control theory to introduce the formalism that will be used throughout this thesis. Then, Section 2.1.2 examines the shift from error-driven feedback laws such as Proportional-Integral-Derivative (PID) controllers to optimization-based frameworks like Model Predictive Control (MPC). It highlights how the shift toward predictive optimization was driven by the need to handle multivariable processes and, to respect hard physical and safety constraints in real-world settings. However, as systems move from controlled industrial conditions to complex and uncertain real-world environments, relying on static physics-based models becomes a limiting factor. Consequently, Section 2.1.3 explores the integration of Machine Learning into the control loop. By analyzing the strengths and limitations of model-free Reinforcement Learning (RL) and Model-Based RL (MBRL), we argue that pure end-to-end learning-based approaches often compromise safety and reliability established by classical methods. Finally, Section 2.1.4 provides a synthesis of the identified research gaps.

2.1.1 Control Theory Formalism

Prior to exploring the motivations for learning-based control methods, the control theory formalism underlying this thesis must be defined. Control theory consists in studying the means of influencing the behavior of a dynamical system to achieve a desired state s_{ref} or trajectory.

The evolution of a physical system over time is governed by its dynamics. In the context of digital control, a discrete-time representation is defined such as the next state s_{t+1} is a function of the current state s_t and the applied action u_t:

$$s_{t+1} = f\left(s_t, u_t\right) \tag{2.1}$$

where $f : \mathcal{S} \times \mathcal{U} \mapsto \mathcal{S}$ represents the transition function. In classical control theory, f is assumed to be a known, stationary first-principles model.

Definition 3. A first-principles model is a mathematical model of a dynamical system derived directly from fundamental physical laws (such as conservation of mass, energy, momentum or Newton's laws). The states and parameters of the model have a clear physical meaning rather than being fitted purely from data [129].

This interaction between the system and the controller is traditionally represented as a closed-loop system as illustrated in Figure 2.1. The controller can perceive the states of the system and act on it through actuators. The sets representing the possible states (respectively

actions) are referred to as the state space $\mathcal{S}$, (respectively action space $\mathcal{U}$). The state space $\mathcal{S}$ represents all possible configurations of the system (e.g joint angles, velocities). The state of the system at time t is denoted as $s_t \in \mathcal{S}$. The action space $\mathcal{U}$ is the set of all possible control inputs (e.g., motor torques, voltage). The action at time t applied on the system is denoted as $u_t \in \mathcal{U}$.

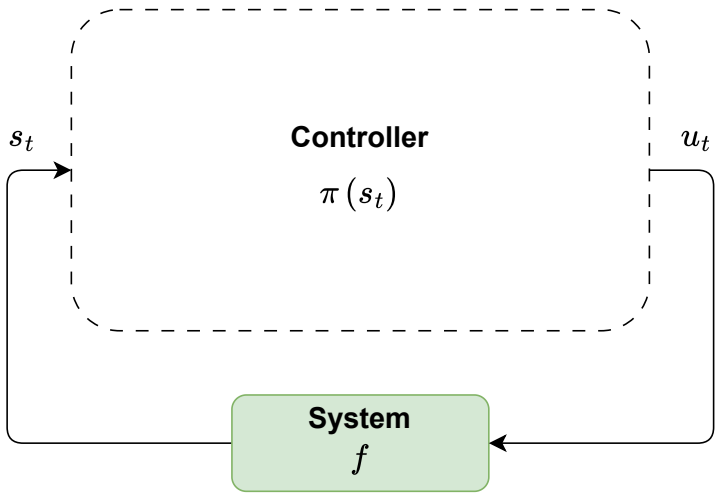


Figure 2.1: Illustration of Closed-Loop Control. The *System* block corresponds to the targeted dynamical system to be controlled. The transition dynamics are described by function f. The state s_t is transmitted to the *Controller*, whose behavioral policy is described by function π. Then, from the s_t the controller choses an action u_t to be applied on the *System*.

The controller follows a goal that is mathematically described by the minimization of a cost function. For a given horizon T (that could be equal to ∞), the objective is to minimize the cumulative cost J

$$J\left(s_{t:t+T}, u_{t:t+T-1}\right) = \sum_{k=0}^{T-1} l\left(s_{t+k}, u_{t+k}\right) + l_T\left(s_{t+T}\right) \tag{2.2}$$

where $s_{t:t+T} \in \mathcal{S}^T$ is a state trajectory, $u_{t:t+T-1} \in \mathcal{U}^{T-1}$ is a sequence of actions, l is the stage cost (e.g. tracking error or energy consumption) and l_T is the terminal cost.

Depending on the task to be accomplished and its requirements, systems can be subject to physical and safety limits. These are expressed as algebraic inequalities such as

$$g\left(s_t, u_t\right) \leq 0 \text{ or } h\left(s_t, u_t\right) = 0 \tag{2.3}$$

These constraints may represent hard limits (e.g. maximum torque) or safety boundaries (e.g. speed limits, minimum distance to human operator).

Finally, the control law π encapsulates the decision-making logic such as

$$u_t = \pi\left(s_t\right) \tag{2.4}$$

The cost function, the constraints and the transition function are part of the standard Optimal Control Problem (OCP), which serves as the template for the controllers discussed in the remainder of this chapter. An OCP is formulated as following:

$$\begin{aligned} \min_{u_{t:t+T-1}} J\left(s_{t:t+T}, u_{t:t+T-1}\right) = \min_{u_{t:t+T-1}} \quad & \sum_{k=0}^{T-1} l\left(s_{t+k}, u_{t+k}\right) + l_T\left(s_{t+T}\right) \\ \text{s.t.} \quad & s_{t+k+1} = f\left(s_{t+k}, u_{t+k}\right) \\ & u_{t+k} \in \mathcal{U},\ s_{t+k} \in \mathcal{S},\ k \in \{0, \ldots, T\} \\ & g\left(s_{t+k}, u_{t+k}\right) \leq 0 \\ & h\left(s_{t+k}, u_{t+k}\right) = 0 \end{aligned} \tag{2.5}$$

The existence and quality of a solution to the OCP defined above depends on the following three properties of the system [173]:

- **Controllability:** This property assesses whether the control input u_t can drive the system from any initial state s_0 to any desired state s_t within a finite time. This property is not always guaranteed, as real-world systems often have fewer actuators than state variables. For example, a car has a steering wheel and an accelerator, but it cannot move sideways directly. Its lateral position cannot be changed independently of its forward motion. In this example, the system is **globally controllable** (the car can go anywhere) but **locally constrained** (the sideways movement needs more steps).

- **Observability:** This corresponds to the ability to **reconstruct the state** s_t solely from available measurements. It is not always guaranteed, as sensors may be noisy or missing entirely. For example, an eco-driving device might lack a sensor for the road slope. If the device cannot infer the slope by observing changes in speed or engine load, the state is partially observable and the efficiency might be compromised.

- **Stabilizability:** Even if a system is not fully controllable, it is considered stabilizable if all the unstable modes can be brought to a steady state using the available actions. This property focuses on whether the controller can prevent the system from blowing-up or crashing. For example, a self-balancing vehicle (e.g. a Segway) is inherently unstable as the center of mass resides above the pivot point. While the actuators are only able to apply horizontal force on the base (making it not fully controllable), the system remains stabilizable because those horizontal movements can be modulated to maintain the vertical equilibrium.

Beyond the internal properties of the system, the effectiveness of a controller is governed by the complexity of the considered system. This is formalized by Ashby's Law of Requisite Variety [12], which states that for a controller to effectively manage a system, its internal variety, i.e the number of possible states, must be at least equal to the variety of the disturbances it aims to compensate for. In other words, a controller must be **at least as complex** as the task of keeping the goal satisfied. An extension of this theorem, the Conant-Ashby theorem [52], states that every good controller of a system **must be a model of that system**.

To ground this formalism in an physical example that satisfies these foundational properties, let's consider the classical pendulum problem [39, 222]. A mass m is attached to a

pivot by a rod of length L. The objective is to control its angular position θ using a motor that applies a torque τ at the pivot (c.f Figure 2.2). The components of the problem are the following:

- **State** s_t: A state is represented by the angle and angular velocity at time t such as $s_t = \left[\theta, \dot{\theta}\right]^\top$
- **Action** u_t: An action corresponds to the application of a torque on the joint of the pendulum.
- **Dynamics** f: The dynamics are derived from Newton's Second Law, where the next state depends on gravity, damping and the applied torque u_t.
- **Constraints** (g, h): An inequality constraint g represents the physical limit of the torque that can be applied by the motor ($g\left(s_t, u_t\right) = |u| - u_{\max} \leq 0$) while an equality constraint h enforces the dynamics ($h\left(s_t, u_t\right) = f\left(s_t, u_t\right) - s_{t+1}$)
- **Objective** J: The cost function penalizes the distance from a target angle (e.g reaching $\theta = 0$) while minimizing the energy u_t^2 consumed by the motor.

The pendulum is a fully controllable, observable and stabilizable system. Since the transition function f is perfectly known and the system possesses these mathematical properties, it is a ***perfect* control problem**. For this reason, it represents a simple baseline system on which control algorithms can be tested before they are applied to more complex high-dimensional systems.

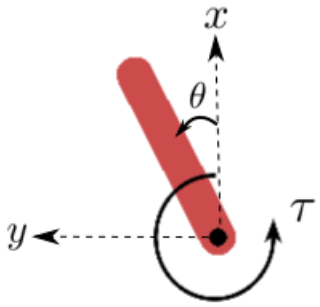


Figure 2.2: Illustration of the Pendulum dynamical system. θ corresponds to the angle of the pendulum bar relative to the vertical axis x. The torque applied to the joint of the pendulum is denoted by τ in the diagram, it corresponds to u_t.

2.1.2 From Reactive Regulation to Predictive Optimization

As shown in the previous section, the objective of control theory is to build a controller that interacts with a system via actuators to achieve a desired behavior. For example, this behavior can be defined by a setpoint or a reference trajectory. Historically, reactive feedback laws have been among the most prevalent approaches to controller design. The industrial standard for such controller is the **Proportional-Integral-Derivative** (PID) [13, 185]. This

controller is essentially error-driven, meaning that it generates an action in response to a deviation e_t between the reference s_{ref} and the measured output [13] (cf. Figure 2.3). Its ease of implementation and effectiveness in various industrial contexts have made it a default controller to try setting up before attempting more sophisticated approaches. However, its effectiveness diminish in a non-stationary environment and it struggles with handling several inputs and outputs, limiting the possibilities in terms of manageable complexity and multi-objective consideration. In addition to that, this type of controller can be considered as myopic because the control action is issued in response to past or current errors. It thus, lacks the ability to anticipate future changes and avoid suboptimal well solutions that can bring the system into a state from which it is impossible to recover [14, 185].

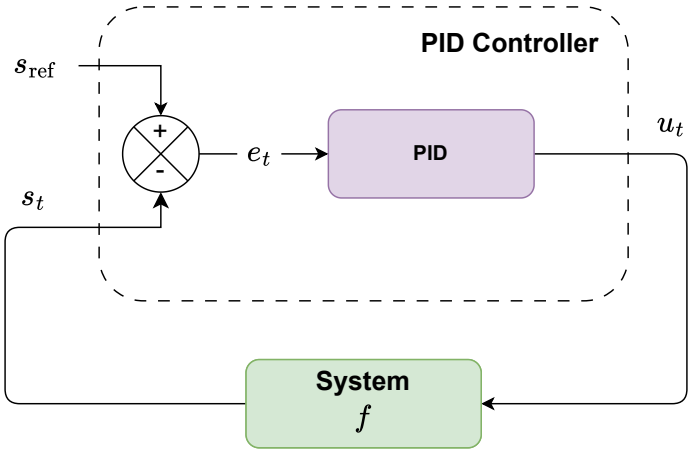


Figure 2.3: Illustration of Proportional Integral Derivative (PID) Control. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by function f. The state s_t of the *System* is transmitted to the *PID Controller*. There, the state s_t is compared to a target state s_{ref}. The *PID* block then produces an action u_t to correct this error.

Control tasks progressively shifted towards the management of more complex, multivariable processes. In this setting, purely reactive laws that do not consider a model of the environment, are inherently limited. To address these limitations, Optimal Control frameworks were developed. Instead of simple gain tuning, the control laws are derived by minimizing a quadratic cost function that coordinates multiple inputs and outputs simultaneously [201]. For example, **Linear Quadratic Regulator** (LQR) [131] (cf. Figure 2.4) provides an analytical solution to the OCP subject to linear dynamics by minimizing a quadratic cost function. It ensures stability under some conditions, and optimality under the assumption of perfect model knowledge giving a control law such as

$$u_t = -Gs_t \tag{2.6}$$

where u_t is the action, s_t the current state of the system and G the gain matrix derived from Ricatti Equations [132].

To handle noisy measurements and partial state observability, the **Linear Quadratic Gaussian** (LQG) [133] controller integrates a LQR with a Kalman filter, yielding an optimal

control policy under Gaussian white noise assumption. However, LQG can lack robustness to model mismatches, i.e when the predictive model of the dynamics does not correspond to the reality anymore. To account for structural uncertainties and external disturbances, H_∞ [79, 248] controller was developed. It treats control design as a minimax optimization problem. Thus, it aims at minimizing the worst-case gain, thereby ensuring robust performance with conservative behavior.

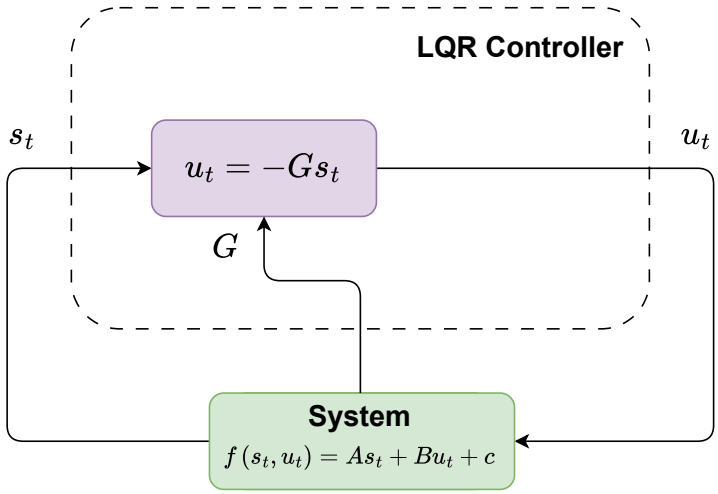


Figure 2.4: Illustration of Linear Quadratic Regulator (LQR) Controller. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by a linear function f defined as $f(s_t, u_t) = As_t + Bu_t + c$. The linear expression of the transition function f is used to derive the matrix G (from the Riccati Equations [131]) defining the control law of the *LQR Controller*. The action u_t of the controller is obtained from this linear control law and applied on the *System*.

In classical LQR or H_∞, the optimization has **no hard constraint**, that is to say, it assumes that actions have infinite range, and the system states can fluctuate without bounds. In reality, physical systems are governed by **hard limits**. For example, an electric motor has a maximum torque it can produce before the current saturates or the windings overheat, and a chemical valve can only transition between fully open (100%) and fully closed (0%) positions. Beyond action constraints, the system states themselves often have safety or structural boundaries. For instance, a robotic arm must avoid joint limit angles to prevent mechanical damage, and a drone must maintain a minimum altitude above the ground to avoid a collision. Ignoring these hard constraints can lead to catastrophic system failure when the control law demands an action that the hardware cannot execute. When these hard constraints are reached, classical feedback laws often require heuristics (e.g anti-windup strategies [85]) which can compromise the stability and optimality of the system [185].

This highlights the need to shift from reactive control with static feedback gains to a predictive framework grounded in optimization. This paradigm transitions from pre-calculated control laws towards the repeated resolution of OCP over a finite future horizon. By embedding a predictive model directly within a mathematical optimization framework (e.g

Quadratic Programming), the controller gains the ability to **proactively plan actions**. In modern robotics and automotive systems, this is primarily achieved through **Model Predictive Control** (MPC) [89], which is the main control framework for online constrained planning [24, 201].

Whereas global optimization methods (e.g. Dynamic Programming) solve the entire control problem beforehand and are computationally prohibited for real-time use, MPC relies on a **receding horizon**. At each timestep t, the controller solves an OCP to find a sequence of future actions $u_{t:t+T-1} = \{u_t, \dots, u_{T-1}\}$. Given the current state s_t, this sequence minimizes a cost function J over a finite prediction horizon T, subject to constraints, as formalized in 2.5.

The defining feature of MPC is the **receding-horizon principle**: only the first control input u_t is applied to the system. After that, the horizon is shifted forward and the optimization is repeated at the next timestep. This control loop is summarized in Algorithm 1. Figure 2.5 illustrates the overall MPC architecture, highlighting the interactions between the environment, the predictive model and the optimization solver.

Algorithm 1: MPC Control Loop

Input: Initial state s_t, prediction horizon T, system model f, cost function J, constraints $\mathcal{C}$

for $t = 0, 1, 2 \dots$ **do**

  Measure or estimate the current state s_t

  $u_{t:t+T-1} \leftarrow$ solve OCP for $(s_t, T, f, \mathcal{C}, J)$

  Apply first action u_t

end

Historically, the model is derived from **first-principles** (e.g kinematic or dynamic equations of motion), assuming that the underlying physics are well-understood and stationary. To ensure that the resulting trajectory is physically feasible, constraints on both actions u_t and states s_t are embedded directly into the optimization problem.

Two types of constraints can be distinguished: **hard** and **soft** constraints [65]. On the one hand, **hard constraints** correspond to conditions that must strictly be satisfied for a solution to be valid. On the other hand, a **soft constraint** is a constraint that can be violated but with an associated penalty, thereby preserving feasibility at the cost of degraded performance.

When the safety or operational requirements allow for constraint relaxation, a hard constraint can be reformulated as a soft constraint by introducing penalty terms in the cost function. One of the most common approach consists in adding a regularization term Ω to the cost function such as

$$J_{\text{global}}\left(s_{t:t+T}, u_{t:t+T-1}\right) = J\left(s_{t:t+T}, U\right) + \lambda\Omega\left(s_{t:t+T}, u_{t:t+T-1}\right) \tag{2.7}$$

where $\Omega\left(s_{t:t+T}, u_{t:t+T-1}\right) \geq 0$ represents the cost associated with violating the constraints $g\left(s_t, u_t\right) \leq 0$ and $h\left(s_t, u_t\right) = 0$. This penalty term is defined such that $\Omega = 0$ when all constraints are satisfied, while $\lambda > 0$ determines the trade-off between performance optimality and constraint enforcement. The choice of both λ and the penalty structure directly impacts constraint satisfaction, numerical conditioning and closed-loop behavior.

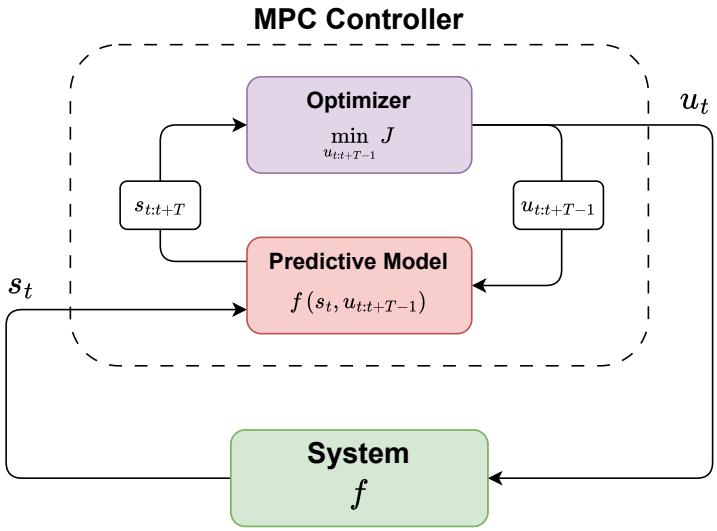


Figure 2.5: Model Predictive Control (MPC) Architecture. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by the function f. The state s_t of the *System* is transmitted to the *MPC Controller*. There, the state s_t is used by the *Predictive Model* to predict trajectories $s_{t:t+T}$ over a receding horizon of length T using the real or an approximation of the transition function f. The *Optimizer* exploits the predictions of the *Predictive Model* to find the optimal sequence of actions $u_{t:t+T-1}$. Then only the first action u_t of the sequence is applied on the *System*.

Solving such constrained optimization problems in real-time requires efficient numerical solvers. Depending on the structure of the dynamics, cost function and constraints, different classes of solvers may be employed. For instance, BFGS (Broyden–Fletcher–Goldfarb–Shanno) [172] can be applied to general nonlinear dynamics and cost functions but do not natively enforce hard constraints without additional mechanisms. In contrast, solvers such as OSQP (Operator Splitting Quadratic Program) [212] are designed to efficiently solve Quadratic Programs (QP) with linear constraints. When the considered dynamics and constraints are linear and the cost function J is quadratic, the use of OSQP is particularly relevant because the MPC problem can be formulated as a QP of the form:

$$\begin{aligned} \min_{U} \quad & \sum_{t=0}^{T-1} \left(s_t^\top Q s_t + q^\top s_t + u_t^\top R u_t + r^\top u_t \right) \\ \text{s.t.} \quad & s_{t+1} = A s_t + B u_t + c \\ & h_{\text{lin}}(s_t, u_t) \leq 0 \\ & g_{\text{lin}}(s_t, u_t) = 0 \end{aligned} \tag{2.8}$$

where

- A, B and c define the linear dynamics.
- h_{lin} and g_{lin} represent the linear inequality and equality constraints.

– the matrices Q, R and vectors q, r define the quadratic cost function.

Constraint satisfaction guarantees in MPC depend on both the **problem formulation** and the **numerical solver** used. Some solvers are designed to strictly enforce hard constraints upon convergence while others provide only approximate solutions that could result in small constraint violations due to early termination, numerical tolerances or limited iteration budgets imposed by real-time computation requirements [44, 114].

Moreover, enforcing hard constraints may lead to infeasible optimization problems in practice. For example, this may be due to model mismatch, disturbances or state estimation errors. Such infeasibility must be handled explicitly at deployment time, either through constraint relaxation, solving a feasibility problem beforehand or leveraging fallback strategies [44, 114].

Despite the effectiveness of MPCs in industry, these performance are contingent upon the accuracy of the used predictive model f. In complex real-world environments, particularly those involving Human-Robot Collaboration (HRC), deriving an analytical model that remains accurate over time is often infeasible. First-principles predictive models struggle to capture stochastic and non-stationary nature of human behavior [113]. This leads to a significant model mismatch that can compromise both performance and safety. To address these limitations, a shift toward **Learning-based Control** is proposed. Learning-based methods offer new possibilities to either augment or replace traditional analytical models with architectures capable of adapting to more complex environments [232, 140].

The following section examines the motivations for transitioning from a first-principles predictive model to a learning-based predictive model within the MPC framework.

2.1.3 From Physics-Based to Learning-Based Control

The effectiveness of Model Predictive Control (MPC) described in the previous section is contingent upon the fidelity of the internal predictive model f. Traditionally, first-principles predictive models are used within MPC. However, for complex control tasks, such models can be difficult to derive analytically without introducing assumptions that may significantly reduce the accuracy of the predictive model. This section exposes the rationale behind the transition from **first-principles** predictive models towards **learning-based** predictive models within MPC.

While first-principles predictive models can be very efficient for well-characterized stationary dynamics, they reach their limits in complex non-stationary environments, particularly in HRC. In these cases, accurate analytical models are either computationally prohibitive or non-existent for describing uncertain and non-stationary phenomena like human behavior.

To address this issue, early research focused on Hybrid MPC leveraging physics-informed learning (cf. Figure 2.6). In this paradigm, a low-fidelity first-principles model (e.g. linear model) used as a baseline, is paired with a statistical learning model such as a Gaussian Process (GP) [238], to compensate for residual errors [15]. This approach preserves part of the structural safety of the physics-based controller. However, since theoretical safety guarantees only apply to the baseline first-principles model, an uncertainty regarding the behavior of the learned model still remains [112]. In addition to that, the baseline model remains fixed which is not suitable to handle non-stationary dynamics.

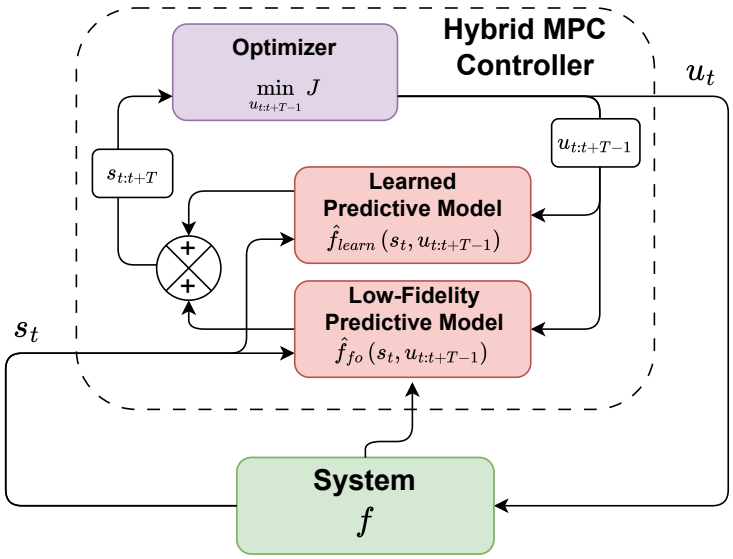


Figure 2.6: Hybrid-MPC Architecture. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by the function f. The state s_t is used by both the *Learned Predictive Model* and the *Low-Fidelity Predictive Model* to predict. The role of the *Learned Predictive Model* (f_{learn}) is to correct the approximation errors of the *Low-Fidelity Predictive Model* (f_{fo}) which is usually a first-order model deduced from the real dynamics of the *System*. Their predictions are combined to obtain a trajectories $s_{t:t+T}$ over a receding horizon of length T. The *Optimizer* exploits the predictions to find the optimal sequence of actions $u_{t:t+T-1}$. Then, only the first action u_t is applied on the *System*.

To open up the field toward more complex control tasks, model-free Reinforcement Learning (RL) has been extensively studied. By interacting directly with the environment, the RL agent learns a control policy π end-to-end by looking to maximize a cumulative scalar reward signal (analogous to minimizing a cost function) without any prior knowledge of the system's dynamics [217] (cf. Figure 2.7).

The success of model-free RL has been demonstrated across diverse and highly complex control tasks on which traditional modeling approaches struggle. Algorithms such as Deep Q-Networks (DQN) [165] and Proximal Policy Optimization (PPO) [200] have shown that controllers learned with RL could master superhuman strategies in discrete and continuous spaces. In the field of robotics, model-free RL has enabled end-to-end control using only pixels as input [165]. In these tasks learned controllers learn to perform agile maneuvers (e.g quadrupeds recovering from falls [149] or high-speed drone racing [135]) or tasks where the aerodynamic or contact forces are too nonlinear to be captured by first-principles predictive models in real-time [103, 117].

Despite these remarkable feats in complex robotics tasks, the application of model-free RL to safety-critical tasks necessitating HRC remains limited due to challenges in ensuring safety, interpretability and reliability during real-world interactions [40]. As highlighted in

[203], these approaches often lack quantitative safety guarantees. In RL, the underlying class of models used are mostly neural networks. Thus, the resulting opaque black box models rely on **reward shaping** (which is analogous to adding a **soft penalty term** to the cost function), to penalize dangerous states, only providing **soft incentives** toward safety rather than hard mathematical boundaries offered by MPC (when using the right solver for optimization). This makes real-world deployment of RL still rare in unstructured safety-critical environments [203]. Furthermore, the trial-and-error learning procedure of model-free RL results in extreme sample inefficiency. It necessitates high-fidelity simulators that often fail to bridge the gap between simulation and reality. This is especially true for tasks that involves humans that can hardly be simulated. Finally, most deep RL implementations results in **frozen policies** that cannot adapt to non-stationarity.

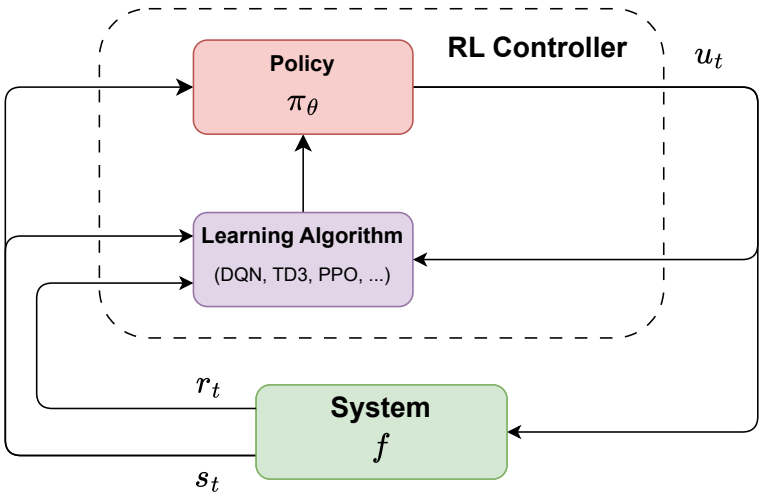


Figure 2.7: Model-free Reinforcement Learning (RL) Control Architecture. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by the function f. The state s_t of the *System* is transmitted to the *Policy* π_θ of the *RL Controller* (usually a Neural Network). The *Policy* outputs the action u_t from a state s_t. It is learned through a RL algorithm (e.g. DQN, TD3, PPO) from the encountered history of state transitions.

In response to the sample inefficiency of model-free RL, Model-Based Reinforcement Learning (MBRL) offers an approach more closely aligned with classical control theory by explicitly learning the system dynamics. Early algorithms such as Probabilistic Inference for Learning Control (PILCO) [60] uses probabilistic models like GPs to account for model uncertainty, significantly reducing the amount of data required for obtaining efficient policies. This was further improved by methods like Probabilistic Ensembles with Trajectory Sampling (PETS) [50], which employed ensembles of neural networks to capture uncertainty, paired with Cross-Entropy Method (CEM) [189] for planning over the learned dynamics.

Involving learned models in control tasks with specific safety requirements, implies taking uncertainty into account to provide rigorous probabilistic guarantees of success in constraint handling [40]. Two types of uncertainty can be distinguished: **aleatoric uncertainty** and

epistemic uncertainty.

Definition 4. Aleatoric uncertainty arises from the inherent natural variations in the studied phenomena. It may be due to random fluctuations, measurement errors, or other unpredictable factors [51, 115].

Definition 5. Epistemic uncertainty stems from a lack of knowledge or complete understanding of the studied phenomenon. This type of uncertainty is closely related to the lack of training data [51, 115].

More recently, research at the intersection of reinforcement learning and control has entered the era of **Latent World Models**, where algorithms such as DreamerV3 [102] and MuZero [198] learn to predict future rewards and dynamics within a compressed **latent space**. By planning entirely inside a learned representation of the environment, these approaches have achieved remarkable performance in tasks with complex high-dimensional observations.

However, despite their predictive power, world models remain complex to implement and necessitate vast amounts of data for their initial training phase. Furthermore, these approaches still lead to frozen policies lacking the lightweight recursive update mechanisms required to adapt to non-stationary dynamics in real-time [203].

Beyond the architectural complexity, learning-based control introduces challenges regarding data availability and reliability. The most prominent one is the **Exploration-Exploitation** trade-off. In a learning paradigm like RL where the controller learns from continuous interaction with the environment, the learning process itself is non-stationary because the policy is regularly updated to improve from its past self. In this non-stationary setting, the controller must occasionally take exploratory actions to gather new data to avoid collapsing into local optimum. In RL literature this is often addressed through intrinsic motivation mechanisms, such as Curiosity-driven exploration [179] or Random Network Distillation (RND) [41], which provide an exploration **bonus added to the reward** for visiting unfamiliar states.

Another closely related challenge is the risk of Out-of-Distribution (OoD) predictions.

Definition 6. Out-of-Distribution (OoD) refers to data points in machine learning that belong to a different probability distribution from the one used to train the model, often called the In-Distribution (ID) data [153].

Definition 7. As opposed to Out-of-Distribution (OoD), In-Distribution (ID) refers to data points in machine learning that belong to the same probability distribution as the one used to train the model [153].

When a learning-based controller encounters a state-action pair that is significantly different from its training data, its predictions can become erratic or overly optimistic, leading the optimizer to select unsafe trajectories. This specific issue is extensively studied in the field of Offline RL, where algorithms aim to learn conservative policies that would stay within the support of the available dataset, thereby avoiding the risks associated with unseen state-action pairs [150]. This highlights the importance of epistemic uncertainty estimation in learned model-based control, providing a means of evaluating the relevance of predictions depending on the input.

The limitations of both physics-informed learning-based control and pure RL necessitate a shift toward MPC with a **continuously learning model** (cf. Figure 2.8). This way, the architectural benefits of MPC regarding safety and constraints handling are preserved. Moreover, replacing the static internal model with an online learning model allows to handle non-stationarity and to be able to adapt to changes in real-time. Unlike the frozen policies obtained with MBRL, model-free RL or the rigid baselines of Hybrid MPC, this approach allows the controller to adjust its predictions in response to a live data stream, enabling it to proactively account for the non-stationary dynamics.

However, implementing such a system with strict safety requirements introduces significant challenges regarding the choice of the learning architecture. Specifically, the model must be capable of fast, stable, incremental updates.

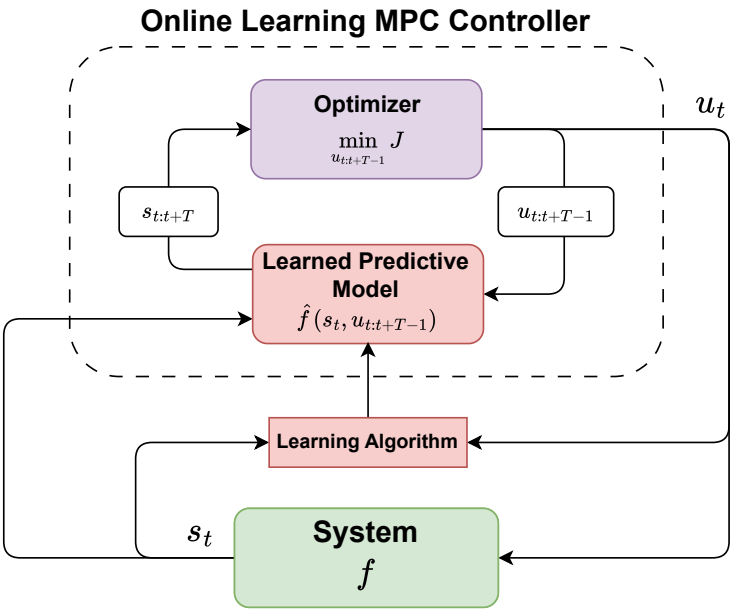


Figure 2.8: MPC with Online Learning Predictive Model. The *System* block corresponds to the targeted dynamical system to be controlled. Its transition dynamics are described by the function f. The state s_t of the *System* is transmitted to the *Online Learning MPC Controller*. There, the state s_t is used by the *Learned Predictive Model* to predict trajectories over a receding horizon of length T. the *Optimizer* exploits the predicted trajectories to find the optimal sequence of actions $u_{t:t+T-1}$. Then, only the first action u_t of the sequence is applied to the *System*. The *Learned Predictive Model* is updated at each iteration through a *Learning Algorithm* using the realized transition composed of the state s_t, the action u_t and the next state s_{t+1}. The next state s_{t+1} is the target to be predicted from the inputs s_t and u_t.

2.1.4 Synthesis and Research Gaps about Control

The previous sections traced the evolution of control architectures from **reactive feedback laws** to the emergence of **predictive optimization**. The transition toward **learning-based control** has been further studied, highlighting how the integration of predictive models learned from data addresses some of the limitations of static, first-principles models in complex and non-stationary environments. However, this transition also reveals a persistent tension between mathematical rigor and adaptive capability. This section synthesizes the reviewed literature to identify the specific research gaps.

One of the main challenges in Human-Robot Collaboration (HRC) is that the dynamics (f) of the system to be controlled often become non-stationary or partially unknown due to the non-stationary nature of human behavior. This leads to model mismatch, where the predictive model no longer aligns with reality. As introduced in Section 2.1.1, the 3 core properties of control, controllability, observability and stabilizability take on a specific meaning regarding the positioning of the works of this thesis:

- **Controllability:** In HRC, controllability is often **shared**. For instance, in the case of speed recommendation as defined in Chapter 1, the human driver acts as a biological intermediary between the controller (i.e the speed recommendation system) and the vehicle. Depending on the driver and his receptivity to recommendation setpoints, the system may become **under-actuated** (fewer independent actuators than degrees of freedom) or **locally uncontrollable** as the controller itself cannot enforce the required acceleration. We argue that to maintain control authority, the controller must be able to anticipate non-receptivity rather than reacting to it.

- **Observability:** In HRC, the human intent is not directly measurable. In this case, HRC systems often face **partial observability** and can necessitate the use of sophisticated state estimators. The control architecture must be robust to incomplete state information, favoring models that can explicitly handle uncertainty.

- **Stabilizability:** In safety-critical contexts, the frozen policies of model-free RL often lack the rigorous mathematical guarantees required for real-world deployment. This thesis argues for the use of MPC because of its structural capacity to explicitly enforce **hard safety constraints**. By leveraging **online replanning** and **specialized solvers** for constrained optimization, MPC provides a systematic way to maintain the system within a safe operational region, even as human behavior introduces non-stationarity.

According to Ashby's Law of Requisite Variety, a controller in the context of HRC as described in Chapter 1, must possess an internal variety matching the uncertainty and non-stationarity introduced by the human operator. Because human behavior is too complex to be captured by first-principles models, learning-based approaches can represent a solution to build accurate predictive models from data [113]. Static frozen models, with a fixed number of parameters, lack this variety.

In learning-based control, a significant research gap can be identified. Most learning-based controllers result in frozen policies after training. While MBRL and Latent World Models offer high variety, they still lack the **lightweight recursive update mechanisms** required to handle the non-stationarity induced by human behavior in real-time.

To satisfy the Conant-Ashby Theorem, the predictive model ($\hat{f}$) must not only exist but must also adapt its own complexity continuously. This motivates the use of **non-parametric methods**.

Definition 8. Non-parametric models are models whose structure and number of parameters are not fixed in advance but are instead determined by the data. Unlike parametric models, their complexity can grow with the size and diversity of the training dataset, allowing them to capture increasingly complex or non-stationary patterns [233].

By leveraging a non-parametric model, the representation capacity scales alongside the data stream. This leads to an adaptive growth of variety in a way that static or frozen models cannot.

To navigate the trade-offs between anticipation capabilities, safety and adaptability, Table 2.1 provides a comparative synthesis of the control frameworks dicussed in this section. Each identified family of controller is evaluated against five criteria:

- **Anticipation**: Distinguishes whether the controller proactively plans for future states (**predictive**) or merely reacts to current errors (**reactive**).
- **Constraint Enforcement**: Distinguishes between **hard** and **soft** constraint enforcement. Guaranteeing **hard** constraint satisfaction constitutes the primary objective of controllers designed for safety-critical applications.
- **Model**: Identifies the nature of the internal transition function ($\hat{f}$) of the predictive model, contrasting analytical first-principles models with data-driven learning architectures.
- **Non-Stationarity Handling**: Indicates the capacity to adjust to changes in dynamics in real-time.
- **Adaptability**: Relates to Ashby's Law of Requisite Variety, describing whether the model's complexity remains fixed (parametric/frozen) or can increase to accommodate the complexity of the task (non-parametric/evolving).

The families of controller presented in Table 2.1 are categorized as follows:

- **Reactive Feedback Laws (e.g. PID)**: Error-driven and myopic systems that lack a predictive model, relying instead on immediate feedback to control a dynamical system.
- **Classical Optimal Control (e.g. LQR, H_∞)**: Predictive methods that minimize a cost function but typically lack mechanisms to enforce hard constraints.
- **Model Predictive Control (MPC)**: An optimization-based control framework that explicitly forecasts future states to compute control actions, enabling safety through hard constraint enforcement when supported by suitable solvers.
- **Model-Free Reinforcement Learning (e.g. DQN, PPO)**: Data-driven controllers that learn complex behaviors through reward maximization and repeated interaction with the environment, typically relying on soft constraints and producing fixed, non-adaptive policies after training.

- **Model-Based Reinforcement Learning (e.g. PILCO, PETS)**: Unlike model-free RL, model-based methods explicitly learn the dynamics to plan or optimize control policies, but often rely on frozen models that limit adaptation to non-stationarity.
- **Online Learning MPC**: This thesis proposes a hybrid framework that combines the structural advantages of MPC with an online, preferably non-parametric, learning model to adapt to evolving system dynamics.

Family	Anticipation	Constraints Enforcement	Model	Non-Stationarity Handling	Adaptability
Reactive Feedback Laws	∅	∅	∅	∅	Fixed
Classical Optimal Control	✓	∅	First-Principles (analytical)	∅	Fixed
MPC	✓	Hard (optimization-based)	First-Principles (or hybrid)	∅	Fixed
Model-Free RL	✓	Soft (reward penalty)	Learning-based	∅	Fixed (frozen weights)
Model-Based RL	✓	Soft	Learning-based	∅	Fixed (frozen weights)
Online Learning MPC	✓	**Hard (optimization-based)**	**Learning-based (online learning)**	✓	**Evolving (non-parametric)**

Table 2.1: Comparison of the families of control architectures identified in the literature review. *Anticipation* evaluates whether the controller proactively plans for future states (predictive ✓) or merely reacts to current errors (reactive ∅). *Constraints Enforcement* distinguishes between **hard** and **soft** constraint enforcement. *Model* identifies the nature of the predictive model, contrasting between analytical models derived from first-principles and data-driver learning-based models. *Non-Stationarity Handling* indicates the capacity to adjust to changes in dynamics. *Adaptability* relates to Ashby's Law specifying whether the model's complexity is fixed or can expand to match the task complexity.

We established Online Learning MPC (cf Figure 2.8) as a promising architectural choice to address the challenges of HRC as it combines structural safety considerations with recursive online model updates. However, the theoretical feasibility of this approach depends entirely on the nature of the predictive model $\hat{f}$. While the MPC framework allows for the use of either parametric or non-parametric models, the requirement for requisite variety in a non-stationary HRC environment favors models that can adapt their own complexity.

Having answered the question of which control architecture is the most promising, the question of which model could fulfill all the criteria has to be addressed. Consequently, Section 2.2 explores the research landscape about Online Machine Learning to identify the modeling methods best suited for providing structural compatibility, uncertainty-awareness, fast recursive updates and accurate predictions required in this hybrid control framework.

2.2 Online Machine Learning for Predictive Modeling

The ability to continuously build accurate and uncertainty-aware representations of non-stationary dynamics is a requirement for effective adaptive control in a Human-Robot Collaboration (HRC) setting. As shown in Section 2.1, the transition from standardized industrial contexts to the complexity of HRC interactions necessitates a Model Predictive Control (MPC) architecture that can plan under uncertainty with a learned model. However, the performance, adaptability and safety of such a framework are contingent upon the properties of its internal predictive model $\hat{f}$.

Traditional machine learning techniques are constrained by the assumption of stationarity. Thus, they usually are not tailored to model non-stationary dynamics. In the context of this thesis, this section aims at answering the following question:

How can a learning architecture continuously model nonlinear, non-stationary dynamics from an online stream of data without losing previously acquired knowledge?

First, Section 2.2.1 introduces Machine Learning to set up the formalism that will be used throughout the course of this thesis. Then, Section 2.2.2 distinguishes between Batch Learning and Online Learning, highlighting the transition from static datasets to continuous streams. Section 2.2.3 exposes the limitations of global models, specifically focusing on the inherent trade-off between model adaptability and knowledge retention as an architectural bottleneck for global approximators like deep neural networks. In response, Section 2.2.4 provides an overview of local learning methods which naturally mitigate this trade-off through spatial isolation. Section 2.2.5 introduces the use of multi-agent systems for online learning where the learning procedure is defined by a set of local social rules. These social rules allow self-organization of a set of agents and the global emergence of learning. Finally Section 2.2.6 provides a synthesis of the identified research gaps.

2.2.1 Machine Learning Formalism

Before delving into the reasoning towards online learning and the use of local multi-agent systems, it is necessary to first establish the formalism adopted in this thesis regarding machine learning. Here, machine learning is used to **approximate the unknown transition function f from an available stream of observations** [107].

To achieve this, the approximation problem if framed within the paradigm of **supervised learning**. At its core, supervised learning consists of learning a predictive function $\hat{f} : \mathcal{X} \mapsto \mathcal{Y}$ that relates an input space $\mathcal{X}$ and an output space $\mathcal{Y}$ based on a set of N labeled examples $\mathcal{D} = \{(x_i, y_i)\}_{i=0}^{N}$[107].

Supervised learning can represent two types of tasks: *regression* tasks where the output space $\mathcal{Y}$ is continuous (e.g. predicting house price or temperatures) and *classification* tasks where the output space $\mathcal{Y}$ is discrete or categorical (e.g. labeling images as cat/dog) [107].

In this thesis, the focus is put on solving regression tasks because the goal, as specified in Section 2.1, is to approximate the unknown transition function f of a dynamical system such as

$$s_{t+1} = f\left(s_t, u_t\right)$$

where $s_t \in \mathcal{S}$ the state of system ($\mathcal{S}$ being a continuous state space) and $u_t \in \mathcal{U}$ the actions of the controller ($\mathcal{U}$ being a continuous action space). The objective is to minimize an empirical risk, defined by a loss function L (such as Mean Squared Error) to ensure that the approximation remains as close as possible to the true observation.

The nature and complexity of the mapping function $\hat{f}$ is defined by its underlying structure which can be categorized into two main families:

- **Parametric Approaches**: These models are defined by a fixed set of parameters θ (e.g. weights in a neural network or coefficients of a linear regression). The complexity of the model is determined beforehand by the number of parameters and remains constant regardless the amount of data collected. These models are computationally stable, meaning that they have predictable computational complexity during learning [5]. However, they suffer from a limited capacity which can lead to underfitting if the true dynamics are more complex than the model's fixed structure [5]. This can happen when the system is subjected to strict hardware constraints.

- **Non-Parametric Approches**: Unlike parametric models, the complexity of non-parametric approaches (e.g. Gaussian Processes (GPs) [238] or k-Nearest Neighbors [213, 246]) is not fixed. Instead, the number of parameters and the model structure can grow and adapt in response to the data [233]. This provides structural adaptability at the cost of less predictable computational cost at learning time [106].

To address the limitations of individual learning models, **Ensemble Learning** leverages multiple models to achieve more accurate predictions and provides better approximations of nonlinear functions. It combines a set of models such that their **errors compensate for each other**, yielding superior predictive performance [63]. During training, multiple models (which may differ from one another) are trained in parallel or sequentially [182]. Promoting diversity among them ensures they do not all capture the same patterns in the data [63]. For a given input, each model produces a prediction. These predictions are then aggregated through a heuristic that either selects or weights them to form the final prediction. The main Ensemble Learning approaches include: **Boosting**, **Bagging** and **Stacking** (c.f. Figure 2.9).

- **Bagging [37]**: Multiple models (usually homogeneous) are trained in parallel on different data subsets to introduce diversity, reduce variance, and improve stability. The final prediction is obtained by averaging (regression) or majority voting (classification).

- **Boosting [81]**: Models are trained sequentially, with each new model focusing on correcting the errors of the previous ones to reduce bias and improve accuracy. The final prediction is a weighted sum of the prediction of each model.

- **Stacking [240]**: A meta-learning approach in which multiple heterogeneous models are trained in parallel. A meta-model is subsequently trained to combine their predictions, leveraging the complementary strengths of the base models.

2.2.2 From Batch to Online Learning

The previous section presented the machine learning formalism that will be used throughout this thesis. This section shifts the focus towards the inner workings of the learning process. Specifically it distinguishes between **Batch Learning** and **Online Learning** paradigms.

In machine learning and deep learning , the Batch Learning paradigm is the most popular and widely used in the industry. In Batch Learning, the learning model has access to a complete dataset and learns from an entire set of examples simultaneously.

However, this paradigm is not sufficient to handle every industrial use case. For example, in deep learning, neural networks require extensive training data and several passes on data to converge [195], which is not compatible with use cases necessitating learning and adapting in real time from a stream of data that might be non-stationary.

The Online Learning paradigm aims to overcome this limitation. Unlike Batch Learning, in Online Learning, the learning model recursively updates the model using only one example at a time, possibly under strict memory and pass constraint (e.g only single pass allowed) [94].

This shift from Batch to Online Learning introduces several new challenges. First, the traditional sequential pipeline comprising data collection, preprocessing, training and testing is replaced by an interleaved architecture where these phases occur simultaneously without clear temporal order. This makes basic operations, such as feature normalization, non-trivial due to unknown global statistics. In this case the model needs to maintain incremental estimates that evolve alongside the data [94], thus introducing a second layer of non-stationarity into the learning process (because those incremental estimates change over time).

Furthermore, because the learning model cannot perform multiple passes over the dataset, it must balance **estimation error** (i.e. variance from seeing finite data subsets) against **approximation error** (i.e., model bias). A higher generalization error is often tolerated for enabling real-time updates [209]. However, this makes online learning models even **less reliable on Out-of-Distribution (OoD)** data due to the **short-term bias** toward recent experience and the lack of global regularization.

The most important distinction between Online and Batch Learning lies in the **assumption of stationarity**. On the one hand, Batch Learning methods assume that the data distribution is fixed and the dataset is exhaustive enough. On the other hand, Online Learning algorithms must be designed to handle non-stationarity that manifests through concept drift [86].

Definition 9. Concept drift in online machine learning refers to the gradual or sudden change in the relationship between input data and target outcomes over time, causing previously trained models to lose accuracy as real-world patterns evolve [86].

To continue being relevant, the learning algorithm should include a mechanism for adaptive learning, incrementally forgetting obsolete concepts while integrating new knowledge. This requirement often leads to a trade-off between model stability and prediction accuracy [87, 205]. A model that converges too firmly, without letting room for plasticity may fail to adapt to concept drifts.

As new architectures are developed to handle these challenges, **Ensemble Learning** seems to offer a way to dynamically add or remove learning capacity to recover from concept drift [94]. This moves adaptation from parameter space to model space.

However, ensembling does not inherently address the challenges of Online Learning. If the underlying models of the ensemble are global approximators, such as neural networks, their internal representations remain susceptible to interference because a single update can perturb the entire parameter space. Consequently, simply adding or removing models from the ensemble does not prevent new knowledge from accidentally corrupting previously learned information. This structural vulnerability leads to a conflict known as the **stability-plasticity dilemma** [163], which is explored in the following section 2.2.3.

2.2.3 Global Models and the Stability-Plasticity Dilemma

The transition from Batch to Online Learning introduces new challenges related to the internal representations of the models. These challenges are formalized as the **stability-plasticity dilemma**.

Definition 10. Global learning refers to machine learning approaches that construct a model capturing the overall data distribution or structure, where parameters are updated using the entire training dataset, and predictions arise from the unified model architecture [242].

Definition 11. The **stability-plasticity dilemma** [163, 231] is a fundamental challenge in both biological and artificial neural systems that involves finding the correct balance between learning new information and retaining old knowledge. It is defined by two competing requirements :

- **Plasticity**: The system must be flexible enough to integrate new knowledge and inputs from the environment.
- **Stability**: The system must be stable enough to prevent the *catastrophic forgetting* of previously encoded data.

In Batch Learning, where global learning models are widely used, the output is decided by the collective activation of all neurons within the system [242]. Because these parameters have global support, the adjustments to internal parameters required to learn a new task or adapt to a concept drift can lead to interferences between the new and old tasks [231]. Indeed, a single update step can potentially have an impact on the entirety of the model. This interference problem is more generally referred to as the **catastrophic forgetting** problem in literature [80, 47, 231].

Definition 12. In the context of machine learning, **catastrophic forgetting** refers to the abrupt and severe degradation of performance on previously learned tasks when a model is trained sequentially on new, potentially conflicting tasks. This usually arises from updates to parameters that are shared for several overlapping tasks that overwrite previous knowledge, disrupting long-term knowledge retention [80, 47, 231].

Simple global models such as linear regressions can be learned online and handle non-stationarity by incorporating discount factors in the learning procedure to prioritize recent observations [120]. These approaches are computationally efficient and can even be used for

solving reinforcement learning problems [214]. However, they are limited by the linearity assumption and fail to capture the complex nonlinear relationships between inputs and outputs in most dynamical systems. Whereas high-capacity approximators like neural networks can provide the necessary representational depth.

Even if neural networks can capture such complexities (nonlinear relationships), they are highly susceptible to **catastrophic forgetting**. Moreover, these models are typically static. This means that their internal complexity (number of layers and neurons) is fixed at design time and cannot adapt to the evolving complexity of the tackled problem. Three main strategies have been proposed to mitigate these effects in global learning models:

- **Regularization Methods**: These attempt to anchor important weights to preserve stability (e.g. Elastic Weight Consolidation) [139] but this often **hurts the plasticity capabilities** of the model. Thus, it prevents it to adapt to significant concept drifts.

- **Replay Methods**: These leverage episodic memory to re-train on past experiences to distillate old knowledge into the new one [47] to prevent forgetting. This trick is extensively used in Deep RL to stabilize an inherently non-stationary learning procedure [165, 197, 9]. However, it **violates the single-pass constraint** often associated with online learning as well as increasing the computational overhead.

- **Parameter Isolation Methods**: These create specialized experts models for different tasks [6, 61]. They tackle the catastrophic forgetting problem by isolating sets of parameters allowing dynamic creation and destruction of relevant experts. While promising, this meta-learning approach often struggles with a problem of **uncontrolled growth of experts** and the lack of a clear spatial logic to trigger the correct expert in continuous state spaces.

As neural networks are widely considered as hard-to-interpret black-box models [245], Tree-based learning models, such as Hoeffding Trees [64], Online Random Forests [192] or Mondrian Forests [145] offer an alternative. They provide an interpretable symbolic structure from data streams. As these approaches struggle with handling concept drift, adaptive variants introducing structural modifications (e.g adding concept drift detectors like ADWIN [28]) have been developed. Despite this, they are prone to structural instability caused by pruning strategies that can induce rough performance loss if not carefully designed [124]. Furthermore, from a control perspective traditional trees are non-differentiable due to hard logical splits. These splits produce a discontinuous output surface with zero gradients making them incompatible with gradient optimization methods. Attempts have been made to tackle this issue but they remain computationally inefficient [110].

To summarize on the limitations mentioned above, **global learning models are not able to cope efficiently with non-stationarity** due to global representations or structural difficulties to handle concept drift. This suggests that a **local learning** model could be more appropriate to handle the challenges inherent to Online Learning.

2.2.4 Local Online Learning

As shown in the previous section, global learning models are unable to cope efficiently with non-stationarity. This motivates the exploration of local learning methods because they

provide an **architectural solution to the stability-plasticity dilemma**.

Definition 13. Local learning refers to machine learning approaches that construct task-oriented models using subsets of the training data, such as local neighborhoods, where parameters are updated incrementally on relevant data points, and predictions airse from context-specific approximations rather than a unified global architecture [242].

In local learning models, the updates are spatially isolated and outputs are determined by specific local parameters [242]. By restricting the influence of a data point to a specific neighborhood, these models achieve a natural form of **Parameter Isolation**. Parameter Isolation enables adaptation to new dynamics in one region of the feature space does not disrupt the knowledge stored in another non-related one [6, 61].

Beyond Parameter Isolation, the adoption of local learning is supported by the requirement for interpretability. In safety-critical systems, interpretable white-box models should be preferred over black-box approximators to ensure trust and safety through transparent inner workings [190]. Local learning naturally provides traceability in the prediction process, provided that the local models are themselves interpretable. In this context, any prediction can be traced back to a specific and localized set of parameters rather than being the result of an opaque global activation.

The foundation of local learning lies in the k-Nearest Neighbors (kNN) algorithm [213, 246]. kNN makes predictions solely based on the most resembling points stored in memory. It can provide an intuitive mechanism to handle non-stationarity in an online learning setting through adaptive neighbor selection [250]. However, it is often limited by

- **Data density** which can lead to biased predictions for non-uniform data distributions.
- **Poor generalization** in sparse regions, where lack of local information degrades accuracy.
- **High computational overhead** induced by the search of nearest neighbors at prediction time. This makes it hard to use kNN in real-time control applications.

To provide smoother approximations, Locally Weighted Regression (LWR) extends the nearest neighbor search by fitting local models to the neighborhood of an input [16]. LWR has been leveraged successfully to learn forward and inverse dynamics models for control [17]. However it suffers from significant drawbacks: this approach is computationally inefficient when the database of stored points grows.

To overcome the inefficiencies of LWR, a set of incremental local expert models is maintained to replace raw data storage. For example Radial Basis Function (RBF) networks [168] are a common approach. The function underlying the data is approximated from the weighted combinations of localized RBFs. However, in RBF Networks, the centers of localized RBFs need to be fixed beforehand, and the initial location of these initial centers plays a crucial role in the model's performance. This is a major drawback for online learning as the training data distribution is not known in advance. Attempts have been conducted to alleviate this issue by designing methods in which neurons can be dynamically added or pruned to match the non-stationarity of a data stream [104, 122].

A notable reference approach in online learning for control is a scalable extension of LWR: Locally Weighted Projection Regression (LWPR). LWPR manages complex, nonlinear dynamics by maintaining a set of local linear models that are updated incrementally. Each linear model is associated with a radial basis function. The final prediction of the model is obtained by computing the average of the closest models weighted by the radial basis function value for the input.

More recent extensions have enabled this type of model to provide uncertainty quantification [162], bridging the gap between frequentist local models and probabilistic models like Gaussian Processes to improve safety guarantees when used in control contexts.

Despite these advancements, LWPR exhibits a structural rigidity arising from the tight coupling between its spatialization and update logics. Because every local unit is governed by the same predefined statistical update equations, the architecture lacks the modularity to:

- **Incorporate heterogeneous local knowledge** (e.g combining linear models with nonlinear approximators).
- **Adapt its internal coordination logic** to specific regions of the input space,matching the modeling complexity of a local expert to the local complexity of the data.

Furthermore, the management of the population of local units (creation and pruning) relies on fixed thresholds that do not account for the relational dynamics between overlapping experts.

Beyond structural rigidity, local learning architectures are inherently vulnerable to the **curse of dimensionality** and the risk of uncontrolled population growth. Without carefully designed mechanisms for expert model creation and pruning, the number of local models can expand exponentially in high-dimensional input spaces. This leads to computational saturation and overfitting.

Definition 14. The curse of dimensionality refers to phenomena where analyzing high-dimensional data becomes exponentially more difficult. As dimensions increase, distances between samples are less meaningful, data volume grows rapidly, and available samples become sparser, requiring exponentially more data for reliable analysis [23].

The structural common points between various local learning systems (e.g. kNN, LWR, RBF Networks, LWPR) is the use of local lower order models and radial basis functions. It suggest a converging architectural philosophy.

In this thesis, an alternative path is explored to mitigate the aforementioned limitations by shifting the focus from purely statistical optimization to multi-agent social self-organization. This approach adopts a bottom-up design philosophy, introducing a clear distinction in terms of learning procedure compared to approaches like LWPR. The system is described at the lowest level, by social rules governing the interactions between local experts. Rather than following a global objective function, local experts adapt, compete or cooperate locally, leading to the **emergence of learning**. This shift represents a transition from centralized function approximation to emergent collective behavior where the population size and expert behavior are regulated by local social dynamics rather than global heuristics.

The following section provides an overview of the literature landscape in the field of Multi-Agent Systems (MAS) for supervised learning.

2.2.5 multi-agent Local Learning

As shown in Section 2.2.4, local learning architectures exhibit several limitations, particularly regarding local experts population management and structural rigidity. These limitations motivate the exploration of an alternative research path, shifting away from statistical optimization towards the emergence of learning through Multi-Agent Systems (MAS).

MAS are popular due to their ability to solve complex problems by breaking them down into simpler subproblems that can be easily addressed by autonomous agents, whether interconnected or not [66]. By defining interaction rules at the individual level, the system can foster emergent global behaviors that are scalable and robust. The use of MAS has been proven effective in fields such as civil engineering [206] or electrical engineering, specifically with issues related to smart grids [188]. More recently, the application of MAS also tackled supervised learning problems, representing a distinct and original trajectory in machine learning research in which learning emerges from the organization of a dynamic set context agents [171]. Each context agent represents a local expert model, specialized in predicting for a given region of the input space.

This multi-agent modeling paradigm is rooted in the Self-Adaptive Context Learning (SACL) paradigm [34, 226]. The learning problem is redefined as the **construction, selection, and adaptation of local models** encapsulated within agents. The objective is then to identify which context agent best represents its neighborhood in the input space.

Early research works applied this paradigm to the control of complex bioprocesses [227] and industrial systems [33]. These works demonstrated that by encapsulating a local context, a control action and a prediction within a context agent, a system could achieve high adaptivity through **cooperative self-organization**.

Definition 15. Cooperative self-organization in multi-agent systems can be defined as the emergent process by which autonomous agents achieve global coordination and adaptive behavior through decentralized, local interactions and rule-based cooperation, without relying on centralized control [204].

However, these early implementations were primarily designed as myopic reactive controllers, without anticipation capabilities. Even these MAS are able to adapt to non-stationarity, they lack the predictive horizon required for the proactive planning necessary in safety-critical use cases.

Then, this approach has been generalized beyond simple regulation. It has been successfully transposed to classification tasks [225, 78]. Recent advancements have leveraged the introspection capabilities of agents to improve generalizability through active learning [57].

Moreover, the architectural modularity permitted by agents, theoretically allows to design heterogeneous systems where the modeling complexity of an individual agent could be tailored to the local complexity of the data [77].

In more recent years, reinforcement learning has been seriously considered to bypass the phase of designing rules that define agent behavior. Indeed, in Multi-Agent Reinforcement Learning (MARL), agent behavior is learning using reinforcement signals obtained by interaction with the environment [43].

Specifically, MARL extends the traditional trial-and-error paradigm of RL to a collective setting where multiple agents learn concurrently to solve a task within the environment. In

this framework, from the perspective of any individual agent, the other learning agents are considered as dynamic components of the environment itself. This interaction introduces non-stationarity, because the reward received for a specific action becomes dependent on the decisions of other agents. Consequently, the learning process becomes an iterative refinement of the joint behavioral policy of agents toward an optimal policy. It requires agents to adapt not only to the environment dynamics but also to the evolving behavior of their peers [43].

It is important to distinguish MARL from supervised learning with context agents. While both rely on a set of local agents, their learning mechanisms differ. In MARL, agent behavior is typically learned end-to-end to maximize a cumulative reward signal, often requiring millions of iterations to converge [43]. In contrast, the context agent approach used in this thesis **combines fixed social interaction rules with immediate, simplified reinforcement signals** (e.g. discrete good/bad feedbacks) rather than complex reward-shaping. This way, learning with context agent can achieve significantly faster convergence and requires fewer iterations to adapt to a data stream.

By shifting from global, static approximators to a bottom-up self-organization of autonomous local experts, the system should gain the structural adaptability required to learn in a non-stationary environment. This multi-agent perspective transforms the learning problem into a self-regulating social process, ensuring that the model's variety can evolve alongside the complexity of the task without suffering catastrophic interferences between old and new knowledge that are typical of global architectures.

2.2.6 Synthesis and Research Gaps about Learning

The transition from Batch Learning with global learning models to local, self-organizing Online Learning highlights a trade-off between generalization capabilities and adaptability to non-stationarity: the stability-plasticity dilemma. In this section, we synthesize the reviewed literature to identify the specific research gaps that motivate the works of this thesis.

In the context of this thesis, the following core challenges of Online Learning: catastrophic forgetting, non-stationarity and uncertainty take on a specific meaning:

- **Mitigation of Catastrophic Forgetting**: Traditional global approximators such as deep neural networks suffer from catastrophic forgetting because for each update, every parameter of the network can be impacted, potentially reshaping the entire knowledge base. They remain structurally vulnerable to forgetting, despite being powerful function approximators, very efficient in Batch Learning. The works of this thesis favor *spatial isolation of parameters through local learning* to alleviate this issue. Restricting updates to specific regions of the input space ensures that newly acquired knowledge does not corrupt representations that correspond to unrelated contexts.

- **Handling of Non-Stationarity**: Human behavior in HRC is inherently non-stationary. Parametric models are limited by a capacity ceiling, i.e. a fixed number of parameters. In this thesis, we argue for the use of *Non-Parametric methods* because they allow the model's complexity to grow and refine alongside the data stream. By treating the model as an evolving population of agents rather than a frozen set of weights, the Ashby's Law of Requisite Variety can be satisfied.

– **Estimation of Epistemic Uncertainty**: To overcome the problem of missing data, safety-critical use cases in HRC require the controller to know when its internal predictive model is unreliable. Unlike global black-box models that may confidently extrapolate in Out-of-Distribution (OoD) regions, learning with local experts provides a natural mechanism for epistemic uncertainty quantification. The spatialization of expert models allows the system to detect knowledge gaps in the input space. For example in multi-agent local learning approaches based on a set of expert agents, if no expert is close for a given input, the system can signal a high epistemic uncertainty (no training data → no expert → no reliable prediction).

Section 2.2 argued for the promising nature of Local Learning approaches for HRC. However, most existing reference approaches (such as LWPR) exhibit a structural rigidity where the spatialization and update logics are tightly coupled. This limits the modularity of such system and prevents the use of heterogeneous models that could allow for more parsimonious local representations. Local learning models are inherently vulnerable to the curse of dimensionality. As the input space dimensionality increases, the volume to be covered grows exponentially. This leads to uncontrolled population overgrowth, computational saturation and overfitting.

To address these limitations, this thesis works explore local learning through Multi-Agent Systems (MAS). This approach does not bypass the curse of dimensionality, but rather introduces a bottom-up agentic design philosophy to the learning process. The spatialization logic is decoupled from the modeling units, allowing for a modular and heterogeneous architecture. In this paradigm, population management is governed by decentralized social rules (cooperative or competitive) rather than fixed global heuristics. Learning with MAS can theoretically be positioned within the Bagging ensemble learning paradigm. Each autonomous agent corresponds to a local learning model specialized on a spatial bootstrap of the data. The system as a whole acts as a decentralized ensemble of local learning models.

However, a gap still remains in the differentiability of MAS approaches and their integration in traditional optimization frameworks. For instance, in [77], the agents are spatialized using orthotopes, leading to non differentiable representations. For an internal predictive model $\hat{f}$ to be used within an MPC framework, it must provide a smooth, ideally differentiable output surface for the optimizer.

Beyond differentiability, the transposition of MAS to continuous reliable control and regression tasks remains less mature than its application to classification or discrete control. Currently, the internal properties of these systems (i.e introspection capabilities) have yet to be fully exploited to improve continuous control with learned models. In addition to that, the curse of dimensionality have yet to be addressed properly for MAS in supervised learning.

Table 2.2 provides a comparative synthesis of the reviewed learning algorithm families. It assesses their ability to meet the requirements of HRC, focusing on:

– **Capacity/Variety**: The ability to adapt model complexity to match evolving data streams.

– **Catastrophic Forgetting Mitigation**: Resilience against interference between newly acquired knowledge and relevant older knowledge.

- **Epistemic Uncertainty Awareness**: The capability to quantify *what the model does not know*, which is crucial for safety-critical HRC with learning models.
- **Non-Stationarity Handling**: Robustness to shifting data distributions and changes in dynamics (concept drifts).
- **Transparency**: Transparency fo the model's inner workings to ensure prediction reliability and interpretability.

Having established the trade-offs in both control architecture (Section 2.1) and online modeling methods (Section 2.2), the final positioning of our works in the research landscape can be done. The following Section 2.3 contrasts the requirements of safety-critical HRC with the capabilities of the reviewed literature to explicitly define the research niche and technical contributions of this thesis.

Family	Capacity / Variety	Catastrophic Forgetting Mitigation	Epistemic Uncertainty Awareness	Non-Stationarity Handling	Transparency
Global Parametric (NN)	Fixed	Weight Regularization / Replay / Ensemble	∅	∅	Opaque
Global Non-Parametric (GP)	Evolving	Global Covariance	✓	∅	Semi-Transparent
Global Tree-based	Evolving	Spatial Isolation	∅	✓	Transparent
Local Non-Parametric Lazy (kNN, LWR)	Evolving	Memory Storage	✓	∼	Transparent
Local Non-Parametric (LWPR)	Evolving	Spatial Isolation	✓	✓	Semi-Transparent
Local MAS	Evolving	Parameter Isolation / Social Rules	✓	✓	Transparent

Table 2.2: Comparison of reviewed learning families for online learning in HRC. *Capacity/Variety* evaluates wheter the model's complexity is fixed or can evolve to match the complexity of the data stream. *Catastrophic Forgetting Mitigation* identifies the mechanism used to prevent interference between the integration of new knowledge and older relevant knowledge. *Epistemic Uncertainty Awareness* distinguishes between models capable of quantifying their own ignorance (✓) and those that cannot (∅). *Non-Stationarity Handling* indicates the robustness to shifting dynamics and concept drifts (✓), including models with limited adaptation (∼). *Transparency* denotes the transparency of the inner workings of the model, ranging from opaque black-box models to fully transparent white box.

2.3 Positioning

Sections 2.1 and 2.2 provided an overview of the literature for the two research topics of this thesis. It revealed that in Human-Robot Collaboration (HRC) there is a trade-off between the necessity for real-time adaptation to non-stationary human behavior and the requirement for reliability for safety-critical control.

As established in Section 2.1, **Model Predictive Contol (MPC)** provides a rigorous framework for handling hard constraints but its effectiveness is bounded by the fidelity of its internal predictive model. In the context of HRC, the non-stationary nature of human behavior often leads to model mismatch, where static first-principle models no longer match reality. This motivates the use of online learning models in MPC architectures.

Section 2.2 demonstrated that although global learning architectures like deep neural networks offer significant representational power, they remain structurally vulnerable to catastrophic forgetting and lack the recursive update mechanisms required for seamless online adaptation. This motivates the use of **local online learning** approaches to mitigate this issue by using spatial isolation of parameters.

The positioning of this thesis lies at the intersection of two research domains:

- **Machine Learning for Predictive Dynamics Modeling**
- **Control with Continuously Learning Predictive Models**.

Within this intersection, we identify a specific research niche in the use of **Multi-Agent Systems (MAS) for supervised online learning**. It shifts from centralized optimization to the decentralized self-organization of a population of local experts by decomposing complex, non-stationary dynamics into manageable local representations. This approach offers a distinct architectural alternative to classical local learning approaches that favors spatial isolation of knowledge representations and introspection capabilities emerging from the organization of the population of agents. This introspection becomes the bridge between the learning and control layers.

The core research question of this thesis is the following:

To what extent can social local learning based on Multi-Agent Systems serve as a foundation for safe control strategies by exploiting model introspection to guide exploration and support reliable decision-making ?

To help visualize the research gaps, Table 2.3 summarizes the positioning of our contributions. In terms of contributions, the remainder of this thesis is structured as following:

- **Chapter 3** introduces the Context Ensemble Local Learning (CELL) formalism and details the iterative development of two specific instantiations: oCELL, which spatializes the local experts with orthotopes and kCELL, which introduces more interactive social learning rules and smoother spatialization through the use of radial basis functions to produce more faithful data representations. These models provide the foundational learning capabilities and the introspection metrics used in following chapters.
- **Chapter 4** transitions toward solving control problems by integrating kCELL into a MPC pipeline. This chapter demonstrates how the model's introspection can be

leveraged to guide exploration and to protect the system during exploitation by avoiding regions of high epistemic uncertainty.

– **Chapter 5** explores prospective directions for the CELL formalism. Through the gpCELL algorithm, an instantiation designed to scale Gaussian Processes (GP) for Online Learning, a step is taken toward learning with a heterogeneous ensemble of local expert agents.

Research Axis	Research Gap / Challenge	Expected Benefit
Online Learning	Catastrophic forgetting and structural rigidity in non-stationary streams.	Enables online adaptation through localized updates; the MAS architecture inherently provides traceability and introspection.
Safe Control	Unreliable MPC planning due to model extrapolation in unknown regions (OoD).	Restricts the optimizer to well-modeled regions to maintain reliability in safety-critical states.
Exploration	Sample inefficiency and the need for high-quality data in unknown dynamics.	Leverages local agent feedback to proactively guide data collection toward high-uncertainty regions.
Scalability	Curse of dimensionality in local spatialization for high-dimensional inputs.	Enables local experts to operate in reduced-dimensionality nonlinear, mitigating the volume explosion.
Heterogeneous Representations	Limited complexity of simple local models for highly nonlinear dynamics.	Integrates non-parametric local models to capture complex dynamics while preserving the benefits of locality.

Table 2.3: Synthesis of research gaps and contributions

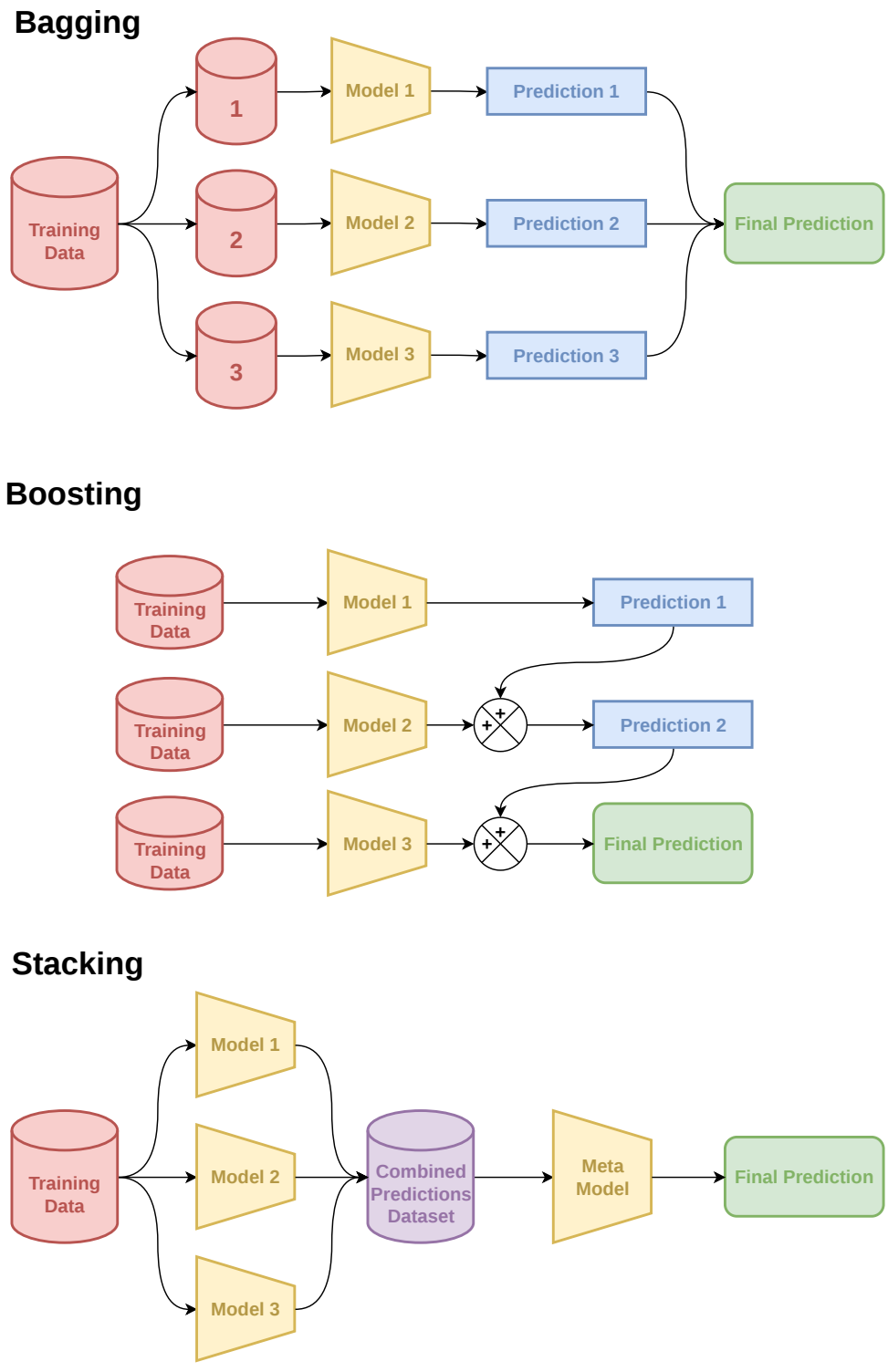


Figure 2.9: Visual comparison of Bagging, Boosting and Stacking

Chapter 3

Context Ensemble Local Learning (CELL)

The literature review presented in previous chapter highlighted a need for reliable learning architectures able to fit into a Model Predictive Control (MPC) framework and to adapt to complex non-stationary environments without forgetting previously acquired knowledge. Traditional approaches using fixed global models often fail to adapt when faced with the shifting behavior of human agents or changing environmental conditions. In this chapter, we explore learning approaches that leverage multi-agent systems, where local experts self-organize based on social feedback.

Building on the Self-Adaptive Context Learning (SACL) paradigm [34] (which is further introduced in Section 3.1), the Context Ensemble Local Learning (CELL) formalism is introduced in this chapter. SACL aims to solve control problems by mapping contexts to actions and their immediate utility. In contrast, CELL explicitly models the underlying system dynamics, i.e. the consequence of actions in a given context. With this approach, the self-organizing ensemble of agents serves as a predictive model that can be integrated into a MPC loop. This represents a shift from model-free control (SACL) to model-based control (CELL within MPC).

The CELL formalism provides a structured framework for designing learning algorithms where introspective properties emerge from the self-organization of agents. These introspective properties facilitate the analysis of the system. In this work, introspection can be defined as follows:

Definition 16. Introspection designates the use of emergent properties, such as the geometric structure and density of agents, to monitor the system's internal state. These properties arise from local self-organizing mechanisms and provide a macroscopic view of the learning process or the reliability of a prediction.

Within CELL, we followed an iterative process through two specific instantiations to refine the learning algorithms, transitioning from discrete to continuous spatial representations. The first instantiation is called Orthotope-based CELL (oCELL). In this instantiation, agents are spatialized using orthotopes (d-dimensional rectangles) in the input space. It allows for the study of prediction accuracy in supervised learning, while exploring the link between the

spatial organization of agents and the traditional machine learning notions of explainability and interpretability.

However, oCELL faces limitations due to the discrete spatialization of agents. Thus, to overcome these limitations, Kernel-based CELL (kCELL) is introduced. This second instantiation of CELL leverages radial basis functions to create smoother continuous representations of the learned dynamics.

Furthermore, kCELL introduces more interactive social mechanisms, enabling agents to adapt more directly to their neighbors. This evolution leads to more faithful data representation and enhanced adaptation capabilities that are demonstrated through experiments on both stationary and non-stationary learning tasks.

More precisely, the contributions of this chaper are threefold:

- We introduce the CELL formalism to frame the design of multi-agent systems based on local expert agents for supervised regression tasks within a control context.
- Through the oCELL instantiation of CELL, we demonstrate the capabilities of such a system to be competitive in terms of predictive accuracy compared to state-of-the-art ensemble learning algorithms on a 2D supervised regression task. We also show that socially emergent properties can be linked to the notions of explainability and interpretability in machine learning.
- Through the kCELL instantiation of CELL, we demonstrate that a system derived from CELL can be able to compete with state-of-the-art online learning algorithms in terms of predictive accuracy on a stationary learning task and to adapt its internal representations in response to changes in dynamics in a non-stationary environment.

The remainder of this chapter is organized to reflect the process that leads to a usable system within MPC context. First, Section 3.1 presents the CELL formalism by defining the core components and social mechanisms of a multi-agent system based on an ensemble of local expert agents. From this theoretical foundation, Section 3.2 introduces the oCELL instantiation of CELL. It provides a comparative experiment on a 2D regression task and introduces the definition of several introspective properties that can be linked to the notions of explainability and interpretability. Then, Section 3.3 describes kCELL, a more robust and expressive instantiation of CELL. It provides a comparative experiment on a stationary modeling task and demonstrates the non-stationary adaptation capabilities of the system on a non-stationary modeling task. Finally, Section 3.4 concludes by summarizing the works presented in this chapter.

3.1 CELL Formalism

The Context Ensemble Local Learning (CELL) formalism provides the theoretical foundations for designing multi-agent systems based on an ensemble of local expert agents governed by social rules that could satisfy the requirements for integration in Model Predictive Control (MPC).

Before detailing the CELL formalism, Section 3.1.1 provides an overview of the theoretical lineage of CELL by describing the Self Adaptive Context Learning (SACL) learning paradigm

and its application within MPC. Then, Section 3.1.2 formalizes the local expert agents and their inner workings and Section 3.1.3 details the inference and learning pipelines. Finally, Section 3.1.4 discusses on the theoretical properties that are brought about by CELL for use within MPC.

3.1.1 From SACL to CELL

The inspiration for the CELL formalism lies in the Self-Adaptive Context Learning (SACL) paradigm [34]. SACL was originally proposed to solve complex control problems by learning to map perceived states, referred to as contexts, to the most effective actions.

The SACL Paradigm In SACL, the controller is defined through two primary entities:

- **The Adaptation Mechanism**: Its goal is to build, maintain and provide reliable knowledge about the current context and potential actions. It correlates the activity of the controller to perceived states of the environment to identify contextual boundaries and model local dynamics.
- **The Exploitation Mechanism**: This entity is responsible for applying actions on the dynamical system. It uses the knowledge provided by the Adaptation Mechanism, alongside domain-specific constraints to decide which control action to apply.

The interactions between these two layers is governed by a continuous information loop and described in Figure 3.1. The Adaptation Mechanism perceives the environment and proposes possible actions, and the Exploitation Mechanism then chooses an action to perform. The latter then emits a feedback that is sent to the Adaptation Mechanism to refine the context-action mapping.

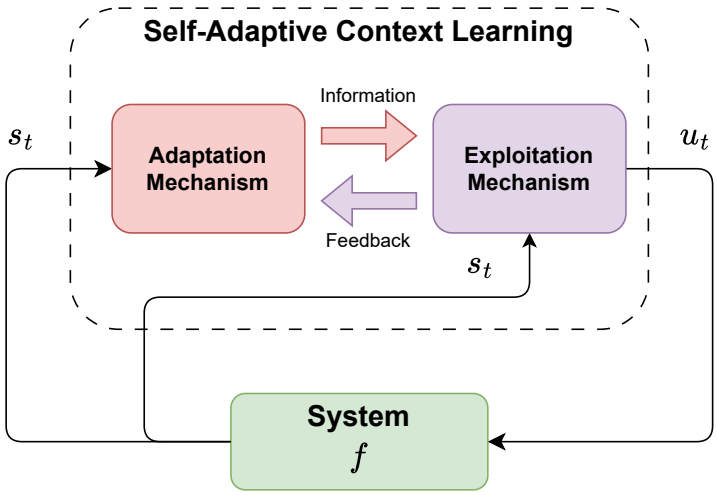


Figure 3.1: The Self-Adaptive Context Learning (SACL) Paradigm

The Adaptation Mechanism relies on a set of self-organizing *Context* agents. These agents are the core adaptive software entities that implement the social learning component of the system. Each *Context* agent is defined by a tripartite structure representing a specific tile of knowledge:

- **Context**: This is a set of validity ranges, one for each input feature. An agent is considered valid only when the perceived state lies within all corresponding ranges. These ranges represent a tile in the input space that allow mapping the current perceived state to a specific action and its effects on the system.
- **Action**: It corresponds to a control command to be applied to the dynamical system (e.g. torque setpoint or "go forward" command).
- **Appreciation**: It corresponds to a forecast of the action's effect on the controlled system. It is often expressed as a confidence or utility value.

Based on the feedback provided by the Exploitation Mechanism, the valid agents that proposed actions can assess the quality of their proposition, allowing them to improve. The SACL architecture enables dynamic cooperative self-organization through the detection and management of Non-Cooperative Situations (NCS). NCS are situations of functional inadequacy regarding agent cooperation, that may occur during the self-organization process when agents deviate from their nominal cooperative behavior due to perception failures or erroneous decisions. The adaptation mechanisms defined to resolve or anticipate these NCS constitute the learning rules governing the collective behavior of *Context* agents.

Despite several successful applications [33, 226], SACL presents two primary limitations:

- **Myopic Behavior**: SACL is mainly designed to operate from instantaneous feedback. This results in a myopic controller because agents are rewarded based on the immediate success of the proposed action. Thus, the controller struggles to anticipate consequences beyond a single time step.
- **Domain-Specific Knowledge**: SACL heavily relies on domain-specific knowledge to design the feedback functions. These functions are used to determine if an agent's Appreciation is erroneous or inexact. This limits the generalizability of the approach.

From Behavioral Control to the CELL Formalism Building on prior works that leveraged SACL principles to design machine learning algorithms rather than control strategies [170, 77], we introduce the CELL formalism.

CELL is a formalism that provides a framework for designing online learning algorithms based on a self-organizing ensemble of local expert agents governed by social rules. Rather than acting as a standalone behavioral controller, CELL focuses on building a predictive model of a dynamical system, intended for integration within a MPC architecture.

The transition from SACL to CELL can be summarized into three shifts in how the multi-agent system is used:

- **From Action Suggestion to Function Approximation**: In SACL, the objective of the multi-agent system is to suggest actions (*control*) and evaluate a utility-based

appreciation. In CELL, the objective of the multi-agent system is to predict the future states of the system (*machine learning*). The goal is to provide a faithful local approximation of the transition function $f : \mathcal{S} \times \mathcal{U} \mapsto \mathcal{S}$, such that

$$s_{t+1} = f(s_t, u_t)$$

where $s_{t+1} \in \mathcal{S}$ is the next state, $s_t \in \mathcal{S}$ is the current state, $u_t \in \mathcal{U}$ the applied action. By focusing solely on the machine learning task, CELL eliminates the need to design domain-specific feedback functions for each individual control problem.

- **From Myopia to Anticipation**: The self-organization of agents in SACL is primarily driven by instantaneous feedback, which limits the controller to a reactive, one-step-ahead horizon. In contrast, CELL operates as a predictive model of the system's future dynamics. When integrated within a MPC framework, it enables the controller to simulate and evaluate multiple future trajectories over a receding horizon. This makes CELL capable of anticipating longer-term outcomes compared to SACL myopic behavior.
- **From External Advice to Tight Coupling**: While SACL treats the multi-agent system as an external advisor providing action proposals, CELL is tightly coupled with the control layer within MPC, as shown in Figure 3.2. The MPC optimizer does not wait for agent propositions, instead, it actively queries the ensemble of local expert agents at high frequency to predict future states or to compute gradients necessary for trajectory optimization. This architecture combines the adaptive, self-organizing strengths of multi-agent systems with the mathematical robustness of optimization-based control.

The following sections detail the CELL formalism, beginning with the definition of its core component: the Local Expert Agents.

3.1.2 Local Expert Agents

The previous section established the relationship between CELL and the SACL paradigm, highlighting the transition from a behavioral control framework to a predictive dynamics model integrated within a MPC architecture. This section defines the notion of Local Expert Agent within CELL.

Each agent $\mathcal{A}_i$ acts as a local expert for a specific area within the input space. An agent is defined by a triplet such as

$$\mathcal{A}_i = \{\phi_i, f_i, \mathcal{M}_i\}$$

where $\phi_i : \mathbb{R}^d \mapsto [0, 1]$ is an activation function, $f_i : \mathbb{R}^d \mapsto \mathbb{R}^m$ is an internal local model and $\mathcal{M}_i$ represents the internal state of the agent.

The **activation function** ϕ_i defines the spatial boundaries in which the agent is considered an expert (i.e. relevant for making predictions). For any given input vector $x \in \mathbb{R}^d$, $\phi_i(x) \in [0, 1]$ returns an activation value. This value is used by a mechanism to select the most relevant agents to predict for x. This mechanism is called a spatial gating mechanism

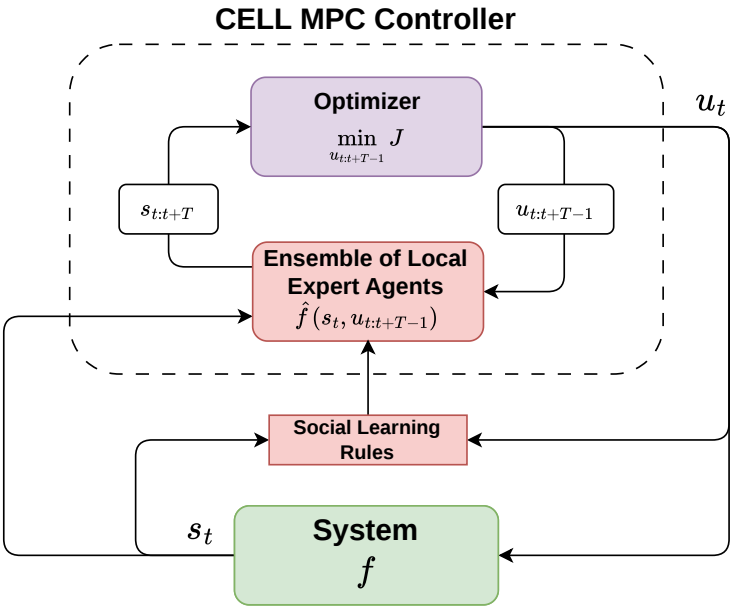


Figure 3.2: CELL predictive model integrated within a Model Predictive Control (MPC) architecture

and it will be fully detailed in Section 3.1.3. An activation value of $\phi_i(x) = 1$ implies that the agent is an expert to predict for x. It means that x is In-Distribution (ID) for agent $\mathcal{A}_i$. Conversely, an activation value of $\phi_i(x) = 0$ implies that x is outside the area of expertise of $\mathcal{A}_i$. It means that x is Out-of-Distribution (OoD). The geometry of this function can be anything and can be adapted to the problem encountered and to specific requirements related to the dimensionality of inputs or computation time (e.g. computing RBF kernels values in x is more computationally heavy than computing intersection between x and a set of rectangles). An example of rectangular activation function is given in Figure 3.3. Ultimately, the activation function answers to the following question:

When should $\mathcal{A}_i$ predict ?

The local **internal model** f_i contains the predictive expertise of the agent, which is relevant locally within the area of expertise defined by the activation function ϕ_i. Each agent $\mathcal{A}_i$ maintains its own set of parameters θ_i to map input values x to predictions values such as:

$$\hat{y}_i = f_i(x, \theta_i)$$

where $\hat{y}_i$ is the prediction of agent $\mathcal{A}_i$ for input x. This formalism is model-agnostic, meaning that virtually any class of learning model could be used as internal model for the agents. Ultimately, the internal model answers to the following question:

What should $\mathcal{A}_i$ predict?

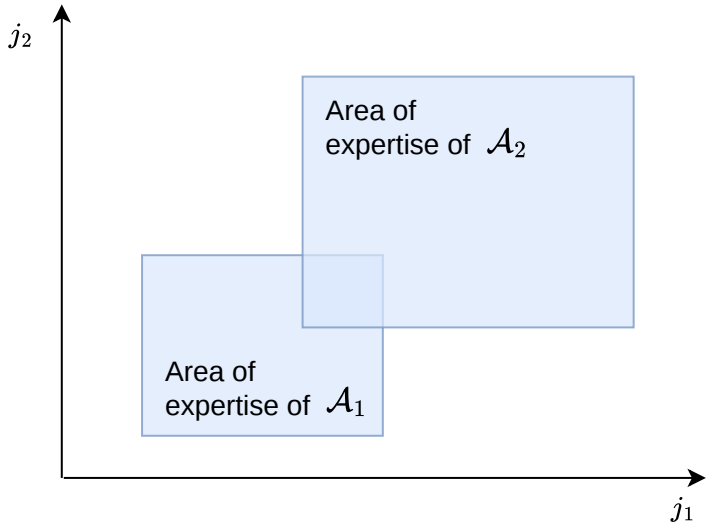


Figure 3.3: Representation of the areas of expertise of 2 local expert agents ($\mathcal{A}_1$ and$\mathcal{A}_2$) where the activation functions ϕ_i correspond to a rectangle. j_1 and j_2 represents the dimensions of the input space .

The **internal state** ($\mathcal{M}_i$) encapsulates metadata about the agent (e.g. local performance metrics, maturity, age, ...) and may also include a memory of the most recently encountered training points (in the form of x, y feature-target tuples).

By separating the spatialization layer (activation function ϕ_i) from the modeling layer (internal model f_i), each component can be independently optimized or replaced. It allows CELL to remain agnostic to both the choice of activation function and the class of internal model employed within local expert agents.

The global prediction of the model is a result of the collective output of the local expert agent that compose it. The state of convergence and learned model representation of the system can be directly gathered by inspecting the internal states $\mathcal{M}_i$ and the spatial distribution of agents across the input space through the parameters of the activation functions ϕ_i.

In addition to these fundamental components, agents are associated with actions that allow them to adapt individually and locally. Two types of actions must be defined in the context of CELL to enable local self-organization:

– **Shape Update**: This action corresponds to updating the parameters of the activation function ϕ_i, which effectively changes the shape of the area of expertise of agent$\mathcal{A}_i$. Depending on the feedback received for its prediction on an input x, the agent may realize either that his area of expertise is either well positioned within the input space or not. For example, if $\mathcal{A}_i$ receives a bad feedback for its prediction on x, it might shrink its area of expertise to not include x anymore. Conversely, if it receives a good feedback, it might expand its area of expertise to include x.

– **Model Update**: It corresponds to updating the parameters θ_i of the internal model f_i to include a point x. For example, it could consist in a gradient descent step.

The next section describes the inference and learning pipelines responsible for agent self-organization and global model prediction.

3.1.3 Operational Pipelines: Inference and Learning

After having formalized the main components that compose an agent in previous section, this section presents **the training loop of a CELL system** and how **the global predictions** are obtained from the set of local expert agents.

A CELL system operates through 2 pipelines: the **inference pipeline**, which produces a global prediction from the ensemble of agents, and the **learning pipeline**, which governs the social self-organization of agents. Both pipelines rely on a **spatial gating mechanism** to manage the locality in the system.

For a given input x, the spatial gating mechanism selects a subset of local expert agents from the ensemble that are the most relevant for prediction. This spatial gating mechanism guarantees **spatial isolation of parameters** inside the learning system. For any input, the spatial gating mechanism outputs two subsets of agents:

– **Activated Agents** ($\mathcal{D}_{\text{activated}}$): Agents for which x falls within their area of expertise (i.e x is ID for activated agents and OoD for non activated agents). These agents are the most relevant agents to contribute to the final prediction.

– **Neighbor Agents** ($\mathcal{D}_{\text{neighbors}}$): The set of neighbor agents is a superset of activated agents $\mathcal{D}_{\text{activated}}$. In addition to including activated agents, it includes agents for which x is not yet ID but is close to their spatial boundaries. The notion of closeness can be defined as a distance threshold or a minimum activation value $\phi_i < \epsilon$. These agents are candidate for updating the shape of their area of expertise to include x and become expert on it.

To obtain the global prediction of the ensemble of local expert agents, the **inference pipeline** aims to select the most relevant subset of agents from an input x, and to aggregate their local predictions. The inference process, illustrated in Figure 3.4, follows four distinct steps:

1. **Distance Evaluation**: The degree of expertise on x of each individual agent is computed using their activation function ϕ_i

2. **Spatial Gating (Expert Selection)**: The spatial gating mechanism filters the ensemble of selected agents to identify the most relevant agents to predict for x, and outputs two sets of agents: the activated agents $\mathcal{D}_{\text{activated}}$ and the neighbor agents $\mathcal{D}_{\text{neighbors}}$. For example, the activated and neighbor agents sets can be filtered based on a threshold on the activation value such as $\phi_i(x) \geq \tau_{\text{activation}}$ or $\phi_i(x) \geq \tau_{\text{neighbor}}$(where $\tau_{\text{activation}} \in \mathbb{R}$ and $\tau_{\text{neighbor}} \in \mathbb{R}$ are two real scalar thresholds).

3. **Local Prediction**: Each selected agent (neighbor or activated depending on the prediction policy defined by the designer of the system) produces a prediction proposal based on its internal model: $\hat{y}_i = f_i(x, \theta_i)$.

4. **Aggregation**: The individual prediction of the most relevant experts are combined using an aggregation function (e.g. weighted average).

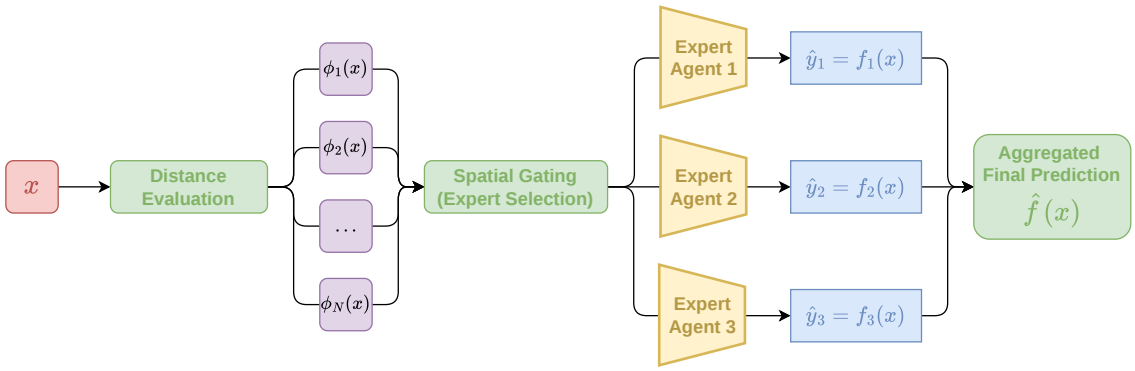


Figure 3.4: Flowchart Diagram of the inference pipeline in a CELL system

The **learning pipeline** governs the evolution of the population of agents through social rules. Unlike traditional gradient descent that updates all parameters globally, the spatial gating mechanism in CELL is leveraged to ensure parameter isolation by exposing only relevant agents to new data. For a given feature-target tuple (x, y), a typical learning step follows five stages as illustrated in Figure 3.5:

1. **Distance Evaluation**: The activation value $\phi_i(x)$ of each agent in the ensemble is evaluated to determine its expertise on input x.

2. **Expert Selection (Gating)**: The spatial gating mechanism identifies the sets of activated agents $\mathcal{D}_{\text{activated}}$ and neighbor agents $\mathcal{D}_{\text{neighbors}}$. This spatial selection ensures that only agents with relevant expertise (or potential for expertise) are candidates for being updated.

3. **Local Prediction**: Each selected agent computes its local prediction $\hat{y}_i = f_i(x, \theta_i)$.

4. **Social Feedback Computation**: Each selected agent receives a feedback that determines its subsequent updates. This signal can encapsulate two components:

   - **Absolute Feedback**: Based on the individual local error, it assesses if the agent's internal model is currently a faithful representation of the local dynamics.
   - **Relative/Social Feedback**: The agent's performance is compared to that of other close agents or to a global baseline (e.g. a running linear regression). This allows social dynamics such as *competition* (where only the best expert is allowed to stay activated for a region) or *cooperation* (where multiple agents refine their boundaries to collectively cover a complex transition).

5. **Application of Learning Rules**: Based on the feedback received and on the agent's internal state $\mathcal{M}_i$, specific social rules are triggered. These rules translates the social feedback into concrete structural or parametric changes.

The **learning rules** within CELL are the primary drivers of self-organization. Each is applied only to a specific set of agents, typically the activated agents $\mathcal{D}_{\text{activated}}$, the neighbors agents $\mathcal{D}_{\text{neighbors}}$ or the k-closest agents. They are triggered by conditions related to the feedback values or the agent's internal memory (e.g. maturity or historical performance). When, triggered, a learning rule performs an action. These actions can be categorized into two types: *Individual Actions* and *Meta-Actions*.

On the one hand, *Individual Actions* are targeted specifically at an agent $\mathcal{A}_i$ to modify an element of its tripartite structure. As specified in Section 3.1.2, we remind that it includes the **Model Update** which adjusts the internal parameters θ_i, and the **Shape Update** which modifies the parameters of the activation function ϕ_i to expand or shrink the area of expertise. On the other hand, *Meta-Actions* are actions that manage the population and affect the composition of the ensemble of agents. **Agent Creation** allows the system to remain non-parametric by spawning new experts in under-represented regions of the input space, while **Agent Destruction** ensures parsimony and prevents uncontrolled population growth by removing redundant or consistently inaccurate agents.

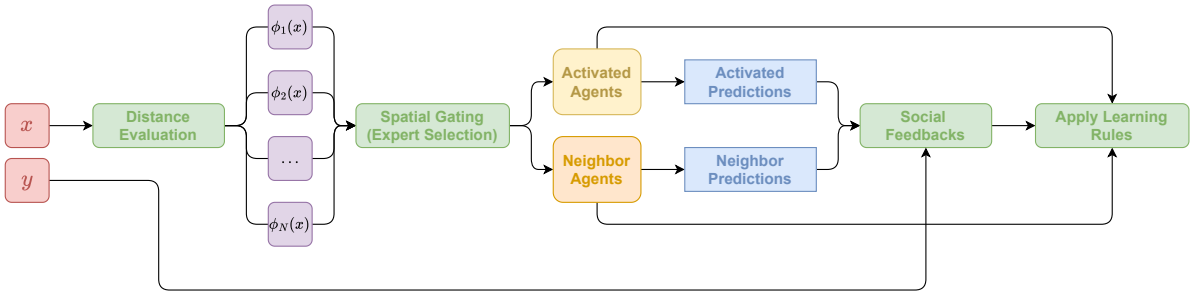


Figure 3.5: Flowchart Diagram of a learning step pipeline in a CELL system

To summarize, the **learning process** of a specific CELL instantiation (such as oCELL or kCELL that will be described later in this chapter) can be fully characterized by a rule table. This table acts as the behavioral blueprint for the ensemble of agents, mapping the current state of the population and the received feedback to a set of discrete actions. By using this structured approach, the designer can precisely define the social dynamics, such as competition for expertise or cooperative space-filling, without relying on global loss functions.

This rule table is defined by four primary columns:

- **Social Context**: The situational state of the ensemble relative to the current input. For example this context can be defined by the number of activated agents ($N_{\text{activated}}$) or neighbor agents ($N_{\text{neighbors}}$).
- **Condition (Trigger)**: The specific feedback value or internal state threshold (e.g., error levels) that activates the rule.
- **Targeted Set**: The set of agents subject to the rule (e.g. $\mathcal{D}_{\text{activated}}$ and $\mathcal{D}_{\text{neighbors}}$).
- **Action**: It corresponds to the action to be performed by the agents in the targeted set or to the meta-action to apply.

The use of a Social Context ensures that the learning process is aware of the local social dynamics. The same error signal might lead to different actions depending on whether an agent is part of a set of experts ($N_{\text{activated}} \geq 1$), acting alone ($N_{\text{activated}} = 1$) or if the input x is within a void ($N_{\text{neighbors}} = 0$). An example of a conceptual rule table using these social indicators is provided in Table 3.1. This formalization ensures that the learning process remains decentralized and modular.

Social Context	**Condition (Trigger)**	**Targeted Set**	**Action**
$N_{\text{activated}} = 0$ $N_{\text{neighbors}} = 0$	$\varnothing$	$\varnothing$	Agent Creation
$N_{\text{activated}} = 0$ $N_{\text{neighbors}} \geq 1$	High error for all	$\mathcal{D}_{\text{neighbors}}$	Agent Creation
$N_{\text{activated}} = 0$ $N_{\text{neighbors}} \geq 1$	Low error	$\mathcal{D}_{\text{neighbors}}$	Shape Update (expansion)
$N_{\text{activated}} \geq 1$	Low error	$\mathcal{D}_{\text{activated}}$	Model Update
$N_{\text{activated}} > 1$	High error	$\mathcal{D}_{\text{activated}}$	Shape Update (retraction)

Table 3.1: Example of learning rule table for a CELL system . Each line corresponds to a learning rule defined by: 1) a *Social Context* representing a condition on the set of local expert agents close to x; 2) a *Condition* on a feedback value like the error (e.g low or high error according to a threshold) or on the characteristics of the agents (e.g size of the area of expertise higher than a threshold value); 3) a *Targeted Set* of agents that will be affected by the rule (e.g. $\mathcal{D}_{\text{activated}}$ or$\mathcal{D}_{\text{neighbors}}$); 4) an *Action* to perform on the agents of the targeted subset.

3.1.4 Discussion on Theoretical Properties

After having established established the relationship between SACL and the CELL formalism, the previous sections formalized its components and detailed the operational pipelines that govern the ensemble of local expert agents. This section discusses the theoretical properties of CELL systems, highlighting how it addresses the challenges of online learning in non-stationary environments and its suitability for integration within a control framework.

A primary challenge in online learning for dynamical system is **catastrophic forgetting** [139], where new information overwrites previously learned knowledge. CELL inherently mitigates this through its spatial gating mechanism, as it ensures **spatial isolation of parameters**. Since learning rules and parameter updates are restricted to the sets $\mathcal{D}_{\text{activated}}$ and $\mathcal{D}_{\text{neighbors}}$, data from one region of the input space cannot easily corrupt the expertise of agents in non-related regions.

While traditional machine learning often relies on top-down global optimization, CELL is inherently multi-agent and adopts a bottom-up design based on social learning. In the context of CELL, the global dynamics model is not a predefined static structure but an **emergent property** of the self-organizing ensemble of local experts. Each agent operates with only partial knowledge of the global problem, **perceives** its local social context, **decides** on a

rule to trigger based on its own feedback, and **acts** to update its internal components. This decentralized decision-making process brings structural adaptability. It allows the model to modify its own internal structure through agent creation or destruction without the computational burden or re-optimizing a global topology. By shifting the complexity from the model's architecture to the social dynamics between agents, the system can autonomously **expand its capacity in response to non-stationary dynamics**.

Furthermore, the formalism is **model-agnostic**, meaning that virtually any class of learning model could be used as the internal prediction model of an agent. This modularity favors a high level of transparency in the inner workings of the learning system. In the remainder of this thesis, we focus on simple, low-complexity models that can converge quickly, such as **linear regression**. Indeed, high-capacity models could be counterproductive because they would not receive enough local data to converge, thereby hurting global performance. Using fast-converging model classes ensures better online learning performance and maintains the white-box nature of the inner workings, which represents a significant advantage over global black box models for safety-critical applications.

Finally, the integration of CELL within an optimization-based control framework like MPC necessitates some properties like **differentiability and fast inference** (both of which are discussed in detail in Chapter 4). While the agents provide local expertise, the aggregation function should ensure the global prediction handles smoothly the transitions between the areas of expertise of agents. If the internal models f_i, the activation functions ϕ_i and the aggregation function are all differentiable, the ensemble of local expert agents produces differentiable predictions. This allows for the extraction of the necessary gradients $\nabla_x f$ (i.e the Jacobians of the dynamics with respect to state and action) for optimizing control trajectories within an MPC framework with a gradient-based solver. This property will be discussed more in Section 4.1.2 of Chapter 4.

The real-time constraints of control loops require the model to perform high-frequency inference. In CELL instantiations, fast inference is facilitated both by the use of low-complexity internal models and by the spatial gating mechanism that selects only a subset of relevant agents for prediction. However, maintaining a fast inference speed becomes challenging as the population grows. Indeed, evaluating the activation value $\phi_i(x)$ of a very large number of agents can rapidly represent a **computational bottleneck**. Consequently, achieving efficient scaling requires a carefully designed spatialization layer to minimize the cost of distance evaluations and ensure that the spatial gating mechanism remains compatible with the timing requirements of the control task to solve. This topic will be discussed in detail in Section 4.1.1 of Chapter 4.

To transition from the theoretical design framework to practical applications, the following sections 3.2 and 3.3 introduce two distinct instantiations of CELL: oCELL and kCELL.

3.2 oCELL: CELL with Orthotope Spatialization

Previously we described the CELL formalism which establishes the operational logic of the ensemble of expert agents while remaining agnostic to the activation function, to the internal model classes and to the social feedback used. Consequently, to transform this theoretical framework into a functional algorithm, one must define the concrete form of the activation

function ϕ_i and the specific set of social rules governing the population. The works presented in this section were published in [31, 30].

The first instantiation of CELL, oCELL, orthotope-based CELL, leverages axis-aligned orthotopes to define agent boundaries.

Definition 17. An **orthotope** is the d-dimensional generalization of a rectangle [130].

This choice builds on previous works on this family of multi-agent systems [57, 77]. It prioritizes computational efficiency and geometric interpretability providing a baseline for evaluating the emergent modeling capabilities of the CELL framework in low dimensional input spaces. oCELL serves as a primary means for demonstrating how emergent introspective properties of this ensemble of local expert agents provide useful insights about the learning process. These introspective properties are shown to be related to the traditional notions of explainability and interpretability in machine learning.

This section is structured as follows. First, Section 3.2.1 defines the tripartite structure of a local expert agent in oCELL, detailing how orthotopic boundaries and local models are mathematically formulated. Section 3.2.2 then presents the behavioral blueprint of the system through its specific learning rules and the resulting rule table. To validate the effectiveness of this approach, Section 3.2.3 provides a comparative analysis of oCELL against state-of-the-art ensemble learning algorithms on a two-dimensional regression benchmark. Afterwards, Section 3.2.4 introduces a set of metrics derived from the spatial distribution of agents, demonstrating how the structure of local expert agents can be exploited to monitor the learning and characterize the underlying data distribution. Finally, Section 3.2.5 presents a critical analysis of oCELL, its properties and limitations that subsequently lead to the development of kCELL in Section 3.3.

3.2.1 Local Expert Agents

In this section, the tripartite structure of oCELL's local expert agents is described in detail. Each agent

$$\mathcal{A}_i = \{\phi_i, f_i, \mathcal{M}_i\}$$

is a specialized local expert that occupies an orthotope-shaped area in the input space, maintains a local linear model and manages a private memory of its past experiences.

The activation function ϕ_i defines the area of expertise in the input space in which an agent considers itself as an expert. In oCELL, for a d-dimensional input space, the area of expertise of agent $\mathcal{A}_i$ is defined by an orthotope $\mathcal{H}_i$ parameterized by a set of lower bounds $l_{i,j}$ and upper bounds $h_{i,j}$ for each dimension $j \in \{1, \ldots, d\}$ such as:

$$\mathcal{H}_i = [l_{i,1}, h_{i,1}] \times \cdots \times [l_{i,d}, h_{i,d}] \tag{3.1}$$

The hard boundaries defined by the orthotopes allow for the use of geometric operations such as volume measurement or intersection testing. The volume $V(\mathcal{H}_i)$ of an orthotope is defined as:

$$V(\mathcal{H}_i) = \prod_{j=1}^{d} (h_{i,j} - l_{i,j}) \tag{3.2}$$

An input $x \in \mathbb{R}^d$ is considered as intersecting an orthotope $\mathcal{H}_i$ if,

$$\forall j \in \{1, \ldots, d\}\,,\ l_{i,j} \leq x_{i,j} \leq h_{i,j} \tag{3.3}$$

Furthermore, an orthotope $\mathcal{H}_1$ intersects another orthotope $\mathcal{H}_2$ if, for all dimension j,

$$\max\left(l_{1,j}, l_{2,j}\right) \leq \min\left(h_{1,j}, h_{2,j}\right) \tag{3.4}$$

where $l_{1,j}$,$l_{2,j}$ and $h_{1,j}$,$h_{2,j}$ denote the lower and upper bounds of respectively $\mathcal{H}_1$ and $\mathcal{H}_2$.

Agents in oCELL are constructed based on the assumption of *uniform knowledge*:

Assumption 3.1 (Uniform Knowledge)**.** The level of expertise of an agent is assumed to be uniform over its area of expertise for any point intersecting it.

Therefore, the **activation function** of an agent $\mathcal{A}_i$ is a binary function $\phi_i : \mathbb{R}^d \mapsto \{0, 1\}$ defined as

$$\phi_i\left(x\right) = \begin{cases} 1 & \text{if } x \text{ intersects } \mathcal{H}_i \\ 0 & \text{else} \end{cases}$$

An input x is said to be In-Distribution (ID) for agent $\mathcal{A}_i$ if $\phi_i\left(x\right) = 1$. This hard boundary ensures the agent's area of expertise is well localized and contiguous.

As the CELL formalism is model-agnostic, oCELL is designed to work with **low-complexity models** to ensure fast updates and convergence of local models in online settings. Each agent $\mathcal{A}_i$ maintains its own local model $f_i\left(x, \theta_i\right)$ where θ_i denotes the local parameters. In the context of this thesis, f_i is typically a local linear regressor:

$$\hat{y_i} = f_i\left(x\right) = \Theta_i \begin{bmatrix} x \\ 1 \end{bmatrix}$$

where $\Theta_i \in \mathbb{R}^{m \times (d+1)}$ is the parameter matrix representing the local affine transformation of agent $\mathcal{A}_i$ and $\hat{y}_i \in \mathbb{R}^m$ denotes the estimated local prediction. With this linear structure, each agent provides a first-order approximation of the dynamics within its area of expertise.

The internal state $\mathcal{M}_i$ tracks the local history of observations. $\mathcal{M}_i$ includes a local memory buffer $\mathcal{B}_i$ that stores the last observed samples (x, y). This buffer is used to update the internal model f_i and has a fixed size to allow forgetting through FIFO (First-In-First-Out) updates.

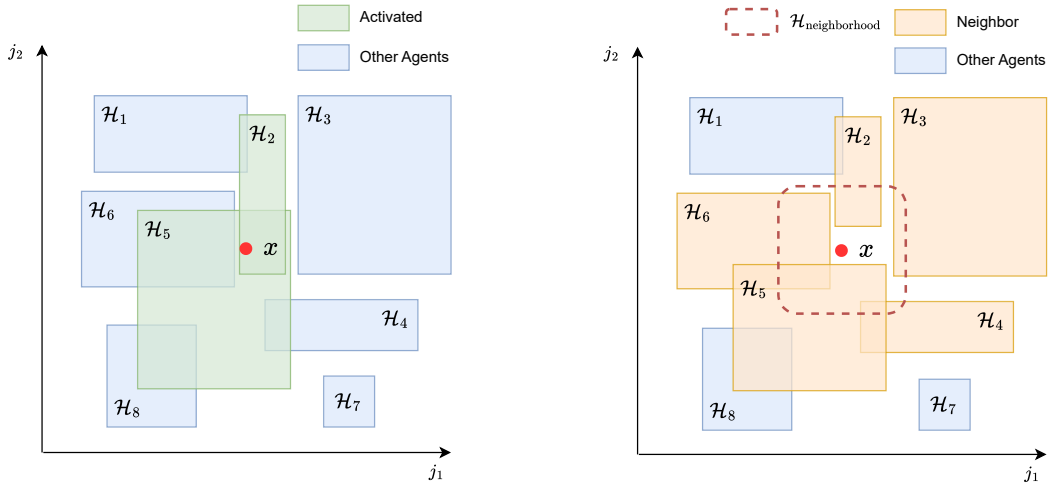
To transform the population of experts into a global model, oCELL uses a **spatial gating mechanism** to select relevant agents. For any input x, the spatial gating mechanism selects two sets of local expert agents (among all the agents in the ensemble):

- **Activated Agents** ($\mathcal{D}_{\text{activated}}$) which contains agents for which $\phi_i\left(x\right) = 1$ (cf. Figure 3.6a)
- **Neighbor Agents** ($\mathcal{D}_{\text{neighbors}}$) which contains agents whose orthotope $\mathcal{H}_i$ intersects a neighborhood orthotope $\mathcal{H}_{\text{neighborhood}}$ centered on x. The size of $\mathcal{H}_{\text{neighborhood}}$ is a hyperparameter of the system. Tuning this parameter allows the designer to adjust the degree of smoothing in the global prediction. (cf. Figure 3.6b)

The global prediction $\hat{y}$ is obtained by an arithmetic mean of the activated agents' proposals

$$\hat{y} = \frac{1}{|\mathcal{D}_{\text{activated}}|} \sum_{i \in \mathcal{D}_{\text{activated}}} \hat{y}_i$$

If $\mathcal{D}_{\text{activated}}$ is empty, the ensemble $\mathcal{D}_{\text{neighbors}}$is selected, and if $\mathcal{D}_{\text{neighbors}}$ is also empty, a fallback mechanism selects the k-closest agents to ensure continuity in the global approximated function. This fallback mechanism prevents prediction failures in regions where the system has limited knowledge (i.e poor agent paving of the area around x).



(a) Activated Agents

(b) Neighbor Agents

Figure 3.6: Example of subsets of agents outputed by the spatial gating mechanism in a 2-dimensional input space. 3.6a illustrates $\mathcal{D}_{\text{activated}} = \{\mathcal{H}_2, \mathcal{H}_5\}$ containing activated agents whose associated orthotopes are intersecting with x. 3.6b illustrates $\mathcal{D}_{\text{neighbors}} = \{\mathcal{H}_2, \mathcal{H}_3, \mathcal{H}_4, \mathcal{H}_5, \mathcal{H}_6\}$ containing the agents that are considered neighbors of x, i.e whose associated orthotopes are intersecting $\mathcal{H}_{\text{neighborhood}}$.

Shape Update An agent $\mathcal{A}_i$ can update its shape by updating the bounds of its associated orthotope $\mathcal{H}_i^t$ at learning step t by a factor α in the direction of an input x. The factor α is a hyperparameter that needs to be tuned. The bounds of $\mathcal{H}_i^t$ are updated to obtain $\mathcal{H}_i^{t+1}$ according to the following relationship

$$h_j^{t+1} = \begin{cases} \left(h_{i,j}^t - l_j^t\right) \varepsilon^{\frac{1}{n}} + l_j^t & \text{if } l_{i,j}^t \leq x_j \leq h_{i,j}^t \\ h_{i,j}^t & \text{else} \end{cases} \tag{3.5}$$

$$l_j^{t+1} = \begin{cases} \left(l_{i,j}^t - h_{i,j}^t\right) \varepsilon^{\frac{1}{n}} + h_j^t & \text{if } l_{i,j}^t \leq x_j \leq h_{i,j}^t \\ l_{i,j}^t & \text{else} \end{cases} \tag{3.6}$$

with

$$\varepsilon = \begin{cases} (1+\alpha) & \text{if expansion} \\ (1-\alpha) & \text{if retraction} \end{cases} \tag{3.7}$$

and n the number of features such that $l_{i,j}^t \leq x_j \leq h_{i,j}^t$.

Thus, the new volume of $\mathcal{H}_i$ is given by the relation:

$$V\left(\mathcal{H}_i^{t+1}\right) = \varepsilon V\left(\mathcal{H}_i^t\right) \tag{3.8}$$

The mechanism of expansion consists in expanding the boundaries of the area of expertise defined by the activation function of the agent, as illustrated in Figure 3.7. Conversely, the mechanism shrinks these boundaries, as shown in Figure 3.8.

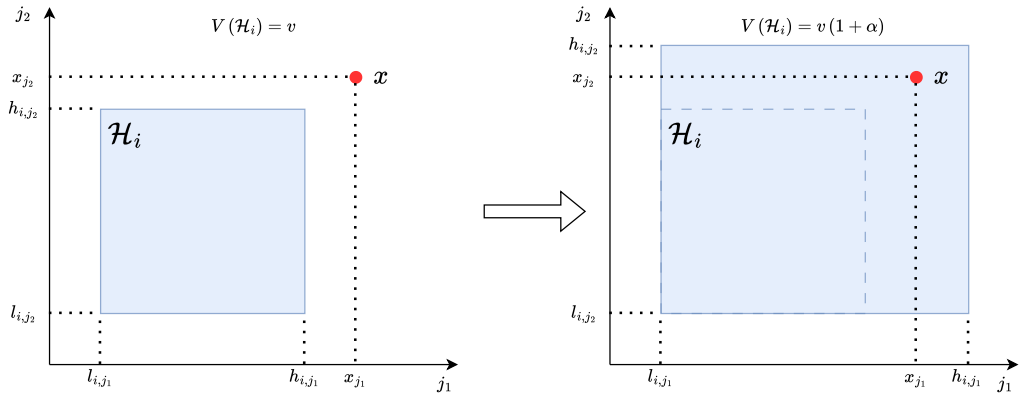


Figure 3.7: Mechanism of agent expansion in oCELL

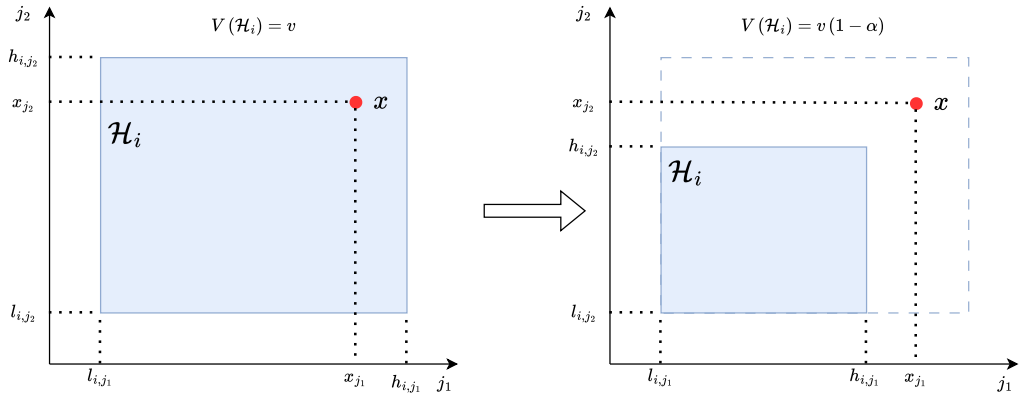


Figure 3.8: Mechanism of agent retraction in oCELL

Model Update An agent $\mathcal{A}_i$ can update its internal model. The new feature-target tuple (x, y) is added to the buffer $\mathcal{B}_i$ and the parameter matrix Θ_i is fitted to the data contained

in $\mathcal{B}_i$ using Ordinary Least Square (OLS) [93] such as

$$\Theta_i = \left(X_{\mathcal{B}_i}^\top X_{\mathcal{B}_i}\right)^{-1} X_{\mathcal{B}_i}^\top y_{\mathcal{B}_i}$$

where $X_{\mathcal{B}_i}$ and $y_{\mathcal{B}_i}$ correspond to the feature and target vectors stored in $\mathcal{B}_i$. In practice, a small regularization term (Ridge Regression) should be used to ensure numerical stability [93] especially when working with a reduced amount of data points.

3.2.2 Learning Rules

After having described the capabilities and properties of the local expert agents in the previous section, this section describes the learning mechanisms and rules that govern the ensemble as a whole to allow the emergence of learning through self-organization. In oCELL, the evolution of the population is governed by social rules that determine how the ensemble processes the continuous stream of feature-target tuples (x_{new}, y_{new}). These rules translate the individual performance of agents and a local social context into discrete actions: updating an agent's model, adapting its shape or managing the population through creation and destruction of agents. The overall flow of the training process is formalized in Algorithm 2.

To determine the appropriate action to apply, each agent $\mathcal{A}_i$ in either $\mathcal{D}_{\text{activated}}$ or $\mathcal{D}_{\text{neighbors}}$ (depending on the social context) receives a feedback based on the quality $L_{\mathcal{A}_i}$ of its prediction. The quality $L_{\mathcal{A}_i}$ is typically calculated as the squared error (or any other error function) between the local proposal $\hat{y}_i$ and the true observation y:

$$L_{\mathcal{A}_i} = \|\hat{y}_i - y\|^2$$

These errors are categorized into three regimes using two sensitivity thresholds τ_{good} and τ_{bad}:

- **Good Prediction**: The agent is a reliable expert for this input ($L_{\mathcal{A}_i} \in [0, \tau_{\text{good}}]$)
- **Inaccurate Prediction**: The agent is relevant but requires model refinement ($L_{\mathcal{A}_i} \in [\tau_{\text{good}}, \tau_{\text{bad}}]$)
- **Bad Prediction**: The agent is not relevant for this region ($L_{\mathcal{A}_i} \in [\tau_{\text{bad}}, +\infty[$)

The learning rules are triggered according to the social context defined by the number of activated agents ($N_{\text{activated}} = |\mathcal{D}_{\text{activated}}|$) and neighbors ($N_{\text{neighbors}} = |\mathcal{D}_{\text{neighbors}}|$). The following paragraphs describe the different social contexts considered, along with the actions or meta-actions they trigger.

Incompetence When $N_{\text{activated}} = 0$ there is a lack of relevant experts to predict for x_{new}. If $N_{\text{neighbors}} \geq 1$, the neighbor agents are candidate to cover the void, i.e to become activated for x_{new}. A neighbor agent that provides *good* or *inaccurate* prediction expands its area of expertise toward x_{new} (**Shape Update**) and updates its internal model (**Model Update**). However, if all neighbors provide *bad* prediction (no satisfactory prediction) or if $N_{\text{neighbors}} = 0$, an **Agent Creation** meta-action is triggered. A new local expert agent centered on x_{new} is spawned into the ensemble with initial dimensions defined by an hyperparameter R.

Inaccuracy When $N_{\text{activated}} \geq 1$, x_{new} is already covered by local expert agents. An activated agent that provides a *good* prediction will peform a **Shape Update** action to expand its area of expertise in the direction of x_{new} with the aim of generalizing. An activated agent that provides an *inaccurate* prediction performs a **Model Update** to refine its internal model. An activated agent that provides a *bad* prediction performs a **Shape Update** to retract its area of expertise away from x_{new} with the aim of excluding that point, specializing the boundaries to maintain local accuracy.

Uselessness When applying too many **Shape Update** actions involving retraction, the area of expertise can evolve into a degenerate shape, becoming far too small to be informative. To prevent uncontrolled population growth and eliminate degenerate agents a destruction rule is applied. Any local expert agent whose volume $V(\mathcal{H}_i)$ falls below a threshold τ_{vol} is removed from the ensemble (**Agent Destruction**).

Table 3.2 provide the behavioral blueprint of oCELL mapping different social contexts and conditions to actions and meta-actions allowing self-organization. The behavior of oCELL is parameterized by a set of hyperparameters that influences local convergence and stability of the emergent model:

- $\tau_{\text{good}}, \tau_{\text{bad}}$: define the error tolerance of local experts.
- α: determines the volume variation coefficient controlling the plasticity of the area of expertise of agents.
- R: defines the initial dimensions for a newly created agent.
- τ_{vol}: is the parsimony threshold on volume for agent destruction.
- memory length: represents the maximum number of data points that can be stored into $\mathcal{B}_i$.

3.2.3 Comparative Study

The previous sections laid the theoretical blueprint of oCELL, detailing how an ensemble of local expert agents can self-organize through social rules to approximate complex dynamics from data. However, the effectiveness of the emergent learning behavior of oCELL must be validated. In this section, we provide an empirical validation of its modeling capabilities, evaluating whether this decentralized, self-organizing learning approach can compete with well established machine learning algorithms on a low-dimensional regression task.

Experimental Setup The primary objective of the experiment presented in this section is to evaluate how effectively the oCELL learning rules allow the ensemble of local expert agents to capture nonlinear dynamics from a dataset. The experiment is conducted on four two-dimensional benchmark functions commonly used in optimization research. These functions were selected due to their diverse topographical challenges, including sharp peaks, flat plateaus and multiple local optima [121]. This choice is motivated by the fact that

Algorithm 2: oCELL

for each $(x_{\text{new}}, y_{\text{new}})$ in the data stream **do**
- $\mathcal{D}_{\text{activated}}, \mathcal{D}_{\text{neighbors}} \leftarrow$ Gating Mechanism(x_{new})
- $N_{\text{activated}}, N_{\text{neighbors}} \leftarrow |\mathcal{D}_{\text{activated}}|, |\mathcal{D}_{\text{neighbors}}|$
- $\forall \mathcal{A}_i \in \mathcal{D}_{\text{neighbors}}$ compute $L_{\mathcal{A}_i}$ from y_{new} and $f_i\,(x_{new})$
- **for** $\mathcal{A}_i \in \mathcal{D}_{\text{neighbors}}$ **do**
  - `// Inaccuracy`
  - **if** $N_{\text{activated}} \geq 1$ **then**
    - **if** $L_{\mathcal{A}_i} \in [0, \tau_{\text{good}}]$ **then**
      - Shape Update (expansion)
    - **end**
    - **else if** $L_{\mathcal{A}_i} \in [\tau_{\text{good}}, \tau_{\text{bad}}]$ **then**
      - Model Update
    - **end**
    - **else if** $L_{\mathcal{A}_i} \in [\tau_{\text{bad}}, +\infty[$ **then**
      - Shape Update (retraction)
    - **end**
  - **end**
  - `// Incompetence`
  - **else if** $N_{\text{activated}} = 0$ and $N_{\text{neighbors}} \geq 1$ and $L_{\mathcal{A}_i} \in [0, \tau_{\text{good}}] \cup [\tau_{\text{good}}, \tau_{\text{bad}}]$ **then**
    - Shape Update (expansion)
    - Model Update
  - **end**
  - `// Uselessness`
  - **if** $V(\mathcal{H}_i) \leq \tau_{\text{vol}}$ **then**
    - Destroy agent
  - **end**
- **end**
- `// Agent Creation`
- **if** $N_{\text{activated}} = 0$ and $N_{\text{neighbors}} \geq 1$ and $\forall \mathcal{A}_i$, $L_{\mathcal{A}_i} \notin [0, \tau_{\text{good}}] \cup [\tau_{\text{good}}, \tau_{\text{bad}}]$ **then**
  - Create new agent centered on x_{new}
- **end**
- **if** $N_{\text{activated}} = 0$ and $N_{\text{neighbors}} = 0$ **then**
  - Create new agent centered on x_{new}
- **end**

end

	Social Context	Condition	Targeted Set	Action
Incompetence	$N_{\text{activated}} = 0$ $N_{\text{neighbors}} \geq 1$	$L_{\mathcal{A}_i} \in [0, \tau_{\text{good}}] \cup [\tau_{\text{good}}, \tau_{\text{bad}}]$	$\mathcal{D}_{\text{neighbors}}$	Model + Shape (expansion) Update
		$\forall \mathcal{A}_i,\ L_{\mathcal{A}_i} \notin [0, \tau_{\text{good}}] \cup [\tau_{\text{good}}, \tau_{\text{bad}}]$	$\mathcal{D}_{\text{neighbors}}$	Agent Creation
	$N_{\text{activated}} = 0$ $N_{\text{neighbors}} = 0$	$\varnothing$	$\varnothing$	Agent Creation
Inaccuracy	$N_{\text{activated}} \geq 1$	$L_{\mathcal{A}_i} \in [0, \tau_{\text{good}}]$	$\mathcal{D}_{\text{activated}}$	Shape Update (expansion)
		$L_{\mathcal{A}_i} \in [\tau_{\text{good}}, \tau_{\text{bad}}]$	$\mathcal{D}_{\text{activated}}$	Model Update
		$L_{\mathcal{A}_i} \in [\tau_{\text{bad}}, +\infty[$	$\mathcal{D}_{\text{activated}}$	Shape Update (retraction)
Uselessness	$\varnothing$	$V(\mathcal{H}_i) < \tau_{\text{vol}}$		Agent Destruction

Table 3.2: Learning rules of oCELL. Each line corresponds to a given learning rule defined by: 1) a *Social Context* representing a condition on the number of activated or neighbor local expert agents ($N_{\text{activated}}$ and $N_{\text{neighbors}}$) around a new input x_{new}; 2) a *Condition* on the feedback $L_{\mathcal{A}_i}$ or on the volume of the area of expertise $V(\mathcal{H}_i)$; 3) a *Target Set* of agents that will be affected by the rule ($\mathcal{D}_{\text{activated}}$ or$\mathcal{D}_{\text{neighbors}}$); 4) an *Action* to perform on the agents of the targeted subset.

state-of-the-art machine learning algorithms [49, 136] partially rely on gradient-based optimization during the learning process. The functions considered are the SSR ($f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right)$), Booth, Goldstein-Price and Beale functions (see Figure 3.9). For each function, a synthetic dataset comprising 8000 feature-target tuples has been generated by uniformly sampling points within the feature space. All input features are normalized to the range $[-1, 1]$.

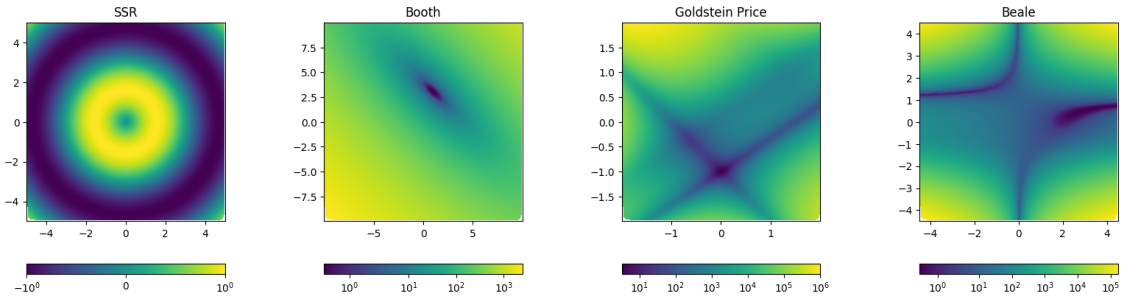


Figure 3.9: Visualization of the 2D functions used to generate the benchmark datasets. The color levels approaching *yellow* correspond to higher function values, while those approaching *blue* correspond to lower ones.

Within this experimental context, using local linear models for each agent's internal model f_i ensures that the nonlinear complexity of the global oCELL model arises from local agent interactions rather than from the representational power of individual models.

oCELL is compared to widely used ensemble and common non-ensemble algorithms. For ensemble methods, we consider both classical approaches such as gradient boosting [83] and

random forests [38], alongside state-of-the-art algorithms: XGBoost [49] and LightGBM [136]. The selected non-ensemble algorithms include decision trees [36], SVMs [46] and linear regression [235].

Hyperparameter optimization is performed via a grid search with the goal of minimizing the mean squared error obtained through 5-fold cross-validation. The resulting best configurations found are reported in Table 3.3. These configurations correspond to the parameter sets exposed by the interfaces of the *scikit-learn* [180], *xgboost* [49] and *lightgbm* [136] python libraries. For oCELL, the hyperparameters were initialized based on the social learning requirements: the initial orthotope side length R, error thresholds τ_{good} and τ_{bad}, the plasticity coefficient α and the memory buffer size.

Results The comparative results, reported in Table 3.4, measure performance across three metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE) and the Coefficient of Determination (R^2).

These results show that oCELL achieves the lowest mean absolute error (MAE) and the highest coefficient of determination (R^2) for all four considered functions. It also obtains the best MSE on the SSR, Goldstein-Price and Booth functions, while XGBoost performs slightly better in terms of MSE on the Beale function. Overall, oCELL demonstrates comparable performances to XGBoost and LightGBM for most functions, while consistently outperforming the non-ensemble models. These results indicate that oCELL effectively captures the nonlinear variations of the underlying functions from the data. As illustrated in Figure 3.10, we can see that the predictions are similar to those shown in Figure 3.9, with a few exceptions: some artifacts in the resulting heatmaps indicate small approximation errors in regions exhibiting nonlinearities.

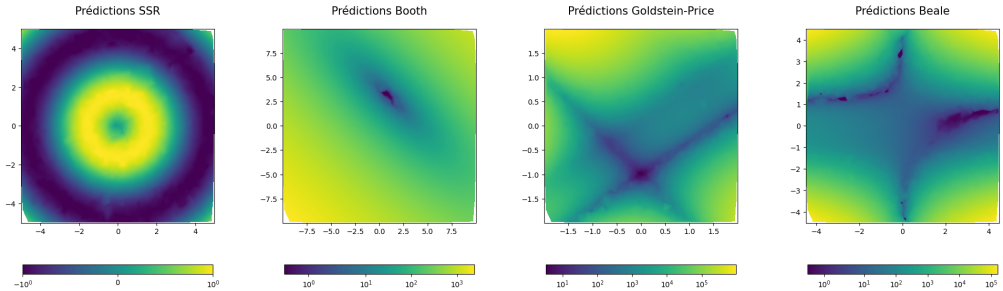


Figure 3.10: oCELL predictions on data not included in the training dataset.

Finally, this experiment has shown that the social learning rules that govern the defined learning rules are sufficient to drive a population of local expert linear agents toward a global accurate representation of the function underlying the data.

3.2.4 Introspective Properties and Uncertainty

In the previous section, the comparative study has established that oCELL could match the predictive accuracy of globally optimized ensemble models. However, one of the main advantage of oCELL lies in its local decentralized multi-agent nature that offers native introspection capabilities. Because the model is composed of spatially-bounded agents that

	hyperparameters	SSR	Booth	Goldstein-Price	Beale
oCELL	R	0.1	0.35	0.1	0.05
	τ_{good}	0.01	0.15	0.01	0.1
	τ_{bad}	0.2	0.2	0.2	0.3
	α	0.4	0.1	0.2	0.2
	memory_length	20	50	200	20
XG Boost	learning_rate	0.1	0.05	0.05	0.1
	max_depth	9	9	9	9
	n_estimators	300	300	300	300
	objective	abs. error	abs. error	squ. error	abs. error
Light GBM	learning_rate	0.1	0.1	0.1	0.1
	max_depth	9	5	9	3
	n_estimators	5	5	5	5
	objective	300	300	200	300
	min_child_samples	huber	l2	tweedie	tweedie
Random Forest	bootstrap	true	true	true	true
	criterion	abs. error	poisson	poisson	poisson
	max_depth	9	9	9	9
	max_features	null	null	null	null
	min_samples_leaf	1	1	1	1
	min_samples_split	2	2	2	2
	n_estimators	300	200	300	300
Decision Tree	criterion	abs. error	poisson	poisson	poisson
	max_depth	9	9	9	9
	max_features	null	null	null	null
	min_samples_leaf	1	2	2	1
	min_samples_split	2	2	2	2
Gradient boosting	learning_rate	0.1	0.1	0.2	0.2
	loss	abs. error	huber	squ. error	huber
	max_depth	9	9	5	5
	max_features	null	null	null	null
	min_samples_leaf	2	4	1	1
	min_samples_split	2	5	2	2
SVM	epsilon	0.1	0.1	0.1	0.5
	gamma	scale	scale	scale	scale
	kernel	rbf	rbf	poly	rbf
Linear Reg.	fit_intercept	true	true	true	true

Table 3.3: Best hyperparameters set found by grid search on 5-fold cross validation results

		R^2	MSE	MAE
Beale	Decision Tree	0,983	6,94e+06	1,07e+03
	Gradient boosting	0,995	2,08e+06	6,19e+02
	LightGBM	0,996	1,61e+06	4,75e+02
	Linear Reg.	-0,001	4,16e+08	1,19e+04
	oCELL	**0,997**	1,34e+06	**2,53e+02**
	Random Forest	0,995	2,02e+06	5,23e+02
	SVM	-0,139	4,74e+08	8,28e+03
	XGBoost	**0,997**	**1,29e+06**	4,11e+02
Booth	Decision Tree	0,994	1,29e+03	2,60e+01
	Gradient boosting	0,999	1,39e+02	7,44e+00
	LightGBM	0,999	1,38e+02	8,09e+00
	Linear Reg.	0,426	1,17e+05	2,63e+02
	oCELL	**1,000**	**9,52e+00**	**9,91e-01**
	Random Forest	0,998	3,57e+02	1,31e+01
	SVM	0,811	3,89e+04	7,85e+01
	XGBoost	**1,000**	8,72e+01	6,01e+00
Goldstein-Price	Decision Tree	0,994	1,02e+08	5,29e+03
	Gradient boosting	**0,999**	2,37e+07	2,62e+03
	LightGBM	**0,999**	1,80e+07	1,82e+03
	Linear Reg.	0,249	1,22e+10	6,79e+04
	oCELL	**0,999**	**1,21e+07**	**7,02e+02**
	Random Forest	0,998	3,98e+07	3,38e+03
	SVM	-0,127	1,83e+10	5,17e+04
	XGBoost	**0,999**	1,46e+07	1,61e+03
SSR	Decision Tree	0,960	1,89e-02	9,30e-02
	Gradient boosting	0,997	1,22e-03	2,30e-02
	LightGBM	0,998	7,68e-04	1,98e-02
	Linear Reg.	0,000	4,77e-01	6,09e-01
	oCELL	**0,999**	**4,97e-04**	**1,39e-02**
	Random Forest	0,978	1,03e-02	6,48e-02
	SVM	0,972	1,32e-02	7,90e-02
	XGBoost	**0,999**	5,93e-04	1,65e-02

Table 3.4: Comparison between oCELL and the reference algorithms (best scores in bold)

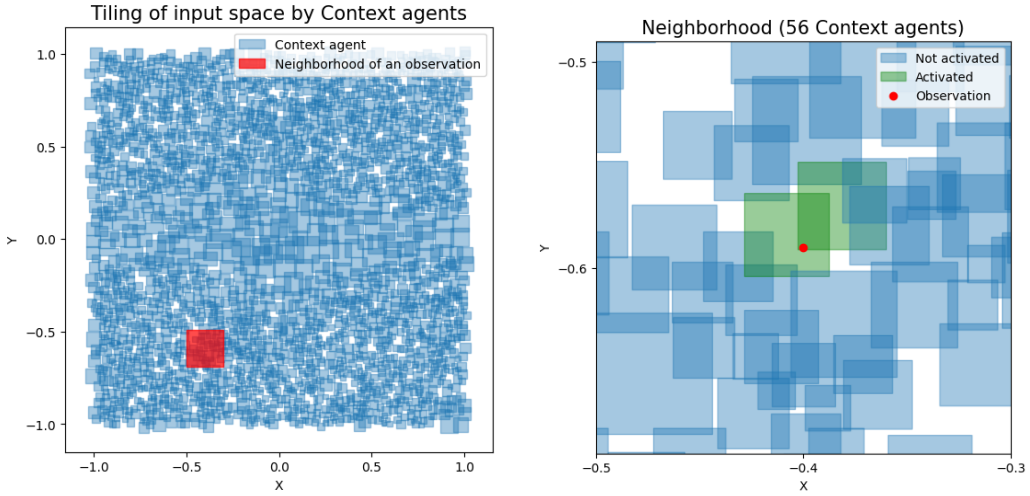
self-organize to local social rules, the resulting **tiling** of the input space represents a **map of the model's knowledge**.

This section demonstrates how the spatial distribution and internal consensus of oCELL agents provide **introspective properties** that are linked to uncertainty and insights about the underlying function's complexity. To illustrate these properties, the Beale function is used as a didactic case study because its topography features both a wide central plateau and sharp gradients. This nonlinear function exhibits regimes of amplitude variations that are very different across its domain of definition. The amplitude of the gradients of the Beale function varies significantly at the boundaries of the definition domain ($-4 \leq x, y \leq 4$), while in the central plateau, the variations in gradient amplitude are more subtle. The Beale function is frequently used as a benchmark for testing optimization algorithms [258, 121].

Before defining introspective metrics, it is necessary to distinguish between the two types of uncertainty. On the one hand, *epistemic uncertainty* represents the uncertainty arising

from a lack of knowledge or insufficient data. In the context of a learning system based on an ensemble of local expert agents like oCELL, it refers to regions of the input space that are uncovered or sparsely explored. Providing more training data to the system can reduce this uncertainty.

On the other hand, *aleatoric uncertainty* represents the inherent variability or noise in the data itself. Even with infinite data, aleatoric uncertainty remains if the underlying function is stochastic or if the measurement of inputs are noisy.



(a) (b)

Figure 3.11: (3.11a) Tiling of the input space (normalized between -1 and 1) by the local expert agents for the Beale function. Each *blue rectangle* represents a context agent and the *red rectangle* is an example of the neighborhood of an observation ($\mathcal{H}_{\text{neighborhood}}$). (3.11b) Activated (*green rectangles*) and Neighbor agents (*blue rectangles*) in the neighborhood of an observation.

Quantifying Epistemic Uncertainty During training, the agents organize themselves in the feature space. Depending on the training dataset and the complexity of the underlying function, the results of self-organization vary. As illustrated in figure 3.11a, the agents' tiling can leave areas of the space uncovered. It is possible that for a given observation, no agent is activated. In such cases, making a prediction requires relying on the proposals of the neighbor or nearest agents. For a given input x, to determine if the prediction is reliable, the coverage index p_{coverage} of the observation's neighborhood is defined as:

$$p_{\text{coverage}}(x) = \frac{V_{\text{neighbors}}}{V(\mathcal{H}_{\text{neighborhood}})} \tag{3.9}$$

where $V_{\text{neighbors}}$ is the volume covered by the agents inside $\mathcal{H}_{\text{neighborhood}}$(orthotope centered on x defining the neighborhood around x).

If $p_{\text{coverage}} \approx 1$, the region is well known, and the prediction is a reliable interpolation. Conversely, if $p_{\text{coverage}} \approx 0$, the region has been underexplored, indicating high epistemic uncertainty. The latter may indicate either gaps in the training data specific to that area of

space, or training that did not proceed as expected for various reasons (strong nonlinearities in the function to approximate, presence of noise, discontinuity, etc...).

The coverage index may be useful to identify missing portions of data in the training dataset, serving as a direct **proxy for estimating epistemic uncertainty**. This is illustrated in Figure 3.12, where oCELL is trained on a dataset that has an entire portion of missing data. When computing the coverage index, the missing portion of data can be identified by locating areas of the input space where the coverage index is low (blue area in the right-hand figure). This experiment shows that the missing portion of data can be identified without having direct access to the training dataset.

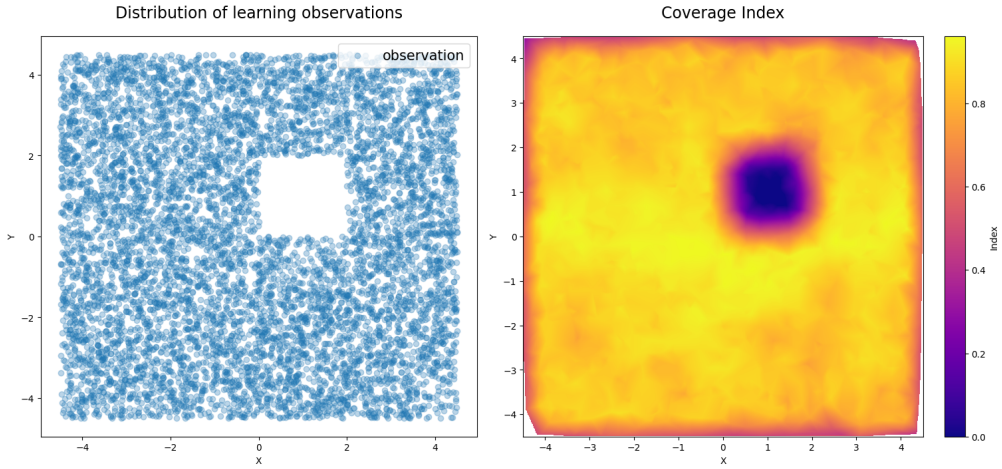


Figure 3.12: Comparison of the distribution map of observations used to train the model (left) and the visualization of the coverage index associated with this model (right).

The coverage index is therefore a quantitative measure of the epistemic uncertainty associated with a prediction. It provides a direct assessment of the model's learned boundaries and limitations. For the development of critical systems, this property is essential to observe in order to assess the risk of extrapolation or interpolation errors.

This measure can be used either locally and globally. Locally, at the individual prediction level, the coverage index quantifies the reliability of the prediction of the model, effectively representing a degree of trustworthiness. Globally, the aggregated coverage index allows for the systematic analysis and visualization of the entire agent organization across the input space. The coverage index represents a tool for understanding the model's inherent structure and mechanisms.

Quantifying Aleatoric Uncertainty When several agents are activated for a single input x, each agent proposes a prediction $\hat{y}_i$. The discrepancy between these proposals represents a measure of aleatoric uncertainty. It quantifies the disagreement among a group of expert agent where all of them consider themselves as relevant to predict. The discrepancy $\sigma_{\text{activated}}$ is defined as:

$$\sigma_{\text{activated}} = \sqrt{\sum_{i \in \mathcal{D}_{\text{activated}}} \frac{\left|\hat{y}_i - \hat{y}_{\text{mean}}\right|^2}{N_{\text{activated}}}} \tag{3.10}$$

where $\hat{y}_{\text{mean}}$ is the mean of the predictions of activated agents.

A high value of discrepancy $\sigma_{\text{activated}}$ suggests either that the local dynamics are highly noisy, or the local linear approximation is struggling to reconcile the local data. As oCELL is agnostic to the model class used as internal model for agents, using a class of models capable of inherently providing a measure of uncertainty on predictions, such as Gaussian Processes (GPs), could robustify this uncertainty estimate.

Analyzing Function Complexity Alongside uncertainty, the structural geometric organization of the population of local expert agents provides insights into the underlying variation regimes of the function underlying the data. In oCELL, the geometry of the ensemble of agents is a direct consequence of the learning processes. Agents expand their area of expertise where the function is easy to approximate and are forced to specialize by shrinking it where it is complex.

The local complexity of the underlying function can be quantified by the concentration of experts in a given region of the input space. The agent density ρ can be defined as the number of agents intersecting a neighborhood $\mathcal{H}_{\text{neighborhood}}$ normalized by its volume such as:

$$\rho = \frac{N_{\text{neighbors}}}{V_{\text{neighborhood}}} \tag{3.11}$$

where $V_{\text{neighborhood}}$ is the volume of the orthotope representing the neighborhood and $N_{\text{neighbors}}$ is the number of neighbor agents intersecting it.

Figure 3.13a shows the density of agents in the input space for the Beale function. By examining the distribution of the average volume of agents in the neighborhood of observations (cf. Figure 3.13b), it can be noted that areas with high agent density contain smaller agents and conversely for low-density areas.

High-density and low-density regions correspond to very different variation regimes of the Beale function. The region containing the largest agents (low-density) corresponds to a plateau where the gradient magnitude of the function varies little. In contrast, the region containing the smallest agents (higher density) corresponds to a region of space where the gradient magnitude of the function varies strongly as one approaches the boundary of the definition domain (Figure 3.13a).

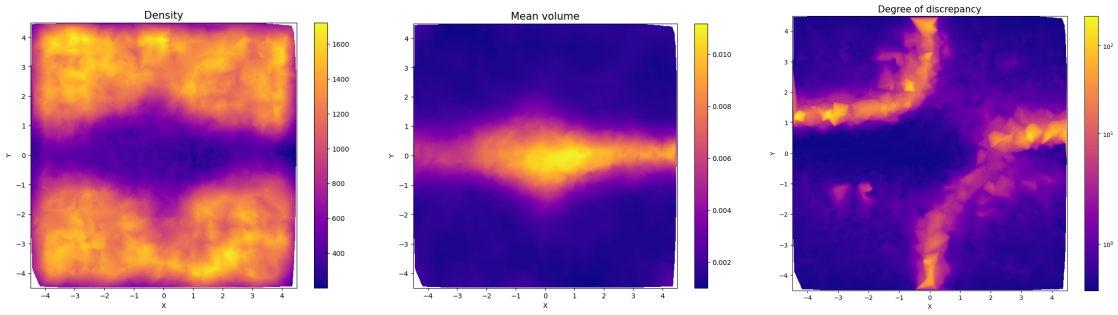
As the activated discrepancy ($\sigma_{\text{activated}}$) measures noise among activated agents, this logic can be extended to the neighbor agents of an input x to detect regime changes in the underlying function. The degree of discrepancy (κ) measures the disagreement among all neighbor agents normalized by the mean prediction:

$$\kappa\left(x\right) = \frac{\sqrt{\sum_{i \in \mathcal{D}_{\text{neighbors}}} \frac{|\hat{y}_i - \hat{y}_{\text{mean}}|^2}{N_{\text{neighbors}}}}}{|\hat{y}_{\text{mean}}|} \tag{3.12}$$

where $N_{\text{neighbors}}$ is the number of agents in the neighborhood and $\hat{y}_{\text{mean}}$ is the average of predictions from the agents in the neighborhood.

If $\kappa \approx 0$, then the predictions of the agents in the neighborhood are very close to each other. This indicates that the underlying function of the data varies little around the observation. Conversely, if κ value is high the agents in the neighborhood make very different

predictions. This suggests that the regime of variation of the underlying function is likely to change abruptly in that area. The degree of discrepancy κ helps identify regions in the input space where approximating the underlying function is the most challenging. For the Beale function, the degree of discrepancy highlights regions of the space with the most complex variations. In Figure 3.13c, the yellow areas represent the highest disagreement between agents.



(a) (b) (c)

Figure 3.13: Visualization of density (3.13a), average volume (3.13b), degree of discrepancy (3.13c) of local expert agents.

The three metrics derived from the self-organizing structure of local expert agents in oCELL offer a view on the model's internal state. These introspection tools reveal capabilities that can be leveraged for critical applications such as fault detection, model debugging or active learning strategies where the system should autonomously decide where more data is needed to reduce uncertainty. Table 3.5 summarizes the metrics that have been introduced in this section, mapping them to their purpose.

Metric	**Target Property**	**Primary Use Case**
Coverage Index (p_{coverage})	Epistemic Uncertainty	Detecting Out-of-Distribution (OoD) inputs or gaps in the knowledge of the model
Activated Discrepancies ($\sigma_{\text{activated}}$)	Aleatoric Uncertainty	Identifying local noise or model-data mismatch.
Density (ρ)	Structural Complexity	Mapping regions of high nonlinearity/gradient.
Degree of Discrepancy (κ)	Variation Regime Instability	Detecting abrupt changes of variation regime in the underlying function.

Table 3.5: Introspective metrics and their purpose

3.2.5 Discussion and Limitations of oCELL

The previous sections established the oCELL instantiation of the CELL formalism. It demonstrated that a population of self-organizing local expert agents could achieve predictive performance comparable to state-of-the-art ensemble algorithms on low dimensional regression tasks while offering a rich set of introspection metrics. In this section, these results are discussed in light of the global objectives of this thesis and critically examine the structural limitations of oCELL that motivate the transition toward smoother representations.

In the comparative study presented in Section 3.2.3, oCELL achieves competitive accuracy levels using only local linear models, demonstrating that global modeling complexity effectively arise from social interactions within the ensemble of local expert agents instead of solely relying on high-capacity internal models.

The introspective metrics derived from oCELL in the previous section, namely the coverage index, the local discrepancies of predictions, the density, and the average volume, offer valuable tools for analyzing the internal structures of the ensemble of agents. We argue that they are linked to the traditional notions of explainability and interpretability in machine learning. To situate oCELL within the field of eXplainable Artificial Intelligence (XAI), the explainability and interpretability denominations should be defined beforehand.

Following [42], we define *interpretability* as the inherent ability to understand how the model works as a whole, focusing on the model's structure and mechanisms, i.e transparency. In contrast, *explainability* is associated with the methodologies used to communicate the reasoning for a specific decision taken by a model.

Definition 18. Interpretability refers to the inherent ability to understand how a machine learning model functions as a whole by focusing on its internal structure and mechanisms. It is often used to describe white-box models that are naturally transparent or designed to be easily understood by humans [42].

Definition 19. Explainability refers to the methodologies used to communicate the reasoning behind a model's output to a human. It is typically applied to complex *black-box* models that are not inherently understandable, requiring post-hoc techniques to make specific decisions or the overall model behavior comprehensible [42].

oCELL operates as a **white-box model** because it partitions the input space into local regions, each governed by a local expert agent that uses an interpretable linear regression as its internal model. Unlike black-box models (e.g. neural networks in deep learning) with opaque inner workings, oCELL relies on simple, transparent linear models for prediction and orthotopes for spatialization. Indeed, the orthotopes offer a clear geometric representation of the areas of expertise and can be interpreted as symbolic "if-then" human-interpretable rules.

The introspection metrics introduced in the previous section can be leveraged for fault detection, model debugging, or smart resampling strategies. In a controller, these metrics act as operational safeguards. The **Coverage Index** (p_{coverage}), **Density** (ρ), and **Degree of Discrepancy** (κ) serve as both interpretability and explainability tools. Because they operate at both local and global scales, they allow to analyze the model's global *map of knowledge* (interpretability) while providing information about why a single prediction might be unreliable due to knowledge gaps or regime changes (explainability). In contrast, the

Activated Discrepancy ($\sigma_{\text{activated}}$) is primarily an explainability metric. Since it is an absolute value tied to the local magnitude of the predictions, it is not informative for analyzing the global structure of the model. It provides a local estimation of aleatoric uncertainty for a specific observation.

For instance, a controller can evaluate its own reliability using p_{coverage}. If an input falls into an uncovered region, the system can autonomously switch to a simpler, more robust control strategy to ensure safety. Similarly, a high κ value acts as a warning sign of local failure of the learning process. Table 3.6 summarizes the scopes and links between the defined metrics and the notions of explainability and interpretability.

Metric	**Scope**	**Explain.**	**Interpret.**	**Functional Insight**
Coverage Index (p_{coverage})	Local + Global	✓	✓	Measures uncertainty and maps knowledge gaps
Activated Discrepancies ($\sigma_{\text{activated}}$)	Local	✓	×	Quantifies local expert disagreement for a specfic input
Density (ρ)	Local + Global	✓	✓	Reveals local complexity and a map of expertise
Degree of Discrepancy (κ)	Local + Global	✓	✓	Identifies variation regime changes and structural instability

Table 3.6: oCELL's introspective metrics and their link to explainability or interpretability. The column *Metric* corresponds to the defined introspective metrics; the column *Scope* indicates whether the corresponding metric should be used at a global or local scale to either study macro or micro behaviors within the ensemble of local expert agents; the columns *Explain.* and *Intepret.* indicate if the corresponding metric can be linked to the traditional notions of Explainability or Interpretability; Finally, the column *Functional Insight*, provides indications on the concrete use of each metric.

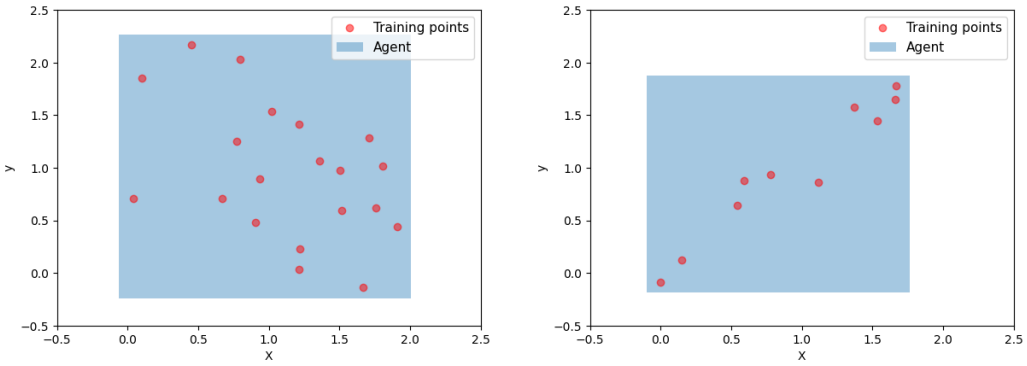
Alongside transparency, the multi-agent nature of oCELL provides functional advantages for regression. Unlike previous works on multi-agent systems based on an ensemble of local expert agents [77], oCELL embraces agent overlaps instead of avoiding them. We argue that for continuous regression tasks, these overlaps are highly meaningful as the consensus derived from multiple local experts helps to smooth the approximated function. This smoothness is an interesting property for using oCELL in Model Predictive Control (MPC), especially when using gradient-based optimizers that require smooth transitions between local models.

Despite those encouraging results, several structural bottlenecks were identified during the experimental phase. These limitations mainly highlight a tension between the simplicity of the orthotope representation of areas of expertise and the complexity of real-world data manifolds.

A primary limitation lies in the **uniform knowledge assumption** on which oCELL is based. oCELL assumes *an agent is equally competent across its entire area of expertise*

delimited by an orthotope ($\mathcal{H}_i$). In practice, the reliability of an agent typically decays toward its boundaries. Currently, agents in oCELL have binary activation functions, meaning that any activated agent is weighted equally in their contribution to the final prediction. To overcome this, weighting approaches based on Euclidean distance to the agent's centroid were explored. However, it did not yield significant improvements. This suggests that a more nuanced weighting should account for both the agent's centroid and the specific geometric support of its area of expertise.

A second limitation lies in the representation of complex data manifolds by orthotopes. Even if using orthotopes is computationally efficient for shape update operations like expansion or retraction, their limited degrees of freedom prevent accurate modeling of intricate manifold geometries. The area of expertise of an agent may occupy a large orthotope, while its training data populate only a small manifold (i.e. a subregion). This can lead to overly broad expertise claims as the model may report high coverage index (p_{coverage}) in regions where it has never actually seen data, as illustrated in Figure 3.14 (side by side comparison of an overgeneralizing agent and a representative agent by visualizing their respective orthotopes (*represented by a blue rectangle*) and training data (*represented by red dots*)). Accurate spatialization and potentially richer base shapes are therefore needed to capture local data structures and prevent overgeneralization.



(a) Representative orthotope (b) Overgeneralizing orthotope

Figure 3.14: Side by side comparison of a representative agent 3.14a against a overgeneralizing agent 3.14b. The *blue rectangle* corresponds to the orthotope associated to the activation function of the agent. Each *red dot* corresponds to a training point seen by the agent.

In addition to that, the social rules of oCELL governing the behavior of agents have raised challenges regarding population stability and resource management. Specifically, the agent destruction mechanism has been proved insufficient to regulate the ensemble of local experts, because it results in local redundancy where so-called parasitic agents persisted without contributing to global accuracy.

Moreover, this problem is exacerbated by the fact that agent creation is done with a single point, slowing local convergence and increasing the likelihood of redundant tiling across the feature space. Then, from a computational perspective, maintaining individual memory buffers for each agent represents a significant overhead that puts a strain on the potential for scaling up to long-term adaptation in real-world conditions.

Furthermore, oCELL treats incoming samples as independent observations, failing to exploit the temporal correlations inherent to data streams. This lack of temporal awareness limits the sample efficiency of the model and its ability to adapt to non-stationarity dynamic environments.

Finally, several hyperparameters remain difficult to tune, particularly the initial side lengths of created agents and the error thresholds (τ_{good},τ_{bad}). These parameters require individual tuning for each input dimension while having a significant impact on final performance, making it hard to scale to higher dimensional inputs.

Taken together, these limitations expose a broader structural issue. oCELL relies on spatialization and prediction aggregation mechanisms that are too rigid. The orthotope representation with hard boundaries is not tailored to support reliable interpolation when agents overlap too much or specialize unevenly. These limitations also highlight the need for inference mechanisms that weigh agents more smoothly according to their estimated expertise level rather than using discrete activation values (i.e. $\phi_i(x) = 1$ for activated and $\phi_i(x) = 0$ for non-activated). These considerations motivate the design of a new CELL system in which spatialization, inference, and adaptation are grounded into smoother data representations.

3.3 kCELL: CELL with Kernel Spatialization

The previous section introduced oCELL, the first instantiation of the CELL formalism. The development of oCELL allowed to establish that a self-organizing population of agents governed by social rules could successfully decompose complex regression tasks into specialized areas of expertise. It showed that the spatial organization of these agents provides valuable introspection signals related to aleatoric and epistemic uncertainties as well as to the traditional notions of interpretability and explainability in Machine Learning. However, oCELL's reliance on rigid orthotope-based areas of expertise limits its ability to build faithful and accurate areas of expertise. While being computationally efficient and logically convenient, this approach becomes a constraint when there is a need for fast adaptation in a non-stationary online setting.

To overcome these constraints, this section introduces kCELL (kernel CELL), the second instantiation of the CELL paradigm presented in this thesis. As oCELL focused on discrete spatial partitioning, kCELL shifts toward a smoother, continuous representation of areas of expertise. Each agent in kCELL uses an activation function in the form of a Radial Basis Function (RBF) kernel.

Specifically, an RBF is a real-valued function whose output depends on the distance from a central point. An example is the Gaussian form:

$$\phi(x) = \exp\left(-\beta \|x - c\|^2\right) \tag{3.13}$$

where c is the central point and $\beta \in \mathbb{R}^+$ determines the kernel width. By defining c as a parameter of the agent's activation function, the kernel value provides a continuous similarity metric between input features and agents, drawing inspiration from the gating mechanisms of RBF Networks [154] and Mixture-of-Experts (MoE) [247]. This change enables the ensemble of local expert agents to express spatial structures more finely, with more degrees of

freedom than what is permitted with orthotopes, and it breaks the assumption of knowledge uniformity over the areas of expertise of agents that were used in oCELL.

In fact, it leads to smoother aggregation of predictions in which agents contribute to predictions proportionally to their kernel-defined relevance, i.e proportionally to their estimated expertise level. This brings the inference closer to the probabilistic weighting strategies used in Gaussian Processes [202].

In addition to that, kCELL represents a step forward the complexification of the social dimension of CELL systems. As oCELL agents cooperate primarily through global context, kCELL introduces more direct social feedback mechanisms. Agents no longer act as isolated local models solely influenced by the social context, but they receive social feedback depending directly on their utility within a local collective of agents. This reinforces the social processes, as agents are informed by their social proximity and relative expertise levels.

These improvements result in improved interpolation quality, more faithful data representation, more stable learning dynamics and a reduced hyperparameter sensitivity. Thus it makes kCELL well-suited for use in control tasks as discussed in later chapters.

The remainder of this section is structured as follows. First, Section 3.3.1 describes the internal structure of kCELL local expert agents including how they are spatialized and their prediction mechanisms. Then, Section 3.3.2 introduces the learning rules that govern the self-organizing behavior of the agents, leading to the emergence of learning from the ensemble. Section 3.3.3 evaluates kCELL's performance in an online stationary modeling task and Section 3.3.4 presents a study of its capabilities in terms of adaptation in non-stationary environments. Finally, Section 3.3.5 provides a critical analysis of kCELL and details the limitations that still subsist despite the improvements.

3.3.1 Local Expert Agents

This section describes the tripartite structure of kCELL's local expert agents. Each agent is defined as:

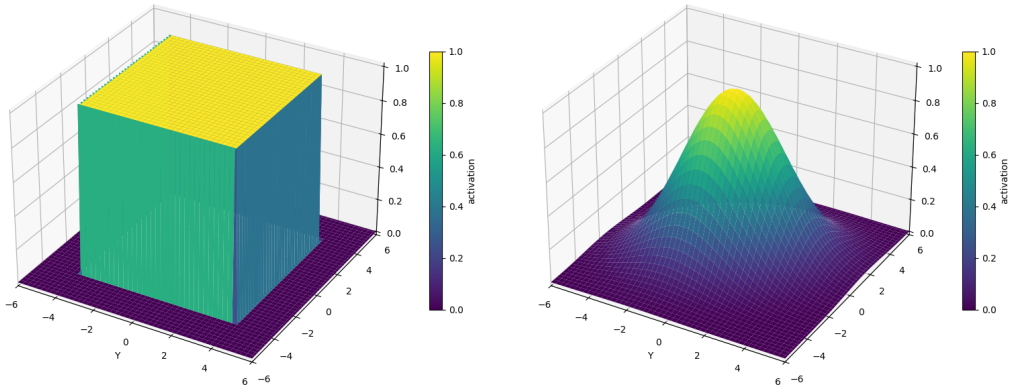
$$\mathcal{A}_i = \{\phi_i, f_i, \mathcal{M}_i\}$$

where ϕ_i denotes its activation function, f_i its internal model, and $\mathcal{M}_i$ its internal state. The agent $\mathcal{A}_i$ is a specialized local expert that represents its area of expertise through a smooth RBF kernel, maintains a local linear model, and manages an internal state to track its performance.

Unlike oCELL which uses a binary activation function, the area of expertise of local expert agents in kCELL are spatialized differently as illustrated in Figure 3.15. A RBF kernel is used as a continuous and smooth activation function to represent the area of expertise in feature space. Compared to using orthotope with hard boundaries, this kernel-based spatialization unlocks more degrees of freedom and makes the learned global function smooth and differentiable. Thus, it makes it more optimization-friendly for use control tasks.

A local expert agent is spatialized in the input space by the mean $\mu_i \in \mathbb{R}^d$ and the covariance matrix $\Sigma_i \in \mathbb{R}^{d\times d}$ of the distribution of its training points. The **activation function** $\phi_i : \mathbb{R}^d \mapsto]0, 1]$ for a given input $x \in \mathbb{R}^d$ is given by the RBF kernel value:

$$\phi_i(x) = \exp\left(-\frac{d_M(\mathcal{A}_i, x)^2}{2l^2}\right) \tag{3.14}$$



(a) Orthotope Activation Surface (oCELL) (b) RBF Activation Surface (kCELL)

Figure 3.15: Shapes in 2D of activation surface of oCELL (3.15a) and kCELL (3.15b) agents. The yellow color corresponds to highly activated regions and blue color corresponds to lower activated regions.

with l the lengthscale of the kernel and $d_M(\mathcal{A}_i, x)$ a measure of similarity between an agent $\mathcal{A}_i$ and an input x. In kCELL this similarity metric is defined as the Mahalanobis distance such as

$$d_M(\mathcal{A}_i, x) = \sqrt{(x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i)} \tag{3.15}$$

In this context, the activation value can be interpreted as the **likelihood that the agent has previously been trained on points similar to** x, reflecting its degree of knowledge. Contrary to oCELL, the knowledge uniformity hypothesis is discarded. Expertise is assumed to be maximized on the center of the agent's area of expertise. Thus, contrary to oCELL, agents in kCELL are constructed based on the assumption of decreasing knowledge:

Assumption 3.2 (Decreasing Knowledge)**.** The level of expertise of an agent is assumed to decrease as the distance from its center increases.

Even with this continuous representation, some measurable mathematical quantities of oCELL can be translated to kCELL such as the notion of volume which remains relevant. As the area of expertise of an agent is not clearly bounded, its volume is tied to the volume of a confidence ellipsoid. The volume $V(\mathcal{A}_i)$ is proportional to the square root of the determinant of the covariance matrix [238]

$$V(\mathcal{A}_i) \propto \sqrt{\det(\Sigma_i)} \tag{3.16}$$

As an instantiation of the CELL formalism, kCELL is model-agnostic but is designed to work with low-complexity models to ensure fast updates and local convergence. Each agent $\mathcal{A}_i$ maintains a local linear regressor as the **internal model** f_i to predict:

$$\hat{y}_i = f_i(x) = \Theta_i \begin{bmatrix} x \\ 1 \end{bmatrix} \tag{3.17}$$

where $\Theta_i \in \mathbb{R}^{m \times (d+1)}$ is the parameter matrix representing the local affine transformation.

The **internal state** $\mathcal{M}_i$ tracks the number of point n_{seen} used to update the shape of the local expert agent $\mathcal{A}_i$. Unlike oCELL's buffer memory, kCELL is designed to be lightweight by retaining as few points as possible. Additionally, $\mathcal{M}_i$ stores a running confidence value $\bar{C}_{\mathcal{A}_i} \in [-1, 1]$ which corresponds to an exponential moving average of the normalized social feedback received by the agent. This confidence value tracks the agent's predictive performance relative to its peers and is used for the destruction mechanism. The precise feedback mechanism is detailed later in Section 3.3.2.

To transform the population of local expert agents into a global model, kCELL uses a spatial gating mechanism to select relevant agents for a given input x. Unlike oCELL, the continuous nature of the activation makes the notion of activated agents blurry. For this reason, the spatial gating mechanism focuses on selecting one subset of local expert agents:

Neighbor Agents ($\mathcal{D}_{\text{neighbors}}$) contains agents for which x is likely to belong to the training dataset, such as $d_M(\mathcal{A}_i, x)^2 \leq \chi^2_{d,\frac{\xi}{100}}$ where $\chi^2_{d,\frac{\xi}{100}}$ denotes the ξth percentile of the chi-squared distribution with n degrees of freedom. The value of ξ belongs to the hyperparameters of the system that needs to be set beforehand.

As stated earlier, kCELL discards the knowledge uniformity hypothesis to better represent the knowledge encapsulated in the ensemble of local expert agents. Through the defined activation function, the further a point is from the agent's center, the lower its activation value. In this continuous context, local expert agents should have an impact on the global prediction that is proportional to their degree of expertise about the input. The global prediction $\hat{y}$ is obtained through a soft-weighted prediction mechanism. Let $\mathcal{D}_k$ denote the index set of the k-closest agents in feature space. The final prediction is given by

$$\hat{y} = \sum_{i \in \mathcal{D}_k} w_i(x) f_i(x) \tag{3.18}$$

where the normalized contribution weights $w_i(x)$ are defined as

$$w_i(x) = \frac{\phi_i(x)}{\sum_{j \in \mathcal{D}_k} \phi_j(x)} \tag{3.19}$$

This formulation ensures that each contributing agent's influence on the final prediction is proportional to its estimated competence.

To maintain its local expertise as new data arrives, each agent $\mathcal{A}_i$ can perform two actions: update the shape of its area of expertise through the parameters of the activation function ϕ_i (Shape Update), or refine its internal model f_i (Model Update).

Shape Update In kCELL, the shape update operation is expressed purely as a mean and covariance estimation problem. Each time a point is ingested by agent $\mathcal{A}_i$, its mean μ_i and covariance matrix Σ_i are updated incrementally using Welford's algorithm [236]

$$\mu_{i\,|\,t+1} = \mu_{i\,|\,t} + \frac{x - \mu_{i\,|\,t}}{n_{\text{seen}} + 1} \tag{3.20}$$

$$\Sigma_{i\,|\,t+1} = \frac{1}{n_{\text{seen}}} \left(\Sigma_{i\,|\,t} + \left(x - \mu_{i\,|\,t}\right) \left(x - \mu_{i\,|\,t+1}\right)^\top \right) \tag{3.21}$$

where n_{seen} is the number of points used by the agent to update its shape. Thus the agents can adapt their area of expertise incrementally without maintaining a buffer of past observations.

Model Update An agent $\mathcal{A}_i$ can update its internal model. To ensure efficiency in online settings and avoid the need for a memory buffer, the parameter matrix Θ_i is updated using Recursive Least Squares (RLS) [109]. RLS naturally handles the notion of forgetting and for a forgetting value of 1 (i.e no forgetting), it converges rapidly to producing the same results as Ordinary Least Squares (OLS). This way, the agents provide a first-order approximation of the dynamics that converges as new data is ingested sequentially from the data stream.

3.3.2 Learning Rules

The previous section detailed the tripartite structure of local expert agents in kCELL (with ϕ_i, f_i, and $\mathcal{M}_i$) that are spatialized using continuous smooth RBF kernels. This section follows this line of thought, describing the social learning rules that govern the collective behavior of the ensemble. While oCELL relies on binary coverage and fixed error thresholds, kCELL introduces a feedback mechanism based on social relative performance and baseline benchmarking. Thus the agents have to *cooperate* to refine the global prediction while *competing* for social survival.

As described in the CELL formalism, the self-organization of the ensemble of local expert agents in kCELL is governed by rules that determine how this ensemble processes the sequential stream of feature-target tuples (x_{new}, y_{new}). The overall flow of the training process is formalized in Algorithm 3. These rules translate the individual performance of agents within a specific social context into discrete actions: updating its internal model, adapting its shape or managing the population through creation and destruction. In order to track an agent's reliability over time, kCELL leverages two types of feedback signals that assess the quality of an agent's contribution.

Ensemble Feedback ($N_{\text{neighbors}} > 1$) When several agents are neighbors of x_{new}, they must cooperate locally to refine the group approximation. The ensemble feedback is defined as the fractional change in error $\Delta_{\mathcal{A}_i}$ when an agent $\mathcal{A}_i$ is left out of the global prediction process

$$\Delta_{\mathcal{A}_i} = \frac{E_{-i} - E}{E} = \frac{|\hat{y}_{-i} - y_{new}| - |\hat{y} - y_{new}|}{|\hat{y} - y_{new}|} \tag{3.22}$$

where E is the prediction error and $E_{-i} = |\hat{y}_{-i} - y_{new}|$ is the prediction error calculated without $\mathcal{A}_i$'s proposal. $\Delta_{\mathcal{A}_i} > 0$ indicates that the agent is a *strong* expert contributing to error reduction while $\Delta_{\mathcal{A}_i} < 0$ identifies *weak* experts.

Baseline Feedback ($N_{\text{neighbors}} \leq 1$) When an agent is alone or no neighbors exist, social comparison is impossible. Instead, the feedback $\Delta'_{\mathcal{A}_i}$ is computed relative to a baseline prediction y_{base}, provided by a running short-term linear predictor that is regularly fitted with Ordinary Least Squares (OLS) on a sliding window buffer $\mathcal{B}_{\text{short}}$ of the most recent data

points. The baseline feedback is defined as

$$\Delta'_{\mathcal{A}_i} = \frac{E_{base} - E_i}{E_{base}} = \frac{|y_{base} - y_{new}| - |\hat{y}_i - y_{new}|}{|y_{base} - y_{new}|} \tag{3.23}$$

where E_{base} is the prediction error of the short term linear predictor, $E_i = |\hat{y}_i - y_{new}|$ is the prediction error of agent $\mathcal{A}_i$. This value assesses whether the agent is more relevant than a simple local trend.

These two feedback signals are used to update the running confidence $\bar{C}_{\mathcal{A}_i}$ stored in the agent's state memory $\mathcal{M}_i$. First, an instantaneous confidence $c_i \in [-1, 1]$ is computed:

$$c_{\mathcal{A}_i} = \tanh\left(k_{\text{steep}} \times \begin{cases} \Delta_{\mathcal{A}_i} & \text{if } N > 1 \\ \Delta'_{\mathcal{A}_i} & \text{else} \end{cases}\right) \tag{3.24}$$

The running confidence is then updated from the value of $c_{\mathcal{A}_i}$ via an exponential moving average

$$\bar{C}_{\mathcal{A}_i\,|\,t+1} = (1 - \lambda)\,\bar{C}_{\mathcal{A}_i\,|\,t} + \lambda c_{\mathcal{A}_i} \tag{3.25}$$

where λ is the smoothing factor, $\bar{C}_{\mathcal{A}_i\,|\,t}$ is the current value of the running confidence and $\bar{C}_{\mathcal{A}_i\,|\,t+1}$ is the next one.

The learning rules are triggered by conditions on the social context and the feedback signals. These rules allow local groups of agents to continuously improve by refining weaker agents (keeping them mobile and locally adaptable), while making stronger agents specialize further within their current area of expertise. The following paragraphs detail the learning rules.

Inaccuracy When $N_{\text{neighbors}} > 1$, several agents are theoretically considered as experts to predict for x_{new}. However, each one of them possesses a different level of expertise for x_{new} depending on the value of their activation function $\phi_i(x)$. The ensemble feedback $\Delta_{\mathcal{A}_i}$ distinguishes those who are currently stronger from those who are weaker. Weaker agents ($\Delta_{\mathcal{A}_i} < 0$) perform a **Model Update** action to refine their local expertise. Conversely, strong agents ($\Delta_{\mathcal{A}_i} > 0$) are already performing well and do a **Shape Update** action to strengthen their local anchoring and improve the collective knowledge.

Specialization When $N_{\text{neighbors}} = 1$, the only neighbor agent cannot be compared to other experts. In this case, it is compared to a short-term linear baseline predictor. If the agent outperforms the baseline ($\Delta'_{\mathcal{A}_i} > 0$), it is considered a relevant local expert and performs both a **Model Update** and a **Shape Update** action. This dual update allows the agent to specialize in its current region by refining its parameters while moving its area of expertise closer to x_{new}. Conversely, if the agent is beaten by the baseline ($\Delta'_{\mathcal{A}_i} < 0$), it remains idle, waiting to reposition itself through future points or for another agent to enter the area to initiate social cooperation.

Incompetence When $N_{\text{neighbors}} = 0$, no agent can be considered as a neighbor of x_{new}. The learning system identifies a lack of local knowledge around x_{new}. In this context, the baseline feedback $\Delta'_{\mathcal{A}_i}$ is calculated for the closest agent within the ensemble. If this agent

outperforms the short-term linear baseline ($\Delta'_{\mathcal{A}_i} > 0$), it performs a **Model Update** and a **Shape Update** to extend its area of expertise toward x_{new} and include the point. However, if the closest agent is outperformed by the baseline ($\Delta'_{\mathcal{A}_i} < 0$), an **Agent Creation** meta-action is triggered. A new local expert agent is spawned, centered on x_{new}. To accelerate local convergence and to avoid initializing with a single point, the new agent is initialized from a short-term sliding window buffer $\mathcal{B}_{\text{ini}}$containing the N_{ini} most recent observations seen by the system. The buffer $\mathcal{B}_{\text{ini}}$ is cleared at each **Agent Creation** meta-action to avoid redundant agent creation. An agent cannot be created until $\mathcal{B}_{\text{ini}}$is full.

Uselessness To prevent uncontrolled population growth and mitigate the proliferation of agents that do not contribute to predictive accuracy, a destruction mechanism based on social survival is used. Unlike oCELL which uses a volume-based threshold, kCELL relies on the running confidence $\bar{C}_{\mathcal{A}_i}$ stored in $\mathcal{M}_i$. If an agent consistently receives negative feedback by either being the weakest of a local group, or by failing to beat the short-term baseline, its confidence will gradually decrease. When $\bar{C}_{\mathcal{A}_i}$ falls below a predefined threshold $\tau_{\text{confidence}}$, the agent is considered useless and an **Agent Destruction** action is triggered. This ensures that the ensemble of local expert agents remains efficient and that only the most reliable experts survive.

Table 3.7 summarizes these learning rules by mapping social contexts and conditions to the corresponding triggered action or meta-action.

	Social Context	**Condition**	**Targeted Set**	**Action**
Incompetence	$N_{\text{neighbors}} = 0$	$\Delta'_{\mathcal{A}_i} > 0$	Closest	Model + Shape Update
		$\Delta'_{\mathcal{A}_i} < 0$	Closest	Agent Creation
Specialization	$N_{\text{neighbors}} = 1$	$\Delta'_{\mathcal{A}_i} > 0$	$\mathcal{D}_{\text{neighbors}}$	Model + Shape Update
		$\Delta'_{\mathcal{A}_i} < 0$	$\mathcal{D}_{\text{neighbors}}$	$\varnothing$
Inaccuracy	$N_{\text{neighbors}} > 1$	$\Delta_{\mathcal{A}_i} > 0$	$\mathcal{D}_{\text{neighbors}}$	Shape Update
		$\Delta_{\mathcal{A}_i} < 0$	$\mathcal{D}_{\text{neighbors}}$	Model Update
Uselessness		$\bar{C}_{\mathcal{A}_i} \leq \tau_{\text{confidence}}$	$\mathcal{D}_{\text{neighbors}}$	Agent Destruction

Table 3.7: Learning rules of kCELL. Each line corresponds to a given learning rule defined by: 1) a social context representing a condition on the number of local experts $N_{\text{neighbors}}$ around a new input x_{new}; 2) a condition on one of the two types of feedback $\Delta_{\mathcal{A}_i}$and $\Delta'_{\mathcal{A}_i}$ or on the running confidence of agents $\bar{C}_{\mathcal{A}_i}$; 3) a target subset of agents that will be affected by the rule (e.g Closest the subset of agents containing only the agent which is closest to x_{new}); and 4) the actions or meta-actions to perform on the agents of the targeted subset.

To visualize the result of a learning and local cooperation of agents, a 1-dimensional example can be used. A small synthetic dataset is generated from the function $f(x) = \sin(x) + \epsilon\mathcal{N}(0,1)$ with $\epsilon \in \mathbb{R}$ the noise scaling factor. Then, online learning is simulated by sequentially feeding the feature-target tuples (x_{new}, y_{new}) one at a time to kCELL. Each point is only seen once during learning. Figure 3.16 presents a visualization of the spatial organization of agents (3.16a) and the resulting predictions (3.16b). We can observe that the agents overlap and do not fit perfectly across their entire area of expertise. At the edges of agents the error is higher than in the center. This is the expected behavior resulting from

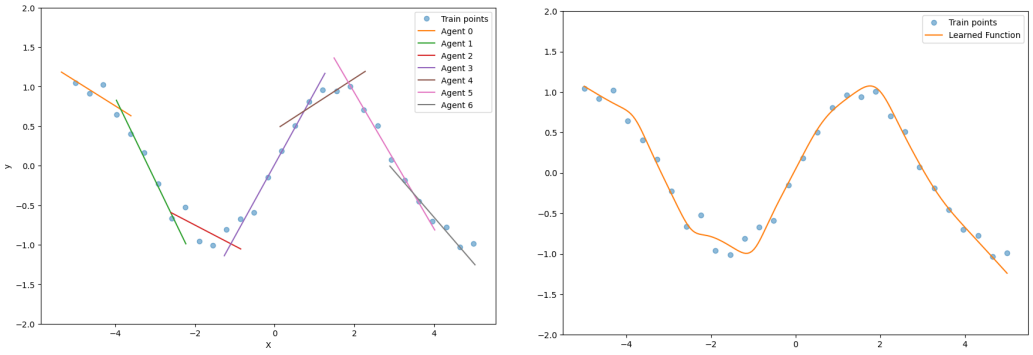
Algorithm 3: kCELL

for each (x_{new}, y_{new}) in the data stream **do**
  $\mathcal{D}_{\text{neighbors}}, \mathcal{A}_{\text{closest}} \leftarrow$ Gating Mechanism(x_{new})
  $N_{\text{neighbors}} \leftarrow |\mathcal{D}_{\text{neighbors}}|$
  `// Incompetence`
  if $N_{\text{neighbors}} = 0$ **then**
    Compute $\Delta'_{\mathcal{A}_{\text{closest}}}$
    if $\Delta'_{\mathcal{A}_{\text{closest}}} > 0$ **then**
      Model and Shape Update
    end
    else
      Create Agent from $\mathcal{B}_{\text{ini}}$
    end
  end
  `// Specialization`
  if $N_{\text{neighbors}} = 1$ **then**
    Compute $\Delta'_{\mathcal{A}_{\text{neighbor}}}$
    if $\Delta'_{\mathcal{A}_{\text{neighbor}}} > 0$ **then**
      Model and Shape Update
    end
  end
  `// Inaccuracy`
  if $N_{\text{neighbors}>1}$ **then**
    $\forall \mathcal{A}_i \in \mathcal{D}_{\text{neighbors}}$, Compute $\Delta_{\mathcal{A}_i}$
    for $\mathcal{A}_i \in \mathcal{D}_{\text{neighbors}}$ **do**
      if $\Delta_{\mathcal{A}_i} > 0$ **then**
        Shape Update
      end
      if $\Delta_{\mathcal{A}_i} < 0$ **then**
        Model Update
      end
    end
  end
  `// Uselessness`
  for $\mathcal{A}_i \in \mathcal{D}_{\text{neighbors}}$ **do**
    if $\bar{C}_{\mathcal{A}_i} < \tau_{\text{confidence}}$ **then**
      Destroy $\mathcal{A}_i$
    end
  end
end

the assumption of non-uniformity in the knowledge within areas of expertise. We observe that the overall predictions approximate closely the function considering the noise added to the data. It demonstrates kCELL's ability to generalize effectively by interpolation, and it highlights the usefulness of a soft-weighted aggregation function.

Finally, from an implementation perspective, the learning behavior in kCELL is governed by the following hyperparameters:

- $\xi \in [0, 100]$: The chi-squared percentile defining a threshold for the gating mechanism to extract the set of neighbor agents from the global population.
- $k_{\text{steep}} > 0$: The steepness of the tanh function for social feedback normalization.
- $\lambda \in]0, 1]$: The smoothing factor for the running exponential moving average of confidence value of agents.
- $\tau_{\text{confidence}} \in [-1, 1]$: The survival threshold for agent destruction. If an agent's confidence goes below this value, it is destroyed.
- $N_{\text{short}} \in \mathbb{N}^+$: The size of the $\mathcal{B}_{\text{short}}$ used to fit the linear baseline predictor.
- $N_{\text{ini}} \in \mathbb{N}^+$: The minimal number of points inside $\mathcal{B}_{\text{ini}}$ needed to create a new agent. With linear regressions, N_{ini} should ideally be greater than the number of input dimensions such as $N_{\text{ini}} > d + 1$.
- $k_{\text{social}} > 0$: The number of closest agents used to compute the social baseline y_{base} when an agent has no neighbors ($N_{\text{neighbors}} = 1$).



(a) Spatial organization (b) Resulting prediction

Figure 3.16: Local cooperation of agents in predicting a noisy sinus function. (3.16a) presents the spatial organization of agents where each colored segment represents the mapping of one agent between feature space and output space. (3.16b) shows the resulting predictions obtained from the aggregation of agents local models, illustrating the interpolation capabilities of the model.

3.3.3 Online Stationary Modeling

The previous sections detailed the tripartite structure of kCELL agents and the social learning rules that govern their collective behavior. This sections moves kCELL from the theoretical formalism to empirical validation by evaluating its capabilities on an online stationary modeling task. Using a high-dimensional robotic benchmark, we demonstrate that the proposed social learning rules allow the ensemble of local expert agents to self-organize into an accurate representation of complex dynamics by only seeing each data point once.

Experimental Setup The experiment is conducted on the SARCOS dataset, a standard benchmark for multi-dimensional robot inverse dynamics [228].

Definition 20. Inverse dynamics modeling is the construction of a model that maps a dynamical system's motion (positions, velocities, accelerations) to the internal forces or actions (e.g. joint torques) required to produce that motion.

The task consists in mapping a 21-dimensional input space (comprising 7 joint positions, 7 velocities and 7 accelerations) to the corresponding joint torques of a seven degrees-of-freedom anthropomorphic robot arm (illustrated in Figure 3.17). The dataset contains 44484 training examples and 4449 test examples. A single pass over the training dataset is performed where each feature-target tuple is presented to the models one at a time and discarded after processing. No feature scaling is applied to the data.

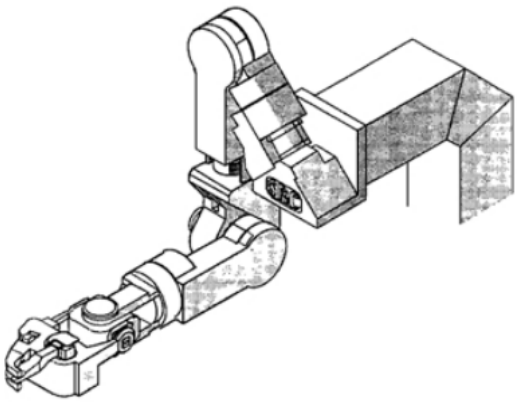


Figure 3.17: SARCOS 7 degrees-of-freedom robot arm as presented in [228]

kCELL is compared to several standard online learning algorithms:

- **k-Nearest Neighbors (kNN)**: a lazy local learning algorithm [246].
- **Linear Regression**: Optimized with Adam optimizer [138]
- **Adaptive Hoeffding Trees**: An online decision tree variant [64].
- **Mondrian Forest**: A highly efficient adaptation of forest tree ensembles for online learning [145].

– **Locally Weighted Projection Regression (LWPR)**: The most direct competitor of kCELL as it uses a similar local linear model spatialization but employs different learning logic [229].

To implement Adaptive Hoeffding Trees, Mondrian Forest, Linear Regression with Adam optimizer and kNN, the python library *RiverML* is used [167]. For LWPR, the original python C-bindings are used. Performing an exhaustive hyperparameter optimization on the whole dataset is impracticable because it is too computationally heavy. For this reason, hyperparameter optimization was conducted via grid search on the first 5000 steps of the stream to minimize the Area Under the Curve (AUC) of the prequential error. The *prequential error* corresponds to the error measured for a new point of the data stream before learning on it.

The performance of the models is evaluated through adaptation speed, measured by prequential error, and accuracy measured by the Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) on the fixed test set.

	Prequential Error AUC	Test RMSE	Test MAE
kNN	2.10	7.42	5.59
Linear Regression (Adam)	1.07	5.62	4.46
Adaptive Hoeffding Tree	2.90	8.44	6.45
Mondrian Forest	**0.70**	2.64	1.86
LWPR	1.04	3.73	2.77
kCELL (ours)	1.72	**1.94**	**0.91**

Table 3.8: Summary table of the final results of the SARCOS comparative experiment. The reported values are the final values obtained after a single pass through the entire dataset. Values in bold correspond to the best value among the different algorithms.

Prequential Error Analysis Figure 3.18 illustrates the evolution of the prequential error smoothed with a window size of 2000 steps. The final AUC for each model is summarized in Table 3.8 in column *Prequential Error AUC*.

As shown in Figure 3.18, the prequential error curve of kCELL remains systematically above those of Mondrian Forest, LWPR and Linear Regression throughout the entire duration of the experiment. This tendency is reflected in the final AUC of 1.72 for kCELL which is significantly higher than the top-performing Mondrian Forest (0.70). This systematic gap in prequential performance seems to highlight a difficulty to extrapolate. This behavior might be linked to the kCELL's implementation of the prediction mechanism in unexplored regions. As the inputs are 21-dimensional, it is highly frequent that a new input x_{new} yields a very low activation score for all agents. Indeed, the Mahalanobis distance used in the activation function of agents is increasingly stretched as the number of dimensions increases. To maintain numerical stability in implementation and avoid dividing by 0 when normalizing the aggregation weights for the global prediction, the individual activation scores are clipped to a minimal value. It leads all closest agents to have the same weight in prediction despite being really far from each other and from x_{new}. This can result in higher errors in uncovered

area of input space. This will be confirmed by further observations of the asymptotic behavior of kCELL.

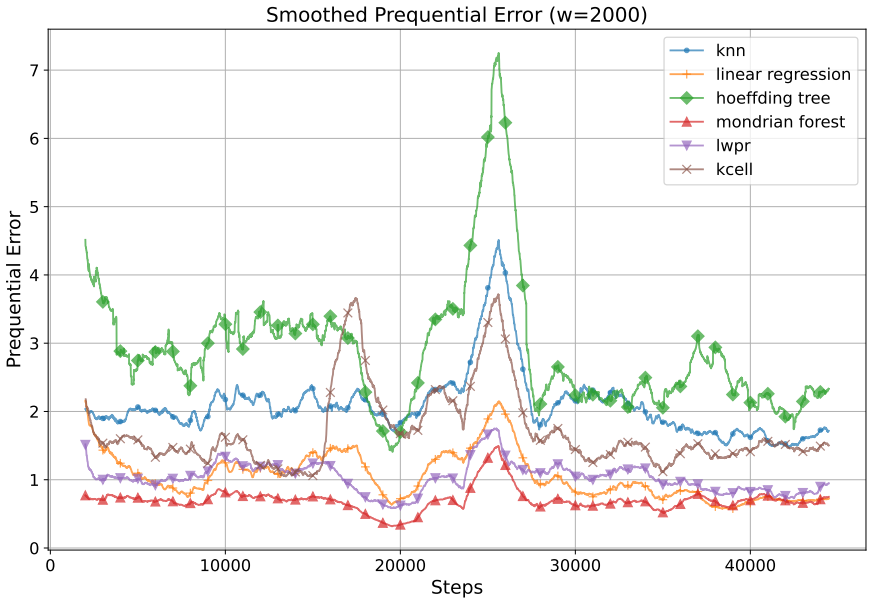


Figure 3.18: Evolution of the prequential error during the learning phase. Each curve is smoothed with a sliding window of 2000 steps of length. y-axis corresponds to the absolute prequential error value and x-axis corresponds to the number of traning steps.

Test Set Accuracy Analysis Figure 3.19 and Figure 3.20 respectively display the evolution of the RMSE and MAE on the test set throughout learning. It can be noted that in the initial 20000 steps, kCELL has a higher RMSE and MAE than all other algorithms. However, the RMSE and MAE decrease consistently as more data is processed. kCELL's error curve crosses the others at approximately 25000 steps for MAE and 35000 steps for RMSE. As shown in Table 3.8, kCELL achieves the lowest final error with an MAE of 0.91 and an RMSE of 1.94.

While kCELL exhibits a higher prequential error during the learning phase, its final performance on the test set is superior to the other algorithms. The steady decrease of the error indicates that the ensemble of local expert agents is effectively organizing into a meaningful representation of the inverse dynamics as the agent population tend to stabilize. kCELL achieves a lower MAE than the other algorithms earlier than it does for RMSE. It suggests that, while kCELL is highly accurate on average, its performance is affected by occasional large errors, to which RMSE is more sensitive.

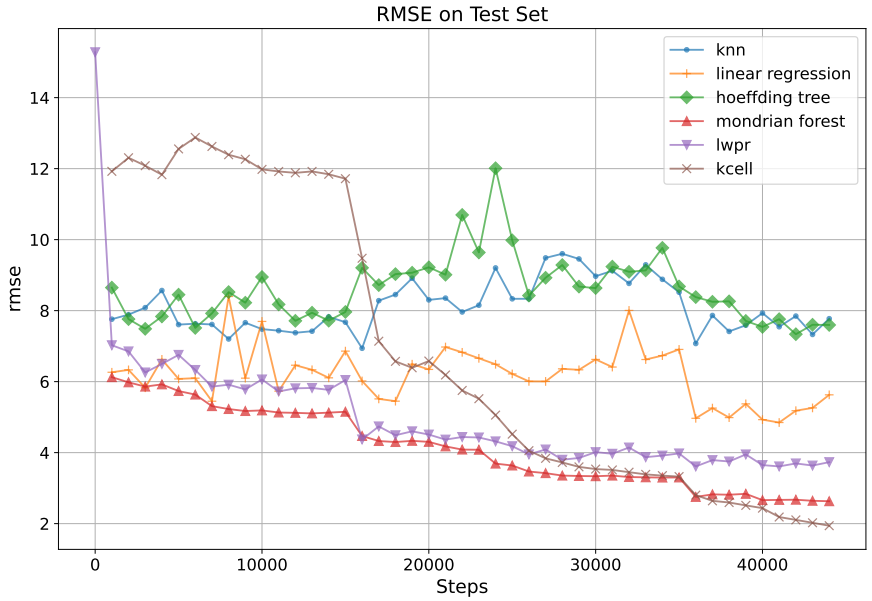


Figure 3.19: Evolution of test RMSE throughout learning. *y*-axis corresponds to the RMSE and *x-axis* corresponds to the number of training steps.

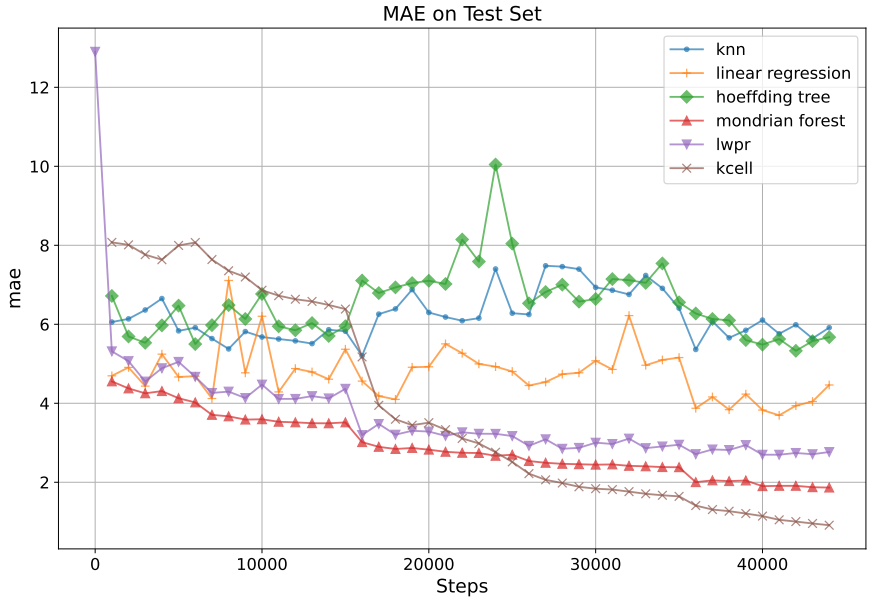


Figure 3.20: Evolution of test MAE during learning. *y*-axis corresponds to the MAE and *x-axis* corresponds to the number of training steps.

Correlation of Activation to Errors To investigate the reliability of the local expert agents, the relationship between the activation values $\phi_i(x)$ and the prediction error is evaluated. Figure 3.21 presents a plot of the prediction errors evaluated on the test set against the activation score of the closest agent.

The analysis reveals a strong negative Spearman correlation of -0.68 (with a p-value $p < 0.001$) meaning that the lower the activation, the higher the error. Indeed, lower activation values, particularly those below 0.2, correspond to significantly higher errors. As the activation value increases toward 1.0, the errors stabilize and reach their minimum. These results support the rejection of the *uniform knowledge* assumption (as in oCELL) in favor of the *decreasing knowledge* assumption (as in kCELL). The activation value represents the agent's level of expertise for a given input. This correlation confirms that the model is aware of its own local limitations through the smooth areas of expertise of local expert agents. This could provide an explanation for the previous observation regarding the differences between the behaviors of the MAE and RMSE curves. The larger errors produced by kCELL may be due to predictions in uncovered regions of the input space.

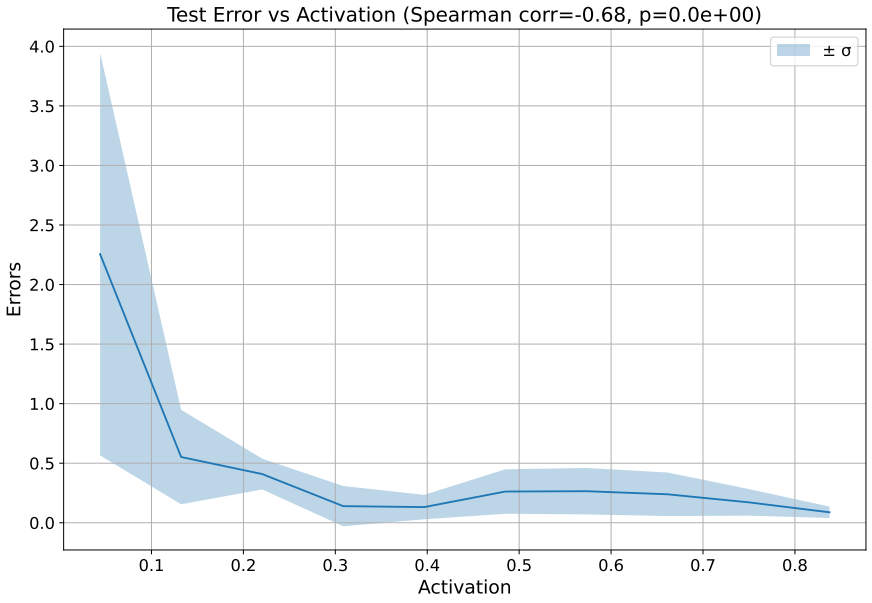


Figure 3.21: Relationship between the activation score $\phi_i(x)$ of local expert agents and the prediction errors on the test set. y-axis corresponds to the absolute error values and x-axis corresponds to the activation value of the closest agent. A Spearman correlation between error and activation equals to -0.68 is found with a p-value $p < 0.001$, suggesting a significant negative correlation.

Controlled Population Growth in kCELL The efficiency of the self-organization process is further analyzed by monitoring the evolution of the population of local expert agents. Figure 3.22 tracks the number of agents in the ensemble throughout the learning process compared to a theoretical maximum. The theoretical maximum is computed from the minimum number of points N_{ini} needed for the system to be able to create an agent. In kCELL, an agent can be created, at most, every N_{ini} steps because the $\mathcal{B}_{\text{ini}}$ buffer is cleared at each new agent creation.

The curve representing the number of agents remains strictly below the theoretical maximum. Furthermore, the gap between the actual number of agents and this theoretical

maximum increases over time. The growth rate of the population of local expert agents is visibly flattening as more data is processed.

This behavior demonstrates that kCELL is not systematically creating new models. Instead the model successfully evaluates its current knowledge map through the areas of expertise of agents, triggering the **Incompetence** rule only when a significant gap in the input space is identified. The flattening of the curve of the number of agent indicates a convergence toward a stable population. This highlights the effectiveness of the population management mechanism (**Uselessness** rule) which prunes redundant or underperforming agents by evaluating their social confidence.

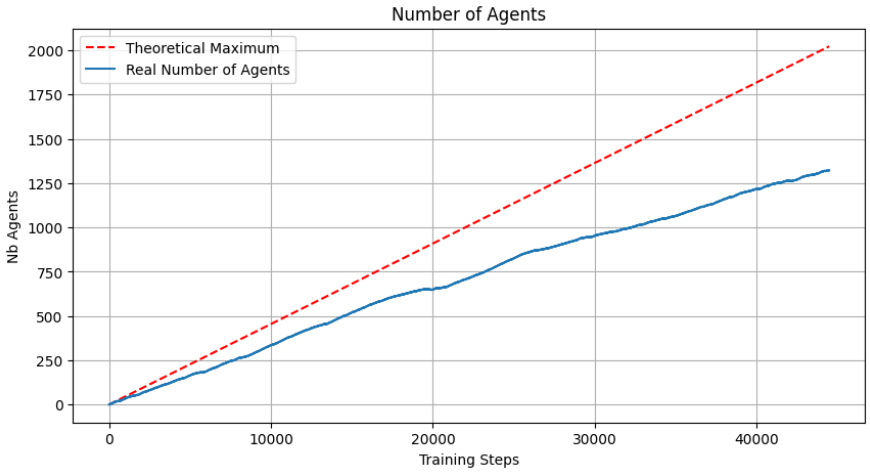


Figure 3.22: Number of agents throughout the learning phase. y-axis corresponds to the number of agents and y-axis corresponds to the learning steps. The red dotted line corresponds to the theoretical maximum number of agents that could be produced by the kCELL system.

Synthesis of the Results The results of this experiment validate the core of the design choices of kCELL for stationary, high-dimensional online modeling. The initial prequential error analysis shows a slower adaptation phase than Mondrian Forest or LWPR. It can be explained by the difficulty of building meaningful high-dimensional spatialization when Mahalanobis distance is stretched. However, in the long run, kCELL exhibits superior modeling performance. By achieving the lowest final MAE and RMSE, kCELL demonstrates that an ensemble of local expert agents coordinated through social learning rules can converge to a more accurate representation of complex robot dynamics than conventional statistical local learning or forest-based methods.

Furthermore, a strong negative correlation between activation scores and errors on predictions has been found. It turns the activation function of agents into a reliable metric for introspection in the model. This ability to quantify epistemic uncertainty on the fly is a useful property for control with learned models. It provides the necessary signals to determine when a prediction can be trusted and when the system is operating in an uncovered region of the input space. This ability will be explored in more detail in Chapter 4.

However, the SARCOS benchmark represents a stationary problem. The mapping is complex and high-dimensional but the underlying physical properties of the robot arm do not evolve during the learning phase. In many real-world applications, particularly in the context of Human-Robot Collaboration (HRC), the system must handle non-stationarity where the target function is susceptible to change over time. In this section, the online stationary modeling capabilities of kCELL have been established. The following section explores how the ensemble of local expert agents reacts within a non-stationary environment.

3.3.4 Adaptation to Non-Stationary Dynamics

The previous section demonstrated kCELL's ability to self-organize and build meaningful representations of complex dynamics to achieve high asymptotic accuracy (compared to baseline online algorithms like LWPR or Mondrian Forests) in a stationary high-dimensional environment. However, real-world control systems often operate in non-stationary conditions where the underlying physical properties or environmental constraints evolve over time. This section presents a study of kCELL's capabilities in terms of online adaptation in the presence of concept drifts.

Experimental Setup To evaluate how kCELL adapts to changing dynamics, two MuJoCo simulation environments [221] are used: **Inverted Pendulum** (cf. Figure 3.23a) and **Hopper** (cf. Figure 3.23b).

- **Inverted Pendulum**: This environment is based on the classic Cart Pole problem [20] presented in Chapter 2.1.1. It consists of a cart that moves with a pole attached to it. The goal is to balance the pole by applying horizontal forces to the cart. The action space is 1-dimensional and the state space is 4-dimensional.

- **Hopper**: The Hopper is a two-dimensional one-legged robot [71] consisting of four main body parts (torso, thigh, leg and foot). The goal is to move forward by applying torques to the three hinges connecting these parts. This represents a higher-dimensional task with a 3-dimensional action space and a 11-dimensional state space.

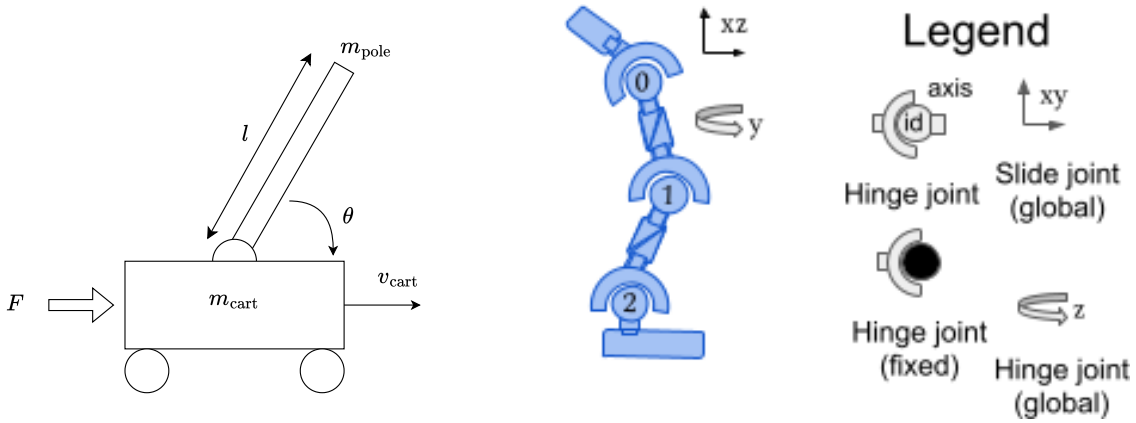
These two environments were chosen to contrast a low-dimensional setting (Inverted Pendulum) with a higher-dimensional one (Hopper). Mahalanobis distances become larger the higher the number of dimensions. Thus, we want to study the impact of increasing dimensionality on the representativeness of the spatialization of agents in kCELL which was one of the main limitations previously identified for oCELL (cf. 3.2.5).

In the context of this experiment the task to solve is a forward modeling task.

Definition 21. Forward dynamics modeling is the construction of a model that maps states (such as positions and velocities) and actions to future state trajectories.

The model learns online from a data stream to estimate the transition function f of the controlled dynamical system, defined by

$$s_{t+1} = f\left(s_t, u_t\right)$$



(a) (b)

Figure 3.23: **The Inverted Pendulum** and **Hopper** dynamical systems. (3.23a) corresponds to the **Inverted Pendulum** where m_{pole}, l, θ are respectively the mass, length and angle of the pole, m_{cart}, v_{cart} are the mass and horizontal velocity of the cart. Finally F correspond to the force applied to the cart which is the action to be controlled to stabilize this dynamical system. (3.23b) corresponds to the **Hopper** where 0 is the joint between the torso and thigh, 1 is the joint between thigh and leg and 2 is the joint between leg and foot.

where s_t is the current state of the dynamical system, s_{t+1} the next state, and u_t is the applied action,

To simulate non-stationarity, three datasets $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$ were collected under three different gravitational acceleration values g_i: *default* gravity ($g_1 = 9.81$), *low* gravity ($g_2 = 4$), *high* gravity ($g_3 = 20$). These datasets were collected using pretrained Reinforcement Learning (RL) controllers trained with Soft-Actor-Critic (SAC) algorithm [101] (with the *stablebaselines3* library implementation [186]). The characteristics of the datasets are summarized in Table 3.9.

Environment	State Dimensions	Action Dimensions	Dynamic Changes	Nb Learning Steps
Inverted Pendulum	4	1	$g_i \in \{9.81, 4, 20\}$	30k
Hopper	11	3	$g_i \in \{9.81, 4, 20\}$	50k

Table 3.9: Characteristics of datasets generated with different values of g. *State Dimensions* stands for the number of features in states, *Action Dimensions* stands for the number of continuous actions, *Dynamic Changes* corresponds the set of g gravitational acceleration values used to generate the datasets, *Nb Learning Steps* corresponds to the number of training steps (the budget) used to train the RL controllers used to generate each dataset (one RL controller per dataset).

Each dataset $\mathcal{D}_i$ is split into a training set $\mathcal{D}_i^{\text{train}}$ and a held-out test set $\mathcal{D}_i^{\text{test}}$. Throughout the learning process, the training datasets are streamed in a sequential order ($\mathcal{D}_1^{\text{train}}$ $\rightarrow \mathcal{D}_2^{\text{train}} \rightarrow \mathcal{D}_3^{\text{train}}$) to emulate abrupt changes in the underlying system dynamics (i.e. non-stationarities). The learned models are regularly evaluated against all three test datasets $\left\{\mathcal{D}_j^{\text{test}}\right\}_{j=1}^{3}$. By comparing the index i of the current training set $\mathcal{D}_i^{\text{train}}$ with the index j of

the test set $\mathcal{D}_j^{\text{test}}$, we assess:

- **Performance** on the currently observed dynamics (i.e. $i = j$);
- **Retention** or **Forgetting** of previously encountered dynamics (i.e. $i > j$).

The Mean Absolute Error (MAE) is computed on each test dataset at every evaluation step, as illustrated in Figure 3.24.

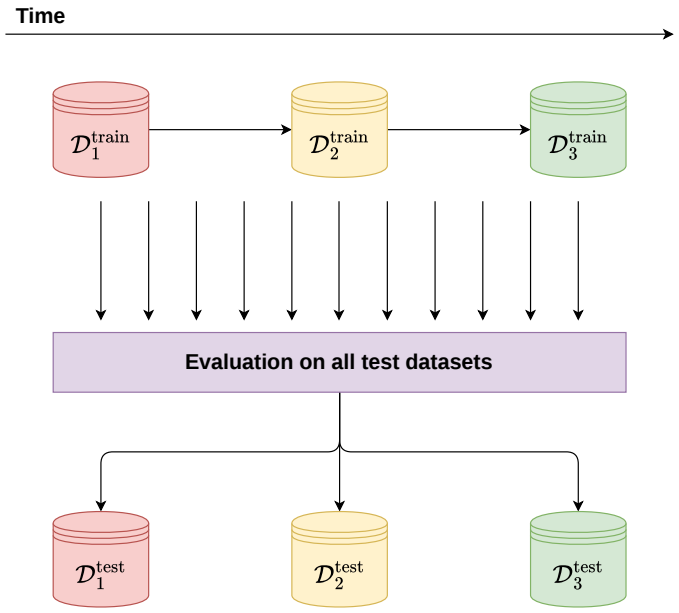


Figure 3.24: Sequential training on datasets $\mathcal{D}_1^{\text{train}}, \mathcal{D}_2^{\text{train}}$ and $\mathcal{D}_3^{\text{train}}$ with periodic multi-dataset evaluation on $\mathcal{D}_1^{\text{test}}, \mathcal{D}_2^{\text{test}}$ and $\mathcal{D}_3^{\text{test}}$.

kCELL is compared to Adaptive Hoeffding Tree [29], **a baseline specifically tailored for non-stationary learning** through integrated concept drift detectors. Unlike Mondrian Forests [145] or LWPR [229] which are primarily designed for stationary dynamics, Adaptive Hoeffding Tree represents a rigorous baseline for active adaptation to non-stationary dynamics.

First, in order to analyze the resulting behaviors, the **prediction error on test datasets** throughout learning is evaluated for both kCELL and the Adaptive Hoeffding Tree baseline. Then, the introspective properties of kCELL, specifically the **agent population growth**, the **age of agents** and **activation values**, are leveraged to provide a more in-depth analysis of the behavior of kCELL in response to changes in dynamics (i.e changes in g value).

Prediction Error Analysis Figures 3.25 and 3.26 illustrate the evolution of Mean Absolute Error (MAE) for both kCELL and Adaptive Hoeffding Tree. The results are organized into three rows, where each row represents the error measured on a specific test dataset $\mathcal{D}_j^{\text{test}}$ throughout sequential training on the train datasets $\mathcal{D}_i^{\text{train}}$. Successful adaptation to the changes in dynamics is characterized by a decreasing MAE within the semi-transparent red

shaded areas that correspond to the training phases where the dynamics of the train dataset $\mathcal{D}_i^{\text{train}}$ correspond to the dynamics of the test dataset $\mathcal{D}_j^{\text{test}}$ $(i = j)$.

For the **Inverted Pendulum** environment (cf. Figure 3.25), kCELL systematically outperforms Adaptive Hoeffding Trees when the training and testing gravity values coincide, achieving a lower MAE throughout each stationary phase. In the **Hopper** environment (cf. Figure 3.26), both learning algorithms achieve comparable MAE levels.

In both simulation environments, kCELL successfully adapts to changes in dynamics. This is indicated by the consistently decreasing MAE when the current train dataset $\mathcal{D}_i^{\text{train}}$ corresponds to the test dataset $\mathcal{D}_i^{\text{test}}$, that is to say when $i = j$ (represented by the semi-transparent red shaded area in Figures 3.25 and 3.26).

A contrast in the retention of prior knowledge is observed by kCELL between the two environments:

- In the **Inverted Pendulum** environment, transitions to new dynamics (e.g. $\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$) lead to a significant increase in MAE for previously encountered dynamics $(i > j)$, indicating a **forgetting** effect.

- In the higher-dimensional **Hopper** environment, kCELL maintains a stable and low MAE on the previously encountered dynamics $(i > j)$. For instance, the error on $\mathcal{D}_1^{\text{test}}$ does not degrade abruptly when the model is exposed to $\mathcal{D}_2^{\text{train}}$and $\mathcal{D}_3^{\text{train}}$, suggesting a **retention** of prior knowledge.

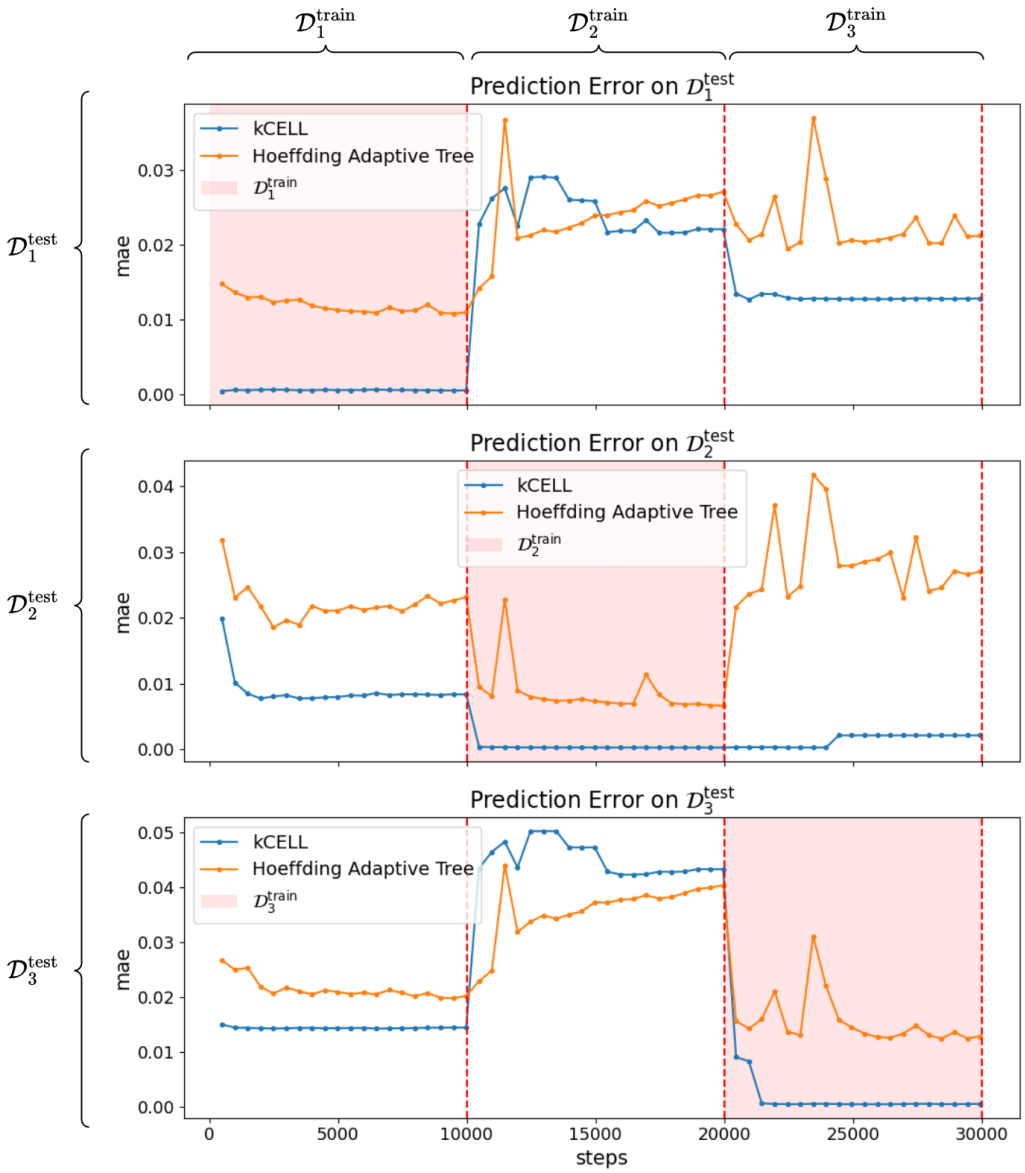


Figure 3.25: Evolution of the Mean Absolute Error (MAE) throughout training for the **Inverted Pendulum**. The x-axis corresponds to the number of points processed (number of steps) and the y-axis corresponds to the MAE. Each row displays the error on a specific test dataset $\mathcal{D}_j^{\text{test}}$ for $j \in \{1, 2, 3\}$. The *vertical red dotted lines* indicate transitions between the train dataset $\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$. The *semi-transparent red shaded areas* represent the training periods where the train dataset $\mathcal{D}_i^{\text{train}}$ corresponds to the test dataset $\mathcal{D}_j^{\text{test}}$ $(i = j)$. The blue curve represents kCELL and the orange curve represents the Adaptive Hoeffding Tree baseline. **Interpretative Note**: A downward trend in a *semi-transparent red shaded area* indicates effective online adaptation to a change in dynamics.

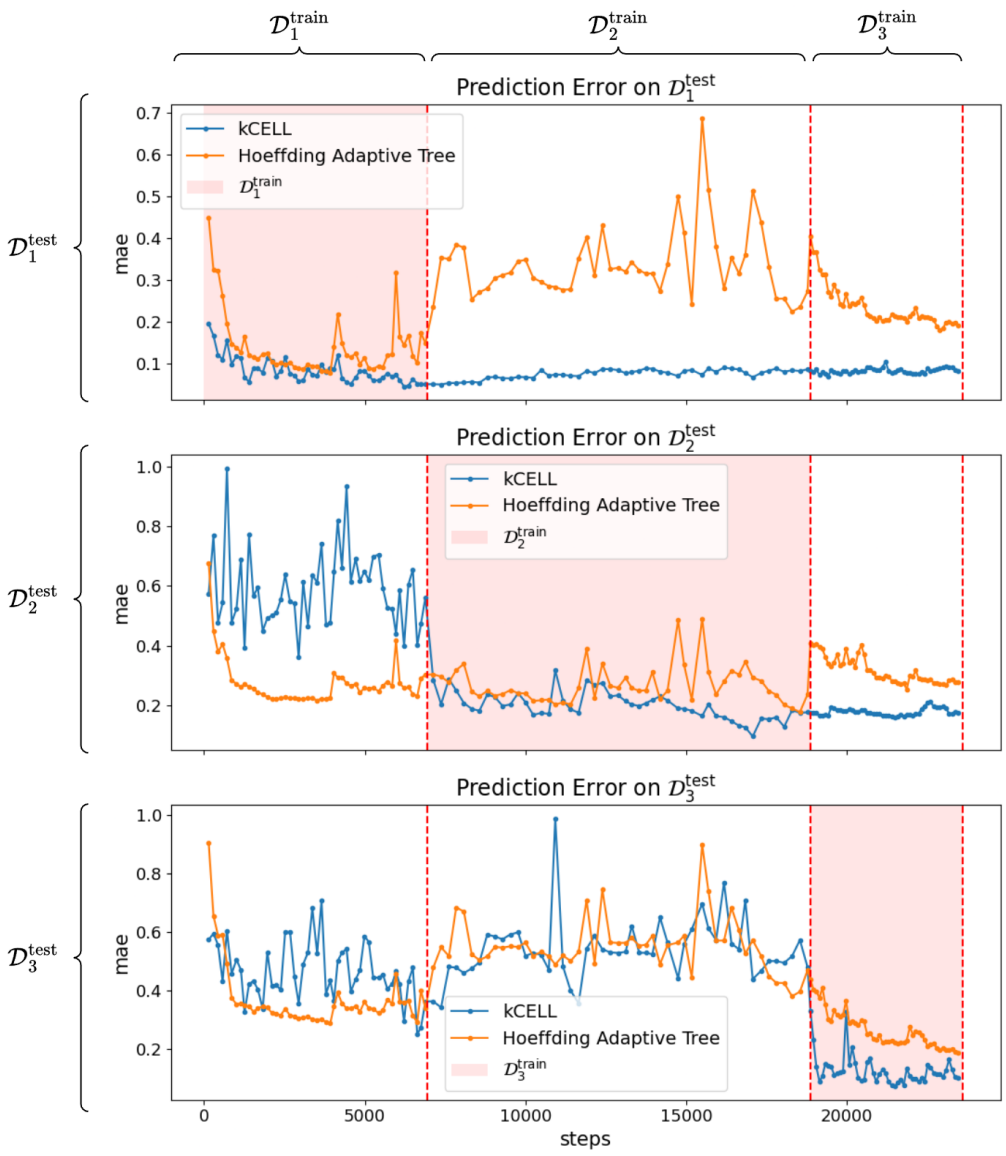
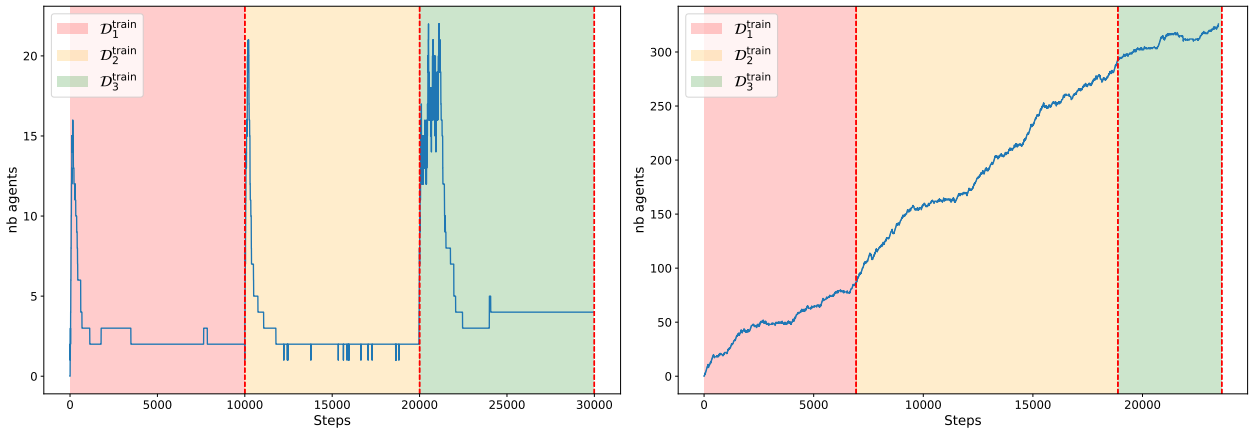


Figure 3.26: Evolution of the Mean Absolute Error (MAE) throughout training for the **Hopper**. The plotting conventions are identical to Figure 3.25.

Structural Evolution of Agent Population To better understand these adaptation patterns, the evolution of the local expert agent population displayed in Figure 3.27 is analyzed.

– **Inverted Pendulum**: Each change in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$) induces a short-term burst in the number of agents following by a reduction of this number and a stabilization. This indicates a restructuration of the population. New dynamics trigger agent creation while the confidence-based mechanism prunes older agents that are no longer relevant to the current dynamics (cf. Figure 3.27a).

– **Hopper**: The number of agents grows monotonically. Distinct growth regimes are observed for each gravity value. The growth is logarithmic for $\mathcal{D}_1^{\text{train}}$(*red area*), becomes linear for $\mathcal{D}_2^{\text{train}}$(*yellow area*), and then stabilizes around 20000 steps for $\mathcal{D}_3^{\text{train}}$(*green area*). This suggests an accumulation of local expert agents rather than a replacement of the existing population (cf. Figure 3.27b).



(a) Inverted Pendulum

(b) Hopper

Figure 3.27: Evolution of the number of agents in kCELL during learning. Each semi-transparent colored area corresponds to a given training dataset / dynamics variation regime. The dotted vertical red lines corresponds to changes in dynamics i.e to a change of train dataset ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$). The x-axis corresponds to the number of point ingested (number of steps) and the y-axis corresponds to the number of agents in the system.

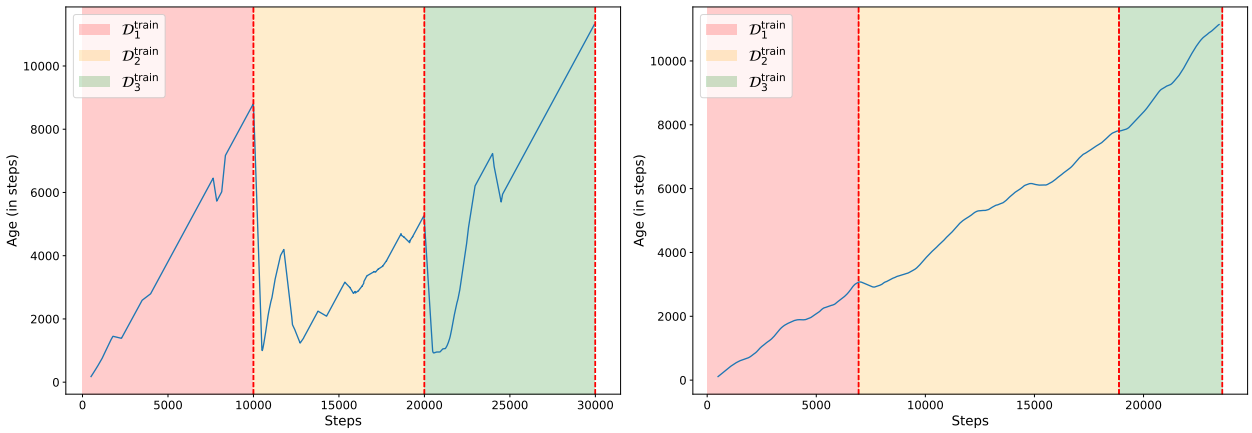
Agent Age Analysis The mean age of agents (cf. Figure 3.28a) provides further insights about these observed differences between the two simulated environments:

– **Inverted Pendulum**: Each change in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$) coincides with an abrupt drop on the mean age of the population, indicating that older agents are replaced by newly created ones. This behavior aligns with the confidence-driven destruction mechanism that favors the survival of agents that are experts for the current dynamics (cf. Figure 3.28a).

– **Hopper**: The mean agent age increases almost constantly with only minor slowdowns following changes in dynamics, indicating that older agents persist over time and are not displaced by newer ones (cf. Figure 3.28b).

These results suggest that dimensionality and the resulting geometric separation of the data play a role in whether kCELL favors forgetting or retention of knowledge.

Agent Activation and Local Expertise The evolution of the maximum activation value of the agents in kCELL, displayed in Figure 3.29, reveals distinct behaviors across the two environments:



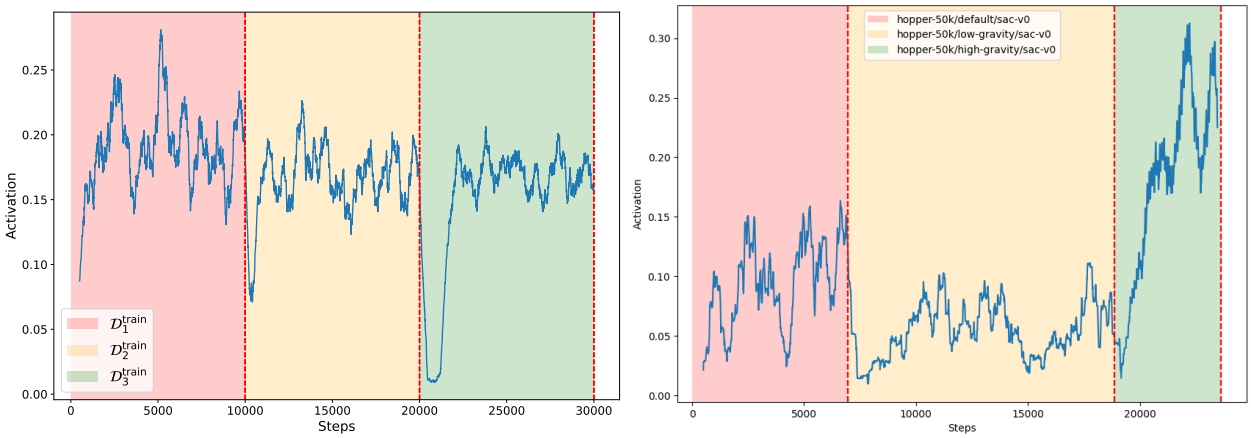
(a) Inverted Pendulum

(b) Hopper

Figure 3.28: Evolution of the age of agents in kCELL during learning (smoothed with a window of 500 steps). Each *semi-transparent colored area* corresponds to a given training dataset $\mathcal{D}_i^{\text{train}}$ (*red* $\rightarrow \mathcal{D}_1^{\text{train}}$, *yellow* $\rightarrow \mathcal{D}_2^{\text{train}}$, *green* $\rightarrow \mathcal{D}_3^{\text{train}}$). The *dotted vertical red lines* corresponds to changes in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$). The x-axis corresponds to the number of point processed (number of steps) and the y-axis corresponds to the mean age of agents (in steps) in the system .

- **Inverted Pendulum**: Each change in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$) results in a sharp drop in the maximum activation value, signaling a loss of relevance of existing local expert agents. Activation values subsequently recover as new agents acquire expertise under the new dynamics (cf. Figure 3.29a).
- **Hopper**: Distinct nominal activation values for each train dataset $\mathcal{D}_i^{\text{train}}$ are observed. While slight drops are noticeable following changes in dynamics, the effects are less pronounced than for the Inverted Pendulum environment (cf. Figure 3.29b).

These results suggest that in the higher-dimensional state-action space of the Hopper environment, the different train datasets $\mathcal{D}_i^{\text{train}}$ occupy more disjoint regions. This separation allows agents trained on $\mathcal{D}_i^{\text{train}}$ to remain specialized without interfering with those trained on $\mathcal{D}_j^{\text{train}}$ where $i \neq j$.



(a) Inverted Pendulum (b) Hopper

Figure 3.29: Evolution of the maximum activation value of agents in kCELL during learning (smoothed with a window of 500 steps). Each *semi-transparent colored area* corresponds to a given training dataset $\mathcal{D}_i^{\text{train}}$ (*red* $\rightarrow \mathcal{D}_1^{\text{train}}$, *yellow* $\rightarrow \mathcal{D}_2^{\text{train}}$, *green* $\rightarrow \mathcal{D}_3^{\text{train}}$). The *dotted vertical red lines* corresponds to changes in dynamic ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$). The x-axis corresponds to the number of point processed (number of steps) and the y-axis corresponds to the maximum activation value of agents.

Knowledge Retention Behavior Analysis Unlike the Inverted Pendulum environment, for the higher-dimensional Hopper environment, a **knowledge retention** effect has been identified (cf. Figure 3.26) during the learning process. This effect can be attributed to the **agent persistence** within the kCELL model (cf. Figures 3.27b and 3.28b), which may be driven by two non-exclusive mechanisms:

- **Adaptive Knowledge Reuse**: Agents trained on earlier datasets $\mathcal{D}_i^{\text{train}}$ might retain partial competence in overlapping regions of the state-action space (i.e. where $\forall\, i \neq j$, $\mathcal{D}_i^{\text{train}} \cap \mathcal{D}_j^{\text{train}} \neq \emptyset$), allowing them to adapt incrementally as new data arrives. In this scenario, population growth is mainly driven by the additional modeling complexity introduced by the change in dynamics.

- **Spatial Separation**: High-dimensional effects may stretch the Mahalanobis distance, yielding almost zero activation values for older agents when changing dynamics, preventing both meaningful adaptation and confidence-based degradation (as older agents are never updated again). In this case, older agents become frozen in previously visited regions of the state-action space that do not overlap with newly discovered ones (i.e $\forall\, i \neq j$, $\mathcal{D}_i^{\text{train}} \cap \mathcal{D}_j^{\text{train}} = \emptyset$).

To disambiguate these two mechanisms, we analyze the activation patterns of the final agent population across the entire training sequence ($\mathcal{D}_1^{\text{train}} \rightarrow \mathcal{D}_2^{\text{train}} \rightarrow \mathcal{D}_3^{\text{train}}$). Figure 3.30 presents a heatmap of these activation values, where the x-axis represents the agents sorted by age (from oldest to youngest) and the y-axis represents the chronological stream of training data (aggregated into bins).

The heatmap reveals a clear block-diagonal structure which provides strong evidence in favor of a **Spatial Separation** mechanism (see Figure A.2 in the Appendix for a comparison with Inverted Pendulum, where no block-diagonal structure can be observed). Agents remain highly activated for data points belonging to the specific dataset on which they were originally created and trained. Indeed, there are very few cross-dataset activations: for instance, agents activated by $\mathcal{D}_1$ samples no not show significant activation values for $\mathcal{D}_2$ or $\mathcal{D}_3$ samples.

This lack of overlap suggests that the observed knowledge retention for the Hopper environment is not due to continuous adaptation of the ensemble of local expert agents, but rather to the spatial separation of several subsets of agents. Each change in dynamics induces the creation of a new subpopulation of agents that coexists with the previous ones without replacing them.

Figure 3.30: Heatmap of the final agents activation values on the whole sequence of training data. The x-axis represents individual agents sorted by age (oldest to youngest), and the y-axis represents the chronological sequence of training data ($\mathcal{D}_1^{\text{train}} \rightarrow \mathcal{D}_2^{\text{train}} \rightarrow \mathcal{D}_3^{\text{train}}$) aggregated into bins of 300 samples. The dotted horizontal red lines corresponds to changes in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$). The colors corresponds to activation values of agents over each aggregated training data bins with yellow meaning high activation and blue meaning low activation.

This interpretation is further supported by a two-dimensional UMAP [161] projection of the training data in Figure 3.31, with samples colored according to the dataset to which they belong (*blue* $\rightarrow \mathcal{D}_1^{\text{train}}$, *yellow* $\rightarrow \mathcal{D}_2^{\text{train}}$, *pink* $\rightarrow \mathcal{D}_3^{\text{train}}$). UMAP is a **nonlinear dimensionality reduction technique** that approximates the local and global structures of high-dimensional data, making it well-suited for visualizing the separation between different dynamical regimes.

The projection reveals a clear geometric separation between the train datasets $\mathcal{D}_i^{\text{train}}$ (see Figure A.1 in the Appendix for a comparison with Inverted Pendulum, in which the UMAP projection shows no separation). Each change in dynamics induces the system to explore a distinct, non-overlapping region of the state-action space. This structural separation provides a geometric explanation for the coexistence of multiple subpopulations of agents: since data from different dynamics occupy largely disjoint regions, agents specialized in previous train datasets remain relevant within their respective subspaces without interfering with those created after subsequent changes in dynamics. This confirms that the observed knowledge retention effect arises from the agents specializing to specific, non-overlapping dynamics profiles. This effect is amplified by the high-dimensional geometry of the state-space of the Hopper environment that stretches Mahalanobis distances.

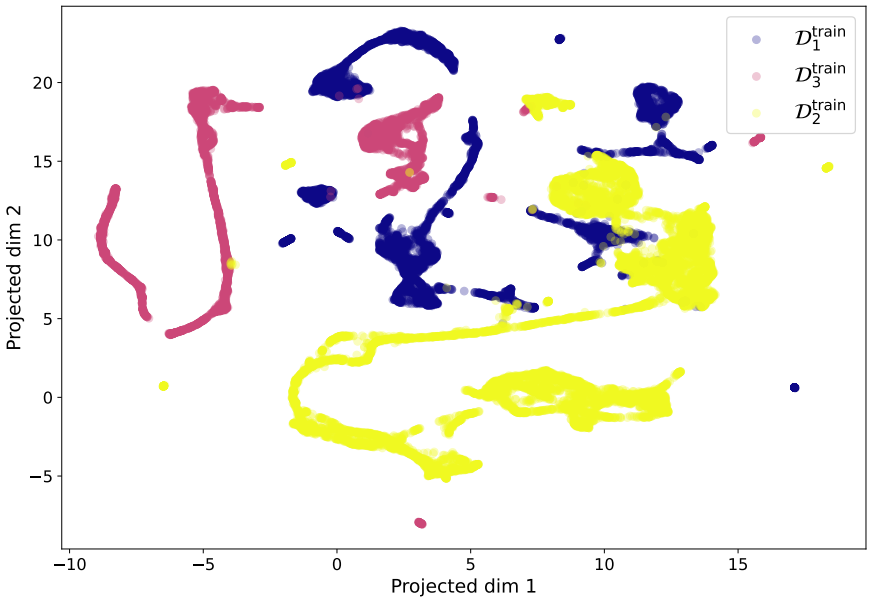


Figure 3.31: UMAP projection of $\mathcal{D}_1^{\text{train}}, \mathcal{D}_2^{\text{train}}, \mathcal{D}_3^{\text{train}}$for Hopper where each point is colored depending on the training dataset it comes from (*blue* $\rightarrow$ $\mathcal{D}_1^{\text{train}}$, *yellow* $\rightarrow$ $\mathcal{D}_2^{\text{train}}$, *pink* $\rightarrow$ $\mathcal{D}_3^{\text{train}}$)

Discussion and Synthesis The results presented in this section suggest that kCELL adapts to non-stationary dynamics through a combination of local specialization and structural plasticity, with behaviors that depend strongly on the dimensionality and geometry of the state-action space (mainly if dynamic profiles spatially overlap or not). In both Inverted Pendulum and Hopper environments, kCELL successfully reduces prediction error on the currently observed dynamics without requiring explicit task or concept drift detection modules.

For the **Inverted Pendulum environment**, adaptation is characterized by a population renewal process. Changes in dynamics induce a rapid loss of relevance for existing agents, leading to their replacement. Knowledge forgetting emerges as a natural consequence of

the overlap between different dynamics profiles in a low-dimensional space. This reflects appropriate structural adaptation rather than failure of knowledge retention.

In contrast, the **Hopper environment** demonstrates kCELL's ability to maintain stable prediction performance across successive dynamics changes through knowledge retention. The existence of well-separated regions in the state-action space allows for the coexistence of multiple generations of agents. Although the Mahalanobis distance may lose its discriminative power as dimensionality increases, the geometric separation in this 14-dimensional state-action space is sufficient to maintain effective specialization of local expert agents. This indicates retention of dynamics-specific expertise rather than passive persistence of obsolete models. Indeed, the occurrence of dynamics changes is unknown beforehand.

Finally, we highlight the ability to analyze kCELL through introspective properties such as the spatial organization and activation patterns of agents as a key advantage in terms of interpretability. These introspective metrics can prove useful to debug, refine learning mechanisms or build fallback strategies when the model detects its own non-reliability. This relates to the introspective metrics presented in Section 3.2.4 (e.g. the coverage index), showing that despite the design differences with oCELL, some related properties remain.

3.3.5 Discussion and Limitations of kCELL

The previous sections detail the architecture of kCELL. kCELL is the second CELL instantiation described in this thesis, designed to address some of the limitations of oCELL. The performance of kCELL were evaluated across different modeling tasks demonstrating its ability to capture complex dynamics in both stationary and non-stationary settings. Now, this section discusses about the differences between oCELL and kCELL and then exposes the inherent limitations and research gaps that remain.

More Advanced Social Interactions The kCELL learning algorithm has been built to address some limitations of oCELL. Thus, it pushes the social learning paradigm further by introducing feedback mechanisms relative to a local collective of agents. Coupled with a socially grounded population management mechanisms, it induce a mix between competition for survival and local group cooperation to improve the accuracy of the global model. These social mechanisms allow a better population management than oCELL which struggled from agent population overgrowth.

Differentiable Architecture Unlike the discrete boundaries of the areas of expertise of agents in oCELL, kCELL leverages RBF kernels to provide smooth and differentiable areas of expertise. This property makes kCELL more suited than oCELL for integration into traditional optimization-based control frameworks like Model Predictive Control (MPC) with gradient-based optimizers.

Refined Spatialization and Introspection kCELL retains the core introspective properties of oCELL despite their design differences. Those properties allow for in-depth studies of the system behavior. It has been demonstrated in the previous section (3.3.4) through the study of kCELL's behavior in an online stationary modeling task. Furthermore, the

smooth spatialization through RBF kernels seems to represent a better proxy for epistemic uncertainty compared to oCELL. The experiments conducted in the previous section (3.3.4) have showed a strong correlation between agent activation and prediction errors, confirming that discarding the uniform knowledge assumption of oCELL was correct. Indeed, prediction errors are significantly slower near the center of agents kernels.

Performance and Robustness kCELL exhibits competitive asymptotic performance with baseline state-of-the-art algorithms such as LWPR or Mondrian Forests in online learning with stationary dynamics while showing controlled agent population growth. Moreover, its ability to adapt to concept drift makes kCELL a robust candidate for modeling the non-stationary dynamics inherent in Human-Robot Collaboration (HRC).

Simplified Hyperparameterization One of the main limitation for oCELL applications was the number of hard-to-tune hyperparameters. kCELL has moved away from rigid error thresholds by introducing social feedbacks relative to a local group of close agents. In kCELL, the goal of agents is not to minimize their own prediction error but to maximize their positive contribution to a local group of agents.

Limitations and Future Works Finally, several limitations relative to kCELL and to the experiments conducted in this section have been identified:

- **Sensitivity to High-Dimensionality**: As kCELL relies on Mahalanobis distance for agent spatialization, it is highly sensitive to the number of dimensions of the input space. As the number of dimensions increases, the Mahalanobis distances stretch and cause agents to become more and more isolated. They become small relative to the input space, which can prevent them from forming effective local neighborhood or to participate in local cooperation schemes. This is a common issue of local learning approaches.
- **Out-of-Distribution (OoD) Predictions**: While kCELL shows strong performance on test datasets, the prediction mechanism for OoD inputs remains a concern. In the stationary modeling experiment (cf. Section 3.3.3) the prequential error was worse than baselines (LWPR and Mondrian Forest), suggesting that the model's predictive behavior outside the regions covered by training data requires further refinement. Indeed, LWPR shows a way better prequential error profiles despite the design similarities with kCELL. This seems to indicate that the prediction mechanism of kCELL could be further improved.
- **Experimental Scope**: The current validation of kCELL is limited. The stationary experiments were conducted on a single dataset without an exhaustive hyperparameter search. In addition to that the non-stationary experiment focused on only one type of concept drift. Future works are needed to evaluate kCELL on more diverse datasets and different drift types (e.g gradual dynamic change instead of sudden dynamic change).
- **Lack of Ablation Studies**: While the combined learning rules of kCELL show encouraging results, a formal ablation study have not been conducted yet to quantify the

individual impact of each learning rule. Future works should prioritize these studies to identify the primary drivers of the model's performance and move from designed intuition to validated proofs.

Through kCELL, we have designed a differentiable, introspective and adaptive learning algorithm that addresses some of the limitations of its predecessor. Some limitations still remain to be addressed but promising research directions have been uncovered.

3.4 Synthesis and Conclusion

This chapter has defined the theoretical and algorithmic foundations of the CELL formalism. By detailing the iterative development from the rigid spatialization of agents in oCELL to the smooth and differentiable, kernel-based spatialization of agents in kCELL, we have demonstrated the modeling and adaptation capabilities of learning systems based on a self-organizing ensemble of local expert agents governed by social learning rules.

Through the experiments presented in Sections 3.3.3 and 3.3.4, we have shown that kCELL effectively addresses the stability-plasticity dilemma in non-stationary ensembles through social population management strategies. However, the critical analysis presented in Section 3.3.5 highlighted significant remaining challenges, particularly regarding the reliability of Out-of-Distribution (OoD) predictions and the curse of dimensionality. Since kCELL is able to estimate when it does not know a point, the OoD predictions problem can be addressed. In high epistemic error regions of the input space, fallback strategies can be used temporarily to allow the learning system to adapt. Thus, the curse of dimensionality becomes a limitation that is to be prioritized in terms of research directions.

In Chapter 4 of this thesis, kCELL is integrated into a control pipeline, and the use of its introspective properties to improve learning-based control are studied. Then Chapter 5 introduces prospective works which explore the model-agnosticity property of CELL to produce more representative models.

Chapter 4

Solving Control Tasks with CELL

The previous chapter has introduced the CELL (Context Ensemble Local Learning) formalism. CELL provides a theoretical framework for designing machine learning systems based on a self-organizing ensemble of local expert agents governed by social rules. The development of two CELL instantiations, oCELL and kCELL, capable of learning online from a stream of data have been detailed. We demonstrated that analyzing the geometry of the agents spatial organization as well as the local social dynamics could provide meaningful introspective properties that can be linked to the traditional notions of explainability and interpretability in machine learning.

These introspective properties may be exploited for improving model-based control with learned models and this chapter investigates how this can be achieved. Thus, in this chapter, the integration of kCELL into a Model Predictive Control (MPC) scheme is explored. In this context, the kCELL model learns online the *a priori* unknown dynamics of the system. It models the transition function $f : \mathcal{S} \times \mathcal{U} \mapsto \mathcal{S}$ (with $\mathcal{S}$ the state space and $\mathcal{U}$ the action space) mapping state-actions to the next state such as

$$s_{t+1} = f\left(s_t, u_t\right)$$

where $s_t \in \mathcal{S}$ is the current state, $u_t \in \mathcal{U}$ is the action applied to the controlled dynamical system and $s_{t+1} \in \mathcal{S}$ is the next state.The control actions are obtained by repeatedly solving an Optimal Control Problem (OCP) over a finite horizon [187].

However, without prior knowledge of the system's dynamics, using a learned model in a control loop introduces several challenges [40] that need to be considered when solving OCPs involving hard constraints related to safety, stability or physical feasibility. This chapter specifically focuses on the following challenges:

– **Real-Time Predictions and Optimization**: In MPC, the predictive model and the optimizer must synergize properly to produce reliable control actions in real-time. Depending on the class of optimizer used, the required properties of the predictive model can vary. For instance, using population-based optimizers requires the predictive model to evaluate thousands of trajectories multiple times per control step. In contrast, using gradient-based optimizers requires the predictive model to be differentiable or locally linearizable.

- **Efficient Exploration of Unknown Dynamics**: A learned model is only as good as its training data [91]. Therefore, good quality data that is representative of as many different state transitions as possible must be collected. When the dynamics of the system to be controlled are unknown *a priori*, an exploration phase must be conducted to collect data. In complex environments, *uninformed* exploration strategies (e.g. actions obtained with random noise) are often data-inefficient and fail to cover the state-action space effectively, leading to knowledge gaps in the internal representation of the learned predictive model.
- **Robust Exploitation through Uncertainty Awareness**: In MPC, the optimizer may exploit the weaknesses of the model. These weaknesses, such as prediction inaccuracies on Out-of-Distribution (OoD) state-actions, can lead the optimizer to explore bad solutions. Indeed, unreliable predictions lead to unreliable optimization results. While data-driven control methods have achieved promising results, they often rely on opaque inner workings and struggle to provide reliable guarantees when operating outside the support of the training data [183, 26].

This chapter demonstrates how the introspective properties of kCELL can be leveraged to address these challenges. The core contributions presented are threefold:

1. We discuss how kCELL can be efficiently integrated into real-time, model-based control frameworks by leveraging its local linearization capabilities for gradient-based optimization. We also propose implementation strategies based on parallel computation or spatial indexing to enable fast trajectory prediction in population-based optimization.
2. We introduce introspection-driven strategies based on agent spatial organization and local disagreement in order to explore a low-dimensional environment without prior knowledge of the dynamics. We show that these strategies lead to more effective coverage of the state-action space than uninformed exploration mechanisms.
3. We introduce an introspection-driven conservative regularization mechanism for trajectory optimization. This mechanism leverages the spatial organization of agents to minimize epistemic uncertainty, to reduce the risk of extrapolation errors, and to improve the robustness of control with learned dynamics.

This chapter is organized as follows. Section 4.1 discusses the computational and differentiability properties of kCELL enabling its use for real-time constrained control within a MPC controller. Section 4.2 focuses on exploration of unknown dynamics and shows how the spatial organization and local interactions of agents can be exploited to guide exploration. Section 4.3 addresses robust exploitation by introducing an introspection-driven regularization for trajectory optimization with kCELL to improve the reliability of the control loop.

4.1 Efficient kCELL for Real-Time Control

As demonstrated in Chapter 3, kCELL is capable of handling non-stationary online learning tasks while exhibiting competitive predictive accuracy with state-of-the-art baseline algorithms such as LWPR [229] or Mondrian Forests [145]. However, its practical deployment in model-based control requires additional computational and structural considerations.

MPC relies on repeated model evaluations within an optimization loop. The use of learned models makes inference speed and mathematical structure important to the feasibility and reliability of control, especially under real-time constraints. Therefore, kCELL must comply with the following requirements:

- **High-Throughput Inference**: Population-based optimization methods such as the Cross-Entropy Method (CEM) [58] require that the model evaluate thousands of candidate trajectories over a single control step. In this context, the computational cost must not only be low but also predictable. A naive sequential lookup of the local expert agents in kCELL becomes prohibitively expensive as the population of agent grows.
- **Differentiability and Local Linearization**: Conversely, gradient-based methods, such as Sequential Quadratic Programming (SQP) [187] require a differentiable model to compute the local linearization of the approximated dynamics. Consequently, the resulting superlinear convergence makes real-time feasibility easier to reach. In comparison, derivative-free methods scale poorly with dimensionality and often fail under tight timing constraints [172].

Hence, kCELL must be implemented to provide both fast batched inference and differentiable, locally linearizable representations. In terms of computational efficiency, as agents in kCELL have a defined spatial support, it allows local expert selection to be greatly accelerated through spatial indexing. In addition to that, expert selection in kCELL is embarrassingly parallel. This property can be exploited to leverage massively parallel execution for efficient batched inference on modern hardware such as GPUs. Moreover, in kCELL the activation functions and linear internal models are differentiable. Thus, kCELL provides analytical gradients and straightforward local linearizations that naturally align with gradient-based optimizers.

Section 4.1.1 discusses strategies that can be implemented to accelerate inference and learning in kCELL, in order to make it compatible with population-based optimizers like CEM. Then, Section 4.1.2 shows how the local structure of kCELL leads to trivial local linearizations of the learned dynamics, making it compatible with constrained MPC formulation that can be solved using SQP.

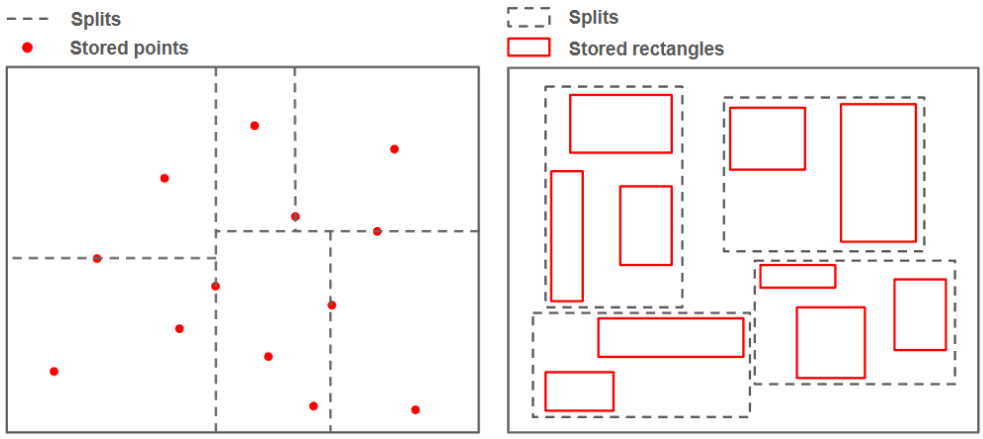
4.1.1 Efficient Implementation

Population-based trajectory optimization algorithms are broadly used in robotics for their robustness to non-convex and non-smooth dynamics or cost functions [45]. This includes stochastic shooting methods such as Model Path Predictive Integral (MPPI) [239] or CEM (which will be used in Section 4.2). These methods rely heavily on parallel evaluation of large population of candidate trajectories at each control step. In this section, we discuss and compare the use of geospatial indexing and massively parallel computing to accelerate local expert selection. Through a simple case study, we show that GPU parallel brute force is more reliable than geospatial indexing for fast inference with kCELL.

Geospatial Tree Indexing

CELL instantiations like kCELL and oCELL always begin a learning step with the selection of neighbor agents. Since the agents are spatially distributed in the feature space, the selection of neighbor agents can be broken down into two components: **distance computation** (to find the set of closest local experts) and **thresholding** of those distances to discriminate neighbors.

Because agents are spatially distributed within the input space, managing the ensemble of agents as a geospatial database can be considered. To speed up nearest neighbor searches, spatial indexing techniques can be leveraged to select the k-closest agents with a reduced complexity. Indexing in geospatial databases is most commonly performed using trees such as KD-tree or R-tree [99]. The **KD-tree** (for k-dimensional tree) is a binary tree data structure. It recursively partitions the input space by splitting along alternating dimensions. It is efficient for range queries and nearest neighbor searches on a **set of points** [25] (cf. Figure 4.1a). The **R-tree** is a hierarchical tree structure designed for indexing multi-dimensional objects like **rectangles or polygons**. Each node of the tree represents a bounding rectangle that encloses child nodes. R-trees are well-suited for range, intersection and nearest neighbor queries in GIS (Geographic Information System) and geospatial databases [100] (cf. Figure 4.1b).



(a) KD-tree (b) R-tree

Figure 4.1: Illustration of KD-tree and R-tree on 2D data

For implementing CELL instantiations, R-trees are preferable to KD-trees, as they are designed to handle polygons and orthotopes. Therefore, agents can be indexed by their bounding orthotope. For kCELL, this corresponds to the orthotope encompassing the agent's confidence interval, i.e the ellipsoid defined by

$$d_M\left(x, \mathcal{A}_i\right)^2 < \chi^2_{d,\frac{\xi}{100}} \tag{4.1}$$

where $\chi^2_{d,\frac{\xi}{100}}$ denotes the ξth percentile of the chi-squared distribution with d degrees of freedom, where d is the number of input features.

Furthermore, R-trees can be incrementally updated, making them compatible with the online learning nature of CELL instantiations [100]. With a spatial index based on a R-tree, the selection process can be broken down into three ordered steps:

1. Searching for the k-closest agents in the R-tree.
2. Computing the similarities between the point and the k-closest agents.
3. Thresholding the distances to discriminate the neighbor agents.

Implementing a R-tree as a geospatial index over the agents reduces the neighborhood searches from $O(n)$ in the naive case, to $O(\log n)$. However, for insertion and deletion, the average time complexity increases from $O(1)$ in the naive case to $O(\log n)$. While geospatial indexing seems less advantageous in terms of insertion and deletion, it should be noted that agent creations and destructions are usually a lot less frequent than neighborhood searches in kCELL.

GPU Parallelization

As the dimensionality of the feature space increases, the structural advantages of the R-tree diminish due to distance stretching and bounding box overlap. As a consequence, the search complexity degenerates to $O(n)$. In such cases, the overhead induced by geospatial indexing offers no advantage over a brute-force approach. To address these limitations, the embarrassingly parallelizable nature of neighborhood search in kCELL (and oCELL) can be exploited through GPU acceleration. By shifting from pointer-based tree structures to dense tensor representations, kCELL (and oCELL) can leverage massively parallel throughput because the selection phase can be executed as a vectorized linear algebra operation.

Memory Layout To maximize the efficiency of GPU usage for agent selection, agent data is stored in a contiguous memory block. For example, for kCELL, two global agent tensors are defined to store the parameters of the activation functions:

$$\mathcal{T}_\mu \in \mathbb{R}^{N\times d},\ \mathcal{T}_\Sigma \in \mathbb{R}^{N\times d\times d}$$

where N is the maximum capacity of the agent pool and d the number of features. $\mathcal{T}_\mu$ stores the means and $\mathcal{T}_\Sigma$ stores the covariance matrices. With this layout, the GPU hardware can easily saturate its bandwith for efficient distance computations.

Dynamic Agent Population In kCELL, the number of agents is dynamic. Agents can be created or destroyed depending on the social interactions between the local expert agents. In a GPU parallelization context with contiguous memory storage, managing a dynamic set of agents requires carefully designed creation and deletion strategies to avoid costly memory allocations.

The most straightforward approach consists in a dynamic reallocation of memory each time an agent is created or destroyed as illustrated in Figure 4.2. In this figure, each line of a tensor corresponds to the memory allocated to one agent. This memory stores the different parameters of this agent.

Although easy to implement, this approach is inappropriate for high-frequency learning loops because of the overhead induced by re-allocating GPU memory and by copying existing data.

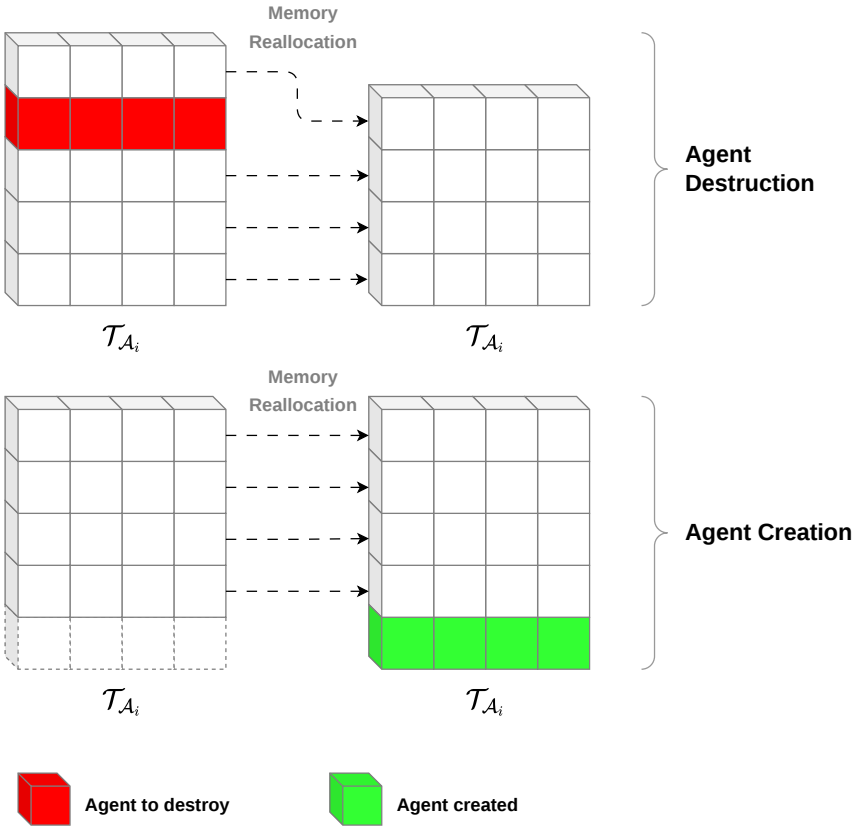


Figure 4.2: Agent creation and destruction in dynamic reallocation regime. Each line of the $\mathcal{T}_{\mathcal{A}_i}$tensor corresponds to the memory allocated to one agent to store its internal parameters.

To obtain a complexity close to $O(1)$, another strategy can be considered. Fixed-capacity tensors are preallocated to store agents data. A Boolean mask $M \in \{0,1\}^N$ is maintained to track which memory slots are currently occupied by an agent.

Agent creation then consists in assigning new agents data to the first index i such that $M_i = 1$. If the agent population exceeds the predefined capacity, standard dynamic resizing strategies can be employed. Conversely, agent destruction sets $M_i = 0$, marking the slot as available to host data of future agents, as illustrated in Figure 4.3. This effectively hides the agent from the selection kernel without moving data or having to free a memory slot.

For agent selection, the mask M is used to filter the active memory slots ($M_i \neq 0$) to retain only relevant distance computations, i.e. those corresponding to existing agents.

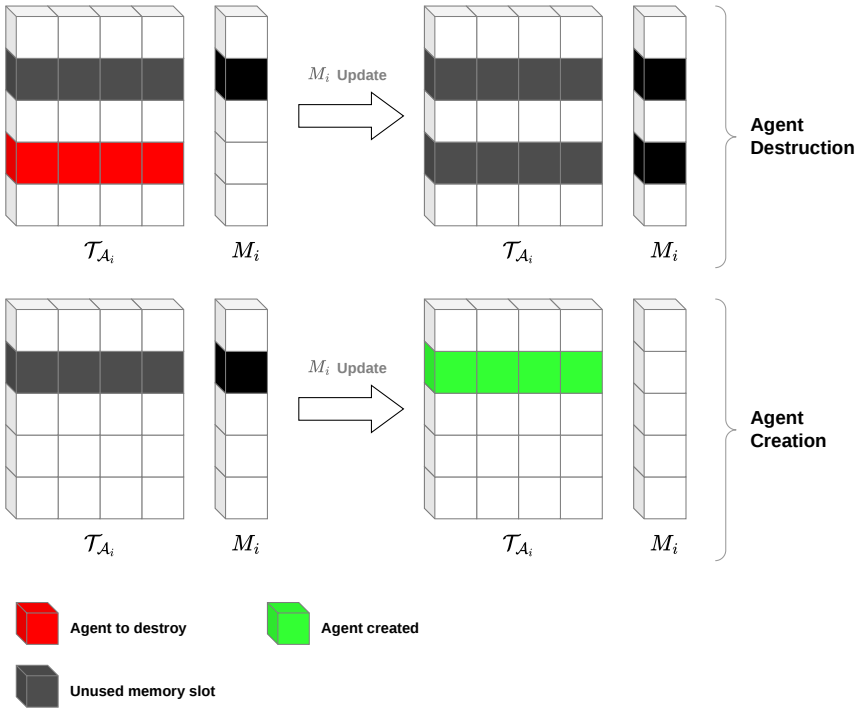


Figure 4.3: Agent creation and destruction in preallocated memory pool with masking. Each line of the $\mathcal{T}_{\mathcal{A}_i}$tensor corresponds to the memory allocated to one agent to store its internal parameters. Agent creation and destruction operations are performed by updating the M_i mask. A *black* cell corresponds to $M_i = 0$ (slot available) and a *white* cell corresponds to $M_i = 1$ (slot taken by an agent).

Comparison between GPU and Tree Selection

The choice of using a geospatial index or GPU parallelization depends on the number of input dimensions, the number of agents and the distribution of training data.

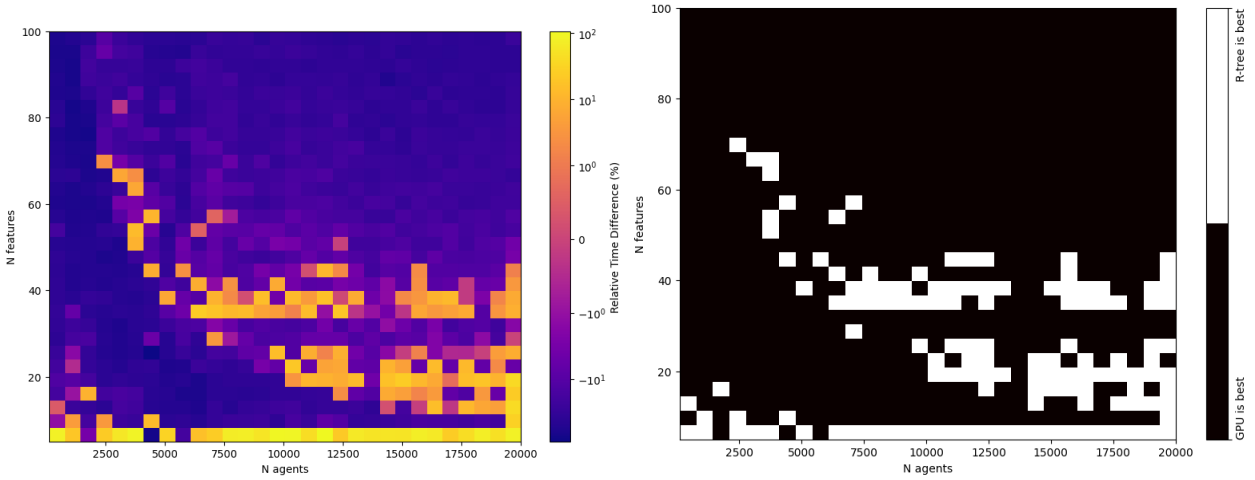
To assesss the efficiency of each of the two approaches, we benchmark neighborhood selection in Python using *PyTorch* [178] for GPU-accelerated computations and the *Rtree* package [92] for efficient R-tree indexing. Both selection strategies are compared on a synthetic nearest-neighbor selection task, varying both the number of input dimensions and the number of agents. As discussed in Chapter 3, in kCELL, the local expert agents are self-organizing according to the encountered data. In real-world scenarios, these data points are rarely uniformly distributed. Therefore, 100 clusters of synthetic agent centroids are generated to emulate high-density manifolds. Compared to uniform sampling which may exaggerate the effective search volume, generating a clustered distribution can allow for a more balanced comparison. Indeed, R-trees main advantage lies in efficiently pruning empty spaces.

From a set of 1000 input points, the time to find the 5-nearest neighbors is measured and averaged across 3 cycles. Figure 4.4a exposes a heatmap of the relative execution time

difference in percentage computed as

$$\Delta_t = \frac{\Delta_{\text{GPU}} - \Delta_{\text{R-tree}}}{\Delta_{\text{R-tree}}} \times 100 \tag{4.2}$$

where Δ_{GPU} and $\Delta_{\text{R-tree}}$ are the average query execution times for respectively the GPU and R-tree approaches. Figure 4.4b illustrates which strategy has obtained lowest execution time in each tested configuration.



(a) Relative Execution Time Difference Heatmap

(b) Winner Binary Heatmap

Figure 4.4: Comparison of the execution times of GPU and R-tree on various $\{n, N\}$ configurations for a clustered distribution of synthetic centroids. contains a heatmap of the relative execution time difference between the two strategies (Δ_t). Shades of yellow correspond to $\Delta_t > 0$, i.e to R-tree being faster. Conversely, shades of blue correspond to $\Delta_t < 0$, i.e to GPU being faster. 4.4a contains a binary heatmap clearly showing in which configuration which strategy was faster. *White* cells corresponds to configurations in which R-tree is faster and *black* cells correspond to configurations in which GPU is faster.

It can be noted that R-tree maintains a competitive edge in low-dimensional regimes ($n \leq 5$). Indeed, R-trees struggle in higher dimensions due to the curse of dimensionality. In higher dimensional regimes, the distinction between nearest and farthest points becomes meaningless due to distance stretching, and pruning becomes ineffective [142].

We can notice that the execution time of the R-tree compared to GPU parallelization does not seem monotonic. In Figures 4.4a and 4.4b, we notice a non linear phase transition area (looking like a "semi-banana") that extends from $N = 2500$ centroids and $n = 70$ features to $N = 20000$ and $n = 50$. Within this region, R-tree is competitive or faster than GPU.

However, when the centroids are uniformly distributed, the competitive advantage of R-trees within the non linear phase transition area fades when the centroids are uniformly distributed, as shown in Figure 4.5. In this case the R-trees are almost never faster than GPU computations. It shows that the structure of data needs to be analyzed beforehand to make sure R-tree is a pertinent indexing option. This is an expected result because R-trees are very efficient to prune large empty spaces. The uniform distribution of centroids does not

exhibit exploitable structures for the R-tree. This is not an issue for the GPU-based approach which relies on brute force computation and is therefore unaffected by data structure.

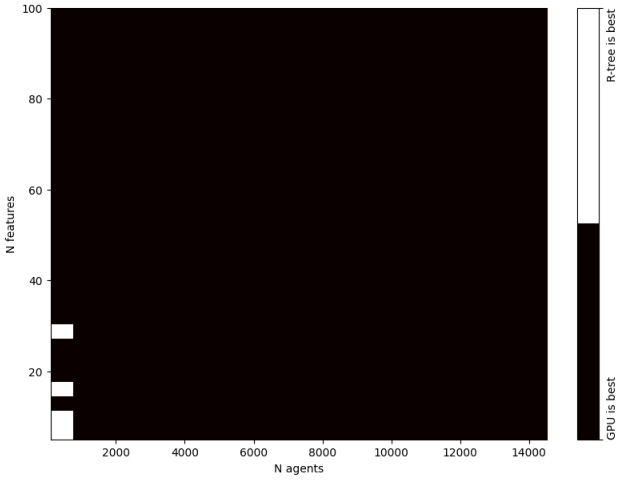


Figure 4.5: Winner binary heatmap with uniformly distributed synthetic centroids. *White* cells corresponds to configurations in which R-tree is faster and *black* cells correspond to configurations in which GPU is faster.

These observations indicate that the performance of nearest neighbor selection using R-trees compared to GPU parallelization is better in low dimensional regimes ($n \leq 5$). It also depends heavily on the underlying structure of data when the number of dimensions increases ($n \geq 5$) due to the **curse of dimensionality**.

From an engineering standpoint, GPU-based selection provides a more predictable and stable latency profile, independent of data distribution. In contrast, R-tree indexing can provide large gains in very specific sparse regimes and its performance is highly sensitive to the data distribution.

If we consider an online learning setting for adaptive control with MPC without prior knowledge of the dynamics of the environment, the data distribution is not known in advance and may be non-stationary. For this reason, the GPU-based brute-force selection should be preferred for its *deterministic latency* and *hardware scalability*. GPU execution time scales predictably with the number of input dimensions d and the number of samples N, whereas R-tree latency exhibits high variance depending on tree depth and splits overlap. Moreover, GPUs naturally handles **batch queries** as a single high-throughput operation, which can be deemed useful for population-based optimization.

The implementation strategies discussed in this section address the primary computational bottleneck of kCELL within MPC controllers, namely the repeated selection and evaluation of local experts under strict real-time constraints. By leveraging either spatial indexing or GPU parallelization for selecting neighbor agents, kCELL can be integrated into population-based trajectory optimizers while maintaining predictable inference and learning latency.

4.1.2 Differentiability and Local Linearization for Gradient-Based MPC

Fast and scalable inference in kCELL is essential for its integration into real-time MPC controllers, especially when using population-based or gradient-based trajectory optimization methods that rely on repeated model evaluations. The previous section discussed leveraging geospatial indexing with R-trees and GPU parallelization to make kCELL compatible with real-time inference and population-based optimization even with a large population of local expert agents.

However, computational efficiency alone can prove insufficient. Population-based optimizers are able to handle non-differentiable or noisy objectives, and they offer better global search capabilities. However, their computational cost per iteration is high. They exhibit slower convergence near optima and they offer fewer theoretical convergence guarantees, unlike gradient-based optimizers [82]. This section argues that kCELL naturally provides local affine approximations of the dynamics that are directly compatible with gradient-based optimization approaches such as Sequential Quadratic Programming (SQP).

Differentiability in CELL instantiations

In kCELL, online learning is performed by maintaining an ensemble of self-organizing local expert agents, each of them mapping the target function locally (cf. Chapter 3). As specified in the CELL formalism, a local expert agent is denoted as

$$\mathcal{A}_i = \{\phi_i, f_i, \mathcal{M}_i\}$$

where $\phi_i : \mathbb{R}^d \mapsto]0, 1]$ is the activation function that maps features to a value representing the level of expertise of $\mathcal{A}_i$ for predicting a given input, $f_i : \mathbb{R}^d \mapsto \mathbb{R}^m$ is the local internal model that maps inputs to target outputs, $\mathcal{M}_i$ is the internal state of $\mathcal{A}_i$.

In CELL instantiations, the global prediction is obtained by aggregating the predictions of the most relevant local experts. If the activation function, the internal model and the aggregation function are differentiable, then, by standard composition rule, the global prediction function is differentiable. In kCELL, the activation function corresponds to a Radial Basis Function (RBF) (typically Gaussian), the internal model is an affine function and the aggregation function is a weighted average. The final prediction is obtained by

$$\hat{f}(x) = \frac{\sum_i \phi_i(x) f_i(x)}{\sum_j \phi_j(x)} = \sum_i w_i(x) f_i(x) \tag{4.3}$$

where $w_i(x)$ are the normalized weights such as

$$\sum_i w_i(x) = 1 \tag{4.4}$$

As all the involved operations are differentiable with respect to x, the derivative of the prediction function of the system is given by

$$\frac{\partial \hat{f}}{\partial x}(x) = \sum_i \left(\frac{\partial w_i}{\partial x}(x) f_i(x) + w_i(x) \frac{\partial f_i}{\partial x}(x) \right) \tag{4.5}$$

This property makes kCELL usable in gradient-based nonlinear optimization frameworks. However, we argue that a principled approach to local linearization can be leveraged by exploiting kCELL's internal structure. In typical model-based control, nonlinear dynamics are linearized via a first-order Taylor expansion to fit the requirements of convex optimization [187]. However, when the predictive model is already an ensemble of local linear structures, applying a Taylor expansion becomes conceptually redundant. For this reason, using kCELL enables structural local linearization instead of relying on standard differentiation.

Local Linearization

In kCELL, the spatial organization of agents and their affine internal model allow to extract local linearization of the approximated function without computing full Jacobians or Taylor expansions.

In MPC, the predictive model predicts the next state s_{t+1} from $x_t = [s_t, u_t]$ where s_t is the current state of the system and u_t is the action to apply. Assuming agents have affine internal models

$$f_i(x_t) = A_i s_t + B_i u_t + c_i \tag{4.6}$$

, the local linearization around an input x_t is given by

$$\begin{aligned} \frac{\partial f}{\partial x}(x) = A(x_t)\, s + B(x_t)\, u + c(x_t) &= \sum_i w_i(x_t)\, f_i(x_t) \\ &= \sum_i w_i(x_t)\,(A_i s_t + B_i u_t + c_i) \\ &= \underbrace{\sum_i w_i(x_t)\, A_i}_{A(x_t)}\, s_t + \underbrace{\sum_i w_i(x_t)\, B_i}_{B(x_t)}\, u_t + \underbrace{\sum_i w_i(x_t)\, c_i}_{c(x_t)} \end{aligned} \tag{4.7, 4.8}$$

This approach offers a significant advantage for control, as it preserves the dynamics as originally learned by the model without introducing an additional differentiation step.

Sequential Quadratic Programming (SQP)

The use of Sequential Quadratic Programming (SQP) is discussed as being an optimization technique that synergizes well with kCELL. SQP is a local optimization method designed for solving nonlinear constrained optimization problems. The optimization problem to solve is a nonlinear program formulated as:

$$\begin{aligned} \min_{u_{0:T-1}} \quad & J(s_{0:T}, u_{0:T-1}) \\ \text{s.t.} \quad & s_{t+1} = f(s_t, u_t) \\ & h(s_t, u_t) \leq 0 \\ & g(s_t, u_t) = 0 \end{aligned} \tag{4.9}$$

where $J : \mathcal{S}^{T+1} \times \mathcal{U}^T \mapsto \mathbb{R}$ is the cost function, $f : \mathcal{S} \times \mathcal{U} \mapsto \mathcal{S}$ is the transition function describing the dynamics of the system, h and g correspond respectively to the inequality and equality constraints.

At each iteration, the original nonlinear program is approximated by a Quadratic Program (QP) in which the objective is quadratic and the dynamics and constraints are linearized around a reference trajectory

$$\begin{aligned} \min_{u_{0:T-1}} \quad & \sum_{t=0}^{T-1} \left(s_t^\top Q_t s_t + q_t^\top s_t + u_t^\top R_t u_t + r_t^\top u_t \right) \\ \text{s.t.} \quad & s_{t+1} = A_t s_t + B_t u_t + c_t \\ & h_{\text{lin}}(s_t, u_t) \leq 0 \\ & g_{\text{lin}}(s_t, u_t) = 0 \end{aligned} \tag{4.10}$$

where A_t, B_t and c_t define the locally linearized dynamics obtained from a first-order Taylor expansion, h_{lin} and g_{lin} correspond respectively to the linearized inequality and equality constraints obtained from a first-order Taylor expansion. The matrices Q_t, R_t and vectors q_t, r_t are usually obtained from a second-order Taylor expansion of the original cost J around a reference trajectory τ_t.

Solving this QP yields a new trajectory τ_{t+1} used to compute a descent direction $\tau_{t+1} - \tau_t$. Since both SQP and kCELL rely on locally valid approximations of the dynamics, the update step must be restricted within the region where the linearization is accurate. To this end, a line search [96] or trust-region strategy [53, 199] can be employed to determine an appropriate step size that keeps linearization error contained and ensures sufficient cost decrease. This mechanism offers stable convergence when optimizing trajectories with a learned model. The process is repeated until convergence.

Due to its spatialized agent-based architecture, kCELL naturally provides the local linearization of the dynamics needed for the local QP. As shown in Equation 4.8, for an input state-action tuple $x_t = [s_t, u_t]$, the local affine model is defined as:

$$s_{t+1} \approx \underbrace{A(x_t)}_{A_t} s_t + \underbrace{B(x_t)}_{B_t} u_t + \underbrace{c(x_t)}_{c_t} \tag{4.11}$$

It provides the matrices needed to linearize the dynamics in the QP. The approximated QP subproblem is fully compliant with the dynamics initially learned by kCELL without relying on explicit differentiation and Taylor-based approximations. Consequently, SQP-based MPC provides a principled and efficient way to exploit kCELL's local structure while being able to enforce hard constraints under the right conditions.

We have discussed about the integration of kCELL into real-time MPC pipelines by exploiting both its computational and structural properties. First the embarrassingly parallel nature of kCELL enables fast trajectory prediction, making population-based optimizers tractable through GPU parallelization. This approach is particularly well suited for large-scale trajectory sampling and global solution exploration with a non-convex and highly nonlinear cost function. Then, we showed that kCELL's agent-based architecture provides both differentiability and explicit local affine models of the learned dynamics, making it suitable for being coupled with a gradient-based optimizer. These properties of kCELL naturally allow the use of SQP, which exploits local linear models to efficiently enforce hard constraints and produce smooth control trajectories.

Beyond computational efficiency, kCELL exposes introspection metrics such as distance-based expertise and local model disagreement that quantify the reliability of predictions. The

remainder of this chapter explores the use of these introspection metrics to improve control in terms of exploration and exploitation under operational constraints.

4.2 Exploration MPC with kCELL

The previous section described the ways of integrating kCELL into real-time MPC, focusing on high-throughput inference, differentiability and local linearization. However, the performance of any model-based controller is bounded by the quality of the learned dynamics. When no prior knowledge of the system dynamics is available, the controller must balance minimizing the task cost with gathering informative data in unknown or poorly modeled regions of the state-action space. This section addresses the second challenge identified in the introduction of this chapter: the **efficient exploration of an unknown environment**.

Conventional exploration methods rely either on heuristic noise processes (e.g Ornstein-Uhlenbeck noise [223]) or on uncertainty-driven mechanisms derived from auxiliary models, such as Bayesian posterior variances in Gaussian Processes [211, 191], intrinsic motivation signals in Curiosity-driven exploration [179], or novelty scores learned through Random Network Distillation [41]. While effective in many situations, these approaches quantify uncertainty through probabilistic inference or learned proxies that are shaped by predefined model architectures and learning schemes.

In contrast, kCELL exposes uncertainty as a structural and geometric property emerging from its ensemble of self-organizing local expert agents. kCELL's spatialized agent-based architecture provides two distinct introspection metrics. These 2 metrics can be directly embedded into the MPC objective function to drive exploration: **distance-based expertise** and **local model disagreement**.

Distance-based expertise quantifies how far a state-action pair lies from regions that are sufficiently covered by agents, providing a geometrically grounded measure of epistemic uncertainty in feature space. **Local model disagreement** captures variability in the predicted dynamics among neighboring agents, providing a local estimate of predictive uncertainty.

Because kCELL operates in an online learning regime, these introspection metrics evolve through constant interaction with the environment and can be evaluated at every MPC iteration. Exploration can therefore be formulated as a deterministic, introspection-based regularization of the control objective, leading to an augmented cost function:

$$J\left(s_t, u_t\right) = \alpha J_{\text{task}}\left(s_t, u_t\right) + \lambda J_{\text{explore}}\left(s_t, u_t\right) \tag{4.12}$$

where J_{task} denotes the task-related cost function, J_{explore} the exploration penalty and $\alpha \in \mathbb{R}$, $\lambda \in \mathbb{R}$ are weighting factors that trade off exploitation and exploration.

4.2.1 Distance-based Expertise

Unlike opaque black-box models, kCELL provides an explicit measure of its degree of knowledge across specific regions of the state-action space through the spatial distribution of local expert agents. To encourage exploration, the model should aim to acquire data lying outside

the currently covered regions. The degree of knowledge can be quantified by evaluating the Mahalanobis distance between a state-action pair and the closest agents.

The objective is to deliberately drive the encountered state-action trajectories away from the currently covered regions of the input space in order to collect informative samples in poorly modeled areas. To achieve this, we introduce a distance-based penalty $p_{\text{dist}}(x_t)$ that augments the MPC cost function by promoting exploration of unknown regions, defined as

$$p_{\text{dist}}(x_t) = -\sum_{i \in \mathcal{D}_{\text{closest}}} w_i(x_t) \left((x_t - \mu_i)^\top \Sigma_i^{-1} (x_t - \mu_i) \right) \tag{4.13}$$

$$= -\sum_{i \in \mathcal{D}_{\text{closest}}} w_i(x_t) \, d_M(\mathcal{A}_i, x_t)^2 \tag{4.14}$$

where $x_t = [s_t, u_t]$, $\mathcal{D}_{\text{closest}}$ denotes the set of the k-closest agents, μ_i and Σ_i are the parameters of their activation function, and $w_i(x_t)$ are the normalized weights obtained from the activation value of each agent, computed as

$$w_i(x_t) = \frac{\phi_i(x_t)}{\sum_j \phi_j(x_t)}$$

Because the penalty is negative, minimizing the augmented MPC cost function promotes trajectories that maximize the distance to the currently well-covered regions. In this context, $p_{\text{dist}} \ll 0$ means that x_t is not represented in the training data. Conversely, $p_{\text{dist}} \approx 0$ indicates that x_t is close to a well-covered region and is therefore likely to be represented in the training data.

4.2.2 Local Disagreement

In well-covered regions, where multiple agents are close in state-action space, the variance between their individual predictions serves as a proxy for local predictive inconsistency. High variance, referred to as **high disagreement**, suggests that the local dynamics are either stochastic or noisy, or that the model has not yet converged to a stable local representation.

The objective is to drive state-action trajectories towards regions of high predictive disagreement, to force the model to either discover new areas or converge within already known regions of the feature space. This mechanism is functionally analogous to uncertainty-driven exploration strategies proposed in [211, 191, 179, 41] but remains fully intrinsic to the kCELL architecture, as it emerges from the social dynamics of the ensemble of local expert agents.

A disagreement-based penalty term $p_{\text{disagree}}(x_t)$ is defined such as

$$p_{\text{disagree}}(x_t) = -\sum_{i \in D_{\text{closest}}} w_i(x_t) \left\| \hat{f}(x_t) - f_i(x_t) \right\|^2 \tag{4.15}$$

where $x_t = [s_t, u_t]$, $D_{\text{closest}}$is the set of the k-closest agents, $\hat{f}(x_t)$ is the global prediction, $f_i(x_t)$ is the prediction of the agent i, and $w_i(x_t)$ are the normalized weights obtained from the activation value of each agent, computed as

$$w_i(x_t) = \frac{\phi_i(x_t)}{\sum_j \phi_j(x_t)}$$

Minimizing this negative term makes the optimizer seek state-actions where the *social consensus* among local expert agents is low. It thereby prioritizes data collection where the model is the most uncertain.

4.2.3 Comparative Study

The previous sections defined two introspection-driven metrics that could guide exploration of a MPC controller including a kCELL predictive model in an unknown environment. These two metrics are: **distance-based expertise** and **local disagreement**. This section moves from the mathematical formulation to empirical validation by evaluating these strategies in an online learning setup. The objective is to assess whether embedding these introspection metrics into the MPC cost function leads to improved coverage of the feature space compared to uninformed or purely exploitative strategies.

Experimental Setup The experiment is conducted on the classical **Pendulum** control task [39] provided by the *gymnasium* library [222]. This benchmark is selected because it exhibits nonlinear dynamics and presents a non-trivial exploration challenge. The task involves controlling a pendulum, initially starting from its stable downward equilibrium, and applying joint torques to swing it up until it reaches and maintains the unstable upright position.

The state space $\mathcal{S}$ is composed of the 2D coordinates of the pendulum's tip $(x, y) = (\cos\theta, \sin\theta)$ and its angular velocity $\dot{\theta}$. The action space $\mathcal{U}$ is composed of the joint torque applied to the pendulum. As mentionned above, the pendulum is initialized at the bottom stable equilibrium ($\theta = \pi$,$\dot{\theta} = 0$) at the start of each episode. No initial state randomization is applied. It ensures that high-energy states (near the upright position) are harder to reach. This setup increases the difficulty of exhaustive exploration for uninformed exploration strategies.

The controller interacts with the environment for a total of 5000 timesteps, divided into 10 episodes of 500 steps. A kCELL model is learned online from the stream of transitions $\{s_t, u_t, s_{t+1}\}$ to model the system dynamics. The kCELL predictive model is embedded in a MPC controller where the state-action trajectory optimization is performed using the **Cross-Entropy Method (CEM)**. A population-based optimizer is selected because its inherent stochasticity introduces variability in the candidate action sequences. This variability is critical during the early stages of kCELL learning. Indeed, it prevents the initialization of new agents with near-zero variance, which could pollute learning by making them directly overspecialized.

To properly evaluate the impact of kCELL's introspection on exploration, four different exploration strategies are compared.

– **Distance-based Exploration**: In this strategy, the cost function only includes the distance-based penalty:

$$J\left(s_{t:t+T}, u_{t:t+T-1}\right) = \sum_{k=t}^{t+T-1} p_{\text{dist}}\left(s_k, u_k\right) \tag{4.16}$$

This incentivizes the controller to maximize the distance between the future state-actions and the currently well covered regions of the state-action space.

– **Disagreement-based Exploration**: In this strategy, the cost function only includes the local model disagreement penalty:

$$J\left(s_{t:t+T}, u_{t:t+T-1}\right) = \sum_{k=t}^{t+T-1} p_{\text{disagree}}\left(s_k, u_k\right) \tag{4.17}$$

This incentivizes the controller to explore regions of the state-action space where the social consensus among local expert agents is low, i.e. where the disagreement between local experts is high.

– **Pure Exploitation**: In this baseline strategy, only the original task cost $J_{\text{task}}$is minimized (no regularization is applied). The task cost function, defined over the entire receding horizon of length T, $J_{\text{task}} : \mathcal{S}^{T+1} \times \mathcal{U}^T \mapsto \mathbb{R}$ maps states $s_{t:t+T} \in \mathcal{S}^{T+1}$ and actions $u_{t:t+T-1} \in \mathcal{U}^T$ trajectories to a scalar cost value. It follows a standard quadratic form such as

$$J_{\text{task}}\left(s_{t:t+T}, u_{t:t+T-1}\right) = \sum_{k=t}^{T} \left(\left(s_k - s_{\text{ref}}\right)^\top Q\left(s_k - s_{\text{ref}}\right) + u_k^\top R u_k\right) + \left(\left(s_T - s_{\text{ref}}\right)^\top Q\left(s_T - s_{\text{ref}}\right)\right) \tag{4.18}$$

where s_{ref} is the target upright position and $Q \in \mathbb{R}^{3\times3}$, $R \in \mathbb{R}^{1\times1}$ are weighting matrices determining the influence of the states and actions on the total cost. Comparing this baseline strategy with the **Distance-based** and **Disagreement-based** strategies should reveal the real contribution of using the proposed regularization methods.

– **Uniform Random Exploration**: In this baseline strategy, control actions are sampled uniformly at random. It represents a purely uninformed exploration strategy that does not require prior parameter tuning (like it is the case for exploration strategies based Ornstein-Ulhenbeck noise process).

All strategies that involve a continuously learning predictive model share the same kCELL architecture and hyperparameter set. These four exploration strategies are evaluated on the same total interaction budget. This way, we ensure that any observed differences most likely arise from the exploration strategy itself. To account for the stochasticity induced by the CEM optimizer and random exploration, all results are averaged over 5 independent seeds.

Quantitative State-Space Coverage To measure the state exploration efficiency, the continuous state space $\mathcal{S}$ is discretized into a regular grid of bins (50 bins per dimension). Each visited state is mapped to its corresponding bin, which is then marked as visited. State-space coverage is then defined as the ratio of visited bins over the total number of bins. The final coverage ratios (averaged over 5 seeds) are reported in Table 4.1.

This table shows that both introspection-driven strategies (distance-based expertise and local disagreement) achieve **significantly higher state-space coverage** than the uninformed random uniform strategy under the same interaction budget, with an increase of approximately 20% in coverage ratio. Conversely, the pure exploitation strategy, where the controller optimizes only the task cost without any exploration regularization, results in the

	Disagreement (ours)	Distance (ours)	Random Uniform	Full Exploitation
Coverage (in %)	**82.7 ± 4**	**80.8 ± 4**	62.4 ± 11	10.5 ± 5

Table 4.1: Measured state space coverage ratio (in %) after 5000 exploration steps. Best scores are highlighted in bold.

lowest coverage. This indicates that, without prior knowledge of the dynamics, exploitation (i.e optimizing the task objective) alone is insufficient to drive the system toward diverse and informative regions of the state-space. Furthermore, the distance-based and disagreement-based strategies achieve comparable final coverage levels, indicating that both introspection metrics are effective for guiding exploration beyond the regions initially supported by the data.

Evolution of Coverage during Exploration Figure 4.6 shows the evolution of state-space coverage over time. The two introspection-driven strategies consistently dominate the random uniform baseline. Pure exploitation seems to remain confined to a narrow subset of states and fails to significantly increase coverage.

The two proposed introspection-driven strategies show comparable growth in coverage over time. Minor differences in variability across seeds are observed but the mean coverage remains similar, suggesting that both strategies effectively promote exploration. Any apparent differences should be interpreted cautiously as they may not be statistically significant. Overall, these results confirm that embedding introspection metrics directly into the MPC objective yields an exploration advantage over uninformed stochastic action sampling.

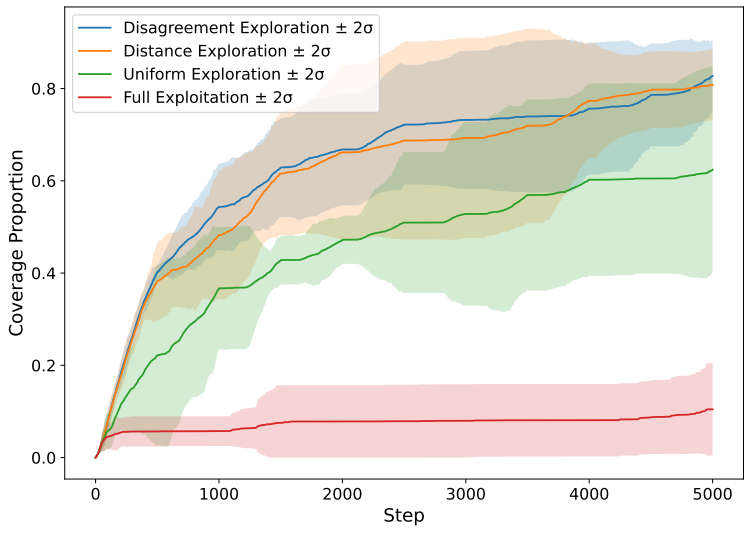


Figure 4.6: Evolution of the state space coverage ratio during exploration phase. x-axis corresponds to the number of steps and y-axis corresponds to the measured state space coverage ratio. The *solid* lines indicate the mean coverage over 5 seeds and the *shaded* area corresponds to the 95% confidence interval (i.e. mean$\pm 2\sigma$).

Spatial Distribution of Visited States Figure 4.7 presents heatmaps of the visited states in the $(\theta, \dot{\theta})$ space after the exploration phase. For all strategies, states corresponding to low angular velocities ($\dot{\theta} \approx 0$) and angles close to the starting configuration ($\theta \approx \pi$) are visited more frequently. This reflects the energy barrier associated with reaching higher energy states. These states require more control effort to be reached (e.g. the top states corresponding to near-upright pendulum configurations ($\theta \approx 0$ or 2π)).

Clear differences emerge between strategies. Both distance-based and disagreement-based explorations **achieve substantially broader coverage**, with frequent visits to high-energy states and large angular velocities. In contrast, the random uniform strategy exhibits sparse and uneven coverage, with large regions remaining unvisited (especially in high energy regions). Pure exploitation remains almost entirely confined near the starting state configuration. It consistently fails to overcome the energy barrier required to explore the upper half of the state-space.

These observations emphasize the limitations of uninformed random exploration in such a dynamical system, demonstrating that introspection-driven cost formulations guide the controller toward informative regions that are otherwise difficult to reach.

To sum up, these observations highlight the limitations of uninformed random exploration in such dynamical systems. They also demonstrate that introspection-driven costs allow the controller to deliberately seek out to reach informative regions that are otherwise difficult to reach due to an energy barrier.

Agent Activation Patterns and Local Geometry Figure 4.8 illustrates the spatial organization of kCELL agents after the exploration phase. Figure 4.8a shows agent activations projected onto the Cartesian coordinates (x, y) of the pendulum tip. Figure 4.8b displays activations in the $(\theta, \dot{\theta})$ space.

For the distance-based and disagreement-based strategies, agent activations cover the entire circular manifold corresponding to the pendulum's reachable positions (cf. Figure 4.8a), where yellow regions indicate higher activation values of the closest agents. This suggests that even high-energy and rarely visited states are locally modeled by dedicated agents. In contrast, with the random uniform and pure exploitation strategies, agent activations are concentrated in the lower-energy regions of the state-space.

A broader diversity in agent sizes can be observed for the introspection-driven strategies. This is most likely due to kCELL agent creation mechanism. This mechanism creates agents from a minimum number of samples to ensure that local models are quickly reliable. In high-velocity regimes or during abrupt torque changes, such as strong braking events, state transitions are more widely distributed, leading to larger initial covariance estimates. These larger agents are not pathological artifacts but instead reflect a specific experience of data (here corresponding to regions of high local variability).

For example, in the $(\theta, \dot{\theta})$ projection (cf. Figure 4.8b), some agents appear elongated along one dimension especially for the Distance-based strategy. An analysis of their covariance matrices reveals that these agents correspond to sharp directional changes in the dynamics, typically induced by large control inputs applied against the direction of motion. As such, agent geometry seems to provide an interpretable signature of local dynamical regimes encountered during exploration. More agents of this type can be observed for the Distance-based strategy. This could be due to a more agressive exploration strategy that prioritizes

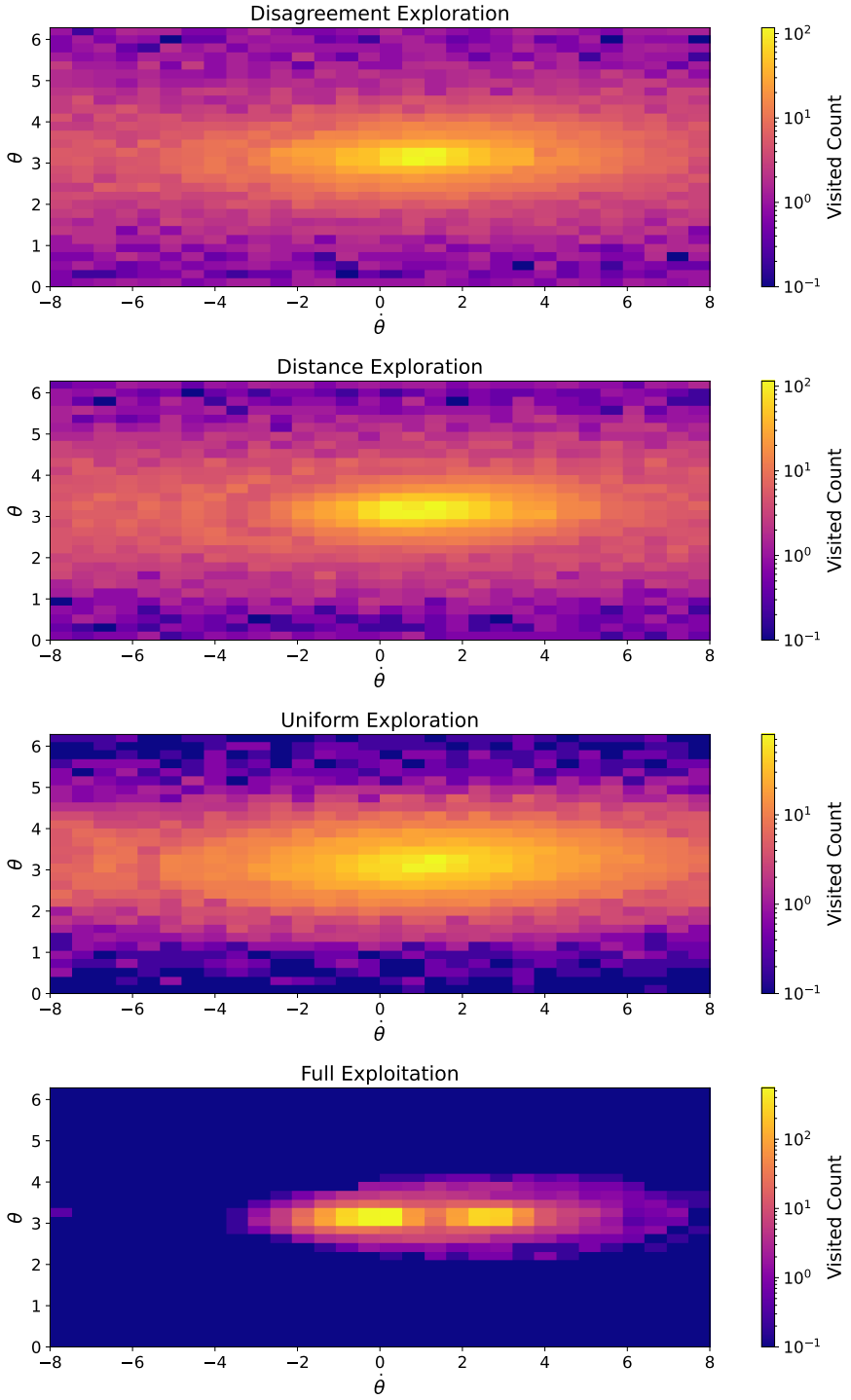


Figure 4.7: Space coverage for the four exploration strategies. The more *yellow*, the more the cell has been visited across the different seeds, conversely for *blue* colored cells. y-axis corresponds to the angle θ and x-axis corresponds to angular velocity $\dot{\theta}$.

more extreme action variations. These activation patterns demonstrate that kCELL does not only covered the state space uniformly but organized local models in a manner that reflects the underlying geometry and variability of the dynamics encountered during exploration.

Agent Population Dynamics Figure 4.9 shows the evolution of the number of agents during exploration. Both introspection-driven strategies lead to a substantially larger number of agents than the random uniform and pure exploration baselines. This increase is positively correlated with the observed improvement in state-space coverage (cf. Figure 4.6).

While both introspection-driven strategies generate more agents overall, the disagreement-based strategy exhibits **higher variability** in agent population across 5 seeds. This is consistent with the fact that disagreement among local agents is more sensitive than raw distance to local modeling inconsistencies. Thus, it could lead the disagreement-based controller to **explore less aggressively** by revisiting badly modeled regions of the state-action space. The more the space is covered, the more the local models can conflict locally and drive exploration to them for consolidation. It could explain the variance difference between the two introspection-driven strategies.

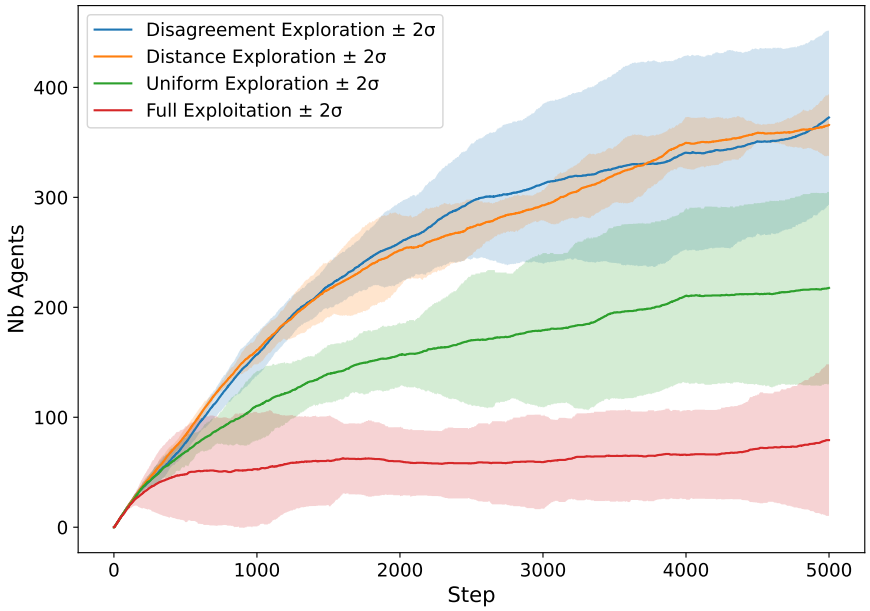
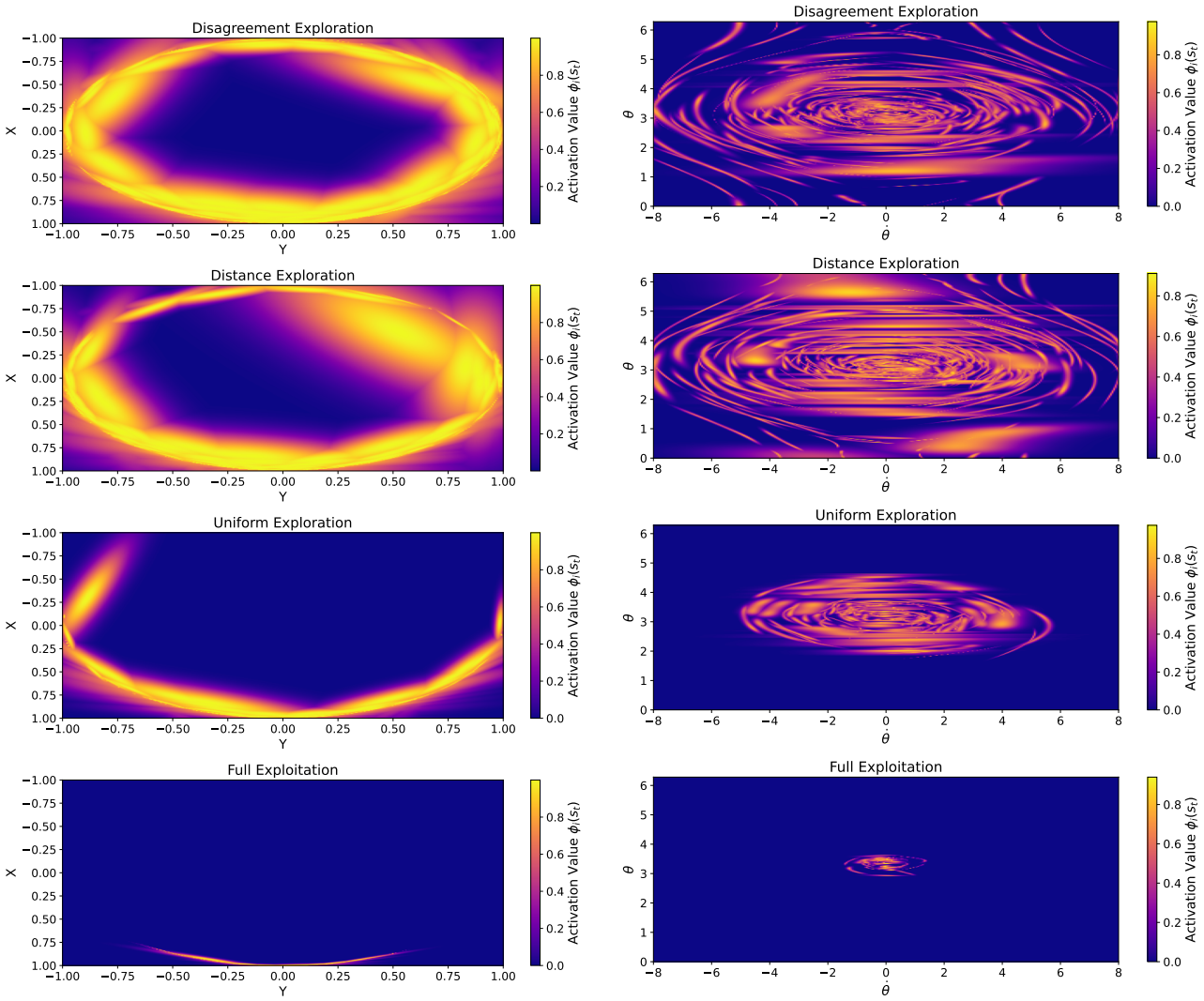


Figure 4.9: Evolution of agent population for the four considered strategies average over 5 seeds. y-axis corresponds to the number of agents and x-axis corresponds to the number of exploration steps. The *solid* lines indicate the mean number of agent over 5 seeds and the *shaded* area corresponds to the 95% confidence interval (mean$\pm 2\sigma$).

Prediction Accuracy and Sample Efficiency To assess whether introspection-driven exploration strategies translate into better models, the predictive accuracy of kCELL after the exploration phase is evaluated. For each exploration strategy, the learned model is evaluated on an exhaustive test dataset crafted specifically to cover a large portion of the state-action space. Model performance is quantified using the Root Mean Squared Error (RMSE) between predicted and ground-truth next states.



(a) (b)

Figure 4.8: Activation patterns of agents for the four considered strategies. From top to bottom, (1) Disagreement strategy, (2) Distance strategy, (3) Random Uniform Strategy, and (4) Full Exploitation strategy. 4.8a shows the activation patterns projected onto the 2D coordinates of the pendulum tip ($x = \cos(\theta)$ and $y = \sin(\theta)$). 4.8b shows the activation patterns projected onto the space of the pendulum angle θ (y-axis) and angular velocity $\dot{\theta}$ (x-axis).

Figure 4.10 presents a bar plot reporting the average RMSE across 5 seeds. We can notice that both introspection-driven strategies achieve **substantially lower prediction errors** than the random uniform and pure exploitation baselines. In contrast, **no statistically significant difference** in RMSE is observed between the distance-based and disagreement-based strategies.

These results indicate that introspection-driven exploration **improves the sample efficiency of online model learning** of a kCELL model within a MPC. While the two introspection metrics lead to slightly different exploration dynamics, they result in comparable models in terms of predictive quality for the same interaction budget. This suggests that the differences observed in state-space coverage are not leading to significant differences in model accuracy under the considered interaction budget and evaluation protocol.

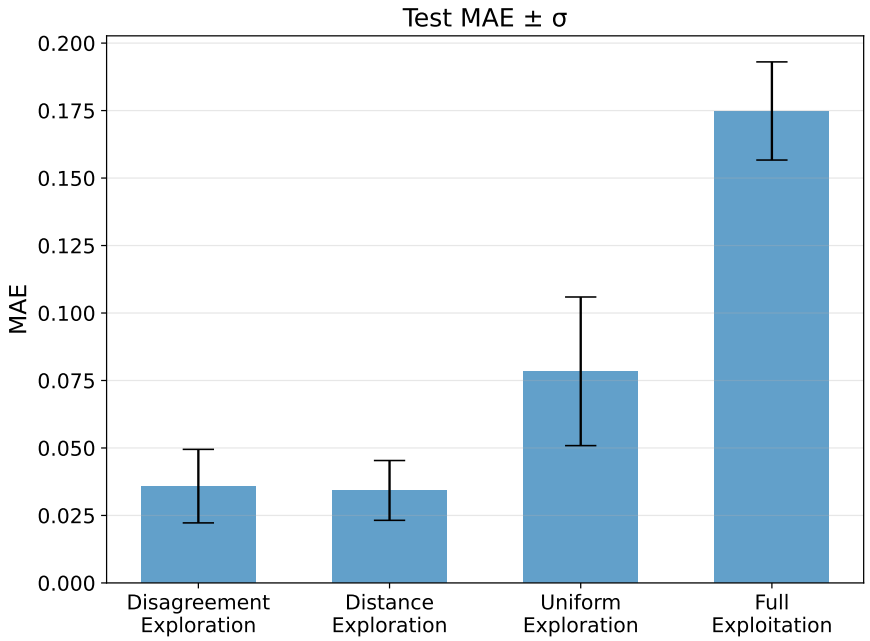


Figure 4.10: RMSE averaged over 5 seeds. y-axis is the RMSE and x-axis ticks corresponds to the exploration strategy.

Global Action Distribution Although distance-based and disagreement-based strategies yield comparable coverage and predictive accuracy, minor differences in exploration dynamics were observed in coverage variability and in agent population statistics across seeds. Those variations are subtle and may reflect stochastic effects of the optimizer rather than systematic differences. To investigate the general behavior of the controller under introspection-driven exploration objectives, the distribution of actions taken during exploration is analyzed.

Figure 4.11 presents Kernel Density Estimates (KDE) of the applied control inputs. We can notice that the resulting **action distributions are very similar** for both introspection-driven strategies. Three dominant modes centered around saturated control actions (-2, 2) and near-zero actions can be observed. This multimodal structure contrasts with the random uniform baseline. It indicates that both introspection-driven strategies induce structured non-uniform action profiles. The presence of repeated extreme actions suggests that introspection-

driven objectives actively encourage saturated actions in order to reach higher energy states rather than relying on diffuse stochastic perturbations.

These results do not allow to tell if the difference between the two introspection-driven exploration strategies are significant. As their global action profiles are fairly similar, the differences may be observable at a more local level.

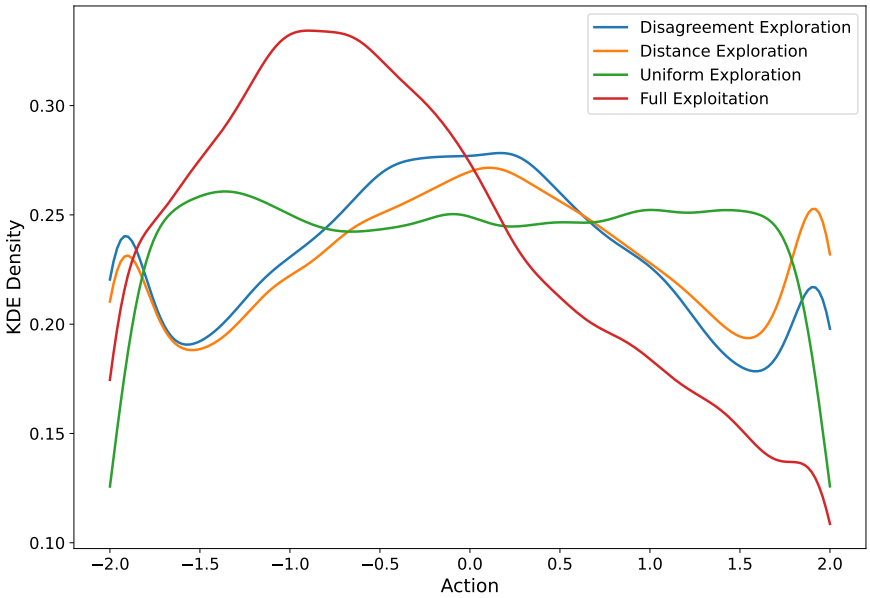


Figure 4.11: Kernel Density Estimates (KDE) of the actions taken during exploration for each considered exploration strategy. The y-axis corresponds to the KDE values and the x-axis ticks correspond to the actions. In the pendulum case, actions correspond to the applied torque value.

Discussion The experimental results presented in this section demonstrate that introspection-driven exploration strategies based on kCELL online modeling (without prior knowledge of the system dynamics) improve exploration efficiency over uninformed, random uniform exploration strategies. By incorporating distance-based expertise and local model disagreement directly into the MPC cost function, the controller is able to consistently escape well-modeled regions to acquire informative data about unknown regions of the state-action space.

Coverage metrics and spatial visualizations show that both introspection-driven strategies lead to a better coverage than random uniform exploration and pure exploitation strategies. Introspection-driven strategies are able to overcome the energy barrier induced by the pendulum environment. High energy configurations are repeatedly visited despite being hard to reach for uninformed strategies. These results confirm that efficient exploration cannot be achieved through randomly uniform actions alone. Instead, it requires additional motivations like model knowledge or uncertainty.

The analysis of global action distributions confirms that both introspection-driven strategies induce structured multimodal action profiles that facilitate exploration of high-energy states through extreme action sampling. The similarities observed in these distributions suggest that the two strategies behave comparably in terms of global action profile. Differences

in coverage or agent population remain unexplained and could arise from more specific local behaviors.

At the local scale, activation patterns and population dynamics provide insights into the underlying mechanisms of introspection-driven exploration strategies. Increased agent creation strongly correlates with improved coverage. This indicates that exploration emerges from the interaction between the introspection objective and kCELL's agent creation process. Furthermore, agent geometry and size variability reflect local properties of the explored dynamics. For example, for high-velocity regimes, created agents tend to be larger. For sharp directional changes, agents display a high variance on the action dimension. Each agent represents a singular experience of data. This highlights one of the key advantage of kCELL, which is its transparency. Indeed, uncertainty and internal model structure are not hidden within opaque parameters but instead manifest explicitly through the spatial organization of agents.

Finally, the observed improvements in predictive accuracy confirm that introspection-driven exploration has concrete benefits for online model learning. Under a fixed interaction budget, introspection-driven exploration strategies lead to more accurate learned dynamics.

Overall, kCELL's architecture allows to extract introspection metrics that can be leveraged successfully to build efficient uncertainty-aware exploration mechanisms without prior knowledge about the dynamics of the environment. Those mechanisms do not rely on external stochastic processes or auxiliary uncertainty models.

Limitations While the results demonstrate the effectiveness of introspection-driven exploration with kCELL, several limitations still stand.

kCELL was introduced to address the overoptimistic spatial representations of oCELL (cf. Section 3.2.5) that originated from the use of an activation function based on an orthotope. While allowing more degrees of freedom and a better representative power, using Gaussian-based spatialization in kCELL introduces its own set of constraints. Agents are spatialized using RBF kernels parameterized by the mean and covariance estimated from the local data distribution. As a result, agent creation requires a minimum variability level to avoid ill-conditioned covariance estimates. Typically, it implies not having zero variance on one dimension of the local data manifold.

Moreover, while population-based optimizers such as CEM introduce stochasticity that partially mitigates this issue by inducing action variability, excessive dispersion of actions can lead to overly extended areas of expertise and interpolation errors. This effect can be observed in Figure 4.8b where some agents appear elongated (especially with the distance-based strategy) due actions being too spread out. These agents still represent valid local experiences but such behavior may pose challenges when extending the approach to more complex control environments.

The Gaussian assumption underlying agent spatialization restricts the ability of kCELL to faithfully represent complex or multimodal feature manifolds that cannot be represented by a single Gaussian kernel. The agent population can approximate such structures through multiple overlapping agents, but this approximation remains indirect. Extending the spatialization mechanism beyond Gaussian assumptions, for example by using Kernel Density Estimates (KDE) [177] or mixture-based representations, could allow kCELL to better capture more complex local manifolds.

This experiment has been restricted to a low-dimensional control problem (3 state dimensions and 1 action dimension). The pendulum environment captures nontrivial exploration challenges such as energy barriers. However, scalability to higher-dimensional systems remains an open question for kCELL. In such settings, agent management, coverage estimation and computational overhead may require additional mechanisms.

Finally, this study considers distance-based expertise and local disagreement as independent exploration objectives. A combined approach was not explored, mainly because these introspection metrics operate on very different scales and have distinct geometrical interpretations. Investigating combinations or adaptive weighting schemes between distance-based and disagreement-based costs could reveal complementary effects and lead to more exhaustive exploration strategies.

Beyond exploration, introspection metrics extracted from kCELL naturally lead to the design of conservative control strategies under learned dynamics. While the studied exploration strategies exploit these metrics to deliberately seek poorly modeled regions, the same quantities can be repurposed during exploitation to bias the controller toward states where the model is locally reliable. In this case, introspection cost no longer act as penalty but as a confidence-aware bonus that encourages trajectories to remain close to existing agents, i.e close to known regions.

4.3 Conservative MPC with kCELL

The previous section demonstrated that introspection-driven exploration strategies derived from kCELL could be highly effective for guiding exploration in an unknown environment. However, different challenges arise during the exploitation phase through the reliability of control actions under imperfectly learned dynamics. Model-based controllers often suffer from compounding errors [40].

Definition 22. Compounding errors refer to the accumulation and amplification of small inaccuracies in sequential predictions, especially over extended time horizons. The process starts with minor inaccuracies in initial predictions within a receding horizon framework like MPC. These errors propagate multiplicatively, as each subsequent prediction uses the imprecise output of the previous one as input, such as:

$$\hat{s}_{t+1} = \hat{f}\left(\hat{s}_t + \epsilon_t, a_{t+1}\right)$$

where ϵ_t is the one-step error, $\hat{s}_t$ and $\hat{s}_{t+1}$ are the states predicted for timesteps t and $t+1$, and $\hat{f}$ is the global prediction function of the approximate model. Over iterations, errors accumulate and compound:

$$\epsilon_{t+T} \approx \prod_{i=1}^{T} \left(1 + \delta_i\right) \epsilon_t$$

where δ_i is the local amplification factor of errors. This can lead to divergent states, with $\hat{s}_{t+T} \gg s_{t+T}$ (or $\hat{s}_{t+T} \ll s_{t+T}$).

Compounding errors are linked to the notion of epistemic uncertainty. Indeed, they arise when the model lacks knowledge about the underlying true dynamics in specific regions

of the state-action space. To mitigate this issue, in [40], the authors discuss the use of Gaussian Processes (GP) [237] or Bayesian Neural Networks [32] to quantify the prediction uncertainty and take it into account in the optimization process to obtain safe conservative control policies.

kCELL's spatialized agent-based architecture provides valuable introspection metrics that can serve as proxies for epistemic uncertainty. These introspection metrics can be directly incorporated into the MPC cost function to regularize the behavior of the resulting policy. This regularization encourages the explored states (during optimization) to remain within known regions of the state-action space, thereby preventing the controller from wandering into unknown states that could induce large compounding errors. This section focuses specifically on distance-based expertise introspection metrics that quantify how far a state-action lies from regions that are sufficiently covered by agents. It provides a geometrically grounded measure of epistemic uncertainty.

To bias the policy toward a more conservative knowledge-aware behavior, an introspection-driven regularization of the control objective is defined. The augmented cost function is defined as follows:

$$J\left(s_t, u_t\right) = \alpha J_{\text{task}}\left(s_t, u_t\right) + \lambda J_{\text{conservative}}\left(s_t, u_t\right)$$

where J_{task} is the task-related cost function, $J_{\text{conservative}}$ the conservative regularization term and $\alpha \in \mathbb{R}$, $\lambda \in \mathbb{R}$ the adjustment weights to trade-off task exploitation and conservatism. In this context we define a conservative policy, as a policy that is biased in order to minimize epistemic uncertainty.

4.3.1 Distance-based Conservatism

Unlike opaque black-box models, kCELL provides an explicit measure of its degree of knowledge about specific regions of the state-action space through the spatial distribution of agents. To encourage conservatism, the controller must keep evaluated solutions and state trajectories close to the regions covered by the local expert agents. In kCELL, the degree of knowledge associated with an input $x_t = [s_t, u_t]$ can be quantified by the Mahalanobis distance to the nearest local experts. kCELL agents are local experts on the function to approximate and the closer an input is to the center of an agent, the most probable the prediction will be accurate (as shown in Section 3.3.3). Therefore, the objective of guiding the optimized trajectories toward the known regions of the state-action space consists of keeping them close to the center of agents.

To achieve this, a distance-based regularization term $q_{\text{dist}}\left(x_t\right)$ is introduced. It augments the MPC cost function by promoting conservative behaviors and is defined as:

$$q_{\text{dist}}\left(x_t\right) = \sum_{i \in D_{\text{closest}}} w_i\left(x_t\right)\left(\left(x_t - \mu_i\right)^\top \Sigma_i^{-1}\left(x_t - \mu_i\right)\right) \tag{4.19}$$

$$= \sum_{i \in D_{\text{closest}}} w_i\left(x_t\right) d\left(\mathcal{A}_i, x_t\right)^2 \tag{4.20}$$

where $x_t = [s_t, u_t]$, $D_{\text{closest}}$is the set of the k-closest agents, μ_i and Σ_i are the parameters of their activation function, and $w_i\left(x_t\right)$ are the normalized weights obtained from the activation

value of each agent, computed as

$$w_i(x_t) = \frac{\phi_i(x_t)}{\sum_j \phi_j(x_t)}$$

In contrast to the exploration penalty p_{dist} introduced in Section 4.2.1, q_{dist} is positive. Consequently, minimizing the augmented MPC objective encourages trajectories that remain close to the currently well-covered regions. In this context, $q_{\text{dist}} \gg 0$ means that x_t is currently not supported by the agents' training data. Conversely, $q_{\text{dist}} \approx 0$ indicates that x_t is close to a known region of the state-action space and is therefore likely to have been encountered during training. As q_{dist} is convex, it can easily be incorporated in a convex cost function for optimization with fast conic solvers like OSQP [212] within the context of Sequential Quadratic Programming (SQP) optimization framework.

4.3.2 Comparative Study

The previous section introduced a distance-based conservative regularization mechanism, derived from a kCELL predictive model. This mechanism is designed to bias the optimization process in a MPC controller toward known state-actions. This section studies the impact of this regularization mechanism on the reliability of the optimization process through a low-dimensional case study. The goal is to assess whether incorporating this regularization term into the MPC cost function yields trajectories on which the local expert agents in the kCELL predictive model exhibit **more expertise** (i.e. higher activation values), **fewer compounding errors** and a possible **tradeoff between conservatism and task resolution performance**.

Experimental Setup The experiment is conducted on the classical Pendulum control task [39] that provides a simple benchmark for assessing how model inaccuracies propagate over a receding horizon. As described in Section 4.2.3, the state space $\mathcal{S}$ is composed of the tip coordinates (x, y) and angular velocity $\dot{\theta}$ of the pendulum.

In the previous exploration study, the model was learned while being used for exploring the environment. In contrast, this experiment uses a frozen kCELL dynamics model to ensure that the obtained results are caused by the introduced conservative regularization mechanism and not by continuous model adaptation. The model is initially trained through a dedicated interaction budget of 30000 steps using an informed stochastic exploration policy based on Ornstein-Uhlenbeck (OU) noise. This ensures that the kCELL predictive model has acquired a solid knowledge base before the evaluation phase.

For this task, the ensemble of local expert agent of the kCELL model is spatialized only in the state space $\mathcal{S}$ (without including actions). This design choice is motivated by empirical observations on the pendulum task. Indeed, spatializing agents jointly over state and action dimensions leads to slower stabilization of agent population during learning, whereas state-only spatialization results in faster convergence of local models and more stable agent coverage. Given the low-dimensional action space and the smooth dynamics, state-based spatialization provides a favorable tradeoff between model predictive accuracy and sample efficiency.

In contrast to the population-based CEM optimizer used for exploration, trajectory optimization is performed here using Sequential Quadratic Programming (SQP) with the OSQP conic solver [212]. A gradient-based optimizer is selected to isolate the effect of the proposed regularization by removing the stochastic variability inherent to sampling-based methods. This allows for a deterministic analysis of how the regularization mechanism influences the convergence of the optimizer toward safe, known regions.

The efficiency of distance-based conservatism is evaluated by **varying the regularization coefficient** $\lambda \geq 0$. The cost function minimized by the SQP optimizer over a receding horizon of length $T = 15$ is defined as

$$J\left(s_{t:t+T}, u_{t:t+T-1}\right) = J_{\text{task}} + \lambda q_{\text{dist}}$$

The task cost $J_{\text{task}}$is defined as a standard quadratic form as presented in Equation 4.18. The following strategies are compared:

- **Baseline Exploitation** ($\lambda = 0$): In this strategy, only the unregularized task cost J_{task} is minimized. This strategy represents a standard greedy controller that trusts the learned model implicitly, assuming it has perfect knowledge of the dynamics.
- **Conservative Exploitation** ($\lambda > 0$): In these strategies, the distance-based regularization term q_{dist} is added to the task cost J_{task}. This incentivizes the optimizer to find trajectories that remain close to the centers of existing local expert agents in the kCELL model. Therefore, the optimizer *stays close to what the model knows.*

To evaluate the robustness and reliability of the controller independently of specific initial conditions, performance is assessed across an exhaustive grid of initial states sampled throughout the $(\theta,\dot{\theta})$ space. For each initial state, the MPC outputs an action sequence using the frozen kCELL model coupled with the SQP optimizer. The resulting predicted trajectory is compared to the ground-truth trajectory produced by the true environment dynamics. This way, the compounding errors effects can be measured as the divergence between the model's prediction and the ground truth over the receding horizon. The considered strategies share identical MPC horizon length, cost matrices (Q and R) for J_{task}, solver parameters and physical constraints bounding the states and actions. By comparing the baseline to various values of λ, the goal is to characterize the safety-performance trade-off induced by the regularization mechanism.

Validation of the Regularization Behavior

To validate the behavior of the regularization term, the mean activation of the closest agents along the optimized state trajectories is measured. Activation of agents serves as a proxy for epistemic uncertainty. Figure 4.12 presents boxplots of the mean activation of agents for each considered value of the regularization coefficient λ.

As λ increases, a clear increase in mean activation can be observed. This increase indicates that the optimizer progressively favors trajectories that remain closer to the centers of kCELL agents. This behavior is consistent with the expected behavior of the regularization term q_{dist}. The optimization landscape is effectively reshaped.

In addition, increasing λ also yields a noticeable reduction in the spread of the activation distributions across initial conditions. This reduced variability indicates that under stronger regularization, trajectories initialized in different regions are increasingly constrained to remain within well-covered regions. This effect suggests that the regularization not only increases overall conservatism but also homogenizes the levels of expertise of the kCELL model over optimized trajectories.

These results validate the behavior of the introspection-driven regularization mechanism. The regularizer acts as intended: it **increases the proximity of optimized trajectories to kCELL agent centers** and reduces variability in observed model expertise across initial conditions.

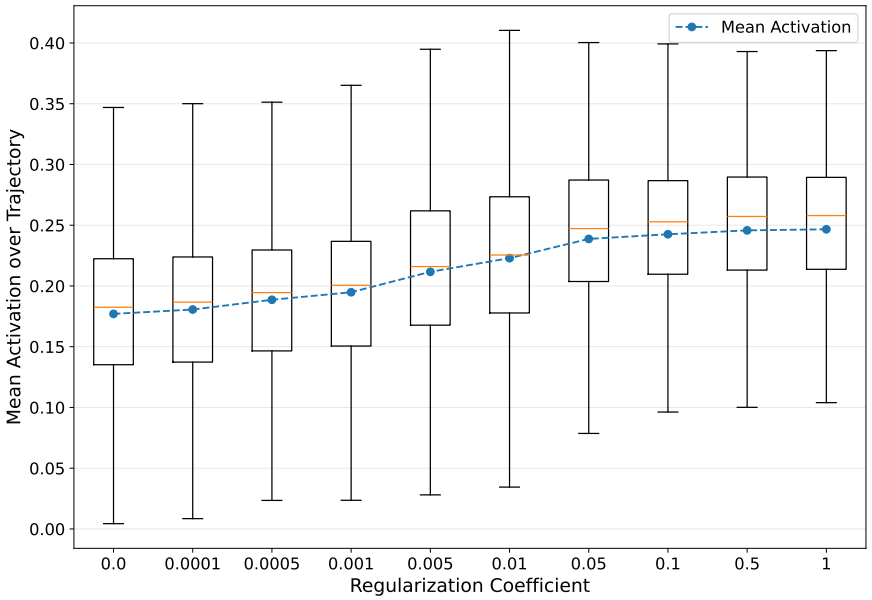


Figure 4.12: Boxplots of the mean agent activation across the states of the optimized trajectories for various values of regularization coefficient λ. y-axis corresponds to the mean activation over optimized trajectories and x-axis corresponds to values of λ. The blue curve corresponds to the global mean and the orange bars correspond to the median value.

Impact on Prediction Accuracy

Let us now study the impact of the regularization coefficient λ on the prediction accuracy. Figure 4.13 presents boxplots of the Mean Absolute Error (MAE) across the states of the optimized trajectories for various values of the regularization coefficient λ. For each value of λ considered in each optimized trajectory, the predicted state transitions produced by kCELL are compared to ground-truth trajectories. Prediction errors are computed and averaged over the receding horizon.

As the regularization coefficient increases, a downward shift in the error distributions can be observed. The higher the values of λ, the lower the median and mean prediction errors. This indicates that trajectories optimized under strong regularization are predicted more accurately on average by the learned model. This behavior is consistent with the core

assumption of kCELL. Recall that this assumption stipulates that regions of high agent activation correspond to areas where local linear models provide the most reliable approximations of the true dynamics.

This reduction of the prediction error does not arise from any modification of the learned kCELL model itself which remains fixed during evaluation. Instead, it is a direct consequence of the optimizer preferentially selecting trajectories that remain within regions of the state space that are well supported by training data. Thus, the regularization acts as a mechanism for **smarter model usage** rather than model improvement.

In addition, the downward shift in MAE distributions when the values of λ increase, is accompanied with a decrease in the spread of the error distributions. This indicates that large prediction errors become progressively less frequent as regularization strength increases. This suggests that distance-based regularization not only improves average prediction accuracy but also reduces the occurence of extreme prediction failures. This effect is particularly relevant in terms of reliability of the model as it seems to successfully prevent, to some extent, the controller to exploit faulty Out-of-Distribution (OoD) trajectories.

Overall, these results confirm that the spatial organization of agents in a kCELL model encodes meaningful information about local model reliability. This information can be effectively exploited during trajectory optimization. By encouraging trajectories to remain within regions of high agent coverage, the proposed regularization yields more accurate and more consistent predictions. This way, it improves the reliability of model-based control with a kCELL predictive model.

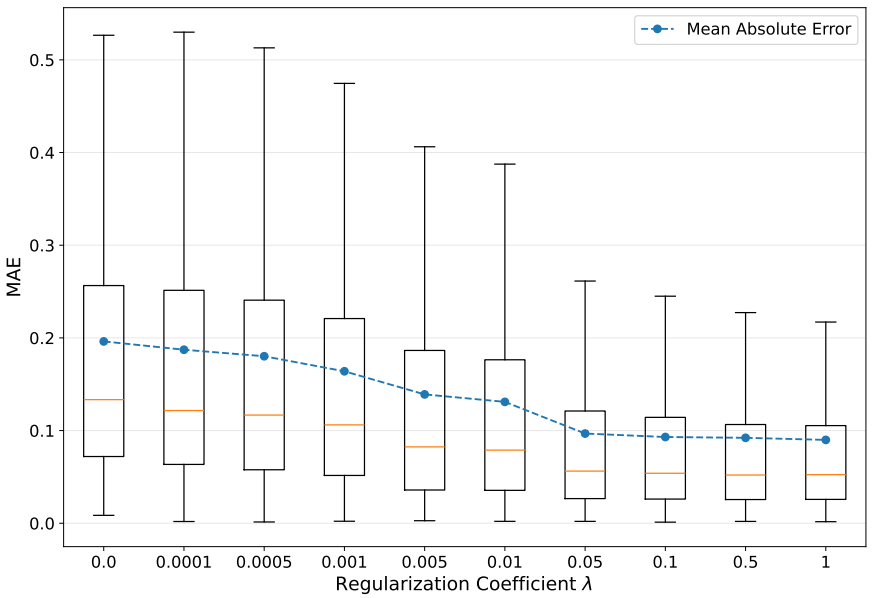


Figure 4.13: Boxplots of the Mean Absolute Error (MAE) across the states of the optimized trajectories for various values of regularization coefficient λ. y-axis corresponds to the mean absolute error over optimized trajectories and x-axis corresponds to values of λ. The blue curve corresponds to the global mean and the orange bars correspond to the median value.

Impact on Compounding Errors

We have seen that the use of learned dynamics models in model-based control is limited by error compounding effects over long prediction horizons. Even small local inaccuracies can compound across successive rollout steps, and rapidly degrade the prediction quality and optimization results in the process [40]. Figure 4.14 shows the evolution of the mean absolute prediction error over the prediction horizon for various values of λ.

In the absence of regularization ($\lambda = 0$), for the baseline (the *darkest blue* line in the plot), the prediction error increases rapidly with the horizon length. This increase reflects the compounding effects of model errors over the receding horizon. As λ values increase, the rate at which prediction error grows is substantially reduced. Stronger regularization leads to consistently lower prediction errors especially for more distant horizon steps. This confirms that the main benefit of the proposed regularization lies in **mitigating compounding errors** rather than solely improving one-step prediction accuracy.

Overall, these results show that this introspection-based regularization term **improves the reliability of long-horizon predictions in MPC**. By maintaining the trajectory within highly covered regions, the error propagation is reduced. It prevents the exponential divergence typically observed in the unregularized case over the receding horizon. It reduces the myopic behavior induced by error-prone rollouts and enhances the practical usability of kCELL for more reliable control policies.

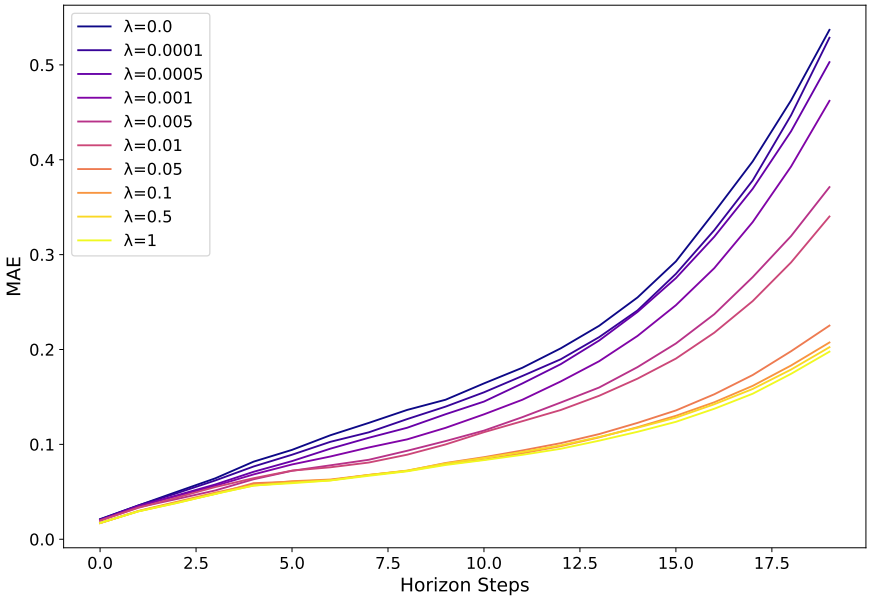


Figure 4.14: Evolution of the Mean Absolute Error (MAE) over the prediction horizon for various values of regularization coefficient λ. y-axis corresponds to the mean absolute error over optimized trajectories and x-axis corresponds to the horizon steps. Each colored curve corresponds to a value of λ. The unregularized baseline case corresponds to the $\lambda = 0$ curve.

Tradeoff between Cost & Accuracy

While the introspection-based regularization term improves the reliability of predictions, such conservatism is expected to impact global task solving performance. Figure 4.15 shows boxplots of the cumulative task cost (J_{task}) across trajectories for various values of λ. As λ increases, the mean cumulative cost slightly increases compared to the unregularized baseline ($\lambda = 0$). This indicates that the controller increasingly sacrifices task optimality in order to stay within regions of the state space that are well supported by the learned kCELL model. In addition, we can notice that the spread of the cost distributions decreases with stronger regularization. This suggests that the conservative behavior leads to more uniform, less optimal, performances across initial conditions.

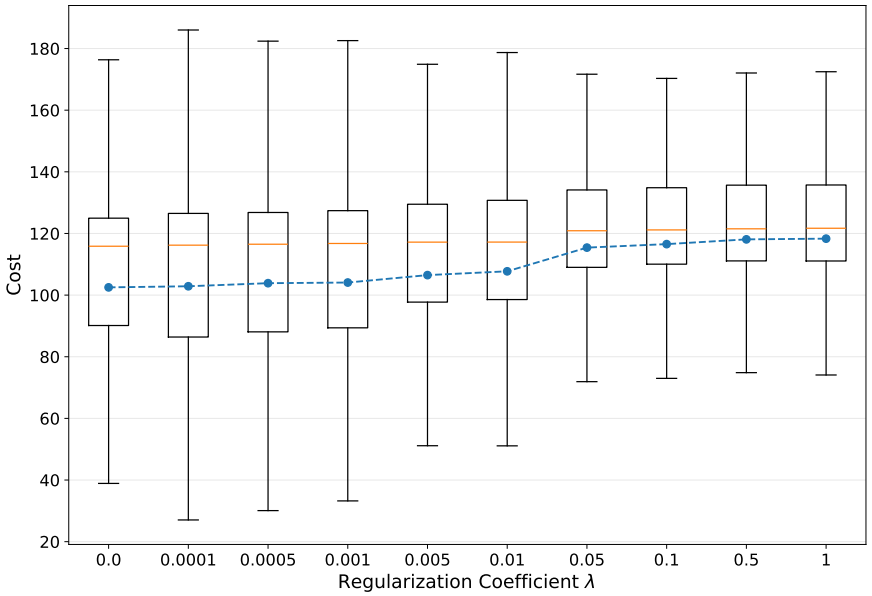


Figure 4.15: Boxplots of the cumulative costs over optimized trajectories for various values of regularization coefficient λ. y-axis corresponds to the accumulated cost over optimized trajectories and x-axis corresponds to values of λ. The *blue* curve corresponds to the global mean and the orange bars correspond to the median value.

Figure 4.16 shows the evolution of the mean relative gap in cumulative cost (vs. the $\lambda = 0$ baseline) over the prediction horizon for various values of λ. Having higher regularization coefficient λ resulted in having higher accumulated costs compared to the unregularized baseline. This effect is more pronounced for longer horizons. It indicates that conservative trajectory selection restricts the optimizer's ability to exploit aggressive control strategies that rely extensively on extrapolation in predictions.

To explicitly link predictive performance to task performance, Figure 4.17 presents a scatter plot of the mean absolute error over the cumulative cost for various values of λ. A nonlinear relationship can be observed if we interpolate a cubic polynomial on those points. Indeed, trajectories associated with lower prediction errors tend to induce higher costs. The interpolated curve exhibits a steep initial decrease in cost as the prediction error increases, followed by a more gradual decrease.

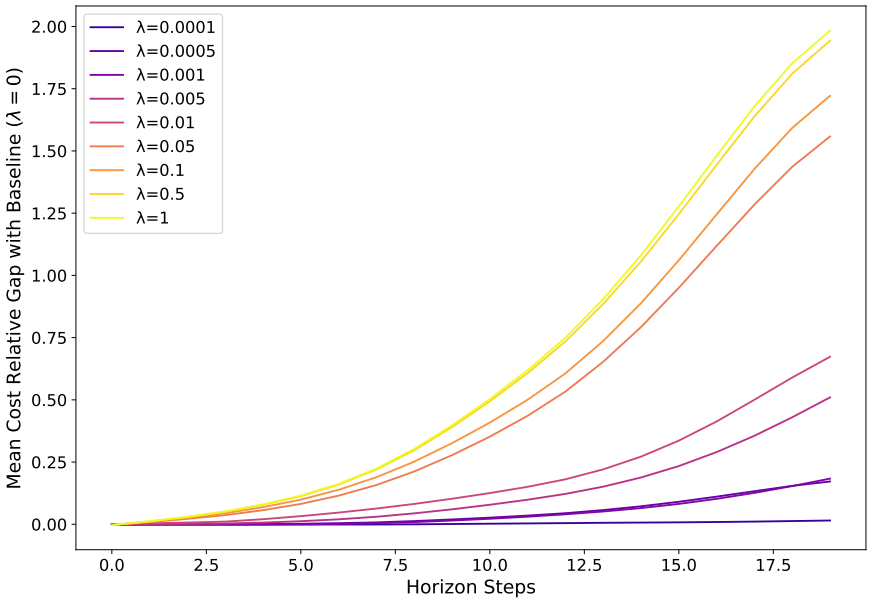


Figure 4.16: Evolution of the mean relative gap in cumulative cost (vs. baseline $\lambda = 0$) over the prediction horizon for various values of regularization coefficient λ. y-axis corresponds to the relative gap over optimized trajectories and x-axis corresponds to the horizon steps. Each colored curve corresponds to a given value of λ.

Overall, for this specific control task, these results reveal an **explicit tradeoff between cost and accuracy** induced by the regularization term. Stronger regularization improves prediction accuracy but reduces error accumulation at the expense of control optimality. The shape of the tradeoff shown in Figure 4.17 suggests that it is possible to find intermediate values of the regularization coefficient λ that yields substantial gains in predictive accuracy with a limited increase in cost. Further works on different control tasks should be carried on to find a practical tuning mechanism for the conservative coefficient.

Discussion The experimental results presented in this section demonstrate that the spatial organization of local expert agents in kCELL can be directly exploited during trajectory optimization to improve the reliability of the controller. Incorporating a distance-based regularization term into the MPC cost effectively bias the optimizer toward the regions of the state space where the learned dynamics are supported by training data.

The results show that this regularization mechanism facilitates **smarter model usage rather than model improvement**. The observed reduction in MAE and the attenuation of compounding errors without any modification of the frozen kCELL model indicate that kCELL introspective properties can be leveraged to inform an optimizer to distinguish between high-confidence interpolations and risky extrapolations. This makes the controller more reliable.

Furthermore, these results provide empirical support for the Decreasing Knowledge assumption at the core of kCELL algorithm (cf. Section 3.3.1). A correlation between agent activation and reduced prediction error can be observed through the effects of the regularization mechanism. This observation is consistent with the results presented in Section 3.3.3

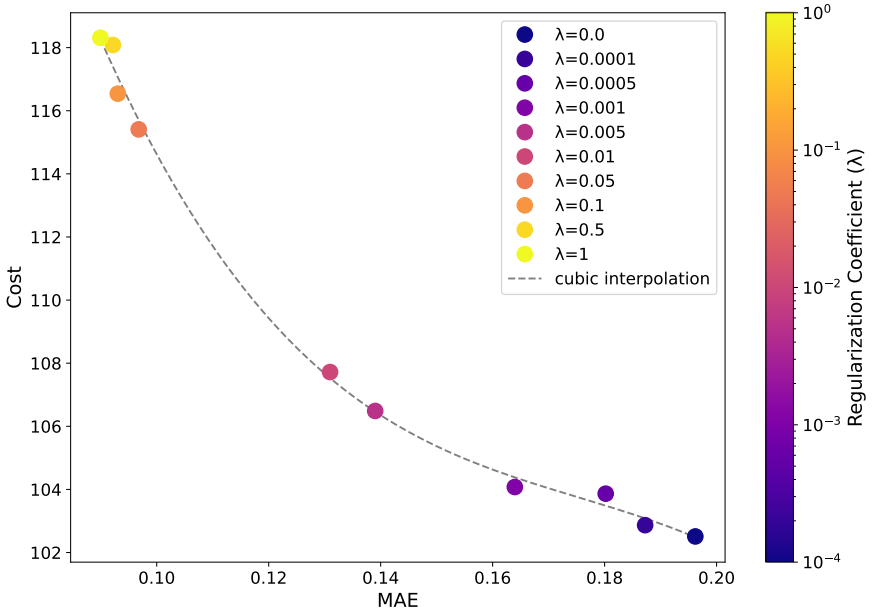


Figure 4.17: Cumulative cost against Mean Absolute Error (MAE). y-axis corresponds to the cumulative costs and x-axis corresponds to the mean absolute errors over trajectories. Each colored point is associated to a value of λ. The *dashed line* corresponds to a cubic interpolation.

and further reinforces the assumption that local linear models become increasingly reliable near their center. The regularization mechanism thus transforms this assumption into a control-relevant advantage.

However, this improved reliability comes at the cost of task optimality. The identified safety-performance trade-off reveals that as λ increases, the controller **increasingly prioritizes known states over minimizing the task cost**. The nonlinear nature of the Pareto frontier illustrated in Figure 4.17 seems promising, as it suggests that substantial gains in predictive reliability could be achieved for a relatively minor increase in cumulative cost. The regularization mechanism acts as a soft feasibility constraint on the validity of the model, considering that unknown regions are either unfeasible or dangerous. This is similar to trust-region methods in nonlinear optimization [53, 199].

Limitations While these experimental results demonstrate the potential of introspection-driven conservatism to improve the reliability of a MPC controller, several limitations remain.

Although the distance-based measure correlates strongly with accuracy, it remains a heuristic proxy for epistemic uncertainty rather than a calibrated probability. Consequently, this approach should be viewed as a risk-reduction mechanism rather than a formal safety guarantee. The efficiency of the regularization is fundamentally dependent on the quality of the local expert agent distribution.

This case study has leveraged a low-dimensional system (the Pendulum) where Mahalanobis distances remain numerically meaningful. In higher-dimensional state-action spaces, agent coverage naturally becomes sparser. The distance to the nearest agent may become

less informative due to the curse of dimensionality. The efficiency of this regularization in more complex high-dimensional systems remains a subject for further investigation. In addition, the effect of the sparsity of the population of agents on the conservative regularization behavior has not been studied. Training multiple models with varying agent densities would be necessary to better characterize how coverage quality impacts the reliability gains.

The decision to spatialize agents solely in the state space was a task-specific choice made to accelerate population stabilization during the learning phase of the frozen kCELL model. This choice limits the expressiveness of agents regarding actions. While this design choice is appropriate for the Pendulum, it may not generalize to more complex high-dimensional dynamical systems. In such cases, regularization in joint state-action space could be required.

Finally, the proposed approach discourages extrapolation into unknown regions but does not provide rigorous barrier certificates or Lyapunov stability guarantees required for safety-critical certification. Given the transparency of the inner workings of kCELL models, this could represent a promising research area for future work.

4.4 Synthesis and Conclusion

This chapter has explored the practical integration of a kCELL predictive model into a model-based controller through the MPC framework. It has bridged the gap between the theoretical foundations laid out with the CELL formalism in Chapter 3 and the concrete challenges of control where no prior knowledge of the dynamics is available. By leveraging the emerging structure of the ensemble of local expert agents of kCELL, we demonstrated how this type of learning model can use its own internal state to improve both data collection (**exploration**) and control reliability (**exploitation**).

Section 4.2 has addressed the exploration challenge in unknown environments. Two introspection metrics derived from a continuously learning kCELL model have been introduced: **distance-based expertise** and local **model disagreement**. The experimental results obtained on the Pendulum task have shown that incorporating these signals into the MPC cost allows the controller to systematically discover complex dynamics that uninformed random strategies fail to reach. This confirms that kCELL's social multi-agent structure provides a reliable mechanism for autonomous unsupervised data acquisition.

Section 4.3 has tackled the reliability challenge of learned models during exploitation. The distance-based expertise signal has been repurposed as a conservative regularization term used to mitigate the error compounding effect. It represents a major challenge in model-based control with learned dynamics. By biasing the optimizer within MPC, toward existing kCELL agents, we have demonstrated a significant reduction in perception errors between the ground truth and the predictive model. This contribution highlights the ability of kCELL to provide a meaningful proxy for epistemic uncertainty (derived from the spatial organization of local expert agents) that can be integrated into a gradient-based optimization loop (e.g. with SQP).

Overall, the conclusions of this chapter support the continuous local expertise assumption at the core of kCELL's design. We have shown that the spatial organization of kCELL agents represents a functional asset that contributes to a *smarter model usage*. The identified trade-off between task performance and predictive reliability provides a promising lead for designing

safer controllers.

Despite these interesting and promising results, significant research gaps remain. Indeed, due to the way the agents are spatialized in kCELL, it suffers from the curse of dimensionality, hurting its scaling capabilities in higher dimensional input spaces because distances are stretching and the notion of neighborhood becomes meaningless. This represents a crucial research direction for learning systems based on an ensemble of local learning agents governed by social rules. We suggest that future works should draw inspiration from how LWPR handles an increasing number of input dimension through Partial Least Squares (PLS) [229]. PLS is a method that projects the data onto a lower-dimensional latent subspace capturing the directions of greatest covariance between inputs and outputs.

Moreover, despite its ability to quantify epistemic uncertainty, kCELL lacks predictive uncertainty estimation. In safe control with Gaussian Processes (GP) for example, predictive uncertainty can be leveraged in the optimization process to robustify control by taking into account the propagation of uncertainties along predicted trajectories. To this end, Chapter 5 explores the modularity of the CELL formalism by proposing a novel instantiation, gpCELL, that leverages GPs as the internal model of local expert agents to introduce principled predictive uncertainty quantification.

Chapter 5

Heterogeneous nonlinear CELL

In the previous chapters, the CELL (Context Ensemble Local Learning) formalism has been established as a robust framework for designing online learning algorithms. Specifically Chapter 3 introduced the first instantiations of this formalism and demonstrated their ability to maintain a stable and informative spatialized representation of knowledge. Building upon these foundations, Chapter 4 has focused on the integration of these learning models into a Model Predictive Control (MPC) pipeline. This integration has highlighted how the introspective properties emerging from the structural transparency of the ensemble of local expert agents, could be used to improve the reliability of the predictive model in an optimization-based control problem, or to guide exploration in an unknown environment.

This chapter moves away from control applications and gets back to the underlying predictive model layer to explore a core property of the CELL formalism: **model-agnosticity**. Because the spatial organization and the local learning layers are decoupled, the agents within an ensemble can theoretically use a wide range of model classes. Exploiting this modularity allows for the design of heterogeneous populations.

Definition 23. In the context of this thesis, a **heterogeneous ensemble of local expert agents** denotes a population of agents whose internal representational capacities vary across the input space, adapting to the complexity of the local training distribution (as illustrated in Figure 5.1).

Traditionally, in multi-agent systems, heterogeneity refers to a mixture of distinct agent classes. For example, some agents would employ simple linear regressions while others would use high-order polynomials or neural networks. While such an approach offers theoretical flexibility, it introduces a significant meta-learning challenge: the model must decide a priori or through complex heuristics which model class is best suited for a specific locality.

To bypass the need for manual model selection while retaining the benefits of diverse local knowledge representation, we propose a novel instantiation of CELL. This instantiation leverages Gaussian Processes (GP) as the internal model for each agent and is called gpCELL. By using the GPs' ability to act as universal function approximators, the expressive power of each agent is not fixed, but emerges from its local training distribution. This approach not only allows the agent to naturally adapt to varying local data complexities but also introduces an estimation of predictive uncertainty. This predictive uncertainty is complementary to the

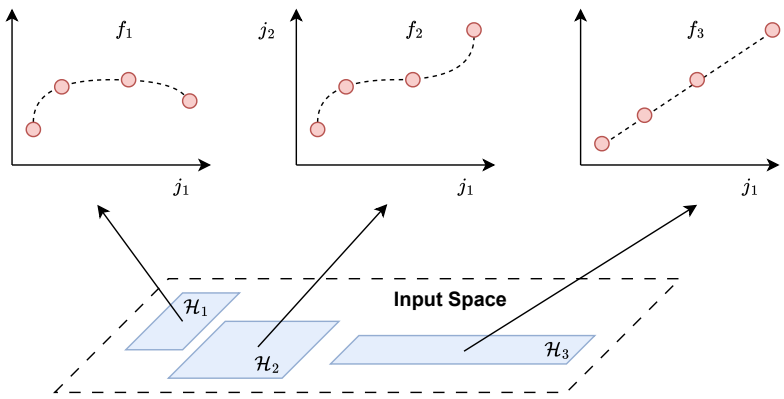


Figure 5.1: Heterogeneous ensemble of local expert agents. Each *blue rectangle* corresponds to the area of expertise of one local expert agent. Each agent has a different type of internal model (f_1, f_2, f_3), as illustrated by the three plots on top of the diagram. The *red dots* correspond to the local training points used for training.

epistemic uncertainty derived from the spatial organization of local expert agents (cf. Chapter 3).

The contributions presented in this chapter are threefold:

- **Scalable Online Gaussian Processes**: We introduce gpCELL, a novel CELL instantiation that bypasses the $O(n^3)$ computational bottleneck of global GPs through a horizontally scaling, distributed architecture.
- **Uncertainty Quantification in CELL**: The integration of GPs in a CELL instantiation enables a more rigorous estimation of predictive uncertainty. We demonstrate through an online modeling task that this uncertainty is empirically well-calibrated.
- **Heterogeneous Agent Design**: Through model selection, gpCELL allows each agent to independently adapt its complexity to the local data manifold. Thus, gpCELL represents a step towards the design of learning systems that take advantage of a heterogeneous ensemble of local expert agents.

5.1 Gaussian Process Regression

Gaussian Process Regression (GPR) is a Bayesian non-parametric framework used for supervised learning. Unlike traditional regression models that estimate fixed parameters for a specific function, GPR provides a **probabilistic distribution over possible functions** that match the observed data.

GPR is widely used for modeling complex stochastic phenomena where uncertainty quantification is as critical as the prediction itself, such as in signal processing [181], in financial modeling [95] or in dynamics modeling for safe control [27].

One of the main advantages of Gaussian Processes (GPs) is that, with an appropriate choice of kernel (e.g. RBF kernel), they can represent arbitrarily complex continuous

functions [238], capturing any variation regime from highly irregular trajectories to smooth differentiable paths.

In GPR, the underlying function $f(x)$ is assumed to follow a GP prior which is defined by a function $m(x)$ (often assumed to be 0 or constant) and a covariance kernel $K(x, x')$ such as:

$$f(x) \sim \mathcal{GP}(m(x), K(x, x')) \tag{5.1}$$

For a supervised learning task with training inputs $X \in \mathbb{R}^{n \times d}$ and noisy observations $y \in \mathbb{R}^n$, assuming a zero mean prior, the joint distribution of both the training data and the prediction $f(x_*)$ at a new test point $x_* \in \mathbb{R}^d$, is a multivariate Gaussian vector:

$$\begin{pmatrix} y \\ f(x_*) \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K(X,X) + \sigma_n^2 I & K(X, x_*) \\ K(x_*, X) & K(x_*, x_*) \end{pmatrix}\right) \tag{5.2}$$

where σ_n^2 represents the observation noise variance. The prediction is performed by conditioning this joint distribution on the observed data y. This results in a posterior distribution for $f(x_*)$ defined by:

- **Predictive Mean**:

$$\mu_* = K(x_*, X)\left[K(X,X) + \sigma_n^2 I\right]^{-1} y \tag{5.3}$$

- **Predictive Variance**:

$$\sigma_*^2 = K(x_*, x_*) - K(x_*, X)\left[K(X,X) + \sigma_n^2 I\right]^{-1} K(X, x_*) \tag{5.4}$$

Learning in GPR does not involve finding specific weights but rather **optimizing the hyperparameters** of the kernel K. This is usually achieved by maximizing the **log marginal likelihood**, which balances the model's fit to the training data against its complexity.

Despite their flexibility, GPs have significant constraints:

- **Computational Complexity**: The inversion of the covariance matrix $[K(X,X) + \sigma_n^2 I]$ has a complexity in $O(n^3)$ with n the number of training data which is impractical for large datasets and for online learning.
- **Memory Usage**: Storing the covariance matrix requires $O(n^2)$ space. This makes GPR hard to scale to large datasets without using approximation methods (like Sparse Gaussian Processes [21]).

5.2 Local Expert Agents

The previous section has presented the Gaussian Process Regression (GPR) and it has emphasized on the $O(n^3)$ computational complexity bottleneck that limits its application on large datasets or for online learning.

To address these limitations, gpCELL applies the principle of **horizontal scaling**: rather than maintaining a single global GP, the system distributes the modeling task across an ensemble of local expert agents, each containing a smaller GP. As a result, each GP learns with fewer data points and can be refitted when needed in a reasonable amount of time. This

section details the tripartite structure of the local expert agents in gpCELL. This tripartite structure follows the architectural principles of the CELL formalism.

Each agent in gpCELL is a local expert that manages a GP and is defined as:

$$\mathcal{A}_i = \{\phi_i, f_i, \mathcal{M}_i\}$$

where ϕ_i denotes its activation function, f_i its internal model, and $\mathcal{M}_i$ its internal state.

The spatialization logic of kCELL is retained in gpCELL. Each agent $\mathcal{A}_i$ represents its area of expertise through a mean $\mu_i \in \mathbb{R}^d$ and a covariance matrix $\Sigma_i \in \mathbb{R}^{d\times d}$. The activation function $\phi_i : \mathbb{R}^d \mapsto]0, 1]$ for a given input $x \in \mathbb{R}^d$ is given by the RBF kernel value

$$\phi_i(x) = \exp\left(-\frac{d_M(\mathcal{A}_i, x)^2}{2l^2}\right) \tag{5.5}$$

with l the lengthscale of the kernel and $d_M(\mathcal{A}_i, x)$ a measure of similarity between an agent $\mathcal{A}_i$ and an input x. $d_M(\mathcal{A}_i, x)$ is the Mahalanobis distance as in kCELL, and it allows gpCELL to remain compatible with the **same spatial gating mechanism as kCELL** (cf. 3.3.1).

Unlike the linear models used in oCELL or kCELL, each agent in gpCELL maintains a **local GP as the internal model** f_i. By using a GP, an agent can provide not only a mean prediction $\mu_{*,i}$ but also a predictive variance $\sigma^2_{*,i}$ representing the uncertainty about the prediction for a given input x. The internal model f_i is defined as:

$$f_i(x) = \begin{pmatrix} \mu_{*,i} \\ \sigma_{*,i} \end{pmatrix} = \begin{pmatrix} K(x, X_{i,\text{train}})\left[K(X_{i,\text{train}}, X_{i,\text{train}}) + \sigma_n^2 I\right]^{-1} y_{i,\text{train}} \\ K(x,x) - K(x, X_{i,\text{train}})\left[K(X_{i,\text{train}}, X_{i,\text{train}}) + \sigma_n^2 I\right]^{-1} K(X_{i,\text{train}}, x) \end{pmatrix} \tag{5.6}$$

where $X_{i,\text{train}}$ and $y_{i,\text{train}}$ correspond to the feature-target dataset of $\mathcal{A}_i$ stored in its internal state $\mathcal{M}_i$.

The internal state $\mathcal{M}_i$ tracks a confidence value $\bar{C}_{\mathcal{A}_i} \in [-1, 1]$. This confidence value is an exponential moving average of received social feedback that is described in detail in Section 3.3.2. It also maintains an internal feature-target dataset $\mathcal{D}_i$ used to retrain the internal model f_i. This dataset contains the features matrix $X_{i,\text{train}} \in \mathbb{R}^{n_{\text{retained}}\times d}$ and the target matrix $y_{i,\text{train}} \in \mathbb{R}^{n_{\text{retained}}}$. This dataset is managed dynamically by the agent and will be presented in the remainder of this section.

GP predictions revert to the specified prior mean function when inputs are far from the training data support, because the covariance kernel decays with distance. Consequently, it can lead to highly unreliable predictions and underestimation of the predictive variance [238]. The aggregation function used to build the global prediction of the ensemble of GP-based local expert agents must take this phenomenon into consideration to avoid taking into account the proposition of an unreliable agent that would pollute the final prediction. To ensure reliability of predictions, gpCELL use a **winner-take-all strategy** for global prediction based on predictive variance. For a given input x, the global prediction $\hat{y}$ is provided by the agent within the set of neighbor agents $\mathcal{D}_{\text{neighbors}}$ that reports the minimum predictive variance:

$$\hat{y} = f_j(x) \;\text{ where } j = \arg\min_{i\in\mathcal{D}_{\text{neighbors}}} \left(\sigma^2_{*,i}\right) \tag{5.7}$$

This aggregation function prioritizes the agent that is the most certain about the local structure of the data. In other words, the selected agent is the one whose internal representation of the underlying function is best aligned with the input x.

Model and Shape Update In gpCELL, the update of the internal model f_i and the shape of the area of expertise (μ_i and Σ_i) are intrinsically linked. Since the shape of the agent is obtained from the distribution of points stored in $\mathcal{D}_i$ and used to retrain f_i, updating $\mathcal{D}_i$ automatically updates the area of expertise of the agent.

When a new point (x_{new}, y_{new}) is processed, the agent must decide if or how this information should be included to its internal model. This is treated as a local model selection problem. The agent evaluates several possible configurations:

- **Nominal**: $\mathcal{D}_i$ remains unchanged and the current internal model is kept without modification.
- **Substitution of point** j: The new point replaces an existing point (x_j, y_j) contained in $\mathcal{D}_i$.
- **Augmentation**: The new point is added to $\mathcal{D}_i$.

Each configuration corresponds to a configuration of the internal dataset $\mathcal{D}_i$ and by extension, to a resulting internal model f_i. The number of configurations $n_{\text{configurations}}$ is equal to:

$$n_{\text{configurations}} = \underbrace{1}_{\text{Nominal}} + \underbrace{|\mathcal{D}_i|}_{\text{Subtitutions}} + \underbrace{1}_{\text{Augmentation}} \tag{5.8}$$

To maintain computational efficiency for large memories, instead of evaluating all possible substitution possibilities by building a new model for each, heuristics can be used to select only one substitution configuration. Examples of heuristics that can be used in this context are given below:

- **Random Selection**: A substitution configuration is selected randomly.
- **Influence-Based Selection**: This heuristic aims to select the point that contributes least to the final prediction of the GP. Points that are highly correlated within the agent's memory tend to have low influence. This means that removing them would have only a minimal impact on the model's predictive performance. The influence is estimated from the inverse of the covariance matrix K. Smaller diagonal entries of K indicate less influential points. The less influential point j^* is obtained by:

$$j^* = \arg \min_{j \in X_{i,\text{train}}} \left[K^{-1}\right]_{jj} \tag{5.9}$$

Using such a heuristic reduces the number of evaluated configurations to

$$n_{\text{configurations}} = 3$$

regardless of the memory size ($\mathcal{D}_i$).

To choose the optimal configurations, the agent computes a variant of the Akaike Information Criterion (AIC) [4] for each possibility:

$$AIC\left(x_{new}, y_{new}\right) = \alpha \left|\mathcal{D}_i\right| - 2 \ln\left(\hat{L}\right) \tag{5.10}$$

where

- α is the weight of the number of points $|\mathcal{D}_i|$,
- $|\mathcal{D}_i|$ corresponds to the number of parameters representing the non-parametric complexity of the model,
- $\ln\left(\hat{L}\right)$ is the log-marginal likelihood of the obtained GP given the training data contained in$\mathcal{D}_i$.

By selecting the configuration that minimizes the AIC, the agent performs a greedy online trade-off between predictive accuracy and model parsimony. When a new configuration is chosen, $\mathcal{D}_i$ is updated accordingly and the internal GP is retrained. Since $\mathcal{D}_i$ has changed, the shape of the area of expertise (μ_i and Σ_i) is automatically updated based on the empirical mean and covariance of the new $X_{i,\text{train}}$. Finally, for stability reasons and to have a bounded complexity, $\mathcal{D}_i$ should have a maximum size N_{max} (beyond which point can only be swapped and not added) and a minimum size analogous to the N_{ini} in kCELL. We clarify that each point beyond N_{max} can only be swapped and not added.

Computational Complexity Considering only one neighbor update, the use of a heuristic that limits the number of GP fits in the model selection process, and a $O(dN)$ complexity for selecting neighbor agents, the complexity of an update step in gpCELL becomes

$$O\left(\underbrace{dN}_{\text{neighbor search}} \times \underbrace{3IN_{max}^3}_{\text{evaluate all config.}}\right) \tag{5.11}$$

where I is the number of iteration required to find the best GP parameters, and N is the number of agents. As demonstrated in Chapter 4, neighbor selection is embarrassingly parallel. Thus, the complexity of this operation can be greatly reduced by using GPU parallelization.

Considering an online learning context where a classical GP is fitted on the whole dataset at each step, the computational complexity of an update is

$$O\left(In^3\right) \tag{5.12}$$

If $N_{max} \ll n$, gpCELL is more practical because it relies on a bounded cubic term, unlike the classical GP whose cubic term is unbounded (scaling with the number of training data). Although this is a simplified example, since methods like Sparse GPs [21] aim to reduce the update complexity, it reveals the computational stability of gpCELL. The complexity of gpCELL scales with the number of agents rather than the number of training points. If the embarrassingly parallel nature of neighbor search is exploited properly, a close-to-bounded update complexity could theoretically be reached.

Example of Single Agent Fit To visualize the learning result for a single local expert agent, we use a 1-dimensional example. A small synthetic dataset is generated from the function $f(x) = \sin(x) + \epsilon\mathcal{N}(0,1)$ with $\epsilon \in \mathbb{R}$ the noise scaling factor that we set to $\epsilon = 10^{-2}$. Online learning is simulated by feeding the feature-target tuples (x_{new}, y_{new}) one at a time to the agent. Each point is only seen once and the agent is configured with an initial memory

size of $N_{ini} = 4$, a parameter weight $\alpha = 1$ and an RBF kernel with a constant prior mean $m(x) = 0$.

Figure 5.2 illustrates the result of learning a single agent on this dataset. Several observations about the agent behavior can be made:

- **Local Anchoring**: The agent does not attempt to cover the entire domain that spans over $x \in [-6, 6]$. It maintains its area of expertise within a localized region. It demonstrates that the model-selection with AIC prevents the model from over-stretching to include points that would require excessive model complexity.
- **Dynamic Memory Management**: The final set of retained points (represented as *red* dots) is larger than the minimal size $N_{ini} = 4$ required to create the agent. Indeed, 5 points have been retained. However, the set of retained points remains sparse over the area of expertise. This means that the agent has used both **substitutions** and **augmentations** to optimize its internal representation.
- **Complexity-Accuracy Trade-off**: The sparsity of the retained points suggests that the agent has selected only the most informative points for obtaining a high-likelihood model. This results in a parsimonious model that captures the local curvature of the underlying function.

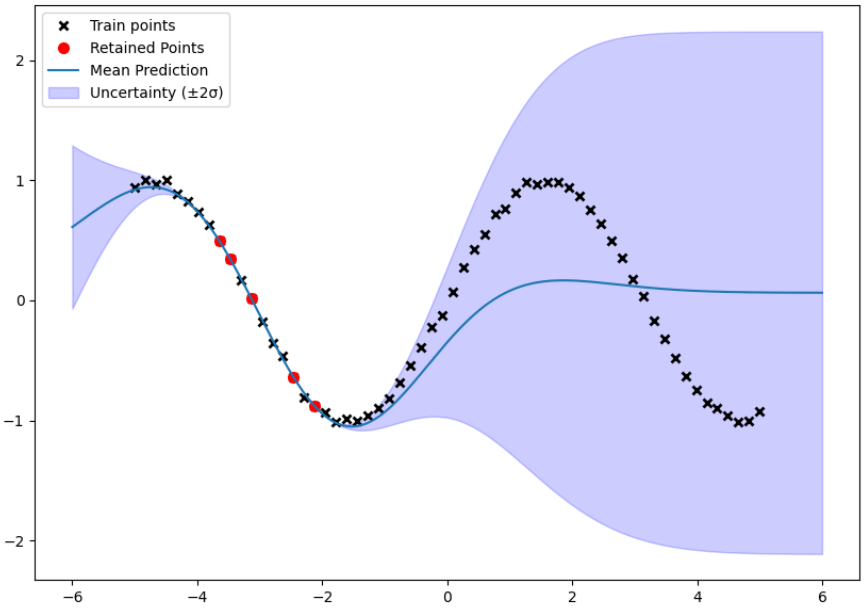


Figure 5.2: Example of the final configuration of a single agent on the function $f(x) = \sin(x) + \epsilon\mathcal{N}(0, 1)$ after learning. Points have been fed to the agent in order from left to right. The y-axis corresponds to the values of $f(x)$, the x-axis corresponds to the values of x. *Train points* are the samples sequentially processed by the agent from left to right; *Inducing points* are the samples retained by the agent at the end of learning and were used to train its internal model. The *blue* curve corresponds to the predictions of the agent.

5.3 Learning Rules

The previous section detailed the tripartite structure of gpCELL agents. It has focused on how GP-based local expert agents manage their local complexity through AIC model selection. This section describes the social learning rules that govern the behavior of the ensemble of local expert agents.

In gpCELL the reliability of an agent $\mathcal{A}_i$ is tracked via its running confidence $\bar{C}_{\mathcal{A}_i} \in [-1, 1]$. This confidence is updated using an exponential moving average of normalized social feedbacks. The agent is compared to a predictive baseline y_{base}, that is derived from the weighted contributions of other experts which are close within the ensemble.

Social Baseline Definition The baseline y_{base} is defined as a weighted average of the predictions $\mu_{*,i}$ (the predictive means) from a specific set of agents $\mathcal{D}_{\text{social}}$. The contribution of each agent is proportional to its activation value $\phi_i(x_{new})$, representing its expertise:

$$y_{base} = \sum_{i \in \mathcal{D}_{\text{social}}} w_i(x_{new})\, \mu_{*,i} \text{ with } w_i(x_{new}) = \frac{\phi_i}{\sum_{j \in \mathcal{D}_{\text{social}}} \phi_j(x_{new})} \tag{5.13}$$

The composition of $\mathcal{D}_{\text{social}}$ depends on the local social context:

- **Neighbor Feedback** ($N_{\text{neighbors}} > 1$): When several agents are neighbors to the given input x_{new}, $\mathcal{D}_{\text{social}}$ contains the neighbor agents, excluding $\mathcal{A}_i$ such as:

$$\mathcal{D}_{\text{social}} = \mathcal{D}_{\text{neighbors}} \setminus \{\mathcal{A}_i\} \tag{5.14}$$

- **Closest Feedback** ($N_{\text{neighbors}} = 1$): When there is only one neighbor to the given input x_{new}, its prediction is compared against the set of the k-closest agents $\mathcal{D}_{k\text{-closest}}$, excluding itself such as :

$$\mathcal{D}_{\text{social}} = \mathcal{D}_{k\text{-closest}} \setminus \{\mathcal{A}_i\} \tag{5.15}$$

  This allows the prediction of the neighbor agent $\mathcal{A}_i$ to be benchmarked against a default broader baseline.

The relative performance of agent $\mathcal{A}_i$ is evaluated as the fractional reduction in error:

$$\Delta_{\mathcal{A}_i} = \frac{|y_{base} - y_{new}| - |\hat{y} - y_{new}|}{|y_{base} - y_{new}|} \tag{5.16}$$

where y_{new} is the ground truth and $\hat{y}$ is the weighted average of predictions of agents within $\mathcal{D}_{\text{social}} \cup \mathcal{A}_i$ such as

$$\hat{y} = \sum_{j \in (\mathcal{D}_{\text{social}} \cup \mathcal{A}_i)} w_j(x_{new})\, \mu_{*,j}$$

$\Delta_{\mathcal{A}_i} > 0$ indicates the agent $\mathcal{A}_i$ is a strong expert that improves the collective prediction, while $\Delta_{\mathcal{A}_i} < 0$ identifies a weak expert whose internal model diverges from the social consensus.

To ensure the stability of the running confidence $\bar{C}_{\mathcal{A}_i}$ against outliers, the raw feedback is normalized into an instantaneous confidence $c_i \in [-1, 1]$ using a hyperbolic tangent function such as

$$c_i = \tanh(k_{\text{steepness}} \Delta_{\mathcal{A}_i}) \tag{5.17}$$

where $k_{\text{steepness}}$ is a hyperparameter representing the steepness of the normalization curve. This ensures that extremely large relative errors do not cause catastrophic drops in confidence. The running confidence is then updated as an exponential moving average of the instantaneous confidence values:

$$\bar{C}_{\mathcal{A}_i|t+1} = (1 - \lambda)\, \bar{C}_{\mathcal{A}_i|t} + \lambda c_i \tag{5.18}$$

where λ is the smoothing factor. This value is used as the primary metric for pruning bad agents in the ensemble.

The learning rules in gpCELL translate the social context and the result of the AIC model selection into discrete actions and meta-actions. The learning rules are detailed in the following paragraphs.

Inaccuracy When $N_{\text{neighbors}} \geq 1$, the closest agent performs its AIC-based selection. If the new point is informative enough to be included in the internal dataset $\mathcal{D}_i$, the internal GP model and the shape of its area of expertise are updated (**Model and Shape Update**).

Incompetence When $N_{\text{neighbors}} = 0$, the system identifies a gap in the local representation of the function to be approximated. The closest agent in the ensemble attempts to ingest the new feature-target tuple (x_{new}, y_{new}) by evaluating the AIC configurations. If the AIC selection results in keeping the **nominal** configuration, it means that an update is likely to corrupt its internal representation, degrading the complexity-accuracy trade-off. In this case, an **Agent Creation** meta-action is triggered to spawn a new agent into the ensemble. This new agent is initialized from the N_{ini} most recent feature-target tuples encountered in the data stream.

Uselessness If an agent's running confidence $\bar{C}_{\mathcal{A}_i}$ falls below a survival threshold $\tau_{\text{confidence}}$, the agent is too inaccurate relative to the social consensus and is removed from the ensemble. This prevents uncontrolled population growth.

Table 5.1 summarizes these learning rules by mapping social contexts and conditions to the corresponding triggered action or meta-action.

Finally, implementation-wise, the learning behavior of gpCELL is governed by a set of hyperparameters that balance local modeling accuracy and global computational efficiency. Some of the hyperparameters of gpCELL are shared with kCELL but new ones are introduced to manage the AIC model selection process of the local GPs. Below are the descriptions of these hyperparameters:

- $\xi \in [0, 100]$: the chi-squared percentile defining the threshold for the spatial gating mechanism to determine the set of neighbor agents.
- $l > 0$: the lengthscale of the RBF kernel used for agent activation. It defines the smoothness of the spatialization.
- $k_{\text{steepness}} > 0$: the steepness of the tanh function used to normalize the social feedbacks $\Delta_{\mathcal{A}_i}$ into the instantaneous confidence c_i.

	Social Context	Condition	Targeted Subset	Action
Incompetence	$N_{\text{neighbors}} = 0$	Model selection leads to **Nominal**	Closest	Agent Creation
	$N_{\text{neighbors}} = 0$	Model selection leads to **Substitution** or **Augmentation**	Closest	Model + Shape Update
Inaccuracy	$N_{\text{neighbors}} \geq 1$	$\varnothing$	Closest	Model + Shape Update
Uselessness	$\varnothing$	$\bar{C}_{\mathcal{A}_i} \leq \tau_{\text{confidence}}$	All	Agent Destruction

Table 5.1: Learning rules of gpCELL. Each line corresponds to a specific learning rule defined by: 1) a social context expressed as a condition on the number $N_{\text{neighbors}}$ of local experts around a new input x_{new}; 2) a condition on the outcome of the model selection process (nominal, substitution, augmentation) or on the running confidence $\bar{C}_{\mathcal{A}_i}$; 3) a target set of agents that will be affected by the rule; and 4) the actions or meta-actions to perform on the agents of the targeted set.

- $\lambda \in]0, 1]$: The smoothing factor for the exponential moving average that updates the running confidence $\bar{C}_{\mathcal{A}_i}$.
- $\tau_{\text{confidence}} \in [-1, 1]$: The threshold for agent destruction. Agents whose confidence falls below this value are pruned from the ensemble.
- α: The complexity penalty weight used in the AIC calculation ($AIC = \alpha\,|\mathcal{D}_i| - 2\ln\left(\hat{L}\right)$). It controls the trade-off between the number of points retained in an agent's memory and its likelihood fit.
- $N_{ini} \in \mathbb{R}^+$: The minimum number of points required in the initialization buffer to create an agent.
- $k_{\text{social}} > 0$: The number of closest agents used to compute the social baseline y_{base} when an agent has no neighbor ($N_{\text{neighbors}} = 1$).

5.4 Online Stationary Modeling

The previous sections have detailed the tripartite structure and learning rules of gpCELL. This section presents an empirical validation of the learning capabilities of this algorithm by comparing it with baseline algorithms on an online stationary modeling task. For this purpose, a high-dimensional robotic dataset is used. We demonstrate that despite the adaptation challenges introduced by using GPs as internal models, gpCELL offers a competitive predictive accuracy with kCELL and provides a seemingly reliable predictive uncertainty quantification.

Experimental Setup To ensure a direct comparison with the results presented in Chapter 3, the experiment is conducted on the SARCOS dataset, a standard benchmark for multi-dimensional robot inverse dynamics [228]. We remind that **inverse dynamics modeling** involves constructing a model that maps a dynamical system's motion to the internal forces or agents required to produce that motion (cf. definition 20). For the SARCOS dataset, the

task involves mapping a **21-dimensional input space** (7 joint positions, 7 velocities and 7 accelerations) to the corresponding joint torques (**7-dimensional output space**) of a seven degrees-of-freedom anthropomorphic robot arm (cf. Figure 3.17 for an illustration).

The dataset contains 44484 training and 4449 test points. A single pass over the training dataset is performed where each feature-target tuple (x_{new}, y_{new}) is presented to the models one at a time. No feature scaling is applied to the data, assuming that the global statistics are unknown beforehand. The goal behind this choice is to test the model's robustness to raw sensor inputs. gpCELL is compared with the same baseline algorithms as in the kCELL comparative study presented in Chapter 3.

For gpCELL, the local GP models are implemented using the GPX implementation of the *Surrogate Modeling Toolbox* library [196]. However, due to the computational complexity of GP updates, gpCELL runs significantly slower than kCELL. Thus, the grid-search hyperparameter optimization procedure used in Chapter 3 was not feasible within a reasonable time. Instead, gpCELL uses the same parameters as kCELL for the spatialization of agents (e.g. lengthscale l, gating threshold ξ). The GP-specific parameters (RBF kernel, constant prior mean and complexity weight in AIC $\alpha = 0.2$) are manually tuned. To maintain a predictable computational complexity suitable for learning from a data stream, two heuristics are applied in the gpCELL implementation:

1. **Influence-Based Selection**: During the AIC model selection phase, only the less influential point of the memory is considered for substitution. The least influential point corresponds to the minimum diagonal entry in the inverse covariance matrix K^{-1}.

2. **Single Agent Updates**: Only the closest agent to the input x_{new} (among neighbors) is updated.

The predictive accuracy of the models is evaluated through adaptation speed, measured by the Area Under the Curve (AUC) of the prequential error (i.e. error computed on a feature-target pair x_{new}, y_{new} before learning on it), as well as the final Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) on the test set. The final results are presented in Table 5.2. Then, the uncertainty quantification capabilities of gpCELL and its agent population growth are analyzed.

Prequential Error Analysis In addition to Table 5.2, Figure 5.3 illustrates the evolution of the prequential error during learning. We can notice that gpCELL exhibits a higher prequential error than kCELL, Mondrian Forests and LWPR. This gap in immediate adaptation capability could be explained by the behavior of GPs in unexplored regions. As the kernel covariance decays with distance, GP predictions revert to the constant prior mean. This can lead to higher errors on Out-of-Distribution (OoD) points before the input space is fully covered by local expert agents.

Asymptotic Accuracy Figures 5.4 and 5.5 show the evolution of RMSE and MAE on the fixed test set. For RMSE, gpCELL becomes more accurate than the baselines LWPR and Mondrian Forest around 42000 steps but remains **slightly behind kCELL**. For MAE, gpCELL shows a slower initial decrease in error than kCELL but achieves the **lowest final**

	Prequential Error AUC	Test MAE	Test RMSE
Linear Regression (Adam)	1.07	4.46	5.63
Adaptive Hoeffding Tree	3.00	5.72	7.87
Mondrian Forest	**0.72**	1.89	2.67
LWPR	1.05	2.77	3.73
kCELL (ours)	1.72	0.91	**1.94**
gpCELL (ours)	2.96	**0.38**	2.07

Table 5.2: Summary table of the final results of the SARCOS comparative experiment whose objective is to evaluate the efficiency of the gpCELL algorithm compared to kCELL and a set of baseline algorithms. The reported values are the final values obtained after seeing the entirety of the dataset once (single pass). Values in bold correspond to the best value among the different considered models.

MAE among all algorithms, crossing the kCELL curve at approximately 35000 steps. This difference in the results obtained for RMSE and MAE can be explained by the fact that RMSE is more sensitive to outliers. gpCELL being better than baselines in MAE suggests a more accurate nonlinear fit on average. On the other hand, the slightly different RMSE hierarchy (kCELL being better than gpCELL) indicates that the model is still prone to **occasional large errors**. It corresponds for example to test points that fall into poorly covered regions of the input space where the local GPs revert to the constant mean prior.

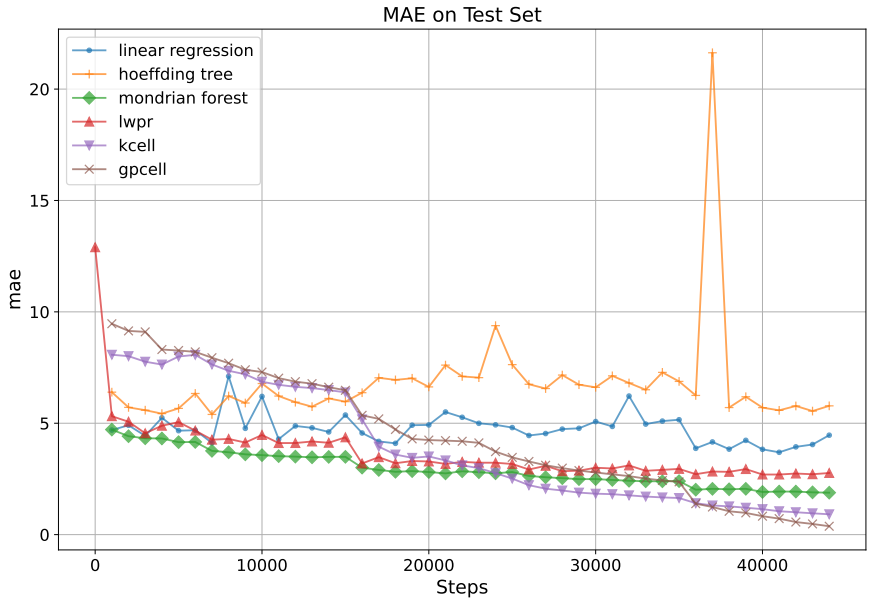


Figure 5.5: Evolution of test MAE throughout learning. y-axis corresponds to the MAE and *x-axis* corresponds to the number of training steps.

Uncertainty Quantification Analysis One of the main advantage of the use of GPs in gpCELL is the ability to provide **predictive variance measurement**. This is complementary with the epistemic uncertainty estimation derived from the spatialization of local expert

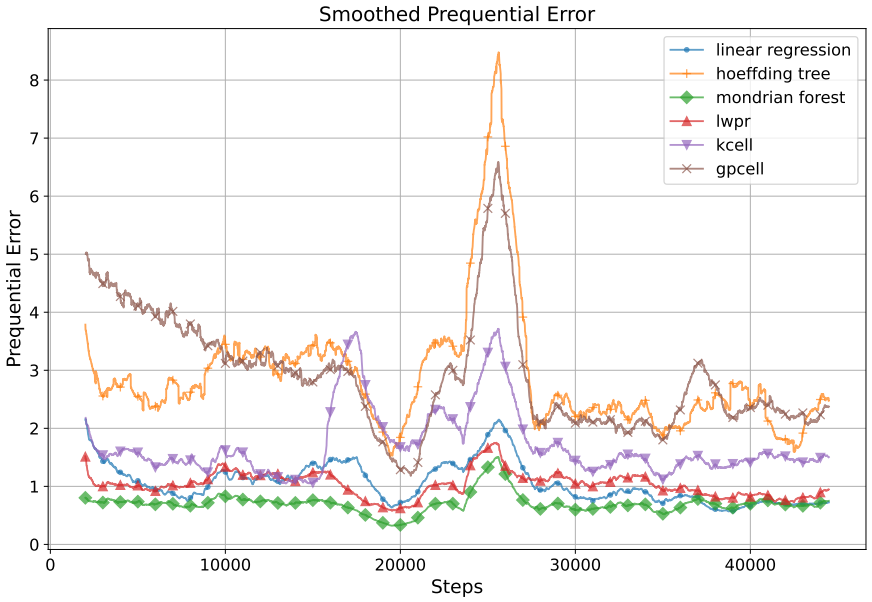


Figure 5.3: Evolution of the prequential error during the learning phase. Each curve is smoothed with a sliding window of 2000 steps. y-axis corresponds to the absolute prequential error value and x-axis corresponds to the number of training steps.

agents. Despite the ability of GPs to provide a well-calibrated uncertainty estimates, the calibration of predictive uncertainty in the ensemble of local expert agents in gpCELL is not mathematically guaranteed, and should be verified experimentally. To this end, the Prediction Interval Coverage Probability (PICP) is measured on the test set. PICP represents the proportion of true observations that fall inside the predicted 95% confidence interval. As shown in Table 5.3, the PICP exceeds 0.95 for every output dimension. This result demonstrates that gpCELL provides **well-calibrated intervals**. It means that the true torque values fall within the predicted 95% confidence range with high reliability.

	joint 1	joint 2	joint 3	joint 4	joint 5	joint 6	joint 7
PICP ($\pm 2\sigma$)	0.96	0.95	0.95	0.95	0.96	0.96	0.95

Table 5.3: Prediction Interval Coverage Probability (PICP). PICP measures the proportion of true observations that fall inside a gpCELL's predicted interval on the test dataset for each output dimension (named joint for compliance with the SARCOS terminology). Here the 95% (2σ) confidence interval is considered.

In addition, the informativeness of the uncertainty metrics is evaluated through Spearman correlation [210].

– **Predictive Variance vs. Prediction Error**: The correlation between the predictive variance of the global prediction of gpCELL and the prediction error is computed. A very strong positive correlation of 0.94 (with $p < 0.001$) is observed. Figure 5.6 shows that prediction errors increase sharply with predictive variance.

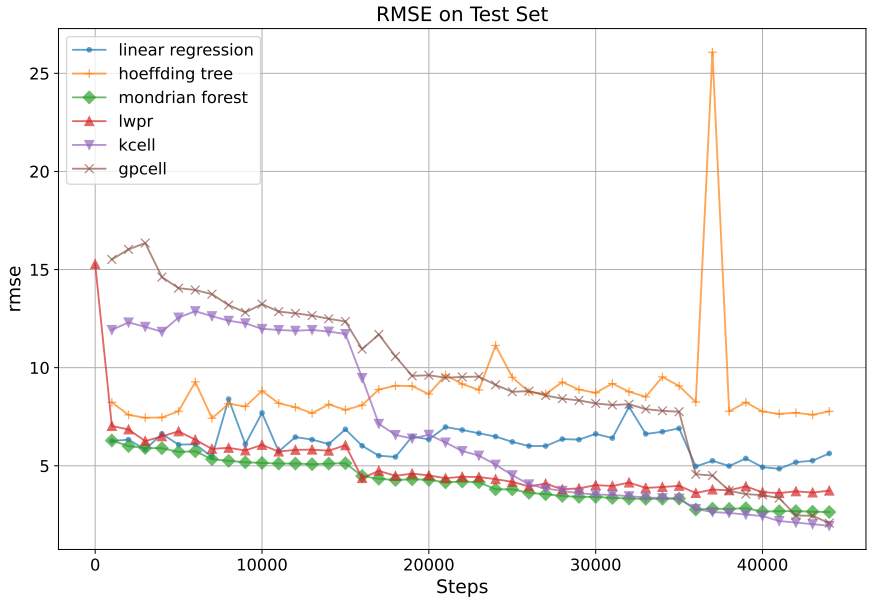


Figure 5.4: Evolution of test RMSE throughout learning. y-axis corresponds to the RMSE and *x-axis* corresponds to the number of training steps.

– **Activation Value vs. Prediction Error**: The correlation between the activation value of the closest agent and the prediction error is computed. The obtained correlation is -0.28 (with $p < 0.001$). While lower than the -0.68 observed for kCELL in Chapter 3, this weak negative correlation remains statistically significant. As shown in Figure 5.7, the lower the activation, the larger the errors. Despite the correlation being weak, it seems that low activation can still serve as a proxy for epistemic uncertainty, signaling when the model is operating in a sparsely covered region fo the input space.

Agent Population Analysis Figure 5.8 shows the evolution of the number of agents during the training phase for kCELL and gpCELL. Both algorithms create fewer agents than the theoretical maximum which corresponds to creating a new agent every N_{ini} steps. Before conducting this experiment, we have hypothesized that the nonlinear modeling capabilities of local GPs would enable for more parsimonious representation: fewer agents covering more complex local manifolds. Unexpectedly, the results indicate that **gpCELL creates more agents than kCELL**. At the end of the training phase, gpCELL has retained approximately **86% of the total encountered points** in agent memories. This confirms that even if the substitution/augmentation logic is discarding 14% of the data. This confirms that gpCELL remains **relatively memory-intensive** compared to kCELL.

Synthesis The results of this comparative study validate empirically the learning capabilities of the gpCELL algorithm. Consequently, the integration of another type of learning model for the design of a CELL learning algorithm is also validated.

The key takeaways from this experiment can be summarized as follows:

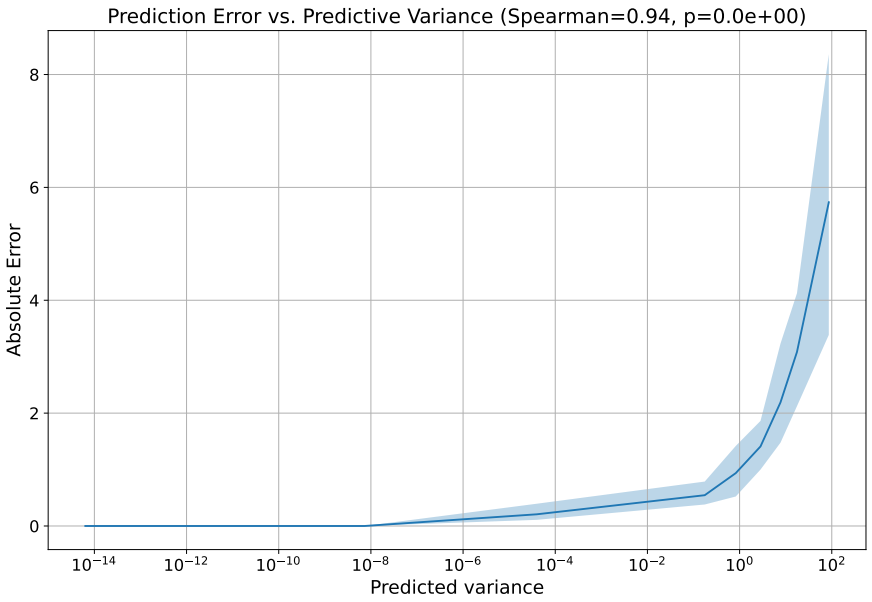


Figure 5.6: Relationship between the predicted variance and the prediction errors on the test set. y-axis corresponds to the absolute error values and x-axis corresponds to the predicted variance. A spearman correlation equals to 0.94 is found with a p-value $p < 0.001$, suggesting a statistically significant positive correlation.

- **Nonlinear Representations**: gpCELL provides a more accurate representation of the underlying function (lowest final MAE). However, this comes at the cost of slower adaptation, higher errors on OoD inputs, and higher memory cost.
- **Uncertainty**: gpCELL provides a well-calibrated predictive variance (95% PICP) which quantifies the reliability of predictions. This variance offers a complementary measure of uncertainty that lies in the output space instead of the input space, as is the case for epistemic uncertainty estimation through the spatial organization of local expert agents.
- **Population Overgrowth**: Contrary to what was expected, the transition to nonlinear internal models did not naturally lead to a more parsimonious population, because gpCELL created more agents than kCELL despite the use of the same spatialization hyperparameters. This suggests that the current learning rules under the chosen hyperparameters lead to an overproduction of agents compared to their individual modeling capabilities.

While gpCELL is more computationally and memory intensive than kCELL, the reliability of uncertainty quantification introduced by the local GPs makes it a promising candidate for safe control tasks.

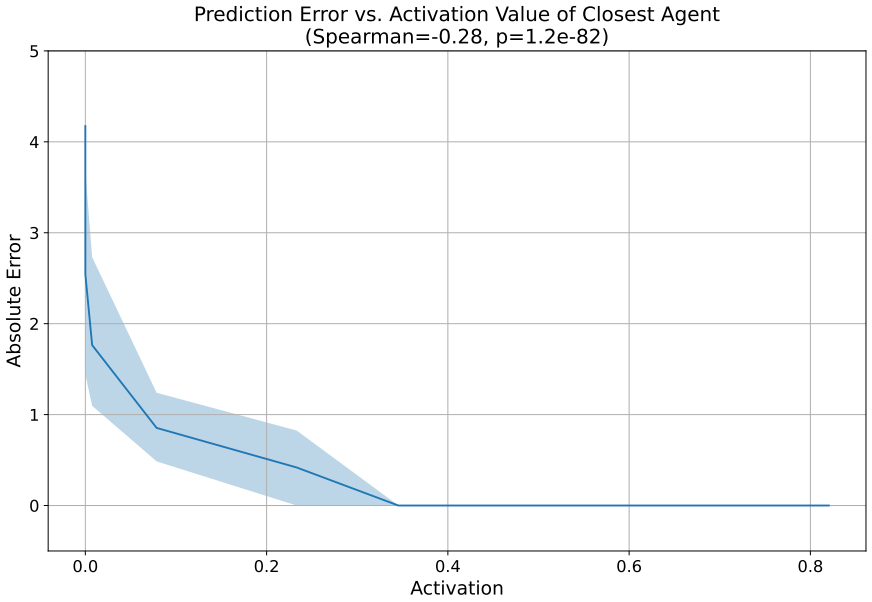


Figure 5.7: Relationship between the activation value $\phi_i(x) \in]0, 1]$ of the closest agent and the prediction errors on the test set. y-axis corresponds to the absolute error values and x-axis corresponds to the activation values. A Spearman correlation equals to -0.28 is found with a p-value $p < 0.001$, suggesting a statistically significant weak negative correlation.

5.5 Conclusions and Limitations

This chapter has detailed early exploratory works regarding the development and validation of gpCELL, a new instantiation of the CELL formalism designed to incorporate nonlinear, non-parametric modeling into local expert agents through GPs. Section 5.1 has given an overview of what is Gaussian Process Regression and how Gaussian Processes are used in Machine Learning. Then, Section 5.2 has provided a formal definition of the tripartite structure of local expert agents following the CELL formalism. It has introduced model selection mechanism based on the Akaike Information Criterion (AIC). AIC allows local expert agents to autonomously manage the complexity of their internal model depending on the local data manifold lying in their area of expertise. Section 5.3 has presented the social learning rules. Finally, Section 5.4 presented the results of a comparative study conducted on the SARCOS benchmark dataset. The results have demonstrated that gpCELL could achieve superior or competitive performance with other baseline algorithms such as kCELL, while providing well-calibred predictive uncertainty estimates.

The development and evaluation of gpCELL yields valuable insights:

- **Online GPs**: By distributing the computational load across an ensemble of local expert agents, gpCELL bypasses the $O(n^3)$ bottleneck of global GPs with a horizontally scaling architecture suitable for data streams.

- **Uncertainty Quantification**: The integration of GPs as the internal model of local expert agents provides a more rigorous estimation of predictive variance in the *output space*. It complements the epistemic uncertainty estimation derived from the spatial

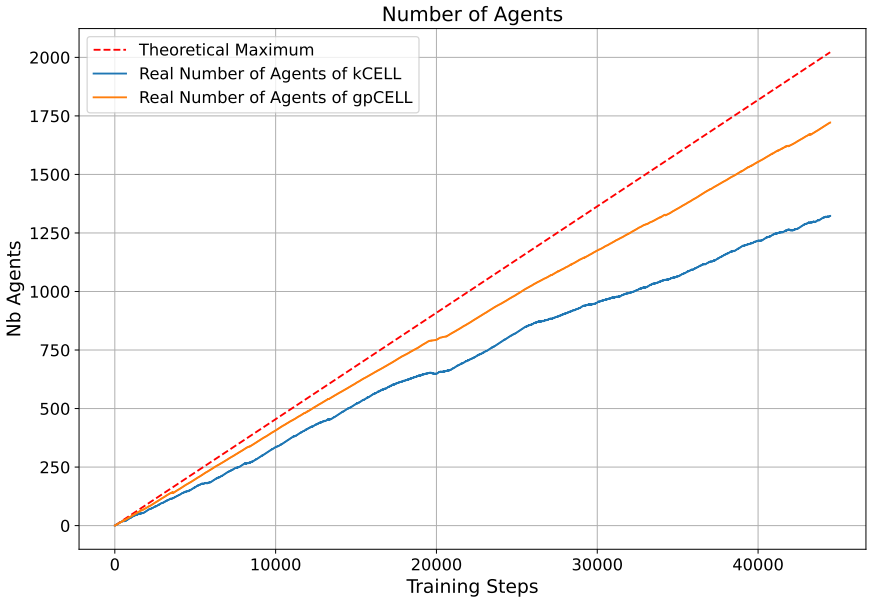


Figure 5.8: Number of agents throughout the learning phase. y-axis corresponds to the number of agents and y-axis corresponds to the learning steps. The red dotted line corresponds to the theoretical maximum number of agents that could be produced by creating an agent every N_{ini} steps. The *blue* line corresponds to kCELL and the *orange* one to gpCELL.

organization of agents in the *input space*.

– **Local Nonlinearity**: The experimental results show that gpCELL achieves the lowest final MAE among the compared baselines. It hints toward potential benefits of using nonlinear internal models able to adapt their complexity based on the local data manifold.

Despite these strengths, the current version of gpCELL reveals several limitations that highlight promising directions for future research.

Population Overgrowth Contrary to what we intended, gpCELL created more agents than its linear counterpart, kCELL. This could be linked to the weakness of the kernel spatialization approach in higher-dimensional input spaces. Indeed, when the number of input dimensions grows, the Mahalanobis distance progressively loses its informativeness. This can lead to an over-fragmentation of the input space where each local expert agent is far from all others. Future works should investigate the *dimensionality reduction* or the use of *more complex data distribution estimators* like Kernel Density Estimation (KDE) for the spatial gating mechanism.

Memory and Computational Efficiency While gpCELL is more efficient than a global GP for online learning, it remains memory-intensive as it retained approximately 86% of encountered data in the memory of agents during the conducted experiment. A dedicated hyperparameter optimization study is needed to evaluate the trade-off between parsimony

and accuracy. Furthermore, the current implementation relies on heuristics such as influence-based point selection and closest-agent only updates to maintain real-time feasability. These heuristics might have biased the experimental results.

Social Mechanisms Unlike in kCELL, gpCELL only uses social feedbacks to destroy agents through updates of individual confidence based on social consensus estimation. This is analogous to a competition for survival among agents. gpCELL currently lacks the incorporation of additional social mechanisms into the agent update logic. More advanced rules could be considered for gpCELL to make agents interact and share knowledge more actively.

Better Baseline Comparison gpCELL is an algorithm that adapts GPs for online learning through the CELL formalism. However, its efficiency has not been compared to established approximation methods such as Sparse Gaussian Processes (SGPs). A broader study on computational and memory efficiency, compared with classical GP adaptations, would help to situate gpCELL more clearly within the state of the art in GP learning.

Chapter 6

Speed Recommendation from an Industrial Perspective

This thesis has been rooted in a specific industrial challenge: the development of speed recommendation strategies to reduce road fatalities and environmental impact by advising drivers of an optimal speed profile they should follow. As established in Chapter 1, the speed recommendation problem can be abstracted as a Human-Robot Collaboration (HRC) problem. This shift in perspective allows the speed recommendation problem to be decomposed into two subproblems: **predicting the vehicle state** when it is driven by a human while subject to external perturbations, and safely **exploiting this predictive model** to provide speed recommendations. The relationship between HRC and Speed Recommendation is illustrated in Figure 6.1.

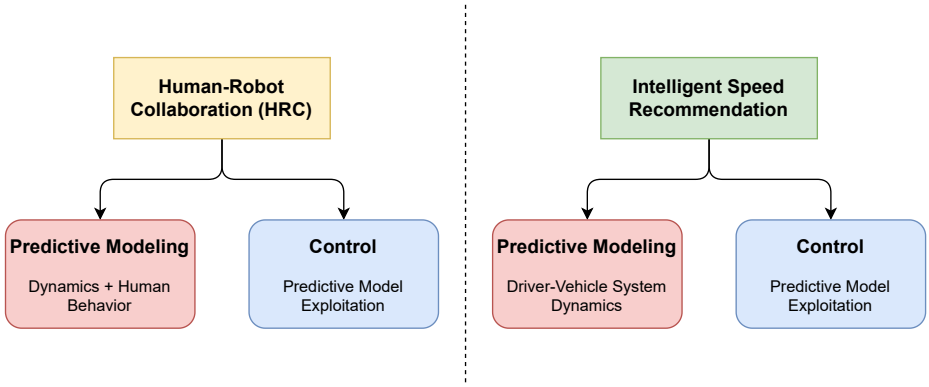


Figure 6.1: Illustration of the relationship between HRC and speed recommendation through the decomposition into two subproblems proposed in this thesis: predictive modeling and control leveraging the predictive model.

In this context, the proposed approach treats speed recommendation as a control problem solved using Model Predictive Control (MPC) coupled with a continuously learning predictive model to handle uncertainty of the driving context, including the potentially non-stationary behavior of the human driver (cf. Chapter 2).

We introduced the CELL (Context Ensemble Local Learning) formalism and developed

online learning algorithms that rely on ensembles of local expert agents. These agents self-organize according to social rules to continuously predict the evolution of the vehicle dynamics influenced by a human driver and other external factors. Chapter 3 and Chapter 4 demonstrated the effectiveness of the developed algorithms for predictive modeling and their integration into a MPC control pipeline, thereby addressing the two subproblems introduced in Chapter 1. Having laid out these theoretical and algorithmic foundations, this chapter returns to the initial motivation of this thesis: speed recommendation.

The goal of this chapter is not to introduce further technical contributions, but rather to provide an industrial perspective on the requirements for safe and efficient intelligent speed recommendation systems. It serves as a positioning chapter, discussing how the works of this thesis on HRC participated in a broader reflection on the progress of the field. While the preceding chapters highlighted the potential of the online learning algorithms derived from CELL, real-world automotive applications require addressing a broader set of requirements, ranging from multi-objective optimization to certifiability (i.e. driving function homologation). We argue that the intrinsic properties of the CELL instantiations, specifically the spatialized local knowledge representation through local expert agents and the introspection capabilities, position this type of approach as an interesting research topic for bridging the gap between **personalized online learning** and **safety-critical industrial standards**.

The remainder of this chapter is structured as follows. First, Section 6.1 identifies the industrial requirements for intelligent speed recommendation, moving from basic systems that inform the driver about the current speed limit, toward systems that pursue multiple objectives such as balancing safety constraints, energy efficiency and driver acceptance. Then, Section 6.2 presents the proposed architecture and explains the rationale for combining MPC with online-learning models, highlighting why this integration offers an effective technical solution for bridging the gap between speed control and speed recommendation. Subsequently, Section 6.3 demonstrates how the contributions of this thesis take a step toward addressing key industrial challenges. Finally, Section 6.4 concludes by discussing the remaining challenges for real-world deployment, highlighting the need for high-fidelity simulation and formal mathematical proofs to ensure the long-term stability of the continuously learning predictive models derived from CELL.

6.1 Industrial Requirements for Speed Recommendation Systems

The technical evolution of speed recommendation systems, such as Intelligent Speed Assistance (ISA), has historically followed a trajectory from passive information retrieval aimed at informing the driver about the current legal speed limit [69, 67] toward active and more intelligent recommendations. Initially, research and industrial efforts were primarily focused on the robust identification of legal speed limits by leveraging digital maps and accurate GPS positioning [18, 56, 55]. Such an approach is purely informative and ensures that the driver remains aware of the current legal speed limits.

As ISA becomes mandatory due to EU regulations [74, 75, 73, 72], modern vehicles are no longer isolated agents but connected entities capable of gathering and processing large amounts of real-time data regarding traffic density, weather conditions, road surface

quality, and more. This shift allows for a transition from static speed limit advisory to intelligent speed recommendation systems advising an optimal speed profile accounting for the dynamic complexity of the road context. As the industrial scope of speed recommendation expands, the deployment of such intelligent systems in a commercial vehicle requires satisfying a rigorous set of requirements which are categorized in the remainder of this section.

Multi-Objective Optimization In contrast to purely advisory ISA systems, intelligent speed recommendation acts as a high-level controller that solves a multi-objective optimization problem. To provide tangible added value for stakeholders such as fleet managers, the system requires the definition of a cost function involving potentially conflicting objectives. Typical examples include:

- **Energy Efficiency**: Minimizing fuel consumption in internal combustion and hybrid vehicles, and minimizing electrical energy use in electric vehicles, through regenerative braking and optimal electric motor torque setpoints.
- **Travel-Time Efficiency**: Reducing trip duration while maintaining a smooth speed profile.
- **Comfort**: Avoiding abrupt acceleration or deceleration, as well as excessive acceleration jerk, which may degrade ride comfort or increase driver cognitive load.

Safety And Legal Guarantees Speed recommendations must be strictly bounded by physical and legal constraints. These act as hard state constraints:

- **Legal Compliance**: Ensuring the recommended speed profile does not exceed applicable legal speed limits, including conditional or temporary regulations.
- **Safety**: Accounting for dynamic elements of the driving context to prevent unsafe operating conditions. Examples include ensuring adequate safety margins in curves, respecting inter-vehicle distances, and adapting to degraded road friction conditions to mitigate the risks of aquaplaning or loss of control.

Overridability and Driver Acceptance As specified in the European regulation [72], the human remains the final decision maker over the recommendation system. Furthermore, as speed recommendation can be formulated as a HRC problem (cf. Chapter 1), driver acceptance is a major factor determining the effectiveness of the recommendation system.

- **Overridability**: As the driver maintains full authority on his vehicle, the speed is recommended and not enforced. This differentiate the speed recommendation problem from the speed control problem in autonomous driving. The driver should be able to *refuse*, *delay* or *accept* the recommended speed at any time.
- **Acceptance**: Acceptance is influenced both by how recommendations are generated and how they are communicated to the driver through a Human-Machine Interface (HMI). Such interface may range from purely visual indications on the vehicle dashboard to more active modalities, such as haptic feedback on the accelerator pedal

[254, 69]. Therefore, the objective is to recommend a plausible speed setpoint and to communicate it in the best way possible to the driver.

Robustness to Uncertainty and Diversity of Road Contexts The recommendation system should maintain its efficiency across a diverse range of Operational Design Domains (ODDs) characterized by high environmental and behavioral variability:

- **Contextual Diversity**: Drivers and vehicles operate in highly different conditions (e.g. urban vs rural, varying weather conditions, traffic congestion). While many driving scenarios occur regularly, others are rare but safety-critical, requiring the system to maintain consistent performance even in low-frequency edge cases.
- **Heterogeneous Traffic Participants**: The presence of diverse road users such as cars, trucks, motorcyclists, cyclists, or even pedestrians, whose hard-to-predict behaviors introduces substantial stochastic variability that must be handled through robust prediction mechanisms.

Certifiability In an industrial context, certifiability can be defined as follows:

Definition 24. Certifiability refers to the formal process required to demonstrate that a safety-critical system conforms to applicable regulatory, safety, and performance standards, thereby authorizing its real-world deployment [219].

Advanced Driver Assistance Systems (ADAS) are subject to several requirements and standards regarding certifiability such as:

- **Functional Safety** (ISO 26262 [119]): Systems must achieve specific Automotive Safety Integrity Levels (ASIL) through rigorous Hazard Analysis and Risk Assessment (HARA), ensuring full traceability throughout the development lifecycle, from requirements to field operational tests.
- **Safety of the Intended Functionality** (SOTIF - ISO/PAS 21448 [2]): This addresses the complexity of ADAS where sensors and algorithms must interact reliably in unpredictable real-world conditions.
- **Scenario-Based Validation**: Deployment requires comprehensive evidence gathered through simulations, Hardware-in-the-Loop (HIL) testing (ISO/TS 22133 [118]) and real-world track trials to validate performance against United Nations (UN) regulations (e.g. UN R171 [224]) for driver assistance systems.
- **Data Integrity and Cybersecurity** (ISO/SAE 21434 [1]): As modern vehicles increasingly rely on V2X (Vehicle-to-Everything) connectivity to collect road data, the recommendation system must remain robust against corrupted or malicious inputs that could results in unsafe speed recommendations.
- **Deterministic Real-Time Performance**: Alongside functional safety, real-time embedded automotive systems must exhibit bounded Worst-Case Execution Times (WCET) and predictable computational load to satisfy hardware constraints of automotive Electronic Control Units (ECUs).

6.2 Proposed Architecture

The previous section exposed a comprehensive set of industrial requirements for the deployment of intelligent speed recommendation systems in real-world conditions. These requirements spanned from multi-objective optimization to strict certifiability constraints. Although substantial progress has been achieved in automated speed control for fully autonomous driving, speed recommendation remains a distinct challenge due to the presence of a human in the loop. This section presents the architecture proposed in this thesis to address the speed recommendation problem.

Distinguishing Recommendation and Control As exposed in Chapter 1, a core distinction must be made between speed control and speed recommendation. In the context of autonomous driving, speed control aims at replacing the human driver, operating in a closed-loop where the controller directly manipulates vehicle actuators (e.g. throttle, brakes, gear box). In contrast, speed recommendation preserves full driver authority over the vehicle. In this context, the human acts as a biological actuator to the system (falling into the shared controllability challenge exposed in Chapter 2). The recommendation system's output is not a control signal directly applied to the vehicle's low level controls, but is a suggested speed setpoint that the driver may choose to *refuse*, *delay* or *accept*.

Despite its importance for road safety and energy efficiency, speed recommendation has been significantly less studied than speed control. Research efforts have historically focused more extensively on automated control, often overlooking the complex modeling requirements associated with human-in-the-loop systems for effective speed recommendation.

To highlight the specific research gaps of the literature regarding speed control and speed recommendation, we re-evaluate the body of literature introduced in Chapter 1 by applying the technical taxonomies established in Chapter 2. This classification allows us to visualize the current state of the art regarding **control and recommendation** strategies (in Table 6.1), and **predictive modeling** through learning approaches (in Table 6.2). It should be specified that Table 6.2 does not include any online learning method like those presented in this thesis (i.e. continuously learning continuously a dynamics model).

As shown in Table 6.1, there is a clear absence of research works exploring the combination between Online Learning approaches and MPC. Furthermore, Table 6.2 reveals that the majority of research relies on **global parametric models** with no existing works, to our knowledge, leveraging a **continuously learning, local non-parametric approaches** to model the dynamics of the driving context.

Families	Papers
Classic ISA	[158, 69, 67, 18, 56, 55, 218]
Advanced ISA	[111, 126, 125, 141, 157, 7, 88, 59, 11, 193, 216, 241, 84, 164, 175, 174, 160, 252, 156]
Reactive Feedback Laws	[111, 22, 3, 59, 241]
Optimal Control	[174, 126, 215, 125, 141, 90, 128, 7, 159, 164, 98, 207, 176, 175]
MPC	[134, 155, 105]
Model-Free RL	[216, 48, 184, 234, 151, 152, 256, 251, 255, 257, 68, 243, 62, 169, 123]
Model-Based RL	[147, 148]
Online Learning MPC	∅

Table 6.1: Classification of papers obtained by conducting a review of literature on speed control and recommendation into different families, each corresponding to a type of strategy. It mirrors the structure of Table 2.1 with two additional families, *Classic ISA* and *Advanced ISA*, to be able to include the Intelligent Speed Assist (ISA) systems that consider the human in the loop. *Classic ISA* corresponds to ISA systems that inform the driver about hte current speed limit, while *Advanced ISA* includes systems that consider additional objectives like fuel consumption minimization or comfort.

To bridge this gap, we propose an architecture that leverages **Model Predictive Control (MPC)** integrated with a **continuously learning predictive model of the driver-vehicle** dynamical system. Specifically, the predictive model is a CELL model based on an ensemble of local expert agents that self-organize through social interaction rules (cf. Chapter 3) as illustrated in Figure 6.2.

The choice of an MPC architecture including a CELL online learning predictive model is motivated by its ability to address the industrial requirements identified in the previous section within a unified mathematical framework.

Multi-Objective Optimization The choice of the MPC framework naturally addresses the multiple, *potentially conflicting objectives*. By defining an appropriate cost function, the system can jointly optimize: energy efficiency, travel time, and comfort for a given driving context to produce a speed recommendation that represents the optimal tradeoff. With an appropriate solver, MPC allows the simultaneous handling of conflicting objectives within a conventional optimization framework.

Safety and Legal Constraints Safety-critical systems must respect hard constraints. Within the MPC loop, *Safety* and *Legal* constraints requirements are handled as hard state and input constraints evaluated over the trajectory predicted by the CELL model. Using appropriate solvers within MPC ensures that the recommended speed profile is inherently

Families	Papers
Global Parametric	[156, 148, 48, 184, 147, 234, 151, 152, 256, 251, 141, 155, 257, 157, 128, 68, 243, 62, 169, 123, 193, 216, 207, 176, 175, 174, 134]
Global Non-Parametric	[215]
Global Tree-based	[84]
Local Non-Parametric Lazy	∅
Local Non-Parametric	[105, 22, 126, 125]
Local Multi-Agent Systems	[164, 11]

Table 6.2: Classification of papers obtained by conducting a review of literature on speed control and recommendation into different families each corresponding to a learning approach used to build predictive models. It mirrors the structure of Table 2.2 that presents a classification of various types of learning models. Here, online learning is not considered specifically, we apply the classification to all encountered learning methods. None of the papers found describe an approach dependent on a continuously learning predictive model as presented in this thesis.

compliant with legal speed limits and physical safety criteria (e.g. curvature-based speed limits or maximum inter-vehicle distances, ...).

Certifiability and Computational Predictability A major challenge for the deployment of learning-based ADAS is the requirement for *Worst-Case Execution Times* (WCET) and *bounded computational complexity* (ISO 26262 [119]). One of the primary advantages of the proposed architecture is that MPC allows the use of battle-tested industrial grade solvers (such as OSQP [212] or qpOASES [76]) that are compliant with these requirements. This ensures that the optimization process finishes within the strict real-time deadlines of embedded automotive hardware.

Driver Acceptance This thesis relies on the following core assumption:

Assumption 6.1. Driver Acceptance intrinsically increases when recommendations align closely with the driver's natural behavior.

In the context of HRC, the machine should incorporate a model of its human partner's intentions to enhance collaborative synergy [54]. Given the previously defined assumption, we hypothesize that if the predictive model accurately captures the specific driving style and intentions of the driver, the recommendations generated will be perceived as more natural and will be more likely to be accepted.

Continuous online learning via the CELL instantiations allows the recommendation system to personalize the predictive model at deployment time. By adapting to the driver's

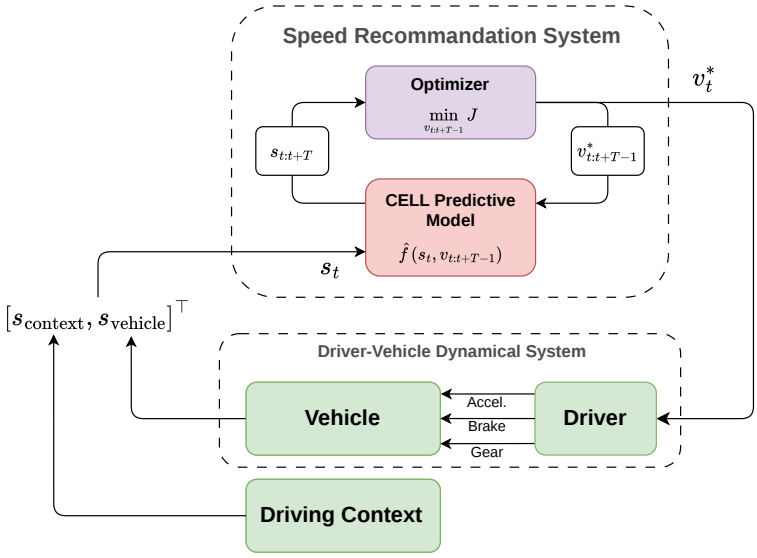


Figure 6.2: Architecture diagram of the proposed architecture for speed recommendation. The *Speed Recommendation System* follows a MPC architecture. It includes the *optimizer* that exploits the predictions from a *CELL predictive model* to obtain the optimal speed recommendation v_t^* from an initial state s_t. The speed recommendation setpoint v_t^* is then transmitted to the *driver* who acts on the vehicle through gear shifts, the brake and accelerator pedal. The initial state s_t is built from the state of the vehicle s_{vehicle} and from the driving context s_{context} such as $s_t = [s_{\text{context}}, s_{\text{vehicle}}]$. The *CELL predictive model* is continuously learning from the successive states. It collects the triplets: $\{s_t, v_t^*, s_{t+1}\}$ and learns to map a function $\hat{f}(s_t, v_t^*) = s_{t+1}$.

non-stationary behavior and specific environmental reactions, the recommendation system minimizes the model-driver mismatch, which represents a step toward a **trust-based collaborative system**.

6.3 Towards Personalized, Safe and Trustworthy Speed Recommendation

The previous section presented the architectural framework proposed in this thesis to address the speed recommendation problem. This section demonstrates how the specific theoretical and algorithmic contributions of this thesis directly address some of the industrial requirements listed in Section 6.1.

Safety through Epistemic Uncertainty Estimation and Introspection A major risk associated with deploying a learned predictive model in safety-critical systems lies in its Out-of-Distribution (OoD) behavior. Some types of learned models, such as neural networks, are known to exhibit systematic overconfidence even in unobserved regions of the state space [97]. In the context of speed recommendation, such overconfidence can result in unsafe or

misleading predictions.

As demonstrated in Chapter 3 and Chapter 4, the CELL instantiations, and more specifically kCELL, mitigate this issue by leveraging the spatial organization of local expert agents to improve the reliability of the learned predictive model within an MPC setting. Given that each agent is responsible for a localized region of the input space, the model can estimate espistemic uncertainty based on the value of activation of the agents. This allows the model to *know when it does not know.*

From an industrial perspective, this introspective property enables the implementation of safety guardrails. When the recommendation system detects high epistemic uncertainty, it can automatically trigger the use of a fallback strategy (e.g. reverting to a simple legal speed limit advisory system) instead of providing a potentially erroneous or unsafe recommendation.

Certifiability The requirement for Worst-Case Execution Time (WCET) is crucial when working with a limited embedded hardware configuration. In Chapter 4, we demonstrated that the kCELL was fully compatible with gradient-based high-performance Quadratic Programming (QP) solvers through its local linearization capabilities.

By integrating kCELL with high-performance solvers such as OSQP [212] or qpOASES [76] within an MPC loop, we leverage solvers with established convergence properties suitable for real-time applications [76, 212]. These solvers are designed for efficient computation in MPC loops and have been analyzed for worst-case complexity under standard assumptions (e.g. moderate receding horizon) [76]. This approach would theoretically allow speed recommendations to be computed within strict computational budget, even with a continuously learning kCELL predictive model. [76, 212]

Contextual Integrity and Parameter Isolation In terms of certifiability, a conflict arises from the need to reconcile continuous online learning with the static and verifiable designs required by ISO 26262 [119]. Global parametric models (e.g. neural networks) pose a significant risk in an online learning setting: online updates may inadvertently cause performance drops in unrelated regions of the state space because any parameter may change. This phenomenon, known as **catastrophic forgetting** (cf. Chapter 2), undermines predictability and hinders certifiability.

The CELL-based models like kCELL mitigate this issue through **Parameter Isolation**. Because learning is local, updates to an agent specialized in "Highway driving" do not alter the behavior of agents specialized in "Urban driving" because the corresponding subsets of local expert agents are spatially separable in the input space (as illustrated in Figure 6.3). This property provides **Contextual Integrity** to the continuously learning predictive model. It ensures that speed recommendations always rely on the model learned for the current driving context.

Trust through Transparency and Interpretability Finally, the industrial requirement for Driver Acceptance is addressed through the inherent transparency of CELL models like kCELL, especially when local experts employ white-box models as internal models (e.g. linear regressions). As discussed in Chapter 3, kCELL provides introspective properties derived

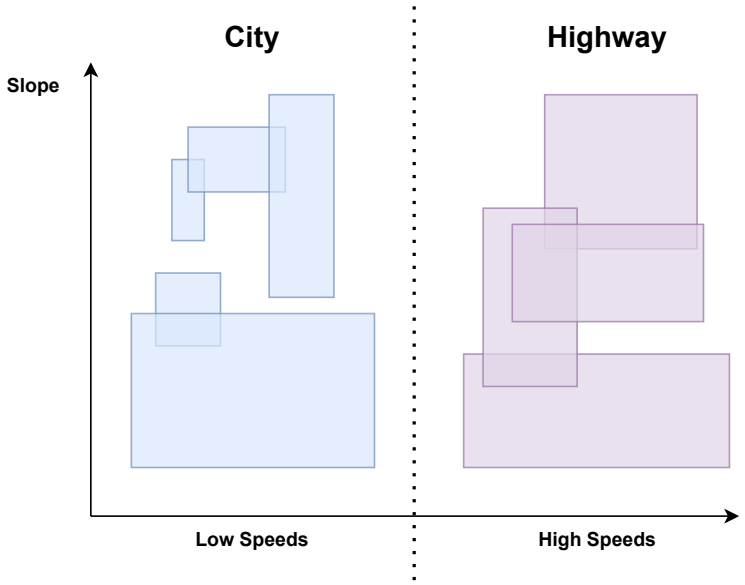


Figure 6.3: Explanatory diagram of Parameter Isolation. The horizontal axis corresponds to the speed and the vertical axis corresponds to the slope. It represents a simplified driving context. The *blue* and *violet* rectangles correspond to the local expert agent spatialized in the state space. This diagram illustrates the spatial separation between two groups of local expert agents that are conceptually unrelated. If the vehicle drives at a low speed, the learning updates will be done on the *blue* group and conversely, at high speed the updates will be done on the *violet* group.

from the spatial organization of agents, which can be linked to the traditional notions of explainability and interpretability.

Unlike global models, the inner workings of a CELL system can be inspected. One can :

- identify which expert(s) contributed to a given prediction,
- determine the driving context under which each local expert was trained,
- inspect feature contributions when white-box internal models are used (e.g. linear regressions),
- rapidly locate and correct faulty or poorly trained agents.

This level of transparency supports both technical auditability (e.g. during reviews requested by a certification authority) and user trust, as the system's reasoning can be made explicit when required. In the context of speed recommendation, such transparency and interpretability are essential for building a trustworthy collaborative relationship between the driver and the speed recommendation system.

6.4 Synthesis and Conclusion

In the previous sections, we established the industrial requirements associated with intelligent speed recommendation and demonstrated how the properties of the CELL models directly contribute towards fulfilling these requirements. Nevertheless, although the proposed approach satisfies several criteria, several challenges remain before our proposed solution can be considered ready for full industrial deployment. This section exposes the remaining limitations and outlines future research directions necessary to bridge the gap between the current algorithmic foundations of our approach, and a mature, market-ready ADAS system.

The Simulation and Validation Gap A key requirement for certifiability is the existence of high-fidelity simulation environments for both pre-training and validation of speed recommendation strategies.

While CELL systems are tailored for online adaptation, industrial deployment requires a cold-start initialization that provides acceptable performance from the moment the system is activated for the first time. This necessitates a pretrained default model capable of functioning before being able to adapt to the driver. Developing such a pretrained model depends on the availability of realistic simulation tools capable of reproducing the diverse conditions encountered by real drivers.

Moreover, developing such simulation environment for speed recommendation is significantly more complex than standard speed control environments. While rigid-body vehicle dynamics and drivetrain energy consumption models can be accurately described by well-established physical models, driver behavior depends on stochastic psychological processes, making it far more challenging to simulate.

Constructing realistic driver behavior models is essential for evaluating the long-term stability of online learning across diverse Operational Design Domains (ODDs), including rare but safety-critical edge cases that cannot be tested safely on public roads. A simulation ecosystem of this kind is therefore a key enabling step toward the certifiability of the proposed approach.

Toward Formal Safety and Convergence Proofs While we demonstrated in Chapter 4 that kCELL is locally linearizable and can be successfully integrated into Quadratic Programming (QP) optimization loops, the current reliability improvements remain mainly empirical.

The introspective properties of kCELL allow to move from *unknown risk* to *measured risk* by quantifying epistemic uncertainty from the spatial organization of the local expert agents. However, industrial certification ultimately requires more steps to reach a *close to zero risk* status through mathematical proofs.

Future works should explore Lyapunov-based formal certifications [127] for CELL instantiations to provide rigorous guarantees regarding the stability of the trajectory optimization process. This is particularly challenging for continuously learning predictive models because the model parameters are varying constantly.

Because kCELL is essentially non-parametric, its complexity depends on the volume of encountered data and on the complexity of the function to approximate. Future works should

also seek to find mathematical bounds on learning convergence and resource usage for such evolving architectures.

Handling More Diverse Data Sources The instantiations of CELL that have been developed in this thesis currently face technical limitations regarding the type of inputs they can handle. Real-world driving contexts involve a mix of continuous features (e.g speed, distance) and categorical features (e.g weather types, road signs, traffic light states). Currently, the developed CELL instantiations have been designed for continuous input spaces. Designing a novel CELL learning algorithm that is able to handle heterogeneous categorical data is essential for achieving robust predictive capacity.

In addition to that, as the dimensionality of the input space grows, the spatialization of agents suffers from the curse of dimensionality. This motivates further research into new mechanisms for agent spatialization to maintain both efficiency and accuracy in high-dimensional problems.

More In-Depth Personalization This thesis has relied on the assumption that driver acceptance increases when the recommended speed profiles align with the driver's behavior. While modeling the driver-vehicle dynamical system as a whole is a meaningful step towards efficient speed recommendation, human driving behavior is strongly influenced by hidden variables such as fatigue, stress, cognitive distraction, or emotional load.

Future improvements should incorporate physiological or behavioral markers that reflect the driver's internal state. A predictive model enriched with such information could greatly reduce uncertainty. This would enable more personalized, context-sensitive recommendations and ultimately foster greater trust and acceptance.

Chapter 7

Conclusion and Future Works

The research work presented in this thesis lies at the intersection of two research domains: **Machine Learning** and **Optimal Control**. This work was initially motivated by the industrial challenge of developing Intelligent Speed Recommendation systems capable of providing context-aware recommendations to drivers, with the goal of improving safety and fuel consumption. By treating the driver and the vehicle as a unified, non-stationary dynamical system, we have abstracted this problem into a **Human-Robot Collaboration (HRC)** problem.

This HRC formulation allows the speed recommendation problem to be decomposed into two subproblems:

- A **modeling** problem, which focuses on learning a predictive model of the uncertain and non-stationary behavior of the human driver
- An **exploitation** problem, which focuses on using this predictive model to generate safe and efficient speed recommendation profiles.

To address the specific challenges of HRC, we have proposed an architecture based on online learning through an ensemble of local expert agents, integrated within a Model Predictive Control (MPC) loop.

To this end, we have developed the Context Ensemble Local Learning (CELL) formalism. CELL provides a theoretical ground for designing learning algorithms where local expert agents self-organize through social rules to maintain a spatialized representation of knowledge. Throughout this thesis, we have proposed and evaluated several instantiations of this formalism, revealing useful emerging introspective properties.

The feasibility of integrating these online learning models into real-time control loops was investigated through case studies on a low-dimensional control task. Those experiments have demonstrated that the introspection properties emerging from the self-organizing ensemble of local expert agents 1) can be leveraged to guide exploration and 2) improve the reliability of control by avoiding regions of high epistemic uncertainty.

The remainder of this chapter is structured as follows. Section 7.1 provides a general conclusion on the research work that have been conducted. Section 7.2 details the specific scientific contributions of this thesis. Finally, Section 7.3 highlights several research avenues and future works perspectives, building upon the works of this thesis.

7.1 General Conclusion

This section provides a retrospective summary of the research work presented in this thesis. It details how each chapter contributed toward building safe and adaptive systems capable of synergizing with humans.

Problem Definition and Motivations Chapter 1 has introduced the industrial context and motivation of this research work related to the development of Intelligent Speed Recommendation. A distinction has been made between **speed control**, i.e automated driving, and **speed recommendation**. As this distinction emphasizes on handling the human in the loop, it has led to the abstraction of the speed recommendation problem into a Human-Robot Collaboration (HRC) problem.

This chapter argues that for such a recommendation device to be effective, it must possess two essential capabilities:

- the ability to continuously **learn and adapt** to the non-stationary, uncertain, intent of the driver;
- and the ability to **reliably exploit that model** within a control framework.

This allowed to identify two primary research axes: **online learning of non-stationary dynamics** and **reliability in learning-based control**.

Literature Review By analyzing the challenges of optimal control with learned models in Chapter 2, we have identified **Model Predictive Control (MPC)** as the most suitable framework for HRC. Indeed, MPC offers a theoretical framework to handle constraints and integrate a continuously updating predictive model. By analyzing the machine learning research axis, we have identified **Catastrophic Forgetting** as a major challenge for online adaptation.

We have deemed **Local learning algorithms** promising to tackle this challenge. These learning methods isolate parameters through spatial decomposition of the learning problem, offering a structural solution to the Catastrophic Forgetting problem. This has led to the exploration of a specific research niche: **self-organizing multi-agent systems for local learning**.

The CELL Formalism and its Instantiations To address the online learning problem, we have introduced the Context Ensemble Local Learning (CELL) formalism in Chapter 3. It provides a **framework for designing online learning algorithms** by treating the learning as a **social process** where local expert agents self-organize to approximate a function.

We have proposed a first instantiation: **oCELL**. oCELL uses a spatialization of local expert agents based on orthotopes and has demonstrated competitive performance with state of the art learning algorithms on a simple 2D benchmark. The development and empirical validation of oCELL has also allowed to identify emerging introspective properties. These properties can be derived from the result of the spatial organization of local expert agents, and they can be linked to the traditional notions of explainability and interpretability in machine learning. However, oCELL revealed some limitations, such as *hard-to-tune hyperparameters*,

the weakness of the *uniform knowledge assumption* and the *inefficiency of orthotopes* in representing complex local manifolds.

In response to the limitations of oCELL, we have developed a new instantiation of CELL: **kCELL**. Instead of using orthotopes for the spatialization of agents, kCELL uses a smooth, continuous approach based on kernels. It also introduces more sophisticated social rules to reduce the dependence on some hard-to-tune hyperparameters. It proved capable of

- adapting to non-stationary environments,
- simultaneously maintaining a transparent spatialization of knowledge, and
- competing with state-of-the-art online learning algorithms in terms of accuracy on a inverse modeling task.

The goal behind the development of kCELL was to build upon oCELL to build a learning algorithm better suited for being integrated into MPC.

Integration into the Control Loop As a robust learning algorithm has been designed through kCELL, Chapter 4 addresses the exploitation problem. We have demonstrated that kCELL could be implemented efficiently for real-time applications using GPU acceleration and geospatial indexing. Moreover, we have showed that the **introspective properties** of the kCELL ensemble of local expert agents can be leveraged to improve

- the learning of the predictive model within a control loop by guiding **exploration** toward unknown but relevant regions; and
- the reliability of the resulting model by safeguarding **exploitation**, discouraging the MPC optimizer from selecting Out-of-Distribution (OoD) trajectories that lie in unknown regions.

Enhancing the Robustness of Learning While kCELL provides a measure of epistemic uncertainty from the spatial organization of local expert agents in the **input space**, it does not provide an uncertainty measure on the predictions of local models in the **output space**. Recognizing this need for more formal uncertainty quantification for robust control, Chapter 5 **returned to the modeling layer** to explore the **modularity** of the CELL systems regarding the model class used by local expert agents. Thus, we have introduced gpCELL, in which local expert agents use **Gaussian Processes (GP) as internal models**. This allows the ensemble of local expert agents to behave like an **heterogeneous ensemble**, as agents can dynamically **adapt their complexity** to the local data manifold. Through empirical validation, we have shown that gpCELL provides well-calibrated predictive variance intervals on its predictions while maintaining the computational advantages of the local learning approach over traditional GP in an online learning task. These confidence intervals can prove essential for safety-critical control using learned models.

Industrial Synthesis Finally, Chapter 6 links the scientific contributions of this thesis back to the original industrial requirements. It evaluates how the proposed architecture, from the CELL formalism to the introspective MPC loop, addresses the specific challenges of driver acceptance, safety and energy efficiency. This synthesis allows to identify the remaining gaps between our current research and a production-ready system, paving the way for future works.

7.2 Contributions

The previous section provided a chapter-wise general conclusion for this thesis work. This section details the scientific contributions of this thesis. Section 7.2.1 presents the conceptual contributions on how the speed recommendation problem has been addressed as a Human-Robot Collaboration (HRC) problem. Then, Section 7.2.2 details the methodological and algorithmic contributions to the field of machine learning. Finally, Section 7.2.3, provides an overview of the contributions to the field of control with learned models.

7.2.1 Conceptual Contributions

The first set of contributions of this thesis works concerns the high-level framing of the speed recommendation problem:

- **Abstraction of Speed Recommendation into HRC**: We have introduced a new perspective on speed recommendation by abstracting it into a HRC problem. This acknowledge the necessity of fully incorporating the human in the loop, making it essential to explicitly model the human behavior.
- **System Architecture for HRC**: Following this abstraction, we have proposed to solve the problem with a modular architecture combining Model Predictive Control (MPC) with an online learning predictive model. This architecture provides the flexibility to adapt to evolving human behavior by regularly replanning the optimal trajectory through optimization. This is necessary for safe recommendations.

7.2.2 Contributions to Online Learning

A significant portion of this thesis focused on the development of online learning algorithms based on a self-organizing ensemble of local expert agents. The contributions are twofold: **Methodological** and **Algorithmic**.

Methodological Contributions We have introduced the CELL formalism for designing online learning systems based on a self-organizing ensemble of local expert agents. The main innovation lies in

- treating learning as a **social process** that incorporates cooperation and competition,
- providing learning and inference pipelines tailored for **use within control loops**.

Algorithmic Contributions

- **oCELL (Orthotope CELL)**: As a first proof of concept, oCELL has demonstrated that spatialized multi-agent systems following the CELL formalism could achieve competitive performance against state-of-the-art learning algorithms on a 2D benchmark. It has also allowed the identification of introspective properties that link the spatial organization of agents to traditional notions of explainability and interpretability in machine learning.

- **kCELL (Kernel CELL)**: We have developed kCELL to provide a smoother and more continuous representation of the function underlying the data. It has introduced *social feedback rules*, where predictive performance is assessed relative to neighbors rather than against an absolute error threshold. This design, enables to set up a competitive survival mechanism to tackle the agent overgrowth problem. Experiments have demonstrated kCELL's ability to adapt to non-stationary dynamics while providing the transparency required for in-depth behavioral analysis, potentially unlocking valuable insights for model debugging.

- **gpCELL (Gaussian Process CELL)**: To address the need for formal predictive uncertainty, we have explored the modularity of CELL by using Gaussian Processes (GP). gpCELL introduced a method based on distributed horizontal scaling to enable the use of GPs in online learning, despite the inherent computational bottleneck associated with a global GP. Through an AIC-based model selection process, agents are able to dynamically adapt their complexity to the local data manifold. A comparative experiment on an inverse modeling task reveals well-calibrated predictive uncertainty intervals and competitive predictive accuracy compared to baseline online learning algorithms.

7.2.3 Contributions to Control with Learned Models

The final set of contributions of our work bridges the gap between learned models and control in three complementary steps:

- **Efficient Integration and Differentiability**: To integrate kCELL into a MPC control loop, we have proposed leveraging GPU parallelization or geospatial indexing for agent selection to achieve real-time performance. Furthermore, we have shown that the trivial local linearization permitted by kCELL's structure naturally fits Sequential Quadratic Programming (SQP) optimizers, as the locality of the optimization aligns with the locality of the learned model.

- **Introspective Exploration for Data Collection**: We have proposed a novel strategy to guide exploration using the emerging properties of the agent ensemble (such as distance-based expertise or local disagreement). Validation on a pendulum control task has shown that this introspective exploration consistently outperforms uninformed random strategies by targeting unknown but relevant regions of the state-action space.

- **Introspective Exploitation for Reliability**: We have introduced a method to safeguard the control process by using introspective metrics such as the distance to the

closest agents to regularize the cost function used by the MPC optimizer. By discouraging the optimizer from selecting OoD trajectories (i.e. those lying far from the center of agents), we have demonstrated increased controller reliability, reducing prediction errors without requiring an external safety layer.

7.2.4 Synthesis of Contributions

This research offers broader contributions to the fields of Multi-Agent Systems (MAS), eXplainable AI (XAI), and Machine Learning.

- **Multi-Agent Systems (MAS)**: We have introduced a formalism that bridges between collective intelligence and function approximation. With the development of the CELL formalism and its instantiations, we have shown that social metaphors such as cooperation, competition, and survival can be effectively embodied as effective learning principles. Furthermore, we have provided a scalable architecture for MAS-based learning through GPU parallelization and geospatial indexing.
- **EXplainable AI (XAI)**: In our works, we have leveraged the spatial organization of local expert agents within a CELL model to show that introspective properties could act as direct and interpretable indicators of uncertainty. This approach contributes to the shift in explainability from post-hoc analysis toward inherently transparent model architectures.
- **Online Learning and Control**: We have addressed the challenge of leveraging a learning model able to handle non-stationary dynamics and a real-time optimization process. By embedding our ensemble of local expert agents into MPC controllers, we have shown that their inherent structural locality could be effectively exploited both to improve computational efficiency through local linearization and to improve reliability of the optimization process. This work represents a step toward the development of autonomous controllers that not only learn from data but also safely and actively balance exploration and exploitation within non-stationary environments.

7.3 Perspectives and Future Works

This final section outlines the future research directions and perspectives opened by the work presented in this thesis. While the CELL formalism and the integration of kCELL within MPC provides a promising foundation for adaptive HRC, several theoretical and practical challenges remain to be addressed regarding both machine learning and control. Section 7.3.1 explores the perspectives on Online Learning with CELL algorithms, while Section 7.3.2 discusses the perspectives on Control with Learned Models incorporating CELL algorithms.

7.3.1 Perspectives in Online Learning with CELL

For Online Learning with CELL algorithms, we highlight two directions for future works.

Ablation and Synergy Studies Future work should focus on quantifying the individual impact of the various social mechanisms in CELL learning algorithms such as the ones introduced with oCELL, kCELL and gpCELL. Systematic ablation studies are required to identify which combinations of rules maximize the learning efficiency and stability. Such systematic approach could be part of an iterative process whose objective would be to optimize the properties of the obtained CELL algorithm regarding constraints like predictive accuracy, sample efficiency or non-stationarity handling.

High-Dimensional Spatialization As the number of input dimensions grows, the spatialization approaches presented in this thesis suffer from the curse of dimensionality. We propose exploring spatialization mechanisms that are robust to increase in dimensionality, such as the use of robust similarity metrics like cosine similarity or local dimensionality reduction. By performing a local Principal Component Analysis (PCA) within each agent's area of expertise (from the parameters of their activation function Σ_i and μ_i), Mahalanobis distances can be computed in this reduced latent space to obtain the value of activation. This would allow agents to adapt their geometry more accurately to the local manifold's actual variance, and potentially use reconstruction errors to discriminate between neighbors.

7.3.2 Perspectives in Control with Learned Models with CELL

For Online Learning with CELL algorithms, we highlight four directions for future works.

Mathematical Guarantees through Trust Regions To robustify the control process, it is essential to derive theoretical safety guarantees. As a first step, we propose defining a trust region around the current state x. Within this trust region, the local linear approximation of the ensemble of local expert agents remains valid within a bounded error ϵ. By solving for the maximum radius r such that

$$\left\| \hat{f}(x) - \sum w_i \hat{f}_i(x) \right\| \leq \epsilon \ \forall x \in \mathcal{B}(x, r),$$

we can provide the SQP optimizer with a dynamic trust-region constraint, ensuring that the trajectory search never relies on inaccurate local linearizations. Because kCELL uses linear models, a closed-form analytical solution for this problem could probably be derived under the right assumptions.

Graph-Based Optimization Initialization Gradient-based optimizers such as SQP are highly sensitive to initial conditions [35]. A promising perspective is to treat the ensemble of agents as a connected graph. An agent $\mathcal{A}_0$ would be connected to an agent $\mathcal{A}_1$ if the area of expertise of agent $\mathcal{A}_1$ intersects with the set of possible predictive outputs of $\mathcal{A}_0$. By searching for a promising chain of successive agents that could potentially connect the current state to the goal, a warm-start trajectory could be generated to initialize the optimizer. This high-level topological planning would guide the low-level numerical optimization away from local minima and provide a feasible (or close-to-feasible) solution if hard constraints are considered.

LQR-based Explainer for Transparency of Decision Making While CELL instantiations like kCELL can provide full transparency in the prediction process, the optimization process often remains a black box. It opacifies the decision-making pipeline, specifically under hard constraints. We propose a post-hoc explanation method based on the Linear Quadratic Regulator (LQR) [131].

Following an optimization with SQP (with linearization of kCELL), the kCELL model can be linearized around the obtained trajectory to compute an unconstrained LQR baseline control law using the Riccati equations. The resulting local linear control law

$$u_t = -G_t s_t$$

provides a transparent linear relationship between the states and actions. By comparing the constrained SQP cost to the LQR cost

$$\Delta J = J_{SQP} - J_{LQR}$$

we can quantify the exact **cost of constraints**. Furthermore, the G_t matrix can be used as a feature importance metric, bringing another justification layer to the decision making process.

Robust Probabilistic MPC Finally, the predictive variance σ_*^2 provided by gpCELL enables the transition from deterministic to probabilistic constraint handling. Indeed, hard constraints $g(s_t, u_t) \leq 0$ can be transformed into chance constraints such as

$$P(g(s_t, u_t) \leq 0) \geq 1 - \alpha$$

where $\alpha \in [0, 1]$ sets the minimal probability at which the constraint must be satisfied.

Using a first-order Taylor expansion and assuming a Gaussian distribution of the predicted state $s_t \sim \mathcal{N}(\mu_t, \Sigma_t)$, this can be reformulated as a deterministic constraint:

$$g(\mu_t, u_t) + \Phi^{-1}(1 - \alpha) \sqrt{\nabla g^\top \Sigma_t \nabla g} \leq 0$$

where Φ^{-1} is the inverse cumulative distribution function. This approach would allow the controller to maintain safety margins that adapt dynamically to the model's confidence. In the context of HRC, it would provide a more reliable synergy with the driver in uncertain environments where safety is critical.

7.4 Final Thoughts

The staggering success of Deep Learning is undeniably tied to the tremendous progress in hardware-accelerated computing, which has unlocked the full potential of large-scale gradient descent. However, while neural networks are very powerful under the right conditions, they are often, and perhaps erroneously, treated as a panacea for all learning problems. As explored throughout this thesis, the black-box nature of deep neural networks and their inherent struggle with catastrophic forgetting present significant hurdles for online learning and safety-critical use cases.

In this research, we have focused on a multi-agent approach to learning that may, at first glance, appear unconventional or even vintage in the era of Deep Learning. By opting for a self-organizing ensemble of local experts rather than a monolithic neural architecture, we have returned to principles of structural modularity and transparency. We remain convinced that revisiting older paradigms through the lens of modern computing is not a step backward but a necessary detour. These approaches offer distinct advantages in domains that still elude the reach of standard neural networks, specifically the real-time, online adaptation to non-stationary dynamics.

Ultimately, this work is an invitation to look beyond the prevailing neural networks. It seeks to pave a promising path for those willing to explore alternative learning architectures based on decentralized multi-agent systems.

Bibliography

[1] Road vehicles — Cybersecurity engineering. Technical Report ISO/SAE 21434:2021, International Organization for Standardization, August 2021.

[2] Road vehicles – Safety of the intended functionality. Technical Report ISO 21448:2022, International Organization for Standardization, June 2022.

[3] Hossam M. Abdelghaffar, Maha Elouni, Youssef Bichiou, and Hesham A. Rakha. Development of a Connected Vehicle Dynamic Freeway Variable Speed Controller. *IEEE Access*, 8:99219–99226, May 2020.

[4] Hirotugu Akaike. A new look at the statistical model identification. *IEEE transactions on automatic control*, 19(6):716–723, 2003.

[5] Constantin Aliferis and Gyorgy Simon. Overfitting, underfitting and general model overconfidence and under-performance pitfalls and best practices in machine learning and AI. *Artificial intelligence and machine learning in health care and medical sciences: Best practices and pitfalls*, pages 477–524, 2024.

[6] Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3366–3375, 2017.

[7] Maazen Alsabaan, Kshirasagar Naik, and Tarek Khalifa. Optimization of Fuel Cost and Emissions Using V2V Communications. *IEEE Transactions on Intelligent Transportation Systems*, 14(3):1449–1461, May 2013.

[8] American Transportation Research Institute (ATRI). An analysis of the operational costs of trucking: 2024 update. Technical report, American Transportation Research Institute, June 2024.

[9] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

[10] Sule Anjomshoae, Amro Najjar, Davide Calvaresi, and Kary Främling. Explainable agents and robots: Results from a systematic literature review. In *18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2019), Montreal,*

Canada, May 13–17, 2019, pages 1078–1088. International Foundation for Autonomous Agents and Multiagent Systems, 2019.

[11] Behrang Asadi and Ardalan Vahidi. Predictive Cruise Control: Utilizing Upcoming Traffic Signal Information for Improving Fuel Economy and Reducing Trip Time. *IEEE Transactions on Control Systems Technology*, 19(3):707–714, April 2010.

[12] William Ross Ashby. An introduction to cybernetics. 1956.

[13] Karl Johan Åström and Tore Hägglund. The future of PID control. *Control engineering practice*, 9(11):1163–1175, 2001.

[14] Karl Johan Åström and Tore Hägglund. *Advanced PID Control*. ISA-The Instrumentation, Systems and Automation Society, 2006.

[15] Anil Aswani, Humberto Gonzalez, S Shankar Sastry, and Claire Tomlin. Provably safe and robust learning-based model predictive control. *Automatica*, 49(5):1216–1226, 2013.

[16] Christopher G Atkeson, Andrew W Moore, and Stefan Schaal. Locally weighted learning. *Artificial intelligence review*, 11(1):11–73, 1997.

[17] Christopher G Atkeson, Andrew W Moore, and Stefan Schaal. Locally weighted learning for control. *Artificial intelligence review*, 11(1):75–113, 1997.

[18] Claus Bahlmann, Martin Pellkofer, Jan Giebel, and Gregory Baratoff. Multi-modal speed limit assistants: Combining camera and GPS maps. In *2008 IEEE Intelligent Vehicles Symposium*, pages 132–137. IEEE, June 2008.

[19] Alessio Baratta, Antonio Cimino, Maria Grazia Gnoni, and Francesco Longo. Human robot collaboration in industry 4.0: A literature review. *Procedia Computer Science*, 217:1887–1895, 2023.

[20] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, (5):834–846, 2012.

[21] Matthias Bauer, Mark Van der Wilk, and Carl Edward Rasmussen. Understanding probabilistic sparse Gaussian process approximations. *Advances in neural information processing systems*, 29, 2016.

[22] Rolando Bautista-Montesano, Renato Galluzzi, Zhaobin Mo, Yongjie Fu, Rogelio Bustamante-Bello, and Xuan Di. Longitudinal Control Strategy for Connected Electric Vehicle with Regenerative Braking in Eco-Approach and Departure. *Applied Sciences*, 13(8):5089, April 2023.

[23] Richard Bellman. Dynamic programming. *science*, 153(3731):34–37, 1966.

[24] Alberto Bemporad. Model predictive control design: New trends and tools. In *Proceedings of the 45th IEEE Conference on Decision and Control*, pages 6678–6683. IEEE, 2006.

[25] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.

[26] Julian Berberich and Frank Allgöwer. An overview of systems-theoretic guarantees in data-driven model predictive control. *Annual Review of Control, Robotics, and Autonomous Systems*, 8(1):77–100, 2025.

[27] Felix Berkenkamp and Angela P Schoellig. Safe and robust learning control with Gaussian processes. In *2015 European Control Conference (ECC)*, pages 2496–2501. IEEE, 2015.

[28] Albert Bifet and Ricard Gavalda. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. SIAM, 2007.

[29] Albert Bifet and Ricard Gavalda. Adaptive learning from evolving data streams. In *International Symposium on Intelligent Data Analysis*, pages 249–260. Springer, 2009.

[30] Clément Blanco-Volle, Nicolas Verstaevel, Stéphanie Combettes, Marie-Pierre Gleizes, and Michel Povlovitsch Seixas. Explainability and interpretability of an ensemble multi-agent system for supervised learning. In *International Conference on Principles and Practice of Multi-Agent Systems*, pages 335–350. Springer, 2024.

[31] Clément Blanco-Volle, Nicolas Verstaevel, Stéphanie Combettes, Michel Povlovitsch Seixas, and Marie-Pierre Gleizes. Explicabilité et interprétabilité d'un système multi-agent ensembliste pour l'apprentissage supervisé. In *Les 32èmes Journées Francophones Sur Les Systèmes Multi-Agents (JFSMA 2024)*, 2024.

[32] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *International Conference on Machine Learning*, pages 1613–1622. PMLR, 2015.

[33] Jérémy Boes and Frédéric Migeon. Self-organizing multi-agent systems for the control of complex systems. *Journal of Systems and Software*, 134:12–28, 2017.

[34] Jérémy Boes, Julien Nigon, Nicolas Verstaevel, Marie-Pierre Gleizes, and Frédéric Migeon. The Self-Adaptive Context Learning Pattern: Overview and Proposal. In *SpringerLink*, pages 91–104. Springer, Cham, Switzerland, December 2015.

[35] Paul T Boggs and Jon W Tolle. Sequential quadratic programming. *Acta numerica*, 4:1–51, 1995.

[36] Friedman Breiman. *Classification and Regression Trees*. Taylor & Francis, Andover, England, UK, October 2017.

[37] Leo Breiman. Bagging predictors. *Machine learning*, 24(2):123–140, 1996.

[38] Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, October 2001.

[39] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[40] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annu. Rev. Control Rob. Auton. Syst.*, (Volume 5, 2022):411–444, May 2022.

[41] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.

[42] Nadia Burkart and Marco F Huber. A survey on the explainability of supervised machine learning. *Journal of Artificial Intelligence Research*, 70:245–317, 2021.

[43] Lorenzo Canese, Gian Carlo Cardarilli, Luca Di Nunzio, Rocco Fazzolari, Daniele Giardino, Marco Re, and Sergio Spanò. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences*, 11(11):4948, 2021.

[44] Mark Cannon and Basil Kouvaritakis. Efficient constrained model predictive control with asymptotic optimality. *SIAM journal on control and optimization*, 41(1):60–82, 2002.

[45] Runqi Chai, Al Savvaris, Antonios Tsourdos, Senchun Chai, and Yuanqing Xia. A review of optimization techniques in spacecraft flight trajectory design. *Progress in aerospace sciences*, 109:100543, 2019.

[46] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3):1–27, May 2011.

[47] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, P Dokania, P Torr, and M Ranzato. Continual learning with tiny episodic memories. In *Workshop on Multi-Task and Lifelong Reinforcement Learning*, 2019.

[48] Jing Chen, Cong Zhao, Shengchuan Jiang, Xinyuan Zhang, Zhongxin Li, and Yuchuan Du. Safe, Efficient, and Comfortable Autonomous Driving Based on Cooperative Vehicle Infrastructure System. *International Journal of Environmental Research and Public Health*, 20(1):893, January 2023.

[49] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. *ArXiv e-prints*, March 2016.

[50] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in neural information processing systems*, 31, 2018.

[51] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. *ArXiv e-prints*, May 2018.

[52] Roger C Conant and W Ross Ashby. Every good regulator of a system must be a model of that system. *International journal of systems science*, 1(2):89–97, 1970.

[53] Andrew R Conn, Nicholas IM Gould, and Philippe L Toint. *Trust Region Methods*. SIAM, 2000.

[54] Ashwin P Dani, Iman Salehi, Ghananeel Rotithor, Daniel Trombetta, and Harish Ravichandar. Human-in-the-loop robot control for human-robot collaboration. *IEEE Control Systems Magazine*, 2024.

[55] Jérémie Daniel and Jean-Philippe Lauffenburger. Multi-level Dempster-Shafer Speed Limit Assistant. In *SpringerLink*, pages 327–334. Springer, Berlin, Germany, 2012.

[56] Jérémie Daniel and Jean-Philippe Lauffenburger. Fusing navigation and vision information with the Transferable Belief Model: Application to an intelligent speed limit assistant. *Information Fusion*, 18:62–77, July 2014.

[57] Bruno Dato. *Apprentissage Permanent Par Feedback Endogène, Application à Un Système Robotique*. PhD thesis, Toulouse 3, 2021.

[58] Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, and Reuven Y Rubinstein. A tutorial on the cross-entropy method. *Annals of operations research*, 134(1):19–67, 2005.

[59] Bart De Vos, Ariane Cuenen, Veerle Ross, Hélène Dirix, Kris Brijs, and Tom Brijs. The Effectiveness of an Intelligent Speed Assistance System with Real-Time Speeding Interventions for Truck Drivers: A Belgian Simulator Study. *Sustainability*, 15(6):5226, March 2023.

[60] Marc Deisenroth and Carl E Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pages 465–472, 2011.

[61] Matthias Delange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Ales Leonardis, Greg Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–1, 2021.

[62] Charles Desjardins and Brahim Chaib-draa. Cooperative Adaptive Cruise Control: A Reinforcement Learning Approach. *IEEE Transactions on Intelligent Transportation Systems*, 12(4):1248–1260, June 2011.

[63] Thomas G. Dietterich. Ensemble Methods in Machine Learning. In *SpringerLink*, pages 1–15. Springer, Berlin, Germany, December 2000.

[64] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000.

[65] Carmel Domshlak, Steve Prestwich, Francesca Rossi, Kristen Brent Venable, and Toby Walsh. Hard and soft constraints for reasoning about qualitative conditional preferences. *Journal of Heuristics*, 12(4):263–285, 2006.

[66] Ali Dorri, Salil S Kanhere, and Raja Jurdak. Multi-agent systems: A survey. *IEEE access : practical innovations, open solutions*, 6:28573–28593, 2018.

[67] R. Driscoll, Y. Page, S. Lassarre, and J. Ehrlich. Lavia – an Evaluation of the Potential Safety Benefits of the French Intelligent Speed Adaptation Project. *Annual Proceedings / Association for the Advancement of Automotive Medicine*, 51:485, 2007.

[68] Yuchuan Du, Jing Chen, Cong Zhao, Chenglong Liu, Feixiong Liao, and Ching-Yao Chan. Comfortable and energy-efficient speed control of autonomous vehicles on rough pavements using deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, 134:103489, January 2022.

[69] Jacques Ehrlich, Michel Marchi, Service Europrojets, Cete Méditerranée, and Christian Leverger. LAVIA, the French ISA project: Main issues and first results on technical tests. *ResearchGate*, 2003.

[70] Rune Elvik. Speed and Road Safety: Synthesis of Evidence from Evaluation Studies. *Transportation Research Record*, 1908(1):59–69, January 2005.

[71] Tom Erez, Yuval Tassa, and Emanuel Todorov. Infinite-horizon model predictive control for periodic tasks with contacts. In *Robotics: Science and Systems*, volume 7, page 73, 2012.

[72] European Commission. Commission Delegated Regulation (EU) 2021/1958 of 23 June 2021 supplementing Regulation (EU) 2019/2144 by laying down detailed rules concerning the specific test procedures and technical requirements for Intelligent Speed Assistance. Official Journal of the European Union, L 409/1, 2021.

[73] European Parliament and Council. Regulation (EU) 2019/2144 of 27 November 2019 on type-approval requirements for motor vehicles and their trailers. Official Journal of the European Union, L 325, 2019.

[74] European Transport Safety Council. Briefing: Intelligent Speed Assistance (ISA). Technical report, ETSC, Brussels, Belgium, 2017.

[75] European Transport Safety Council. Briefing: Intelligent speed assistance (ISA). Technical report, ETSC, November 2018.

[76] Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: A parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6(4):327–363, 2014.

[77] Thibault Fourez. *Analyzing Human Mobility with an Ensemblist Multi-Agent Context Learning System*. PhD thesis, Université Toulouse Capitole, 2024.

[78] Thibault Fourez, Nicolas Verstaevel, Frédéric Migeon, Frédéric Schettini, and Frederic Amblard. An ensemble Multi-Agent System for non-linear classification. *ArXiv e-prints*, September 2022.

[79] Bruce A Francis and John C Doyle. Linear control theory with an \bfH_∞ optimality criterion. *SIAM Journal on Control and Optimization*, 25(4):815–844, 1987.

[80] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999.

[81] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences*, 55(1):119–139, 1997.

[82] G Friedl and M Kuczmann. Population and gradient based optimization techniques, a theoretical overview. *Acta Technica Jaurinensis*, 7(4):378–387, 2014.

[83] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, October 2001.

[84] Ioannis Galanis, Iraklis Anagnostopoulos, Priyaa Gurunathan, and Dona Burkard. Environmental-Based Speed Recommendation for Future Smart Cars. *Future Internet*, 11(3):78, March 2019.

[85] Sergio Galeani, Sophie Tarbouriech, Matthew Turner, and Luca Zaccarian. A tutorial on modern anti-windup design. *European Journal of Control*, 15(3-4):418–440, 2009.

[86] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence*, pages 286–295. Springer, 2004.

[87] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM computing surveys (CSUR)*, 46(4):1–37, 2014.

[88] Citlalli Gámez Serna and Yassine Ruichek. Dynamic Speed Adaptation for Path Tracking Based on Curvature Information and Speed Limits. *Sensors*, 17(6):1383, June 2017.

[89] Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice—A survey. *Automatica*, 25(3):335–348, 1989.

[90] Amir Ghiasi, Xiaopeng Li, and Jiaqi Ma. A mixed traffic speed harmonization model with connected autonomous vehicles. *Transportation Research Part C: Emerging Technologies*, 104:210–233, July 2019.

[91] Amirata Ghorbani and James Zou. Data shapley: Equitable valuation of data for machine learning. In *International Conference on Machine Learning*, pages 2242–2251. PMLR, 2019.

[92] Sean Gillies. Rtree: Spatial indexing for Python GIS.

[93] Gene H Golub and Charles F Van Loan. *Matrix Computations*. JHU press, 2013.

[94] Heitor Murilo Gomes, Jesse Read, Albert Bifet, Jean Paul Barddal, and João Gama. Machine learning for streaming data: State of the art, challenges, and opportunities. *ACM SIGKDD Explorations Newsletter*, 21(2):6–22, 2019.

[95] Joan Gonzalvez, Edmond Lezmi, Thierry Roncalli, and Jiali Xu. Financial applications of Gaussian processes and Bayesian optimization. *arXiv preprint arXiv:1903.04841*, 2019.

[96] Luigi Grippo, Francesco Lampariello, and Stefano Lucidi. A nonmonotone line search technique for Newton's method. *SIAM journal on Numerical Analysis*, 23(4):707–716, 1986.

[97] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

[98] Lulu Guo, Hong Chen, Bingzhao Gao, and Qifang Liu. Energy management of HEVs based on velocity profile optimization. *Science China Information Sciences*, 62(8):1–3, August 2019.

[99] Ralf Hartmut Güting. An introduction to spatial database systems. *the VLDB Journal*, 3(4):357–399, 1994.

[100] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, pages 47–57, 1984.

[101] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

[102] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[103] Nadia Ghezaiel Hammouda, Rashid Khan, Loghman Mostafa, Veyan A Musa, Husam Rajab, Narinderjit Singh Sawaran Singh, Walid Aich, and Khalil Hajlaoui. Application of deep reinforcement learning for aerodynamic control around an angled airfoil via synthetic jet. *Scientific Reports*, 2025.

[104] Honggui Han, Qili Chen, and Junfei Qiao. Research on an online self-organizing radial basis function neural network. *Neural computing and applications*, 19(5):667–676, 2010.

[105] Jihun Han, Antonio Sciarretta, Luis Leon Ojeda, Giovanni De Nunzio, and Laurent Thibault. Safe- and Eco-Driving Control for Connected and Automated Electric Vehicles Using Analytical State-Constrained Optimal Solution. *IEEE Transactions on Intelligent Vehicles*, 3(2):163–172, February 2018.

[106] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. Dynamic neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(11):7436–7456, 2021.

[107] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, volume 2. Springer, 2009.

[108] Ezra Hauer. Speed and Safety. *Transportation Research Record*, 2103(1):10–17, January 2009.

[109] Simon S Haykin. *Adaptive Filter Theory*. Pearson Education India, 2002.

[110] Hussein Hazimeh, Natalia Ponomareva, Petros Mol, Zhenyu Tan, and Rahul Mazumder. The tree ensemble layer: Differentiability meets conditional computation. In *International Conference on Machine Learning*, pages 4138–4148. PMLR, 2020.

[111] Abrar Hazoor, Alessandra Lioi, and Marco Bassani. Development of a Novel Intelligent Speed Adaptation System Based on Available Sight Distance. *Transportation Research Record*, April 2021.

[112] Lukas Hewing, Kim P Wabersich, Marcel Menner, and Melanie N Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3(1):269–296, 2020.

[113] Petter Holme and Fredrik Liljeros. Mechanistic models in computational social science. *Frontiers in Physics*, 3:78, 2015.

[114] Mehdi Hosseinzadeh, Bruno Sinopoli, Ilya Kolmanovsky, and Sanjoy Baruah. Robust-to-early termination model predictive control. *IEEE transactions on automatic control*, 69(4):2507–2513, 2023.

[115] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning*, 110(3):457–506, March 2021.

[116] Qinaat Hussain, Hanqin Feng, Raphael Grzebieta, Tom Brijs, and Jake Olivier. The relationship between impact speed and the probability of pedestrian fatality during a vehicle-pedestrian crash: A systematic review and meta-analysis. *Accident Analysis & Prevention*, 129:241–249, 2019.

[117] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.

[118] International Organization for Standardization. Road vehicles — Test object monitoring and control for active safety and automated/autonomous vehicle testing — Functional requirements, specifications and communication protocol. Technical Report ISO 22133:2026, ISO, Geneva, CH, 2026.

[119] International Organization for Standardization (ISO). Road vehicles – functional safety – part 1: Vocabulary. Technical Report ISO 26262-1:2018, ISO, Geneva, Switzerland, 2018.

[120] Andrew Jacobsen and Ashok Cutkosky. Online linear regression in dynamic environments via discounting. *arXiv preprint arXiv:2405.19175*, 2024.

[121] Momin Jamil and Xin-She Yang. A literature survey of benchmark functions for global optimisation problems. *ArXiv*, abs/1308.4008, 2013.

[122] Lijie Jia, Wenjing Li, Junfei Qiao, and Xinliang Zhang. An online self-organizing radial basis function neural network based on Gaussian Membership. *Applied Intelligence*, 55(6):1–17, 2025.

[123] Liming Jiang, Yuanchang Xie, Nicholas G. Evans, Xiao Wen, Tienan Li, and Danjue Chen. Reinforcement Learning based cooperative longitudinal control for reducing traffic oscillations and improving platoon stability. *Transportation Research Part C: Emerging Technologies*, 141:103744, August 2022.

[124] Xiangkui Jiang, Chang-an Wu, and Huaping Guo. Forest pruning based on branch importance. *Computational Intelligence and Neuroscience*, 2017(1):3162571, 2017.

[125] Felipe Jimenez, Juan Luis Lopez-Covarrubias, Wilmar Cabrera, and Francisco Aparicio. Real-time speed profile calculation for fuel saving considering unforeseen situations and travel time. *IET Intelligent Transport Systems*, 7(1), March 2013.

[126] Kun Jin, Xinran Li, Wei Wang, Xuedong Hua, and Weiyi Long. Energy-optimal speed control for connected electric buses considering passenger load. *Journal of Cleaner Production*, 385:135773, January 2023.

[127] Ming Jin and Javad Lavaei. Stability-certified reinforcement learning: A control-theoretic perspective. *IEEE access : practical innovations, open solutions*, 8:229086–229100, 2020.

[128] Qiu Jin, Guoyuan Wu, Kanok Boriboonsomsin, and Matthew J. Barth. Power-Based Optimal Longitudinal Control for a Connected Eco-Driving System. *IEEE Transactions on Intelligent Transportation Systems*, 17(10):2900–2910, March 2016.

[129] Curtis C Johnson, Tyler Quackenbush, Taylor Sorensen, David Wingate, and Marc D Killpack. Using first principles for deep learning and model-based control of soft robots. *Frontiers in Robotics and AI*, 8:654398, 2021.

[130] Norman W Johnson. *Geometries and Transformations*. Cambridge University Press, 2018.

[131] Rudolf Emil Kalman. When is a linear control system optimal? 1964.

[132] Rudolf Emil Kalman et al. Contributions to the theory of optimal control. 5(2):102–119, 1960.

[133] Rudolph E Kalman and Richard S Bucy. New results in linear filtering and prediction theory. 1961.

[134] Md. Abdus Samad Kamal, Masakazu Mukai, Junichi Murata, and Taketoshi Kawabe. Model Predictive Control of Vehicles on Urban Roads for Improved Fuel Economy. *IEEE Transactions on Control Systems Technology*, 21(3):831–841, June 2012.

[135] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976):982–987, 2023.

[136] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 2017.

[137] Yunjong Kim, Kawon Kang, Nuri Park, Juneyoung Park, and Cheol Oh. Reinforcement learning approach to develop variable speed limit strategy using vehicle data and simulations. *Journal of Intelligent Transportation Systems*, 29(3):251–268, 2025.

[138] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

[139] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[140] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.

[141] Ilya V. Kolmanovsky and Dimitar P. Filev. Terrain and Traffic Optimized Vehicle Speed Control. *IFAC Proceedings Volumes*, 43(7):378–383, July 2010.

[142] Nikolaos Kouiroukidis and Georgios Evangelidis. The effects of dimensionality curse in high dimensional knn search. In *2011 15th Panhellenic Conference on Informatics*, pages 41–45. IEEE, 2011.

[143] Michael Krause and Klaus Bengler. Traffic Light Assistant-Driven in a Simulator. *Proceedings of the 2012 International IEEE Intelligent Vehicles Symposium Workshops*, June 2012.

[144] Hadas Kress-Gazit, Kerstin Eder, Guy Hoffman, Henny Admoni, Brenna Argall, Rüdiger Ehlers, Christoffer Heckman, Nils Jansen, Ross Knepper, Jan Křetínskỳ, et al. Formalizing and guaranteeing human-robot interaction. *Communications of the ACM*, 64(9):78–84, 2021.

[145] Balaji Lakshminarayanan, Daniel M Roy, and Yee Whye Teh. Mondrian forests: Efficient online random forests. *Advances in neural information processing systems*, 27, 2014.

[146] Mohammad Lavasani, Xia Jin, and Yiman Du. Market penetration model for autonomous vehicles on the basis of earlier technology adoption experience. *Transportation Research Record*, 2597(1):67–74, 2016.

[147] Heeyun Lee, Kyunghyun Kim, Namwook Kim, and Suk Won Cha. Energy efficient speed planning of electric vehicles for car-following scenario using model-based reinforcement learning. *Applied Energy*, 313:118460, May 2022.

[148] Heeyun Lee, Namwook Kim, and Suk Won Cha. Model-Based Reinforcement Learning for Eco-Driving Control of Electric Vehicles. *IEEE Access*, 8:202886–202896, November 2020.

[149] Joonho Lee, Jemin Hwangbo, and Marco Hutter. Robust recovery controller for a quadrupedal robot using deep reinforcement learning. *arXiv preprint arXiv:1901.07517*, 2019.

[150] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

[151] Jie Li, Xiaodong Wu, Min Xu, and Yonggang Liu. Deep reinforcement learning and reward shaping based eco-driving control for automated HEVs among signalized intersections. *Energy*, 251:123924, July 2022.

[152] Chunyu Liu, Zihao Sheng, Sikai Chen, Haotian Shi, and Bin Ran. Longitudinal control of connected and automated vehicles among signalized intersections in mixed traffic flow with deep reinforcement learning approach. *Physica A: Statistical Mechanics and its Applications*, 629:129189, November 2023.

[153] Weitang Liu, Xiaoyun Wang, John Owens, and Yixuan Li. Energy-based out-of-distribution detection. *Advances in neural information processing systems*, 33:21464–21475, 2020.

[154] David Lowe and D Broomhead. Multivariable functional interpolation and adaptive networks. *Complex systems*, 2(3):321–355, 1988.

[155] Jiaqi Ma, Jia Hu, Ed Leslie, Fang Zhou, Peter Huang, and Joe Bared. An eco-drive experiment on rolling terrains for fuel consumption optimization with connected automated vehicles. *Transportation Research Part C: Emerging Technologies*, 100:125–141, March 2019.

[156] Víctor Corcoba Magaña, Mario Muñoz Organero, Juan Antonio Álvarez-García, and Jorge Yago Fernández Rodríguez. Design of a Speed Assistant to Minimize the Driver Stress. *ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal*, 6(3):45, September 2017.

[157] Grant Mahler, Andreas Winckler, Seyed Alireza Fayazi, Martin Filusch, and Ardalan Vahidi. Cellular communication of traffic signal state to connected vehicles for arterial eco-driving. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–6. IEEE, October 2017.

[158] G. Malaterre and F. Saad. CONTRIBUTION A L'ANALYSE DU CONTROLE DE LA VITESSE PAR LE CONDUCTEUR : EVALUATION DE DEUX LIMITEURS. *CAH ETUD ONSER*, (62), October 1984.

[159] Andreas A. Malikopoulos, Seongah Hong, B. Brian Park, Joyoung Lee, and Seunghan Ryu. Optimal Control for Speed Harmonization of Automated Vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 20(7):2405–2417, September 2018.

[160] Yukimasa Matsumoto and Daisuke Tsurudome. Evaluation of Providing Recommended Speed for Reducing CO2 Emissions from Vehicles by Driving Simulator. *Transportation Research Procedia*, 3:31–40, January 2014.

[161] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[162] Franziska Meier, Philipp Hennig, and Stefan Schaal. Incremental local gaussian regression. *Advances in Neural Information Processing Systems*, 27, 2014.

[163] Martial Mermillod, Aurélia Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects, 2013.

[164] Oussama Messaoudi and Ammar Lahlouhi. An agent-based inter-vehicle cooperative robust car-following model for longitudinal control under uncertainty. *International Journal of Computer Applications in Technology*, September 2018.

[165] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[166] Iman Tahbaz-zadeh Moghaddam, Moosa Ayati, and Amir Taghavipour. Cooperative adaptive cruise control system for electric vehicles through a predictive deep reinforcement learning approach. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, 238(7):1751–1773, 2024.

[167] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, et al. River: Machine learning for streaming data in python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

[168] John Moody and Christian J Darken. Fast learning in networks of locally-tuned processing units. *Neural computation*, 1(2):281–294, 1989.

[169] A. M. Naveen, Roopa Ravish, and Shanta Ranga Swamy. Distributional Reinforcement Learning For Automated Driving Vehicle. In *2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon)*, pages 1–6. IEEE, October 2022.

[170] Julien Nigon. *APPRENTISSAGE ARTIFICIEL ADAPTÉ AUX SYSTÈMES COMPLEXES PAR AUTO-ORGANISATION COOPÉRATIVE DE SYSTÈMES MULTI-AGENTS*. PhD thesis, 2017.

[171] Julien Nigon, Nicolas Verstaevel, Jérémy Boes, Frédéric Migeon, and Marie-Pierre Gleizes. Smart is a matter of context. In *International and Interdisciplinary Conference on Modeling and Using Context*, pages 189–202. Springer, 2017.

[172] Jorge Nocedal and Stephen J Wright. *Numerical Optimization*. Springer, 2006.

[173] Katsuhiko Ogata. Modern control engineering. 2020.

[174] Engin Ozatay, Simona Onori, James Wollaeger, Umit Ozguner, Giorgio Rizzoni, Dimitar Filev, John Michelini, and Stefano Di Cairano. Cloud-Based Velocity Profile Optimization for Everyday Driving: A Dynamic-Programming-Based Solution. *IEEE Transactions on Intelligent Transportation Systems*, 15(6):2491–2505, May 2014.

[175] Engin Ozatay, Umit Ozguner, and Dimitar Filev. Velocity profile optimization of on road vehicles: Pontryagin's Maximum Principle based approach. *Control Engineering Practice*, 61:244–254, April 2017.

[176] Engin Ozatay, Umit Ozguner, John Michelini, and Dimitar Filev. Analytical Solution to the Minimum Energy Consumption Based Velocity Profile Optimization Problem With Variable Road Grade. *IFAC Proceedings Volumes*, 47(3):7541–7546, January 2014.

[177] Emanuel Parzen. On estimation of a probability density function and mode. *The annals of mathematical statistics*, 33(3):1065–1076, 1962.

[178] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[179] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, pages 2778–2787. PMLR, 2017.

[180] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Andreas Müller, Joel Nothman, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine Learning in Python. *ArXiv e-prints*, January 2012.

[181] Fernando Pérez-Cruz, Steven Van Vaerenbergh, Juan José Murillo-Fuentes, Miguel Lázaro-Gredilla, and Ignacio Santamaria. Gaussian processes for nonlinear signal processing: An overview of recent advances. *IEEE Signal Processing Magazine*, 30(4):40–50, 2013.

[182] Robi Polikar. Ensemble Learning. In *SpringerLink*, pages 1–34. Springer, New York, NY, New York, NY, USA, January 2012.

[183] Krupa Prag, Matthew Woolway, and Turgay Celik. Toward data-driven optimal control: A systematic review of the landscape. *IEEE access : practical innovations, open solutions*, 10:32190–32212, 2022.

[184] Pinpin Qin, Fumao Wu, Shenglin Bin, Xing Li, and Fuming Ya. High-Accuracy, High-Efficiency, and Comfortable Car-Following Strategy Based on TD3 for Wide-to-Narrow Road Sections. *World Electric Vehicle Journal*, 14(9):244, September 2023.

[185] S Joe Qin and Thomas A Badgwell. A survey of industrial model predictive control technology. *Control engineering practice*, 11(7):733–764, 2003.

[186] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[187] James Blake Rawlings, David Q Mayne, Moritz Diehl, et al. Model predictive control: Theory, computation, and design, vol. 2. *Madison, WI: Nob Hill Publishing*, 2017.

[188] Gregor Rohbogner, Simon Fey, Pascal Benoit, Christof Wittwer, and Andreas Christ. Design of a multiagent-based voltage control system in peer-to-peer networks for smart grids. *Energy Technology*, 2(1):107–120, 2014.

[189] Reuven Y Rubinstein and Dirk P Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer Science & Business Media, 2004.

[190] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.

[191] Alberto Viseras Ruiz and Calin Olariu. A general algorithm for exploration with gaussian processes in complex, unknown environments. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3388–3393. IEEE, 2015.

[192] Amir Saffari, Christian Leistner, Jakob Santner, Martin Godec, and Horst Bischof. Online random forests. In *2009 Ieee 12th International Conference on Computer Vision Workshops, Iccv Workshops*, pages 1393–1400. IEEE, 2009.

[193] Yuichi Saito, Fumio Sugaya, Shintaro Inoue, Pongsathorn Raksincharoensak, and Hideo Inoue. A context-aware driver model for determining recommended speed in blind intersection situations. *Accident Analysis & Prevention*, 163:106447, December 2021.

[194] Tajudeen Abiola Ogunniyi Salau, Adebayo Oludele Adeyefa, and Sunday Ayoola Oke. Vehicle speed control using road bumps. *Transport*, 19(3):130–136, January 2004.

[195] Iqbal H Sarker. Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions. *SN computer science*, 2(6):1–20, 2021.

[196] P. Saves, R. Lafage, N. Bartoli, Y. Diouane, J. Bussemaker, T. Lefebvre, J. T. Hwang, J. Morlier, and J. R. R. A. Martins. SMT 2.0: A surrogate modeling toolbox with a focus on hierarchical and mixed variables gaussian processes. *Advances in Engineering Sofware*, 188:103571, 2024.

[197] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

[198] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[199] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning*, pages 1889–1897. PMLR, 2015.

[200] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *ArXiv e-prints*, July 2017.

[201] Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: An engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5):1327–1349, 2021.

[202] Matthias Seeger. Gaussian processes for machine learning. *International journal of neural systems*, 14(02):69–106, 2004.

[203] Francesco Semeraro, Alexander Griffiths, and Angelo Cangelosi. Human–robot collaboration and machine learning: A systematic review of recent research. *Robotics and Computer-Integrated Manufacturing*, 79:102432, 2023.

[204] G Di M Serugendo, M-PG Irit, and Anthony Karageorgos. Self-organisation and emergence in MAS: An overview. *Informatica*, 30(1), 2006.

[205] Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends® in Machine Learning*, 4(2):107–194, 2025.

[206] Shahaboddin Shamshirband, Nor Badrul Anuar, Miss Laiha Mat Kiah, and Ahmed Patel. An appraisal and design of a multi-agent system based cooperative wireless intrusion detection computational intelligence technique. *Engineering Applications of Artificial Intelligence*, 26(9):2105–2127, 2013.

[207] Daliang Shen, Dominik Karbowski, and Aymeric Rousseau. Fuel-Optimal Periodic Control of Passenger Cars in Cruise Based on Pontryagin's Minimum Principle∗∗This research was sponsored by the U.S. Department of Energy (DOE) Vehicle Technologies Office (VTO) under the Systems and Modeling for Accelerated Research in Transportation (SMART) Mobility Laboratory Consortium, an initiative of the Energy Efficient Mobility Systems (EEMS) Program. *IFAC-PapersOnLine*, 51(31):813–820, January 2018.

[208] Mel Siegel. The sense-think-act paradigm revisited. In *1st International Workshop on Robotic Sensing, 2003. ROSE'03.*, pages 5–pp. IEEE, 2003.

[209] Steve Smale and Yuan Yao. Online learning algorithms. *Foundations of computational mathematics*, 6(2):145–170, 2006.

[210] Charles Spearman. The proof and measurement of association between two things. 1961.

[211] Niranjan Srinivas, Andreas Krause, Sham M Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. *arXiv preprint arXiv:0912.3995*, 2009.

[212] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[213] Charles J Stone. Consistent nonparametric regression. *The annals of statistics*, pages 595–620, 1977.

[214] Alexander Strehl and Michael Littman. Online linear regression and its application to model-based reinforcement learning. *Advances in Neural Information Processing Systems*, 20, 2007.

[215] Chao Sun, Jacopo Guanetti, Francesco Borrelli, and Scott J. Moura. Optimal Eco-Driving Control of Connected and Autonomous Vehicles Through Signalized Intersections. *IEEE Internet of Things Journal*, 7(5):3759–3773, January 2020.

[216] Ming Sun, Weiqiang Zhao, Guanghao Song, Zhigen Nie, Xiaojian Han, and Yang Liu. DDPG-Based Decision-Making Strategy of Adaptive Cruising for Heavy Vehicles Considering Stability. *IEEE Access*, 8:59225–59246, March 2020.

[217] Richard S Sutton, Andrew G Barto, et al. *Reinforcement Learning: An Introduction*, volume 1. MIT press Cambridge, 1998.

[218] Van-Bao Ta, Michel Verge, Nazih Mechbal, Guichon Dominique, and Salim Srairi. Development of a Dynamic Intelligent Speed Adaptation (ISA). *Procedia - Social and Behavioral Sciences*, 48:2111–2120, January 2012.

[219] Florian Tambon, Gabriel Laberge, Le An, Amin Nikanjam, Paulina Stevia Nouwou Mindom, Yann Pequignot, Foutse Khomh, Giulio Antoniol, Ettore Merlo, and Francois Laviolette. How to certify machine learning based safety-critical systems? A systematic literature review. *Automated Software Engineering*, 29(2):38, 2022.

[220] Fergus Tate, O Carsten, and UK ISA. Implementation scenarios, 1997.

[221] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.

[222] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[223] George E Uhlenbeck and Leonard S Ornstein. On the theory of the Brownian motion. *Physical review*, 36(5):823, 1930.

[224] United Nations Economic Commission for Europe (UNECE). Uniform provisions concerning the approval of vehicles with regard to Driver Control Assistance Systems (DCAS). Technical Report UN Regulation No. 171, UNECE, Geneva, Switzerland, 2021.

[225] Nicolas Verstaevel. *Self-Organization of Robotic Devices through Demonstrations*. PhD thesis, Université de Toulouse, Université Toulouse III-Paul Sabatier, 2016.

[226] Nicolas Verstaevel, Jérémy Boes, Julien Nigon, Dorian d'Amico, and Marie-Pierre Gleizes. Lifelong machine learning with adaptive multi-agent systems. In *9th International Conference on Agents and Artificial Intelligence (ICAART 2017)*, volume 1, pages 275–286, 2017.

[227] Sylvain Videau, Carole Bernon, Pierre Glize, and Jean-Louis Uribelarrea. Controlling bioprocesses using cooperative self-organizing agents. In *Advances on Practical Applications of Agents and Multiagent Systems: 9th International Conference on Practical Applications of Agents and Multiagent Systems*, pages 141–150. Springer, 2011.

[228] Sethu Vijayakumar, Aaron D'souza, Tomohiro Shibata, Jörg Conradt, and Stefan Schaal. Statistical learning for humanoid robots. *Autonomous Robots*, 12(1):55–69, 2002.

[229] Sethu Vijayakumar and Stefan Schaal. Locally weighted projection regression: An o (n) algorithm for incremental real time learning in high dimensional space. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000)*, volume 1, pages 288–293. Morgan Kaufmann Burlington, MA, 2000.

[230] Jayant Vyas, Debasis Das, and Santanu Chaudhury. DriveBFR: Driver behavior and fuel-efficiency-based recommendation system. *IEEE Transactions on Computational Social Systems*, 9(5):1446–1455, 2021.

[231] Liyuan Wang, Xingxing Zhang, Qian Li, Mingtian Zhang, Hang Su, Jun Zhu, and Yi Zhong. Incorporating neuro-inspired adaptability for continual learning in artificial intelligence. *Nature Machine Intelligence*, 5(12):1356–1368, 2023.

[232] Xiaomei Wang, Yingqi Li, and Ka-Wai Kwok. A survey for machine learning-based control of continuum robots. *Frontiers in Robotics and AI*, 8:730330, 2021.

[233] Larry Wasserman. *All of Nonparametric Statistics*. Springer, 2006.

[234] Marius Wegener, Lucas Koch, Markus Eisenbarth, and Jakob Andert. Automated eco-driving in urban scenarios using deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, 126:102967, May 2021.

[235] Sanford Weisberg. *Applied Linear Regression*. John Wiley & Sons, Ltd., Chichester, England, UK, January 2005.

[236] Barry Payne Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics : a journal of statistics for the physical, chemical, and engineering sciences*, 4(3):419–420, 1962.

[237] Christopher Williams and Carl Rasmussen. Gaussian processes for regression. *Advances in neural information processing systems*, 8, 1995.

[238] Christopher KI Williams and Carl Edward Rasmussen. *Gaussian Processes for Machine Learning*, volume 2. MIT press Cambridge, MA, 2006.

[239] Grady Williams, Andrew Aldrich, and Evangelos A Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017.

[240] David H Wolpert. Stacked generalization. *Neural networks*, 5(2):241–259, 1992.

[241] Yina Wu, Mohamed Abdel-Aty, Ling Wang, and Md Sharikur Rahman. Combined connected vehicles and variable speed limit strategies to reduce rear-end crash risk under fog conditions. *Journal of Intelligent Transportation Systems*, 24(5):494–513, September 2020.

[242] Tiantian Xie, Hao Yu, and Bogdan Wilamowski. Comparison between traditional neural networks and radial basis function networks. In *2011 IEEE International Symposium on Industrial Electronics*, pages 1194–1199. IEEE, 2011.

[243] Guangfei Xu, Xiangkun He, Meizhou Chen, Hequan Miao, Huanxiao Pang, Jian Wu, Peisong Diao, and Wenjun Wang. Hierarchical speed control for autonomous electric vehicle through deep reinforcement learning and robust control. *IET Control Theory & Applications*, 16(1):112–124, January 2022.

[244] Nan Xu, Xiaohan Li, Qiao Liu, and Di Zhao. An overview of eco-driving theory, capability evaluation, and training applications. *Sensors*, 21(19):6547, 2021.

[245] Zhenjie Yang, Xiaosong Jia, Hongyang Li, and Junchi Yan. A Survey of Large Language Models for Autonomous Driving. *ArXiv e-prints*, November 2023.

[246] Kai Yu, Liang Ji, and Xuegong Zhang. Kernel nearest-neighbor algorithm. *Neural Processing Letters*, 15(2):147–156, 2002.

[247] Seniha Esen Yuksel, Joseph N. Wilson, and Paul D. Gader. Twenty years of mixture of experts. *IEEE transactions on neural networks and learning systems*, 23(8):1177–1193, 2012.

[248] George Zames. Feedback and optimal sensitivity: Model reference transformations, multiplicative seminorms, and approximate inverses. *IEEE Transactions on automatic control*, 26(2):301–320, 2003.

[249] Jin Zhang and Jingyue Li. Testing and verification of neural-network-based safety-critical control software: A systematic literature review. *Information and Software Technology*, 123:106296, 2020.

[250] Shu Zhang, Wolfgang Jank, and Galit Shmueli. Real-time forecasting of online auctions via functional k-nearest neighbors. *International Journal of Forecasting*, 26(4):666–683, 2010.

[251] Yi Zhang, Ping Sun, Yuhan Yin, Lin Lin, and Xuesong Wang. Human-like Autonomous Vehicle Speed Control by Deep Reinforcement Learning with Double Q-Learning. In *2018 IEEE Intelligent Vehicles Symposium (IV)*, pages 1251–1256. IEEE, June 2018.

[252] Lihua Zhao, Ryutaro Ichise, Seiichi Mita, and Yutaka Sasaki. An Ontology-Based Intelligent Speed Adaptation System for Autonomous Cars. In *SpringerLink*, pages 397–413. Springer, Cham, Switzerland, February 2015.

[253] Rui Zhao, Kui Wang, Wenbo Che, Yun Li, Yuze Fan, and Fei Gao. Adaptive cruise control based on safe deep reinforcement learning. *Sensors*, 24(8):2657, 2024.

[254] Xiaohua Zhao, Wenhui Dong, Jia Li, Qiqi Liu, and Yunjie Ju. How does the mural decoration of the long tunnel sidewall affect the driver's speed control ability? *Tunnelling and Underground Space Technology*, 130:104731, December 2022.

[255] Feng Zhu and Satish V. Ukkusuri. Accounting for dynamic speed limit control in a stochastic traffic environment: A reinforcement learning approach. *Transportation Research Part C: Emerging Technologies*, 41:30–47, April 2014.

[256] Meixin Zhu, Xuesong Wang, and Yinhai Wang. Human-like autonomous car-following model with deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, 97:348–368, December 2018.

[257] Meixin Zhu, Yinhai Wang, Ziyuan Pu, Jingyun Hu, Xuesong Wang, and Ruimin Ke. Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving. *Transportation Research Part C: Emerging Technologies*, 117:102662, August 2020.

[258] Juntang Zhuang, Tommy Tang, Yifan Ding, Sekhar Tatikonda, Nicha Dvornek, Xenophon Papademetris, and James S. Duncan. AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients. *ArXiv e-prints*, October 2020.

Glossary

Aleatoric Uncertainty	Aleatoric uncertainty arises from the inherent natural variations in the studied phenomena. It may be due to random fluctuations, measurement errors, or other unpredictable factors [51, 115].
Catastrophic Forgetting	In the context of machine learning, catastrophic forgetting refers to the abrupt and severe degradation of performance on previously learned tasks when a model is trained sequentially on new, potentially conflicting tasks. This usually arises from updates to parameters that are shared for several overlapping tasks that overwrite previous knowledge, disrupting long-term knowledge retention [80, 47, 231].
Certifiability	Certifiability refers to the formal process required to demonstrate that a safety-critical system conforms to applicable regulatory, safety, and performance standards, thereby authorizing its real-world deployment [219].
Concept Drift	Concept drift in online machine learning refers to the gradual or sudden change in the relationship between input data and target outcomes over time, causing previously trained models to lose accuracy as real-world patterns evolve [86].
Cooperative Self-Organization	Cooperative self-organization in multi-agent systems can be defined as the emergent process by which autonomous agents achieve global coordination and adaptive behavior through decentralized, local interactions and rule-based cooperation, without relying on centralized control [204].
Curse of Dimensionality	The curse of dimensionality refers to phenomena where analyzing high-dimensional data becomes exponentially more difficult. As dimensions increase, distances between samples are less meaningful, data volume grows rapidly, and available samples become sparser, requiring exponentially more data for reliable analysis [23].
Epistemic Uncertainty	Epistemic uncertainty stems from a lack of knowledge or complete understanding of the studied phenomenon. This type

	of uncertainty is closely related to the lack of training data [51, 115].
Explainability	Explainability refers to the methodologies used to communicate the reasoning behind a model's output to a human. It is typically applied to complex black-box models that are not inherently understandable, requiring post-hoc techniques to make specific decisions or the overall model behavior comprehensible [42].
First-principles model	A first-principles model is a mathematical model of a dynamical system derived directly from fundamental physical laws (such as conservation of mass, energy, momentum or Newton's laws). The states and parameters of the model have a clear physical meaning rather than being fitted purely from data [129].
Forward Dynamics	Forward dynamics modeling is the construction of a model that maps states (such as positions and velocities) and actions to future state trajectories.
Global Learning	Global learning refers to machine learning approaches that construct a model capturing the overall data distribution or structure, where parameters are updated using the entire training dataset, and predictions arise from the unified model architecture [242].
Human-Robot Collaboration	Human-Robot Collaboration (HRC) is defined as the process where a human and a robot work together as a team, through a shared workspace and a shared task, to achieve a common goal by leveraging the complementary strength of both actors [203].
In-Distribution	As opposed to Out-of-Distribution (OoD), In-Distribution (ID) refers to data points in machine learning that belong to the same probability distribution as the one used to train the model [153].
Instrospection	Introspection designates the use of emergent properties, such as the geometric structure and density of agents, to monitor the system's internal state. These properties arise from local self-organizing mechanisms and provide a macroscopic view of the learning process or the reliability of a prediction.
Interpretability	Interpretability refers to the inherent ability to understand how a machine learning model functions as a whole by focusing on its internal structure and mechanisms. It is often used

	to describe white-box models that are naturally transparent or designed to be easily understood by humans [42].
Inverse Dynamics	Inverse dynamics modeling is the construction of a model that maps a dynamical system's motion (positions, velocities, accelerations) to the internal forces or actions (e.g. joint torques) required to produce that motion.
Local Learning	Local learning refers to machine learning approaches that construct task-oriented models using subsets of the training data, such as local neighborhoods, where parameters are updated incrementally on relevant data points, and predictions airse from context-specific approximations rather than a unified global architecture [242].
Non-Parametric	Non-parametric models are models whose structure and number of parameters are not fixed in advance but are instead determined by the data. Unlike parametric models, their complexity can grow with the size and diversity of the training dataset, allowing them to capture increasingly complex or non-stationary patterns [233].
Non-Stationarity	Refers to a dynamical system whose underlying transition dynamics are not constant over time.
Orthotope	An orthotope is the n-dimensional generalization of a rectangle [130].
Out-of-Distribution	Out-of-Distribution (OoD) refers to data points in machine learning that belong to a different probability distribution from the one used to train the model, often called the In-Distribution (ID) data [153].
Stability-Plasticity Dilemma	The stability-plasticity dilemma [163, 231] is a fundamental challenge in both biological and artificial neural systems that involves finding the correct balance between learning new information and retaining old knowledge. It is defined by two competing requirements: – **Plasticity**: The system must be flexible enough to integrate new knowledge and inputs from the environment. – **Stability**: The system must be stable enough to prevent the *catastrophic forgetting* of previously encoded data.

Appendix

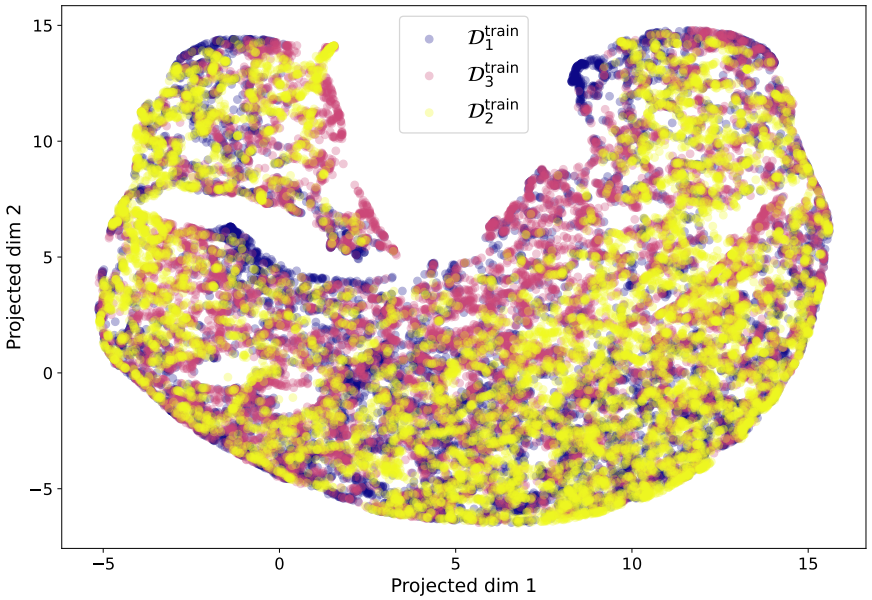


Figure A.1: UMAP projection of $\mathcal{D}_1^{\text{train}}, \mathcal{D}_2^{\text{train}}, \mathcal{D}_3^{\text{train}}$ for Inverted Pendulum where each point is colored depending on the training dataset it comes from (*blue* $\rightarrow \mathcal{D}_1^{\text{train}}$, *yellow* $\rightarrow \mathcal{D}_2^{\text{train}}$, *pink* $\rightarrow \mathcal{D}_3^{\text{train}}$)

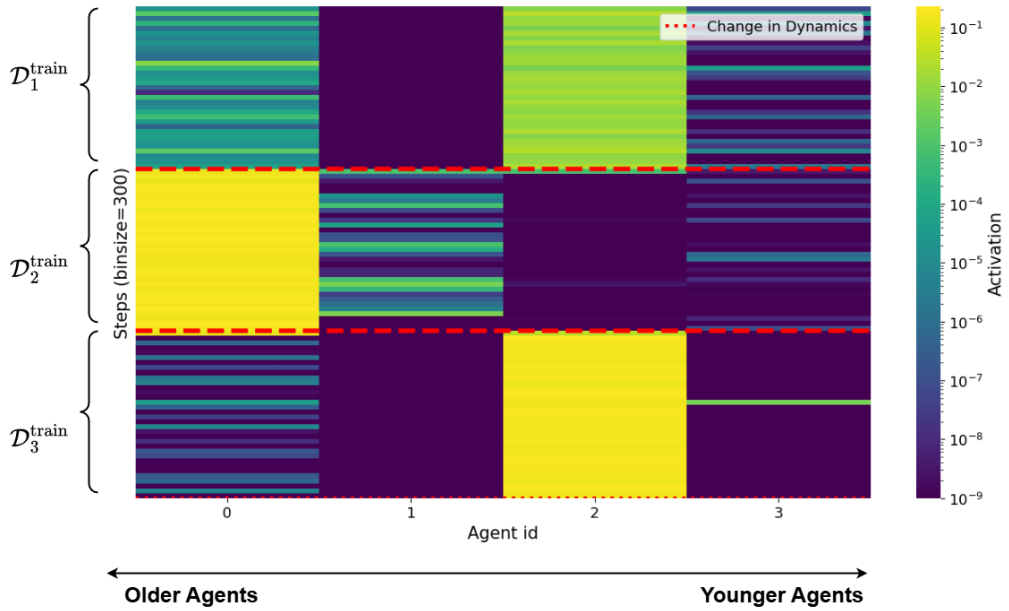


Figure A.2: Heatmap of the final agents activation values on the whole sequence of training data for the Inverted Pendulum. The x-axis represents individual agents sorted by age (oldest to youngest), and the y-axis represents the chronological sequence of training data ($\mathcal{D}_1^{\text{train}} \rightarrow \mathcal{D}_2^{\text{train}} \rightarrow \mathcal{D}_3^{\text{train}}$) aggregated into bins of 300 samples. The dotted horizontal red lines corresponds to changes in dynamics ($\mathcal{D}_i^{\text{train}} \rightarrow \mathcal{D}_{i+1}^{\text{train}}$). The colors corresponds to activation values of agents over each aggregated training data bins with yellow meaning high activation and blue meaning low activation.